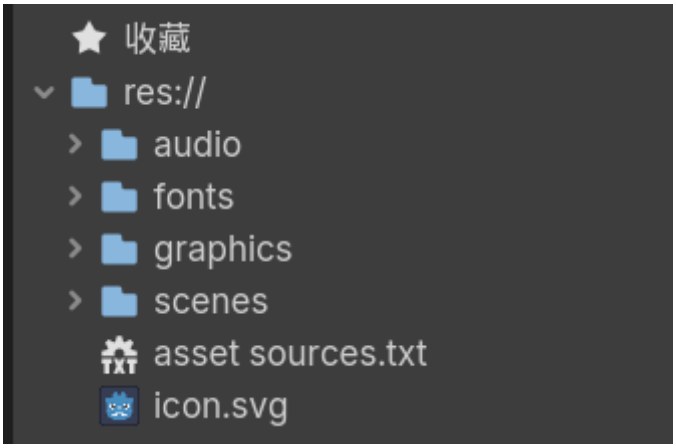
Godot学习笔记

1 青蛙过河类游戏

上

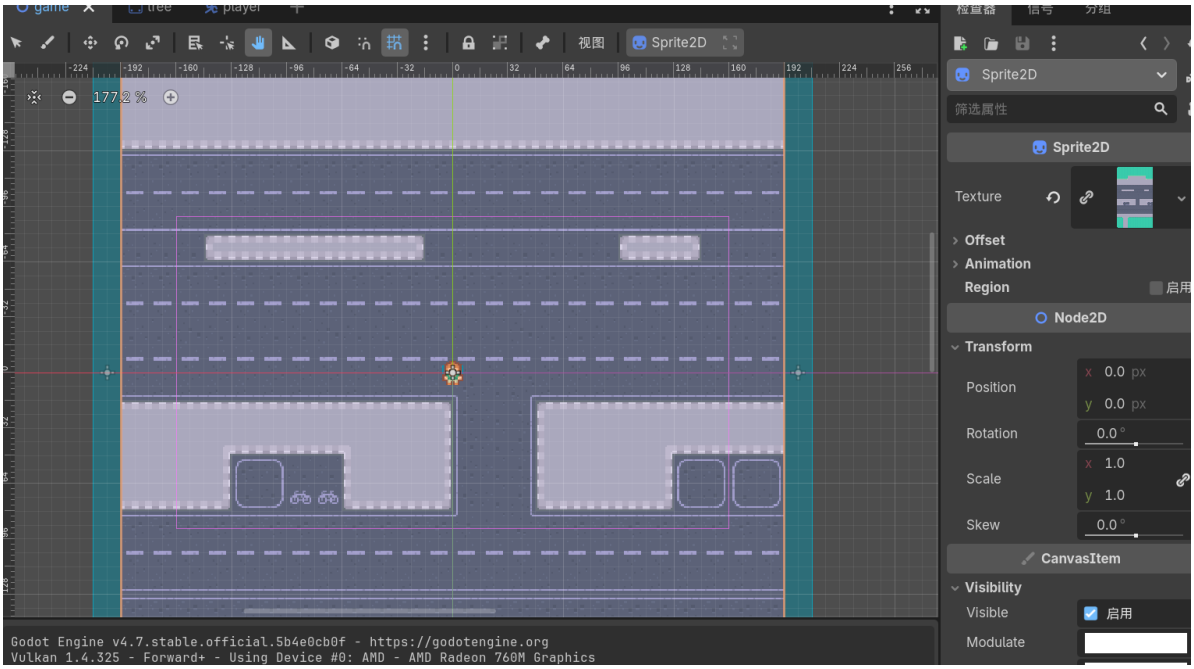
1.项目搭建

打开项目文件，音频、字体、图片等资源已准备好，自行创建scenes文件夹，用于存放场景。

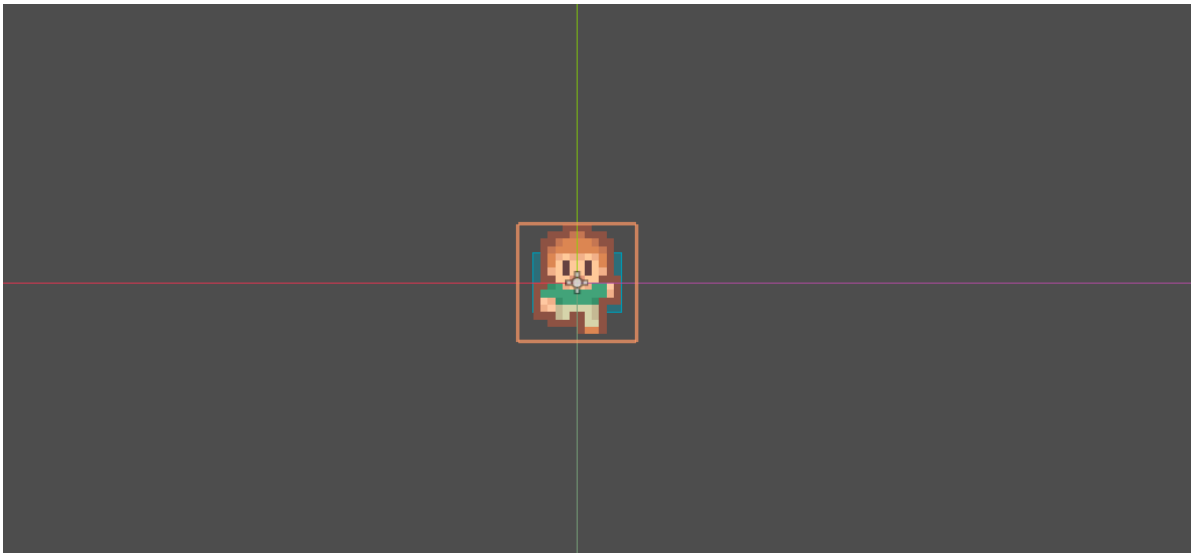


2.基础入门

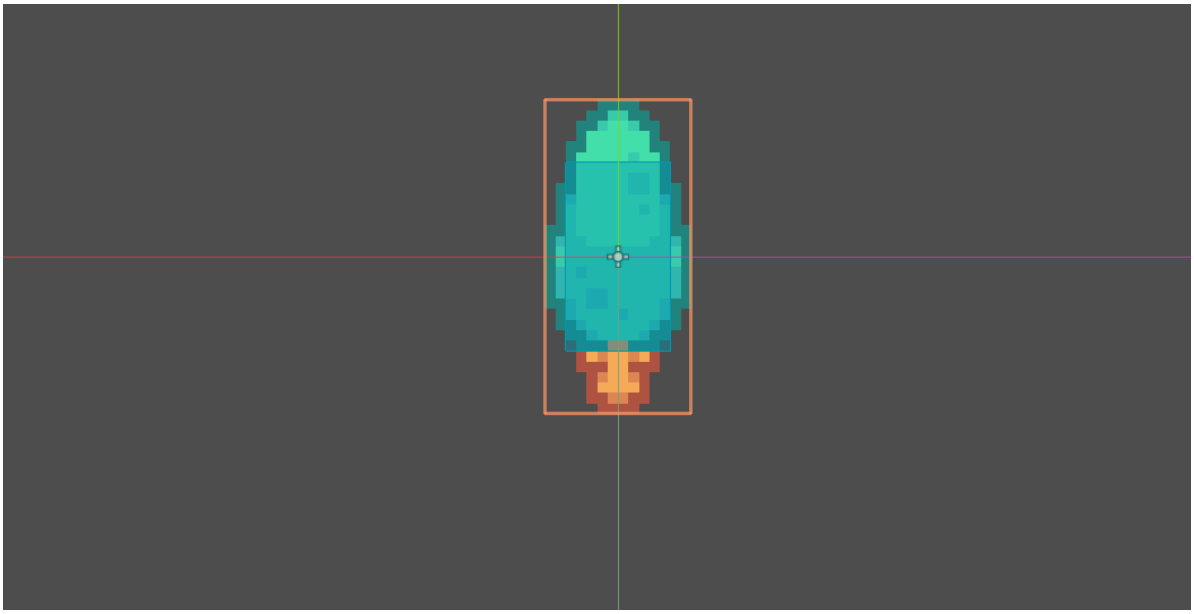
创建Game场景作为游戏的主体场景，游戏的所有内容都将显示在该场景中。在Game场景以Node2D作为根节点，创建Sprite2D子节点，将graphics中的map.png拖动到节点的Texture属性上，将该节点作为地图



同理，创建player场景，作为角色的场景，同样创建Sprite2D子节点，将graphics中player下的player_simple.png拖动到Texture属性上。

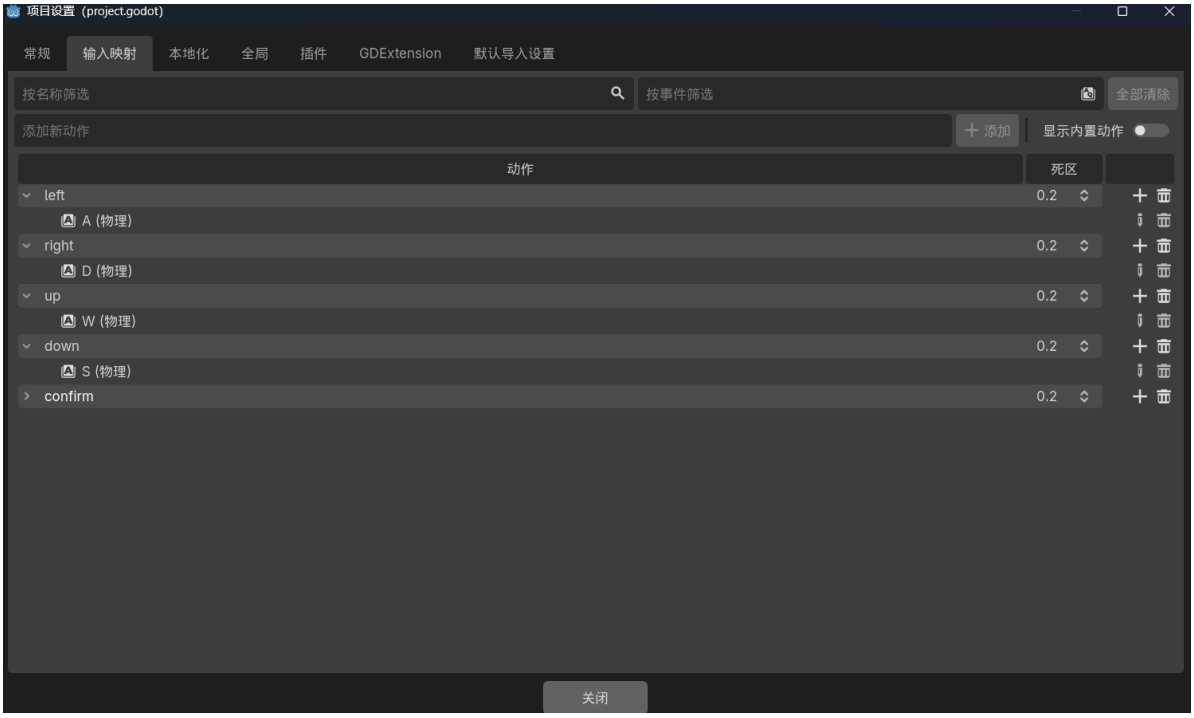


还可以再从graphics中object文件中挑选一些树木之类的物体，创建一个tree场景，后续可以用于测试碰撞箱。



3.获取玩家输入

点击项目，再点击项目设置，在顶部标签页中选择输入映射，添加"left","right","up","down"动作，可快速通过输入监听来绑定按键，分别绑定a、d、w、s四个键作为控制四个方向的按键



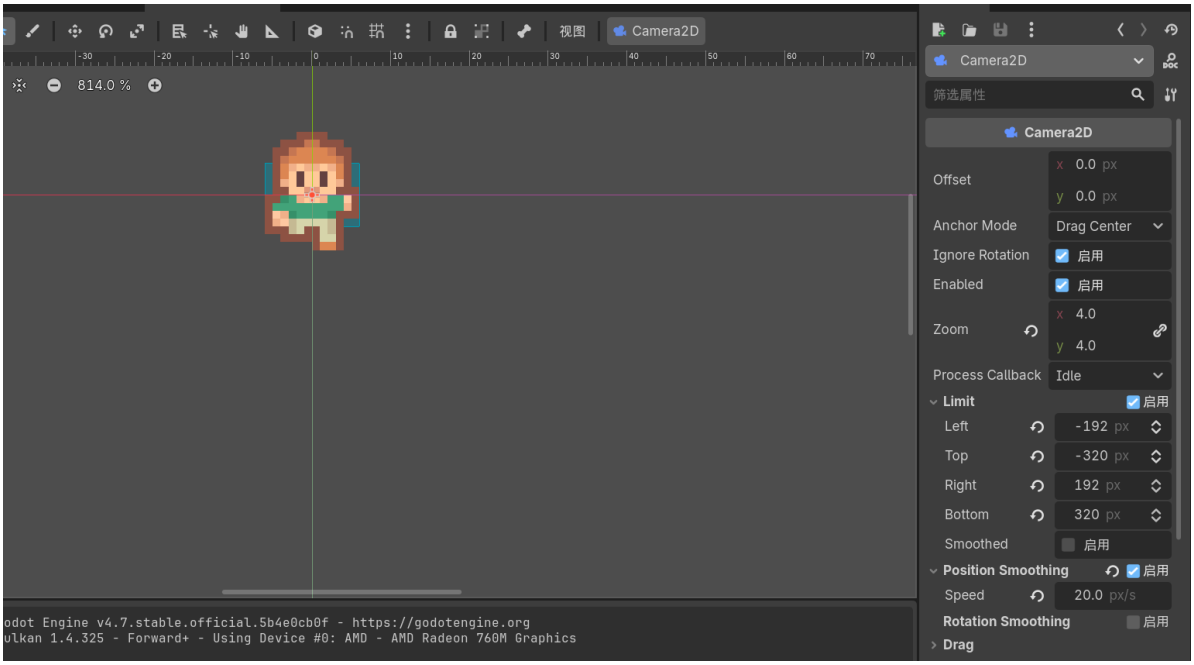
在player场景中对玩家节点添加脚本，设置direction变量作为方向变量，添加speed变量作为预设速度变量。添加_physics_process函数，通过Input.get_vector方法来接收用户的输入，move_and_slide函数根据velocity速度向量来移动角色。至此实现角色获取玩家输入移动的功能，添加"confirm"动作，绑定空格键，每次按下空格键，都将后台在输出"跳"，且每按一次都之输出一次，不会持续输出，使用Input.is_action_just_pressed方法实现，代码如下。

```
func _physics_process(_delta: float) -> void:
    direction = Input.get_vector("left","right","up","down")
    velocity = direction * speed
    animation()
    move_and_slide()

    if Input.is_action_just_pressed("confirm"):
        print("跳")
```

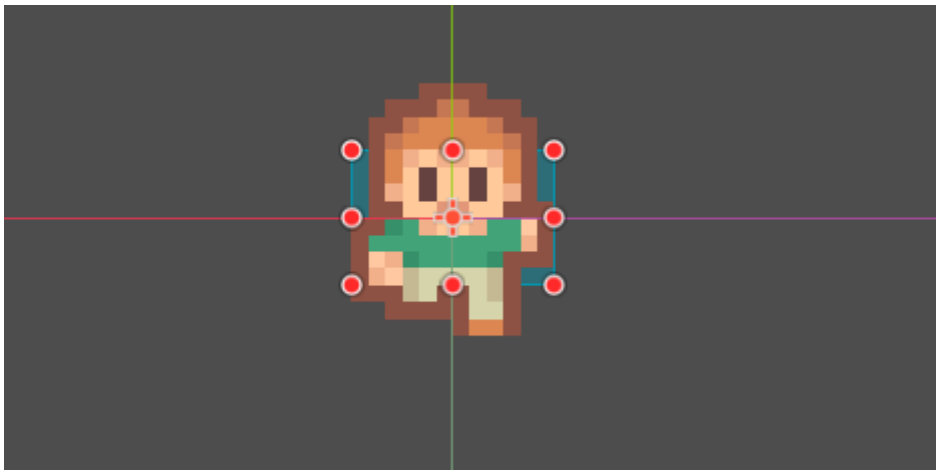
4.摄像机系统

player场景中添加Camera2D节点，它会使镜头自动跟随父节点移动。在Camera2D节点的Zoom属性设置视野的缩放，Limit属性中设置视野的边界，可以使用标尺来测量地图的详细位置以便精确地设置边界。推荐勾选Position Smoothing选项，使镜头移动更加平滑，根据实际情况设置平滑效果的速度。

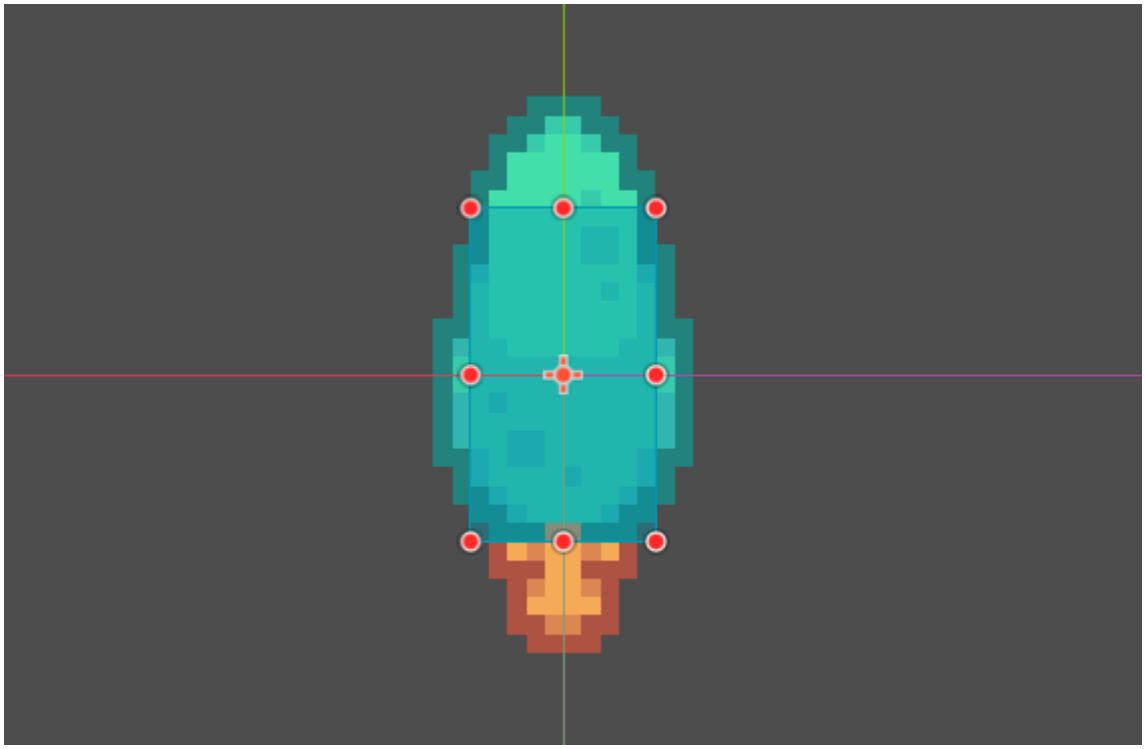


5.物理引擎

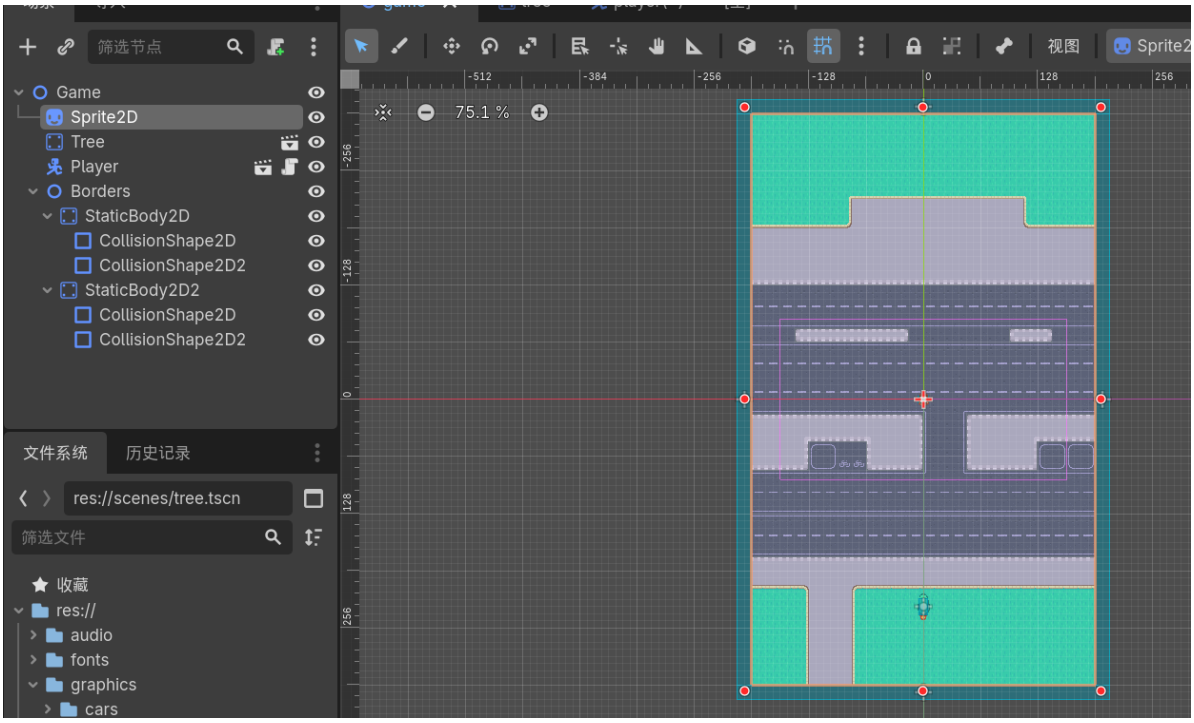
首先设置角色的物理模型，将player根节点修改为CharacterBody2D类型节点，下添加子节点CollisionShape2D，Shape属性设置为矩形，拖动位置并调整大小使其覆盖住角色的主要部分，角色的物理模型就设置好了，如图。



同理，为tree场景设置物理模型，将tree的根节点修改为StaticBody2D类型，添加CollisionShape2D子节点并设置为矩形，覆盖住树的主要部分，如图。



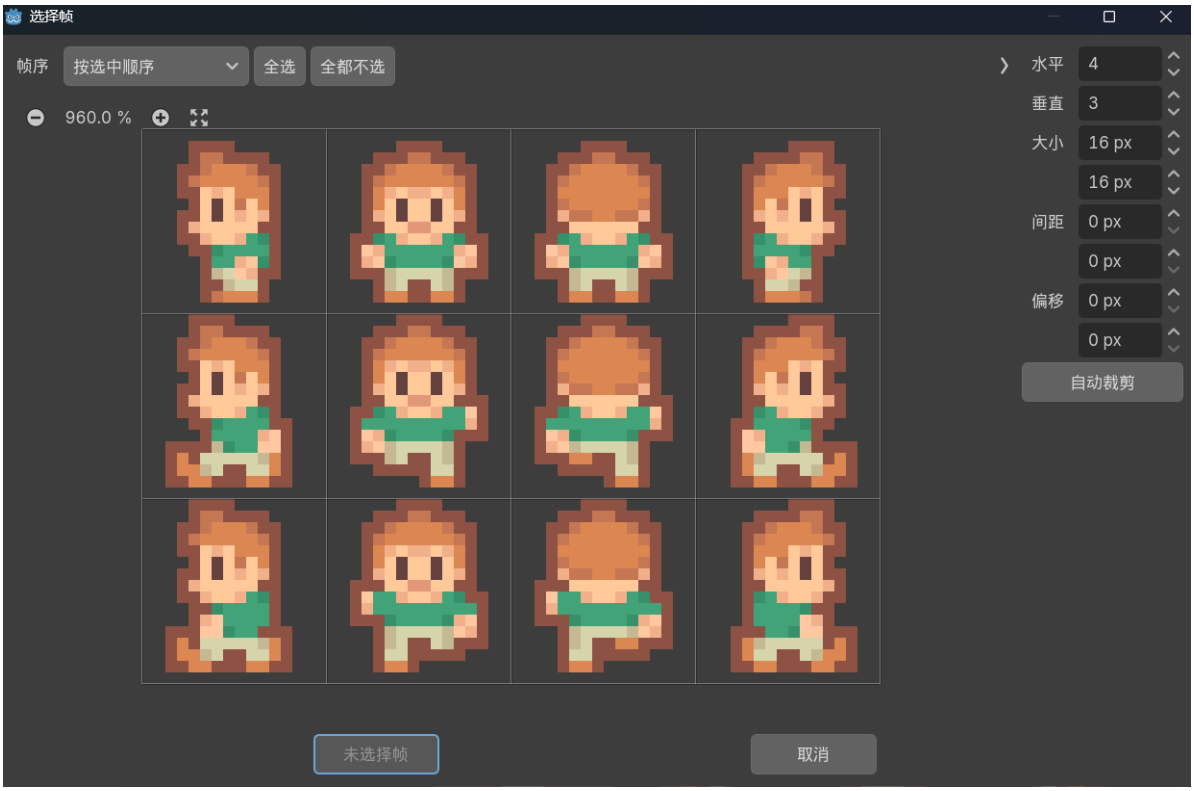
同样的，使用CollisionShape2D节点可以为地图设置边界。先创建Node2D节点Border，为其添加StaticBody2D子节点，再为StaticBody2D节点添加两个CollisionShape2D节点，将其调整到合适的大小，覆盖住地图的左右边界。上下边界和左右边界不能在同一个StaticBody2D节点中，因为它们的变化是相关联的，所以上下边界需要挂载在另外的StaticBody2D节点中，如图.



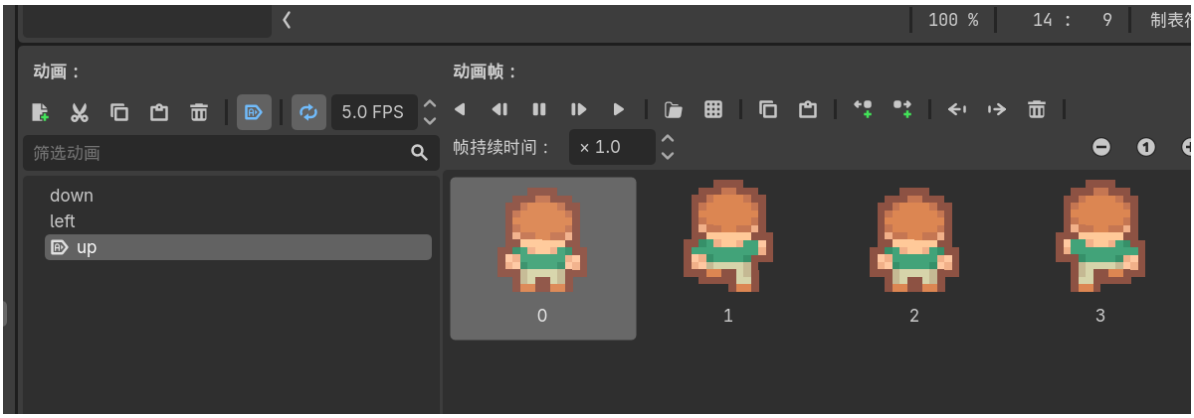
至此，游戏的物理引擎已设置完毕。在Game节点中实例化Player和Tree，检查它们的碰撞箱是否正常工作。

6.动画制作

将player场景中的Sprite2D节点改为AnimatedSprite2D节点，Sprite Frames属性右侧下拉箭头点击新建SpriteFrames，在SpriteFrames窗口中设置。先点击添加动画，添加”left“、”up“、”down“，因为向右移动可以通过操作flip_h属性反转左移动动画实现，所以不需要添加”right“。点击“从精灵表中添加帧”按钮，选择graphic文件夹中player下的player_tileset.png打开，如图所示。



设置好对应的水平和垂直行列数，Godot会自动划分好帧，按需求选择后，调整帧顺序使画面流畅，将”加载后自动播放“和”动画循环“选中，如图。



三种状态的动画设置好后，打开玩家节点的脚本player.gd，编写函数animation，具体代码如下。

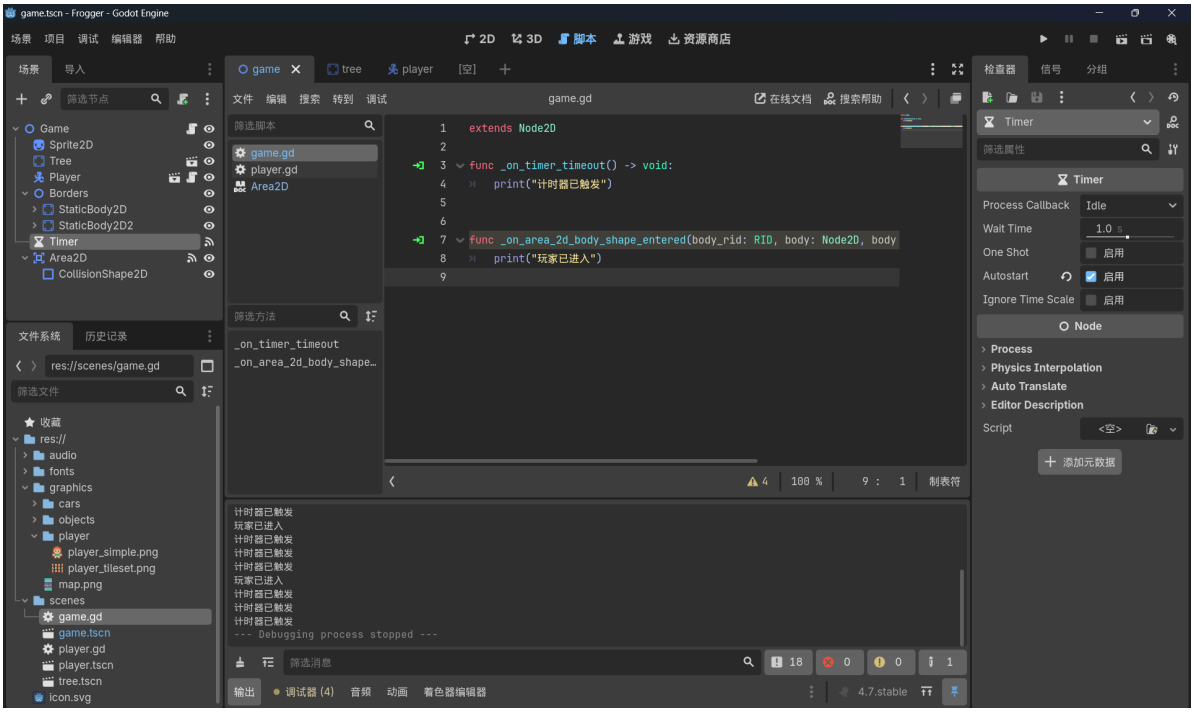
```
func animation():
    if direction:
        $AnimatedSprite2D.flip_h = direction.x > 0#若方向向右，将画面翻转
        if direction.x != 0:
            $AnimatedSprite2D.animation = 'left'#当水平移动，使用left动画
        else:f
            $AnimatedSprite2D.animation = 'up' if direction.y < 0 else 'down'
    else:
        $AnimatedSprite2D.frame = 0#当无用户输入时，动画定格在当前动画的第一帧静止
```

7.信号通信机制

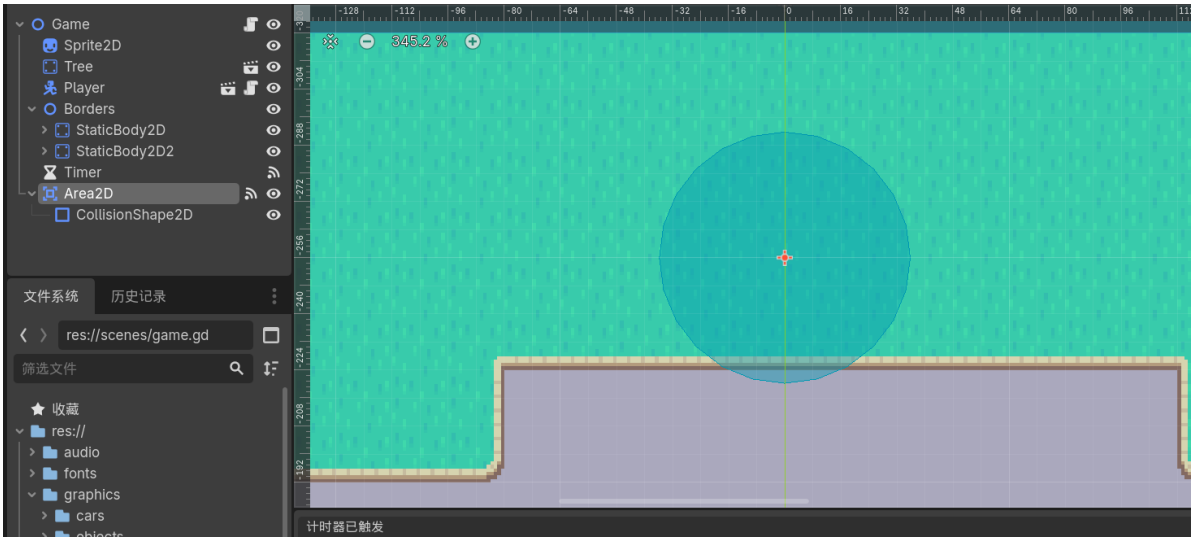
要制作移动的车辆，需要信号相关设置

在game场景中添加timer节点，添加后选中timer节点，可以在信号页面看到timeout信号，需要将其连接到脚本。

在game场景中添加脚本后，双击timeout信号，将其链接到game脚本中。在game脚本中设置on_timer_timeout( )方法，先设置为每隔一秒在控制台打印"计时器已触发"。即计时器的Wait Time属性设置为1.0s，Autostart设置为启用。运行游戏可见控制台每秒打印文本。

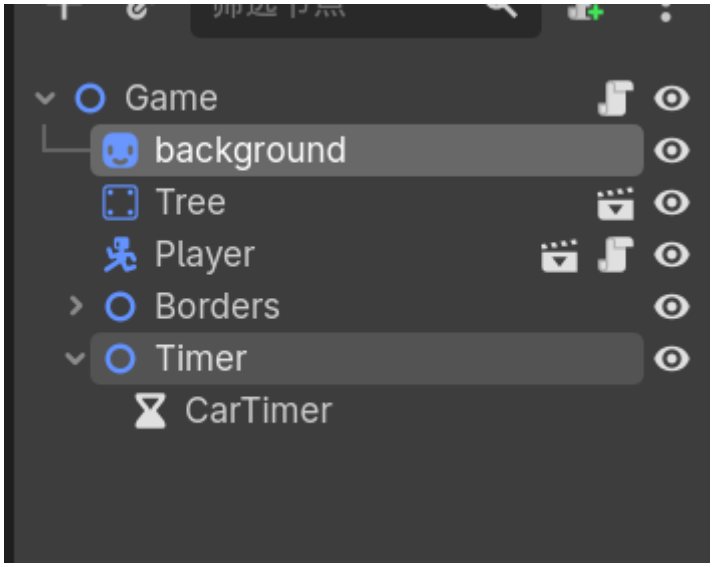


另外可以设置Area2D节点，添加CollisionShape2D节点，选中节点查看信号，我们选用body_shape_entered信号，绑定到game脚本，在_on_area_2d_body_shape_entered方法中设置，每当角色进入区域，即打印"玩家已进入"。



运行游戏可见控制台正确打印。

为让项目结构更加清晰，我们在game场景新建Node2D节点，命名为Timer，将原本的Timer节点作为其子节点，命名为Cartimer，将Area2D节点命名为FinishArea2D并移入Border中。将Cartimer的timeout信号断开与game的连接。整理后项目更加简洁，如图。



下

8.车辆生成

要实现车辆生成功能，先创建Car场景，同样使用Sprite2D节点设置其外观，从res://graphics/cars/中任意选择一个图片拖入Sprite2D节点的Texture属性中。再为Car添加CollisionShape2D节点，设置其物理模型。

通过计时器加脚本的方式将车辆实例化到game场景中，点击timer中Cartimer的timeout信号，将其链接到game中，在game脚本中使用PackedScene类型创建变量car_scene，使用preload方法选择car场景为其赋值。

```
var car_scene: PackedScene = preload("res://scenes/car.tscn")
```

在_on_car_timer_timeout方法中，将car_scene实例化并赋值给变量car，后面可以用as指明变量类型，方便生产代码补全。

```
var car = car_scene.instantiate() as Area2D
```

使用add_child(car)方法将车辆添加到实例中。

此时车辆可以在game场景中被正确实例化，接下来需要完善车辆的运动逻辑。

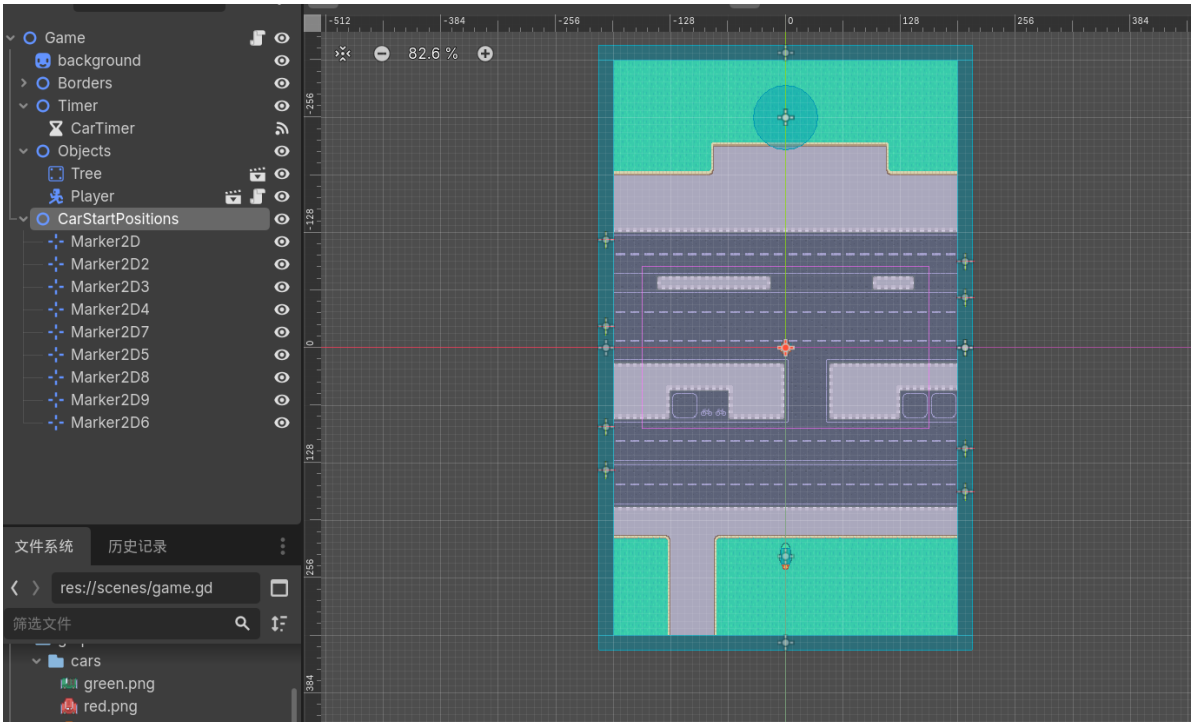
创建car脚本，内容编写如下，车辆运动方向预设为向左，只需在ready函数中添加条件判断，若车辆初始位置的x 坐标小于0，即车辆在屏幕左侧生成，直接将direction.x设置为1，即可设置将运动方向设置为向右。

```
var direction = Vector2.LEFT
var speed = 2

func _ready() -> void:
    if position.x < 0:
        direction.x = 1
        $Sprite2D.flip_h = true

func _process(delta: float) -> void:
    position += direction * speed
```

还需要设置车辆生成位置，确保车辆在地图两侧正确位置生成，可以使用Godot内置的Marker2D节点来指定位置。先在game场景中创建一个新的Node2D节点，命名为CarStartPositions，用于存放所有初始位置，随后新增Marker2D节点，拖拽到地图的各个生成位置，如图。



在game脚本中对_on_car_timer_timeout方法进行修改，创建pos_marker变量，在CarStartPositions中随机挑选位置生产车辆。为确保项目的结构性和整洁性，在game场景中创建Objects对象用于存放所有实例化对象，把Player和Tree拖入其中，将add_child（car）修改为$Objects.add_child(car)，此后生产的车辆对象也会在Object节点下，项目更加整洁。

需要注意，因为ready函数是在车辆节点被添加进场景树的瞬间执行，是在坐标赋值之前执行，因此将车辆添加到场景的操作$Objects.add_child(car)需要在赋值后执行，否则ready函数的判断不会生效，车辆在左侧生成后会向左行驶i，不会出现在画面中。这是经常被忽略的一个时序问题。完整脚本如下。

```
#game.gd
extends Node2D

var car_scene: PackedScene = preload("res://scenes/car.tscn")

func _on_finish_area_2d_body_entered(body: Node2D) -> void:
    print("玩家已进入")


func _on_car_timer_timeout() -> void:
    var car = car_scene.instantiate() as Area2D
    var pos_marker = $CarStartPositions.get_children().pick_random() as Marker2D
    car.position = pos_marker.position
    $Objects.add_child(car)#在位置赋值后添加，确保ready函数正确判断方向
```

```
car.gd
extends Area2D

var direction = Vector2.LEFT
var speed = 2

func _ready() -> void:
```

```
    if position.x < 0:
        direction.x = 1
        $Sprite2D.flip_h = true

func _process(delta: float) -> void:
    position += direction * speed

func _on_visible_on_screen_notifier_2d_screen_exited() -> void:
    queue_free()
```

车辆按照计时器每秒生成，因此一定时间后，车辆虽然会驶出地图，但依然存在，场景内车辆对象过多会降低游戏性能，导致卡顿等，因此需要设计车辆的销毁机制。

在Car场景中添加节点VisibleOnScreenNotifier2D，该节点用于检测对象在屏幕中是否可见，调整其大小使其将car覆盖。选中VisibleOnScreenNotifier2D节点，在信号中选择screen_exited信号与car场景连接，即对象在屏幕中不可见时发送信号，在_on_visible_on_screen_notifier_2d_screen_exited（）方法中设置对象销毁操作，如下。

```
func _on_visible_on_screen_notifier_2d_screen_exited() -> void:
    queue_free()
```

至此，车辆生成功能已设置完毕。

9.帧间隔时间

在游戏运行时，很多函数例如process函数是每帧执行一次，由于不同设备性能不尽相同，有的电脑可以每秒30帧运行，有的电脑可以每秒100帧运行，这个时候，使用car脚本中proess中原本的写法，将导致运行效果不同，speed设置为2时，每秒30帧运行的设备上车辆速度是60单位/秒，而每秒100帧设备上车辆速度为100单位/秒，导致游戏在不同设备上运行效果极差。

```
#修改前代码
func _process(delta: float) -> void:
    position += direction * speed
```

我们需要使用deltatime来使3不同设备上游戏运行效果一致。deltatime指当前帧和上一帧的时间间隔，单位是秒，在每秒30帧的设备上，deltatime的值为1/30，在每秒100帧的设备上deltatime的值为1/100，因此，在原先的位置计算公式中再乘delta，即可消除不同帧率对游戏运行效果的影响。

```
func _process(delta: float) -> void:
    position += direction * speed * delta
```

因为修改了公式，speed也应该相应的修改，将speed的值设置为100.

delta及其重要，每当涉及修改物体位置，物体旋转角度，或是Sprite2D这类节点的基础属性时，都应当使用delta，确保帧率不会影响物体速度。

10.碰撞检测

想要实现车辆碰撞场景，可以使用body_entered信号，但在car场景脚本中编写该逻辑不太实用，最好将游戏中的主要逻辑在game场景脚本中编写，因此，可以使用方法进行绑定。写法如下。

```
car.connect("body_entered",go_to_tile)
```

使用connect方法将信号绑定在car对象上，需要传入两个参数，第一个时str型，代表信号名，第二个是不带括号的方法名，代表信号触发后要执行的方法。

编写go_to_tile的方法内容进行测试，要注意方法需要预留出godot中对应信号会自动传入的参数位置，在本例中body_entered信号会自动传入Node2D类型参数body

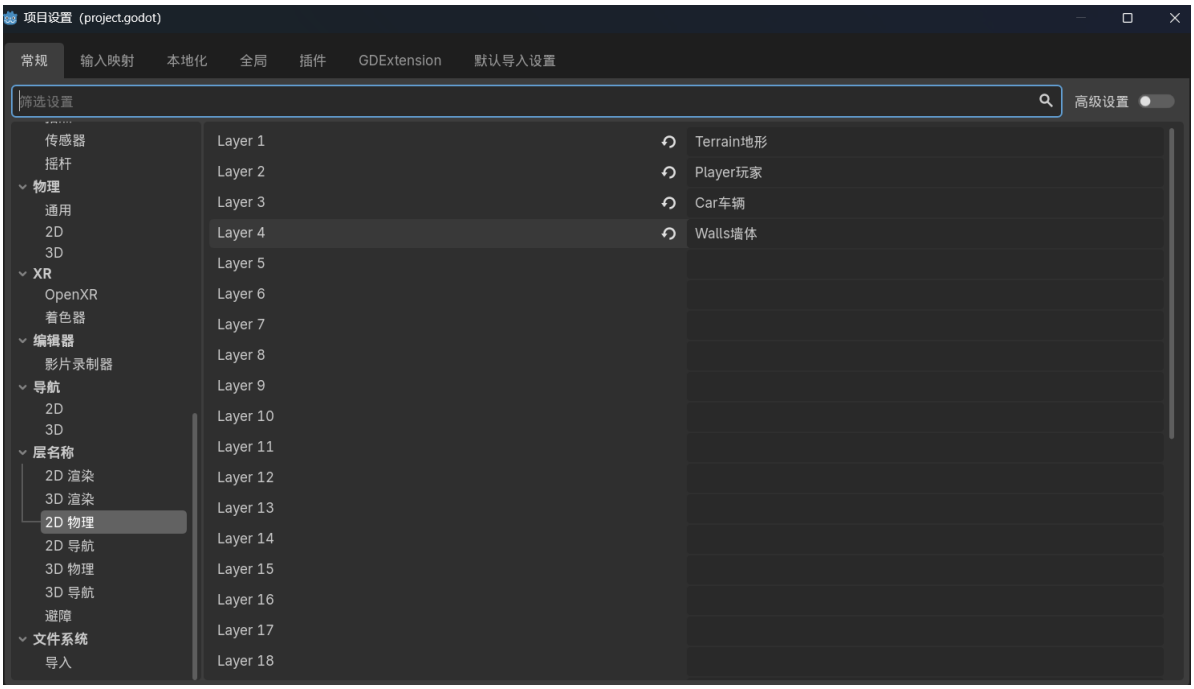
```
func go_to_tile(body):
    print(body)
    print("player car collision")
```

测试发现，控制台会出现大量无关碰撞，这是因为body_entered会监听会检测任意类型物理主题，导致车辆与边界的碰撞也会被检测，为了解决该问题，我们需要使用“碰撞分层”。

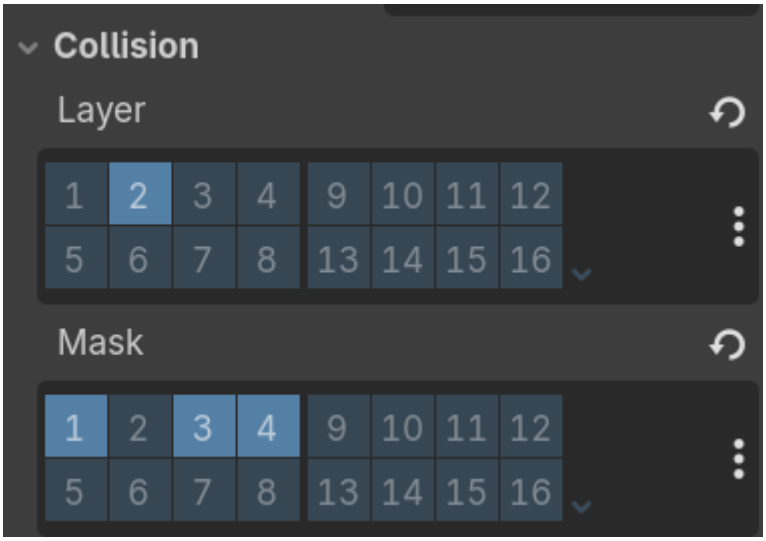
每个场景都有Collisoin属性，打开可以看到Layer属性和Mask属性，Layer代表当前场景属于的层级，Mask代表当前场景能够检测到的层级。



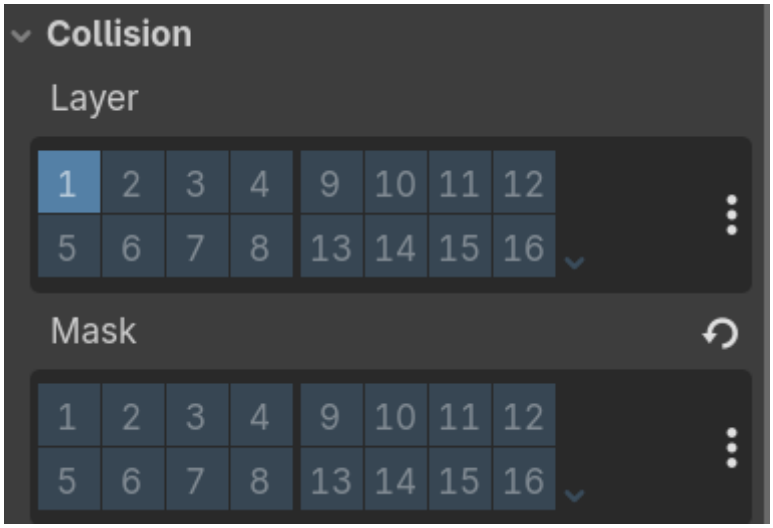
打开项目设置，在“层名称”中找到“2D物理”可以自定义每一个层级的名称，使项目更易理解。将第一层到第四层分别命名为“Terrain（地形）”、“Player（玩家）”、“Cars（车辆）”、“Walls（墙体）”，如图。



通过Collision属性配置，可以定义场景之间的碰撞。如对Player场景进行配置，玩家属于Player，需要可以识别地形、车辆、墙体，故Player场景设置如图。



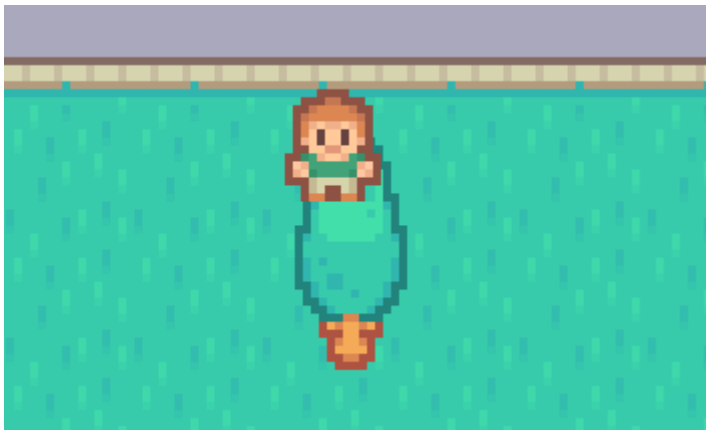
再例如Tree场景，属于地形，而树木自己不需要检测任何物体，因此树木设置如图。



11.关卡搭建

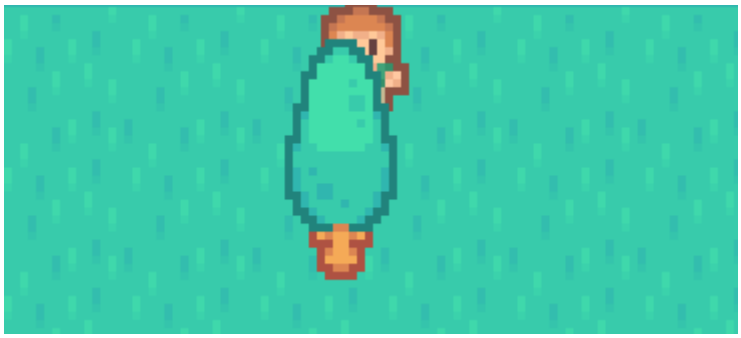
在实现关卡搭建前，我们需要先解决一个问题，即在实际游戏运行时，场景树的顺序决定，Tree先生成，Player后生成，因此玩家始终显示在树木之上。但我们追求的的效果应该是玩家在树下方时，玩家在树前显示，玩家在树上方时，玩家被树遮挡。

image-20260727130015653



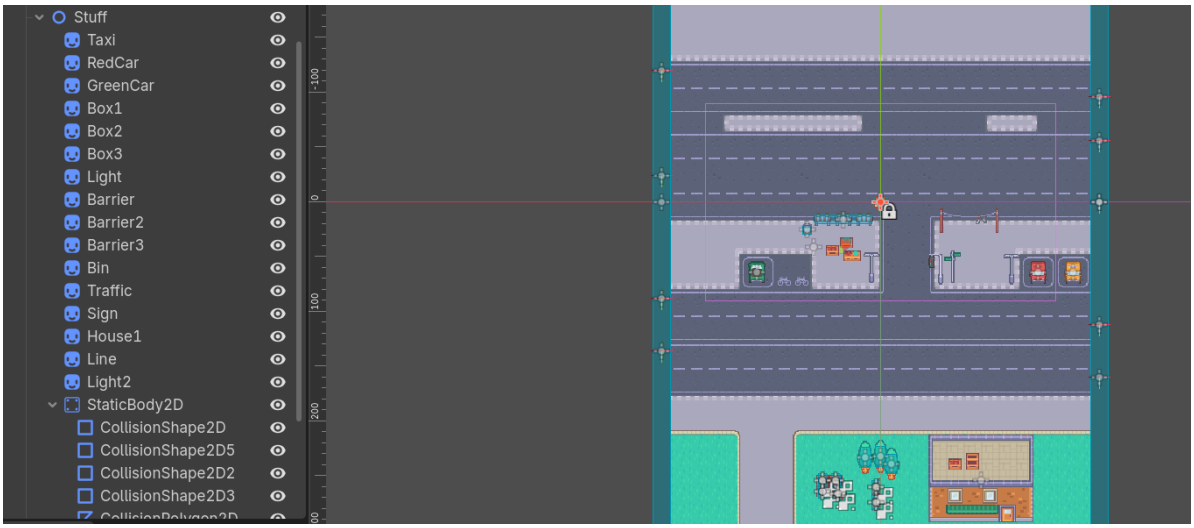
为解决该问题，需要使用Y轴排序，即根据物体的Y轴数值，判断哪个物体在渲染顶层。要开启Y轴排序，先选中Game场景中Object节点，随后在检查器中的Ordering中将Y Sort Enabled选项勾选。再运行游戏，显示效果已正常.





目前Game场景中还是略显单调，我们可以在场景中添加更多物体，使场景更加丰富。

有三种添加方式，第一种就是像添加树一样，单独新建场景，再将其添加到Game场景中。第二种则是直接在Objects节点下新建StaticBody2D节点，拖动图片到场景中为其添加贴图，再为其添加CollisonShape2D节点。这种方式更加快捷。第三种方式则是，我们可以建立一个Stuff节点，先批量拖动图片到场景指定位置中，再在Stuff节点下新建StaticBody2D节点，StaticBody2D节点支持挂载多个CollisionShape2D节点或CollisionPolygon2D节点，为每个图像添加其物理体积，即可批量制作复杂场景。另外不要忘记开机Stuff节点的Y轴排序功能。

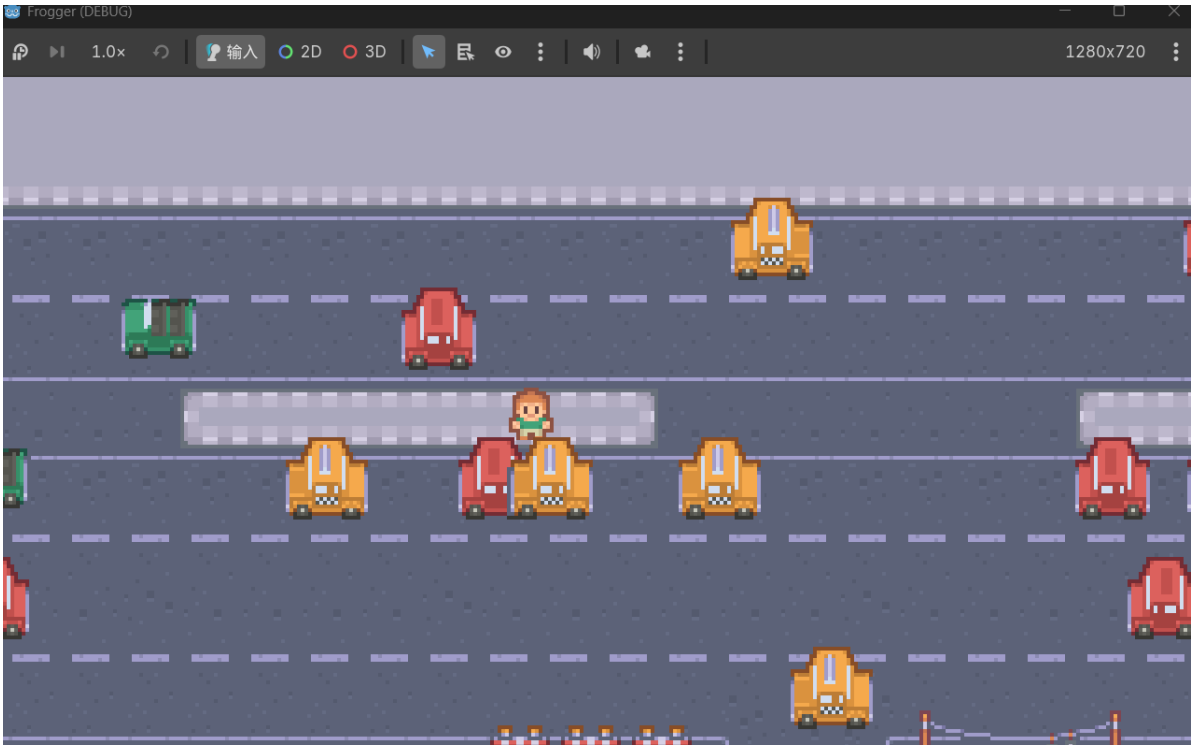


我们还可以发现，目前场景中只会生成单一颜色的车辆，要生成多种颜色车辆，我们可以在Car脚本中将三种车辆贴图读取到内存中，再随机选择生成。具体实现如下。

```
var colors = [#存放三种不同车辆贴图
    preload("res://graphics/cars/green.png"),
    preload("res://graphics/cars/red.png"),
    preload("res://graphics/cars/yellow.png")]

func _ready() -> void:
    if position.x < 0:
        direction.x = 1
        $Sprite2D.flip_h = true
    $Sprite2D.texture = colors.pick_random()#车辆生成时，随机选择一种颜色
```

修改好后，再修改CarTimer中的Wait Time为0.15，使效果更加直观，如图。

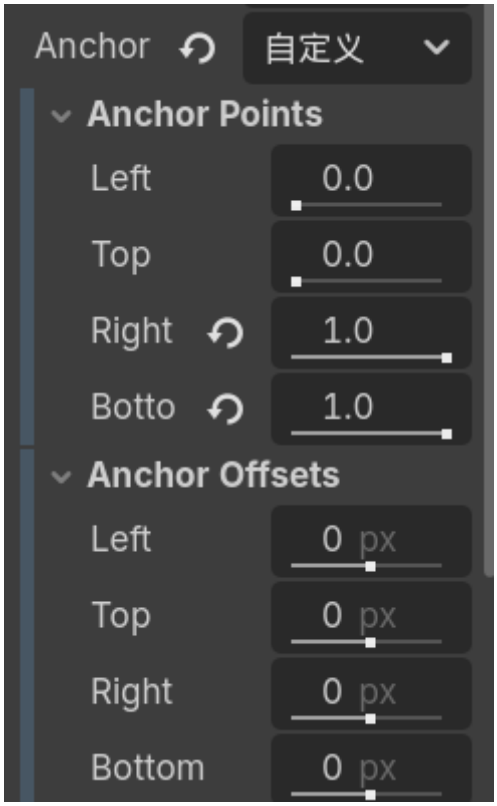


12.UI布局

在之前，我们一直使用的是Node2D类节点，接下来，需要使用Control节点类型来完成标题界面的制作。

新建Control类型场景，添加ColorRect类型节点。

用来制作UI的节点和Node2D节点最大的不同点是定位方式有Position型和Anchors型，Anchors型不采用坐标来确定节点的位置，而是相对于窗口固定。可以在ColorRect节点的Layout属性中切换定位方式，随后设置Anchor为自定义，可以根据需要在Anchor Points中具体配置锚点。 将屏幕的左边界视为0，右边界视为1，上边界视为0，下边界视为1，通过将“Left“、”Top“、”Right“、”Botto“属性修改为0到1范围1内的数字，即可控制节点的相对位置。

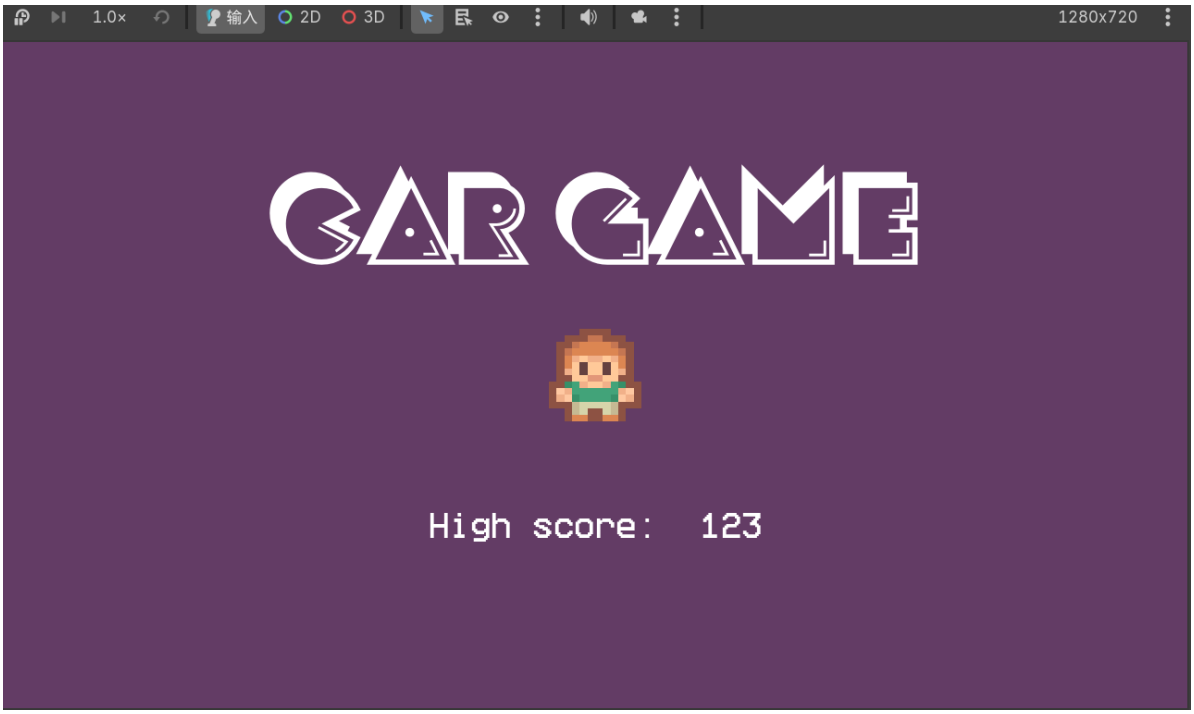


将ColorRect的Anchor Points的”Left“和”Top“设置为0，”Right“和”Botto“设置为1，用于标题界面的背景。

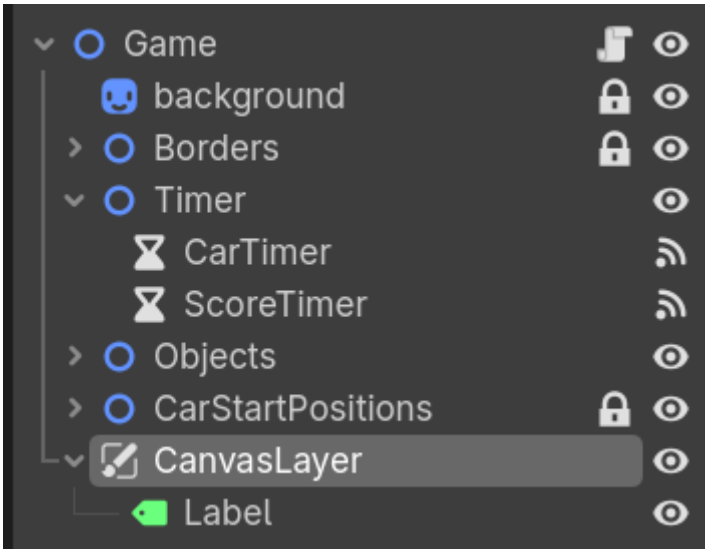
再在Control下添加一个Label节点，Text属性设置标题”Car Game“，可以在Theme Overrides属性中设置字体样式和大小，并设置其Layout属性使其置于画面中间上方，作为游戏名称。

中间则放置一个TextureRect节点，将player_simple.png拖入其Texture属性中，调整Layout属性中的Transform中的size以改变其大小，并通过锚点预设使其居中。

最后添加一个Label节点用于显示High score，操作同游戏名称。最终效果如图。



接着，我们需要在game场景顶部添加一个计分文本显示，为了让文本始终显示在屏幕的上方固定位置，我们需要使用CanvasLayer节点。在主场景中添加CanvasLayer节点，在CanvasLayer节点下添加Label用于显示当前分数。

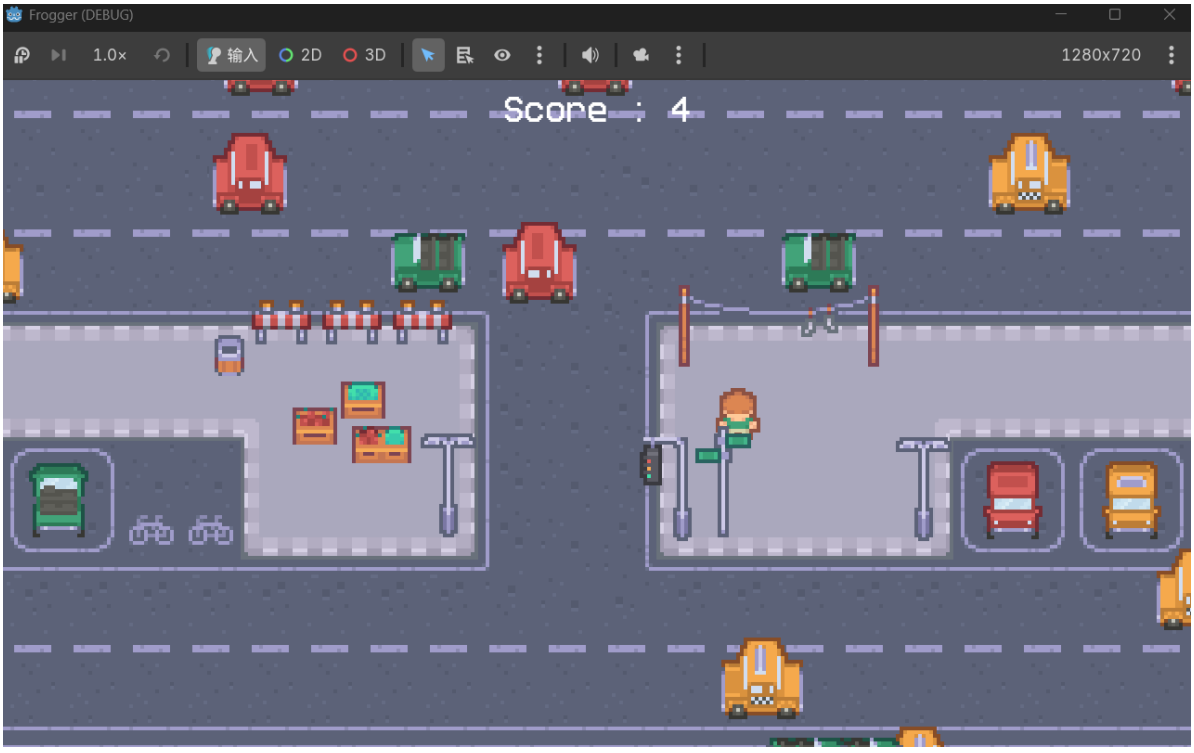


编辑好Label的样式和初始内容后，在Timer中添加一个计时器节点ScoreTimer，用于每秒更新分数，勾选ScoreTimer的Autostart，并设置等待时间为1秒。将该计时器的timeout信号绑定到game场景中。

打开game脚本，新增一个int型变量score，用于记录当前分数，在_on_score_timer_timeout方法中设置时间显示逻辑，如下。

```
func _on_score_timer_timeout() -> void:
    score += 1
    $CanvasLayer/Label.text = "Score :" + str(score)
```

最终效果如图。



13.场景切换过渡动画

要实现三种情况下游戏在标题界面和主场景间的切换，第一是玩家被车撞或进入FinishArea2D，则切换到标题界面，并记录最高分数。第二是玩家在标题界面按下空格，切换到游戏主场景。第三是

第一种切换，修改之前创建的_on_finish_area_2d_body_entered函数，可以使用get_tree方法先获取场景树，再使用change_scene_to_file方法切换到指定参数的场景，运行发现，可以正常跳转，但调试器报了如下错误。



这是因为Godot在场景切换时会在物理帧调用期间移除CollisionObject碰撞体节点，这种操作可能引发多种潜在错误，解决方法也很简单，在_on_finish_area_2d_body_entered方法中使用call_deferred方法指定在空闲期间调用change_scene方法，随后我们新建change_scene方法，在change_scene方法中编写场景切换逻辑，如下，这样虽然麻烦，但可以避免大量潜在问题，一般涉及到场景切换操作时，都应用这种方式。

```
func _on_finish_area_2d_body_entered(_body: Node2D) -> void:
    call_deferred("change_scene")

func change_scene():
    get_tree().change_scene_to_file("res://scenes/title.tscn")
```

之前在"碰撞检测"部分我们使用car.connect将body_entered信号绑定到car，并会在触发时调用go_to_file方法，我们只需要修改go_to_tile方法，让其同样直接调用call_deferred方法即可，如下。

```
func _on_car_timer_timeout() -> void:
    var car = car_scene.instantiate() as Area2D
    var pos_marker = $CarStartPositions.get_children().pick_random() as Marker2D
    car.position = pos_marker.position
    $Objects.add_child(car)
    car.connect("body_entered",go_to_tile)

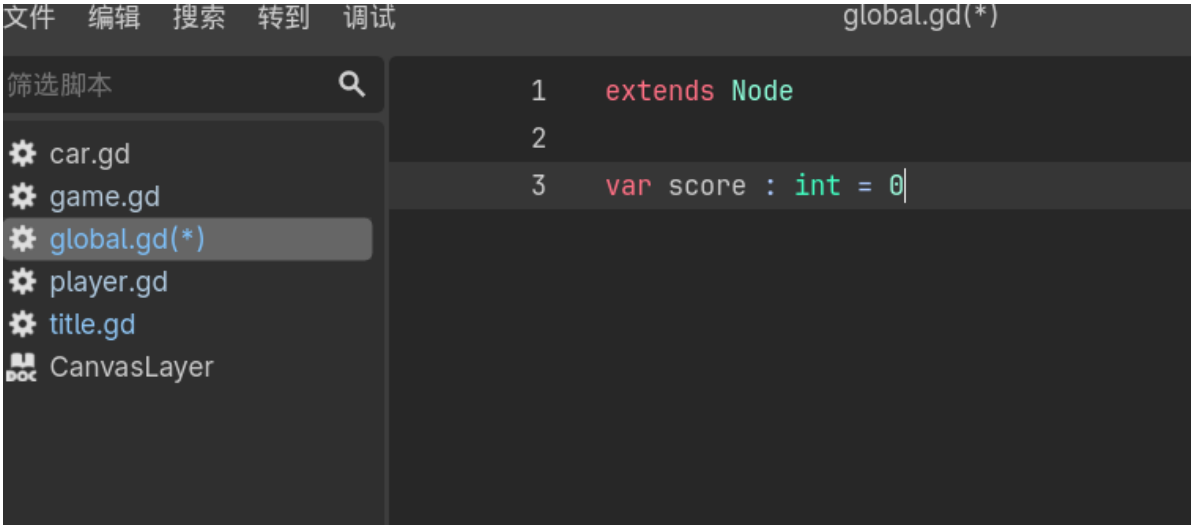
func go_to_tile(_body):
    call_deferred("change_scene")
```

需要注意的是，在Godot中，方法中没有被使用到的参数将会触发Godot的警告，可以在该参数前添加下划线防止触发警告。

第二种切换是在标题场景，我们添加title场景的脚本，在title脚本的process方法中添加判断，若玩家在标题场景，并按下空格键，即跳转到game场景，之前在"获取玩家输入"部分已将空格绑定为confirm，此时可以直接使用，具体如下。

```
func _process(_delta: float) -> void:
    if Input.is_action_just_pressed("confirm"):
        get_tree().change_scene_to_file("res://scenes/game.tscn")
```

两个场景互相跳转已完成，但在title场景中的最高分文本还是假数据，需要实现记录并显示最高分的功能。因此我们需要使用全局变量。先添加一个以基础Node节点作为根节点的场景，将其命名为Global，我们不会在该场景内编写复杂的逻辑，仅用于存放全局变量。为Global挂载脚本，在脚本内新建Score变量，初始值为0，如图。



打开项目设置，在"全局"标签页中选择Global场景，即可将Global设置为全局场景，默认勾选全局变量选项。即无需将Global在别的场景中实例化，也可以访问其中的数据。

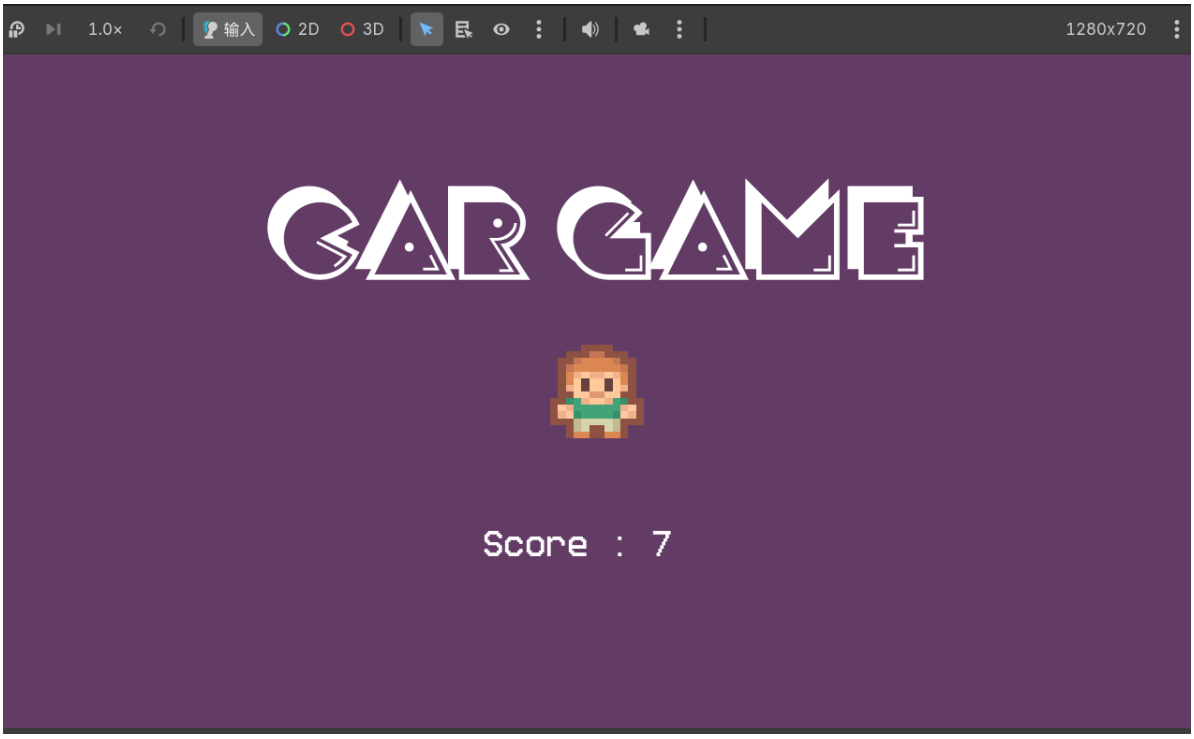
在game脚本中修改_on_finish_area_2d_body_entered方法，走到终点后将当前的score赋值给Global.score，如下。

```
func _on_finish_area_2d_body_entered(_body: Node2D) -> void:
    call_deferred("change_scene")
    Global.score = score
```

再编写title脚本中的ready方法，将label标签内显示文字改为Global.score,如下。

```
func _ready() -> void:
    $Label2.text = "Score : "+str(Global.score)
```

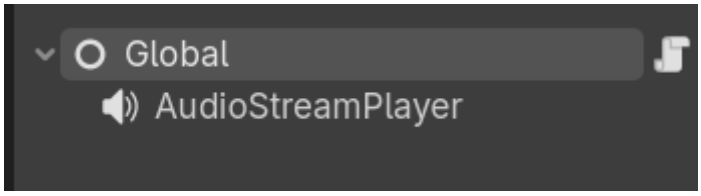
最终运行效果如图。



14.音效与音频系统

整个项目的最后一个部分是添加音频文件，Godot中提供AudioStreamPlayer、AudioStreamPlayer2D和AudioStreamPlayer3D三种音频节点，顾名思义，后二者和前者的区别在于，AudioStreamPlayer2D会受到2D空间的影响，AudioStreamPlayer3D会受到3D空间的影响。

先添加背景音乐，为确保背景音乐在Game场景和Title场景都能播放而且不中断，我们将AudioStreamPlayer节点设置在global场景中。



将背景音乐文件拖入节点的Stream属性中，勾选Autoplay选项，Parameters中勾选Looping开启循环播放。还可以设置Volume d属性以调整音量。

车辆音乐类似，选用AudioStreamPlayer2D节点，添加到Car场景中，同样将车辆音效拖入Stream属性中，启用Looping，设置Max Dist（即音频仍可听到的最大距离）。

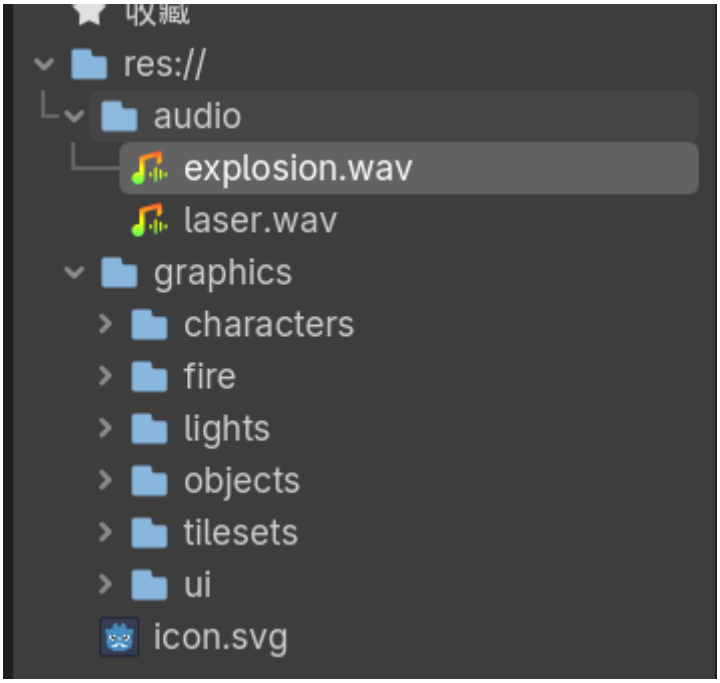
至此，该游戏已制作完成。

2 横板闯关类游戏

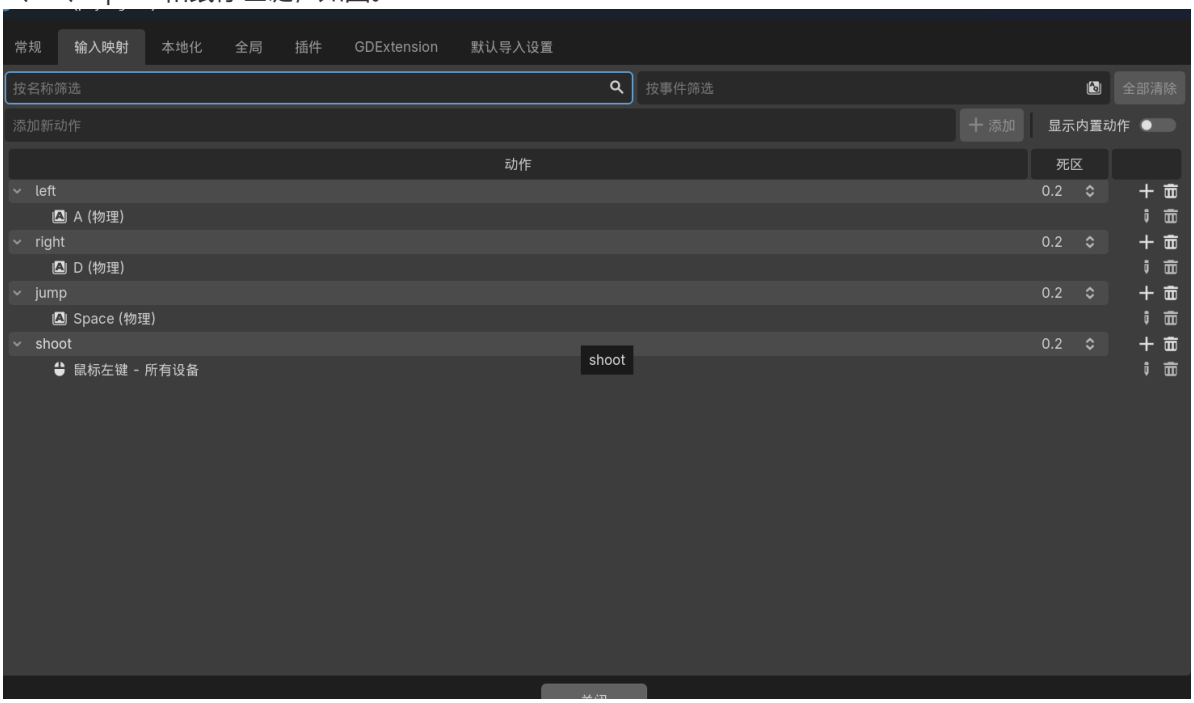
上

1.工程环境搭建

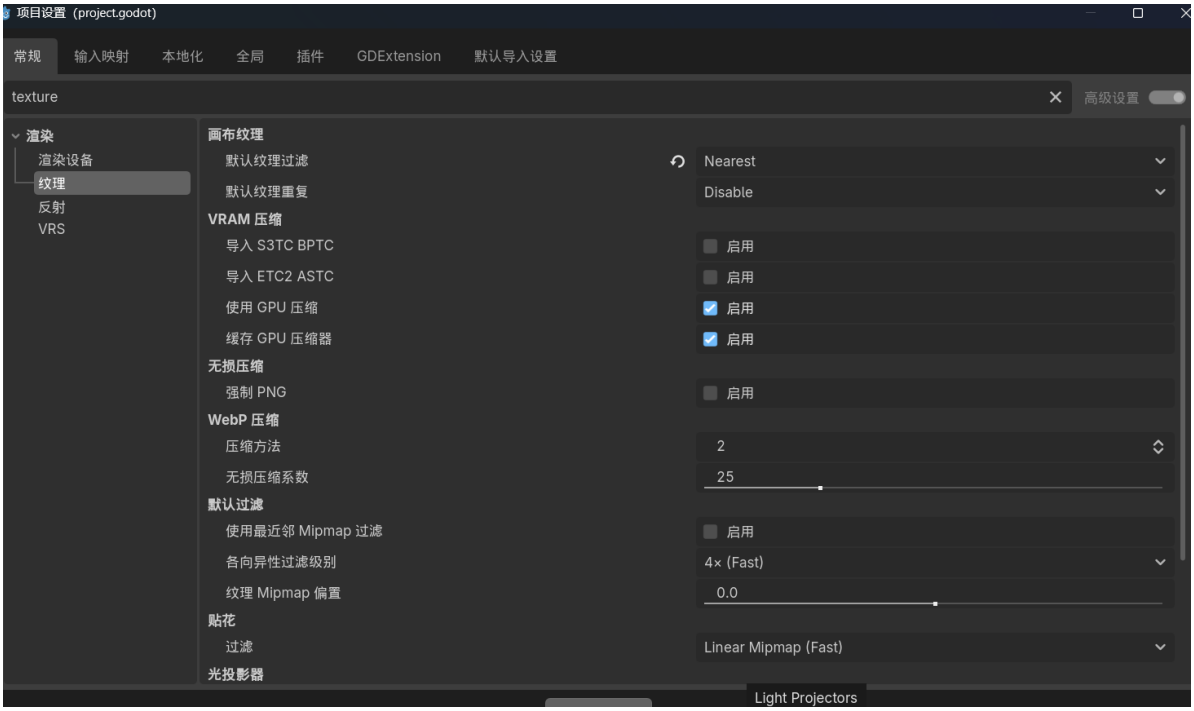
先导入准备好的项目，可以看到其中有准备好的美术及音乐资源，这是后续要用到的项目资源。



接下来，打开项目设置，在输入映射面板中配置好“left”、“right”、“jump”和“shoot”的按键，分别绑定为A、D、Space和鼠标左键，如图。



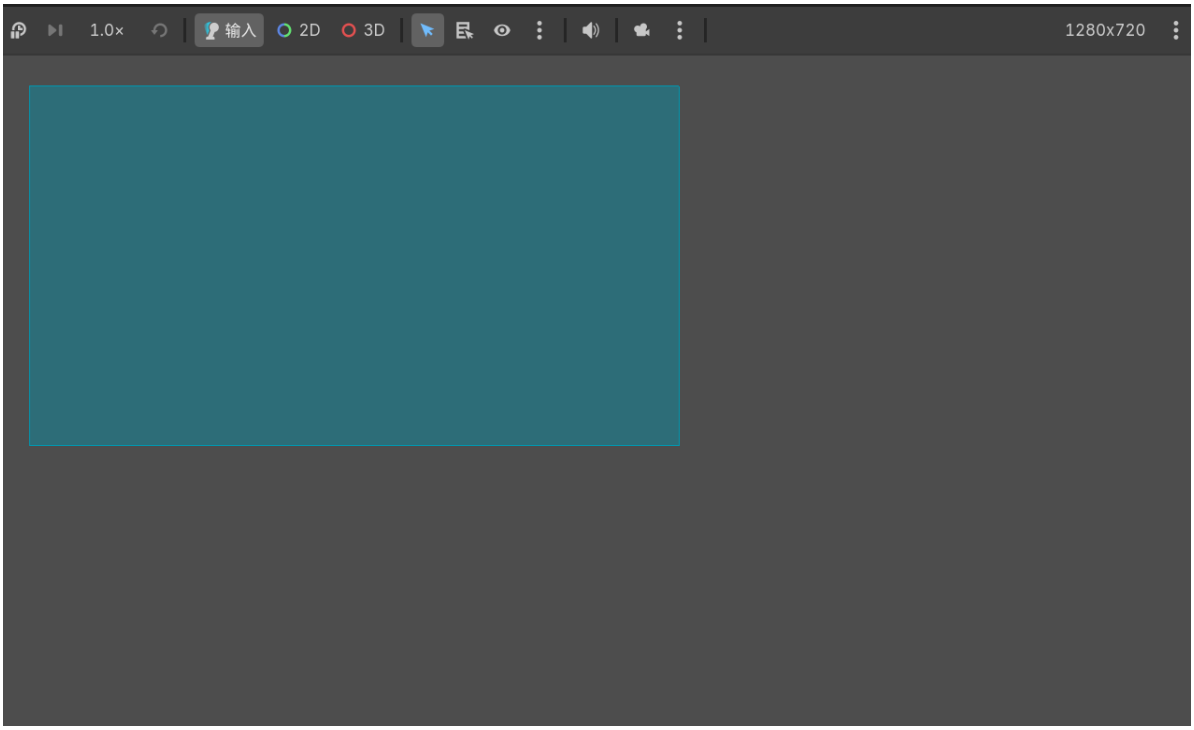
在项目设置的窗口设置中，将大小改为1280x720，适配主流屏幕，另外，在渲染的纹理设置中，将画布纹理的默认纹理过滤设置为Nearest，适用于像素游戏开发。



接下来搭建基础内容，新建scenes文件夹用于存放场景，scenes文件夹下新建levels文件夹，用于保存主关卡场景，新建Node2D场景Level，将其保存到levels文件夹中。

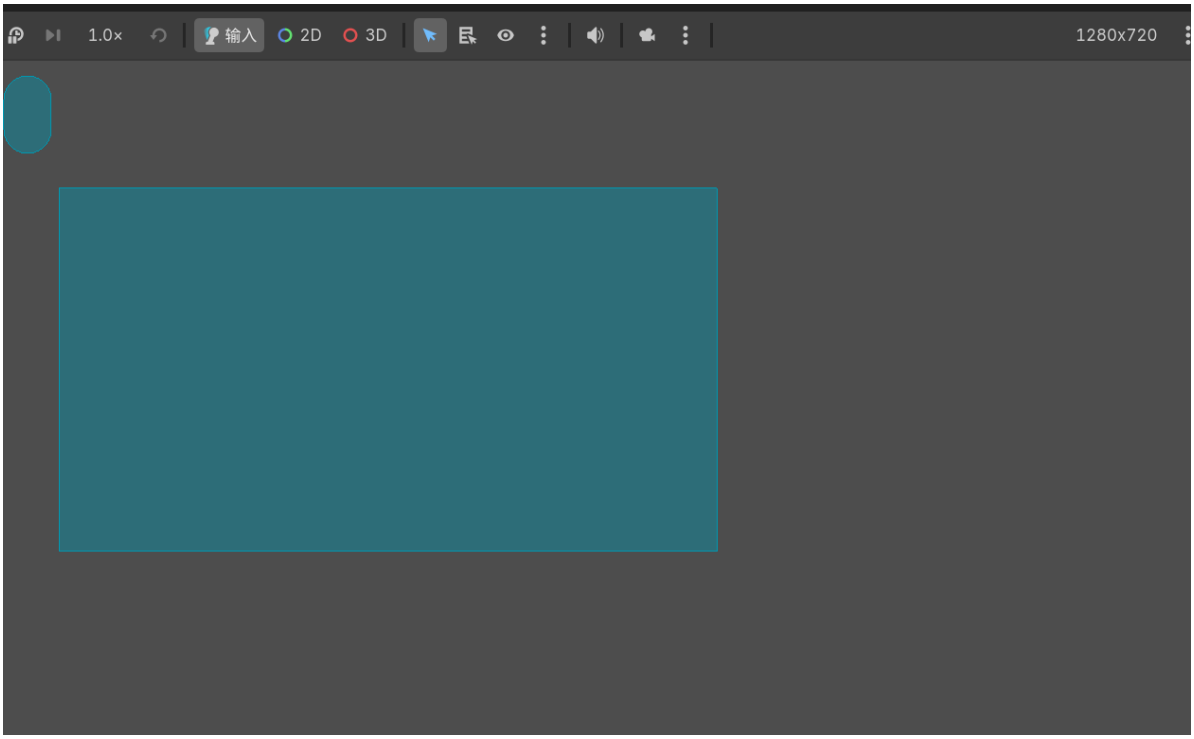


在Level场景中，新建一个StaticBody2D节点，其下挂载一个CollisionShape2D节点，设置为矩形，拉伸到合适大小，拖到屏幕中，用作后续测试。打开调试面板，勾选显示碰撞区域，运行当前场景，如图所示。



接下来新建一个场景，根节点选择CharacterBody2D，命名为Player，用于作为玩家主体。在scenes文件夹中新建一个entities文件夹，角色和后续的敌人都放入该文件夹。为Player添加CollisionShape2D场景，选用CapsuleShape2D形状，先保存。

在Level场景中添加一个Node2D节点，命名为Entities，在其下将Player实例化，运行效果如图所示。



我们先初步实现Player的简单移动逻辑，在Player场景中挂载脚本，编辑脚本的_physics_process方法，如下，即可实现Player匀速向右移动的效果，这只是初步测试效果，后续会大幅优化。

```
extends CharacterBody2D
var direction : Vector2 = Vector2.RIGHT
var speed : int= 50

func _physics_process(_delta: float) -> void:
    velocity = direction * speed
    move_and_slide()
```

至此，工程环境搭建已完成。

2.角色移动逻辑

接下来实现角色左右移动和跳跃的逻辑。

横板闯关类游戏不需要读取上下方向的输入，只需要处理左右移动，再额外实现跳跃操作，下落效果靠重力实现，将direction变量删除，改为direction_x变量，类型为float，velocity.x = direction_x * speed，再新建get_input函数，在其中处理玩家输入，并在physics_process函数中调用。

```
var direction_x : float
var speed : int= 50

func get_input():
    direction_x = Input.get_axis("left","right")

func _physics_process(_delta: float) -> void:
    get_input()
    velocity.x = direction_x * speed
    move_and_slide()
```

运行后按A和D可以看到角色根据输入移动，接下来实现跳跃相关机制。

在get_input函数中添加判断，若按下jump按键，修改角色的y轴速度，如下。

```
func get_input():
    direction_x = Input.get_axis("left","right")
    if Input.is_action_just_pressed("jump"):
        velocity.y = -200
```

测试发现按下空格键后角色正常向上移动，但无法下落，因此还需要添加一个重力处理函数apply_gravity，设置velocity的y值每秒自增10，在_physics_process中调用apply_gravity。运行效果发现角色可以正常下落，完整逻辑如下。

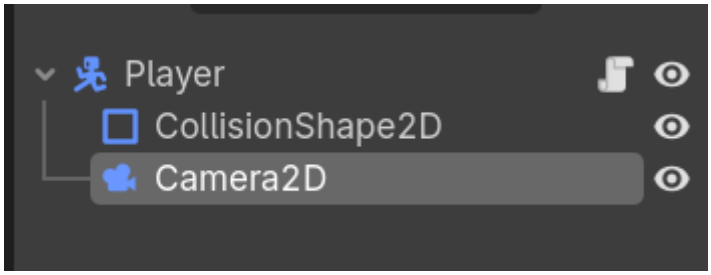
```
extends CharacterBody2D
var direction_x : float
var speed : int= 50

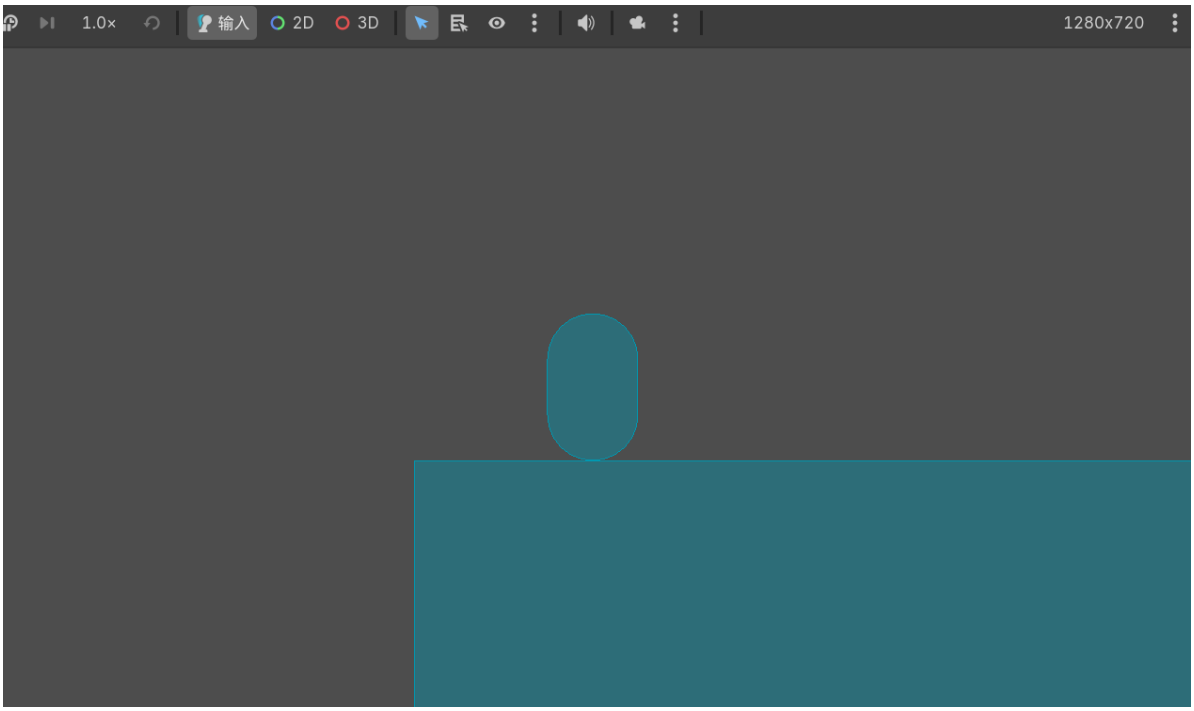
func get_input():
    direction_x = Input.get_axis("left","right")
    if Input.is_action_just_pressed("jump"):
        velocity.y = -200

func apply_gravity():
    velocity.y += 10
```

```
func _physics_process(_delta: float) -> void:
    get_input()
    velocity.x = direction_x * speed
    apply_gravity()
    move_and_slide()
```

接下来还需要为玩家节点添加相机组件，设置缩放为3，运行后效果观感好了很多。





还需要修改起跳力度，让角色跳得更高。新增整型变量jump_strength和gravity，初始值都为10，跳跃判断逻辑修改为velocity.y = -jump_strength，重力逻辑修改为velocity.y += gravity * delta，让apply_gravity函数接收delta参数，杜绝程序处理加速度受帧数影响。

这里有个小技巧，可以在ump_strength和gravity变量定义时添加上@export注解，这样这两个变量就会在选中player场景时在检查器显示，可以在游戏运行时随时修改，方便频繁调试。修改jump_strength为400，gravity为1000，这样跳跃效果较为舒适。完整脚本如下。

```
extends CharacterBody2D
var direction_x : float
var speed : int= 50
@export var jump_strength := 10
@export var gravity := 10

func get_input():
    direction_x = Input.get_axis("left","right")
    if Input.is_action_just_pressed("jump"):
        velocity.y = -jump_strength

func apply_gravity(delta):
    velocity.y += gravity * delta#处理重力加速度，不是直接赋值速度，要使用delta
```

```
func _physics_process(delta: float) -> void:
    get_input()
    velocity.x = direction_x * speed
    apply_gravity(delta)
    move_and_slide()
```

3.子弹发射系统

先编写射击操作雏形：当按下鼠标左键时，控制台输出“shoot”，设置射击冷却为0.5秒。

首先在get_input函数中添加条件判断，若检测到输入信号”shoot“，输出“shoot”，如下。

```
if Input.is_action_just_pressed("shoot"):
        print("shoot")
```

还需要添加射击冷却，在Player场景中添加计时器ReloadTimer，Wait time设置为0.5s，One Shot启用，当射击后，开启计时器，将条件判断修改为检测到“shoot”信号且计时器剩余时间为0，具体如下。

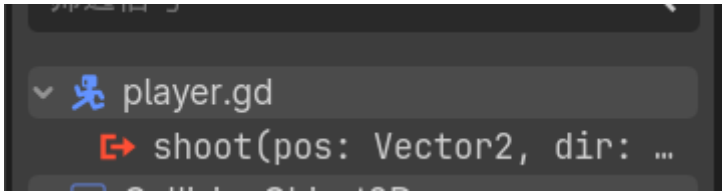
```
    if Input.is_action_just_pressed("shoot") and $ReloadTimer.time_left == 0:
        print("shoot")
        $ReloadTimer.start()
```

接下来制作子弹效果，这样就能实现最基础的射击功能。为了这个功能，我们需要学习自定义signal信号，也就是自己创建信号并发送。

首先，在player场景内部，当玩家按下射击键时，需要生成一颗子弹，但存在一个问题：子弹要和其他实体交互，就必须在level场景内部，但玩家的输入逻辑在player内部。所以我们要把这段输入逻辑关联到真正的level场景上。

在level场景中挂载一份脚本，我们的目的是捕获在player节点发出的信号。

在player中新建信号类型变量shoot，参数为pos: Vector2和dir: Vector2，定义完成后可在信号页看到该信号。



将其关联到level场景，打开level脚本，编辑_on_player_shoot函数，先将其触发后活动设置为打印位置和方向两个参数，方便测试。

在player脚本中修改射击判定条件后的执行内容，将打印“shoot”改为shoot.emit(position,get_local_mouse_position())（即发出该信号，并传入当前角色位置和鼠标点击方向），get_local_mouse_position()是Godot内置的返回鼠标点击坐标的方法，返回的坐标是以当前根节点也就是Player为原点的相对坐标。完整写法如下。

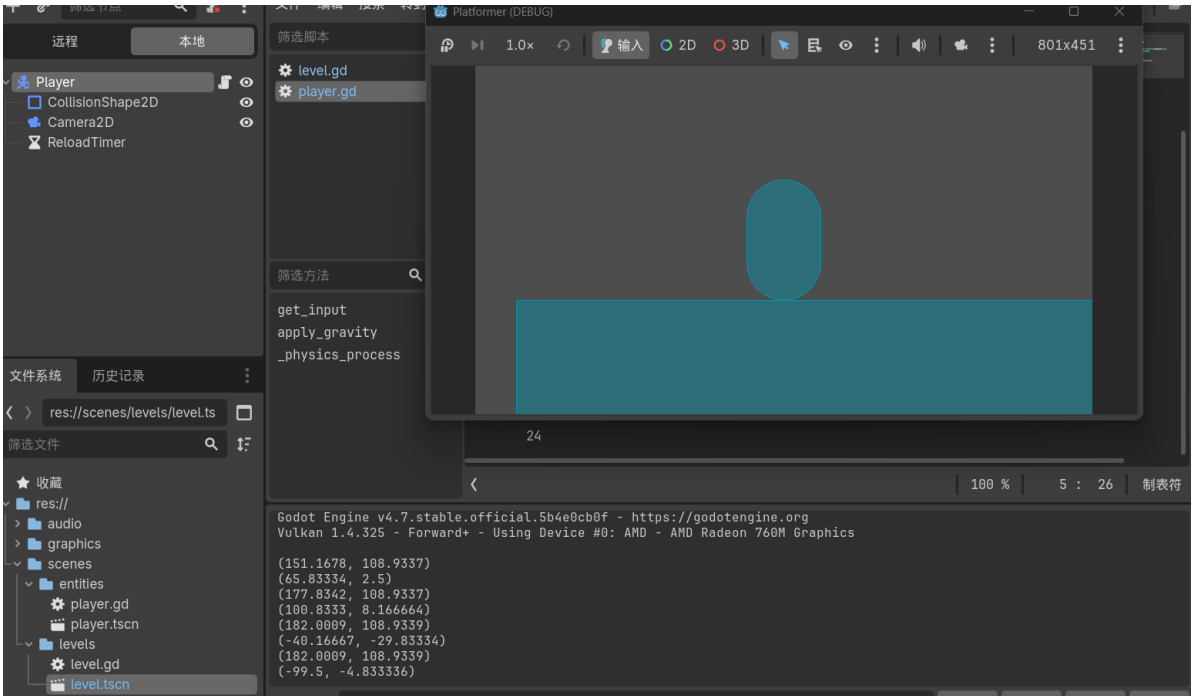
```gdscript
#player.gd
signal shoot(pos: Vector2,dir: Vector2)

func get_input():
    direction_x = Input.get_axis("left","right")
    if Input.is_action_just_pressed("jump"):
        velocity.y = -jump_strength
    if Input.is_action_just_pressed("shoot") and $ReloadTimer.time_left == 0:
        shoot.emit(position,get_local_mouse_position())
        $ReloadTimer.start()
```

```gdscript
extends Node2D

func _on_player_shoot(pos: Vector2, dir: Vector2) -> void:
    print(pos)
    print(dir)
```

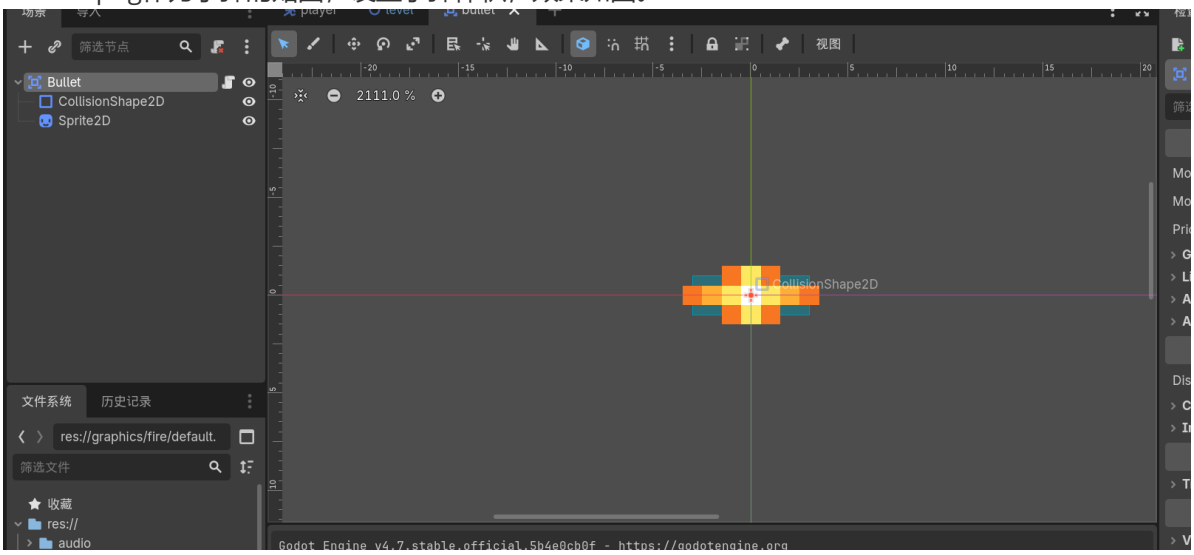
运行可以看到控制台正确输出当前位置和鼠标点击方向，如图。



修改shoot.emit(position,get_local_mouse_position())为shoot.emit(position,get_local_mouse_position().normalized())，即使用normalized方法对向量进行归一化，防止向量长度影响子弹速度。

接下来新建bullet场景，当玩家射击时生成这个子弹。

在bullet场景中挂载CollisionShape2D节点和Sprite2D节点，选取graphics文件夹中fire中的default.png作为子弹的贴图，设置子弹体积，效果如图。



在level场景中新建Node2D类型节点Bullets，用于存放所有子弹实体.

编辑level脚本，先使用preload方法预载bullet到场景中，再修改_on_player_shoot函数，实例化bullet对象，将其添加到Bullets节点下，随后在调用bullet的setup方法，传入参数。具体如下。

```
extends Node2D
var bullet_scene = preload("res://scenes/bullets/bullet.tscn")

func _on_player_shoot(pos: Vector2, dir: Vector2) -> void:
    var bullet = bullet_scene.instantiate() as Area2D
    $Bullets.add_child(bullet)
    bullet.setup(pos,dir)
```

在bullet场景上挂载脚本，在脚本中编写setup方法。子弹的生成位置应该为传入的pos加上方向加一定偏移量，防止子弹在角色内部生成。具体如下。

```
var direction: Vector2
func setup(pos: Vector2,dir: Vector2):
    position = pos + dir * 16
    direction = dir
```

在physics_process函数中设置子弹的移动，如下。

```
func _physics_process(delta: float) -> void:
    position += direction * 30 * delta
```

至此子弹发射系统已初步完善。

4.玩家角色动画

该功能的实现需要依靠AnimationPlayer动画播放器，先关闭调试选项中的“显示碰撞区域”，为Level场景中的StaticBody2D新增Sprite2D节点，暂时将Godot图标调整大小将作为地图纹理，方便测试。

制作角色动画分为上半身和下半身两部分，先制作与地面接触的下半身。

在player场景中新建sprite2D类型节点“Legs”，设置属性texture为res://graphics/characters/player.png，在Sprite2D节点的Animation属性中设置Hframe为8，Vframe为3。

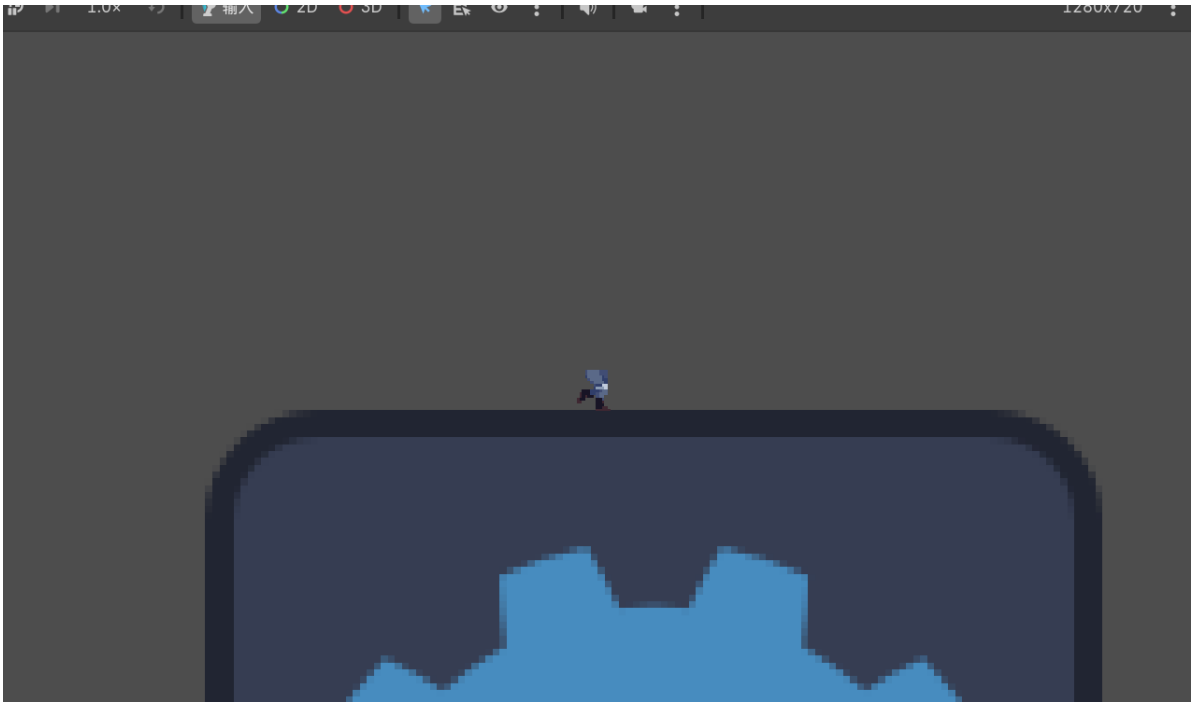
再添加为Player添加一个AnimationPlayer节点，在节点面板中点击动画选项新建一段动画，命名为‘run’，创建完成后出现时间轴，点击加号添加属性轨道，弹出的窗口中选择Legs节点，再选择frame属性，完成后就可以在时间轴中编辑动画，按住Ctrl并滚动滚轮可以对时间轴进行缩放。

修改动画长度为1.4秒，启用”动画循环“



接下来在时间轴中添加动画关键帧，推荐开启“为时间线光标应用吸附”功能。

右键点击时间轴后选择“插入关键帧”，将Value值设置为9，意为在该时间点插入Legs中frame值为9的帧，随后间隔0.2秒后插入Value为10，以此类推直到value为15，整段动画制作完成，开启“加载后自动播放”，运行游戏观察效果。别忘了在视图中拖动Legs使其贴紧Player中心点，并设置体积使其贴合腿部动画。



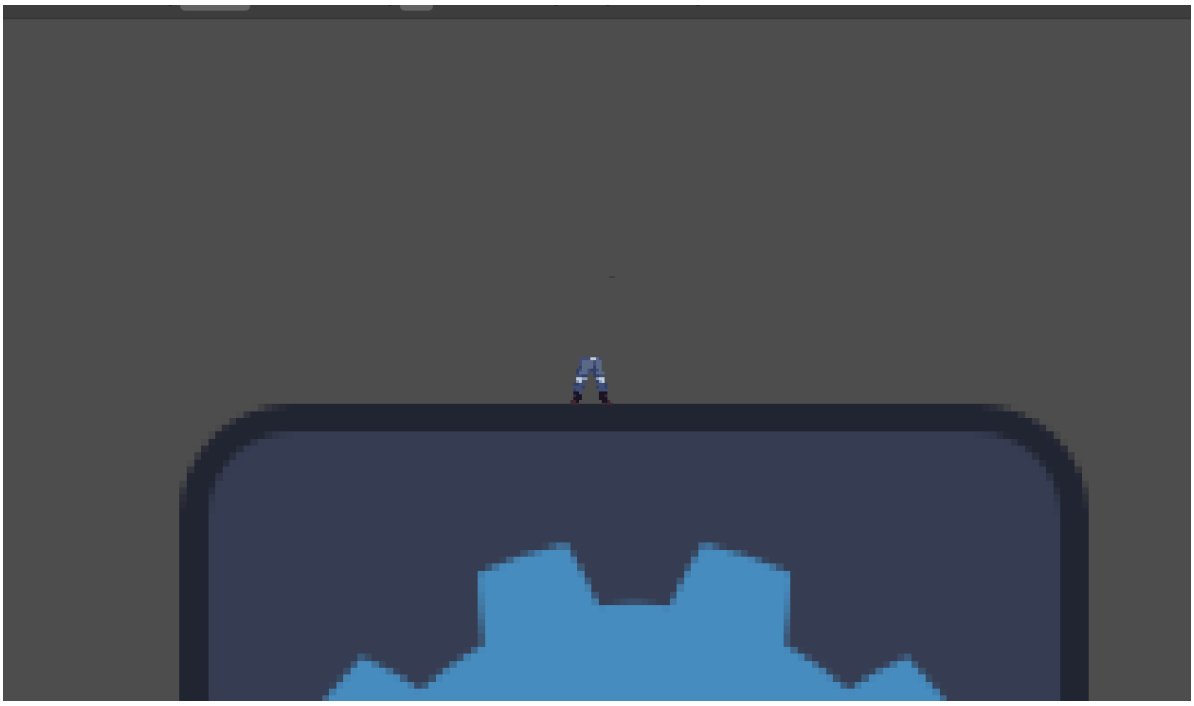
新增一个动画，命名为“idle”，用于玩家未输入时的静止画面。同样选择属性轨道，选择Legs节点并选择frame属性，只需添加一个关键帧，Value为16。同样开启”动画循环“和”加载后自动播放”。

接下来设置动画播放逻辑，在player脚本中新增animation方法，编写动画播放逻辑，并在physics_process方法中末尾调用该方法。具体如下。

```
func _physics_process(delta: float) -> void:
    get_input()
    velocity.x = direction_x * speed
    apply_gravity(delta)
    move_and_slide()
    animation()

func animation():
    $Legs.flip_h = direction_x < 0
    $AnimationPlayer.current_animation = 'run' if direction_x != 0 else 'idle'
```

运行游戏可以看到，输入方向为左时，动画已正常反转。没有输入时显示停止动画。



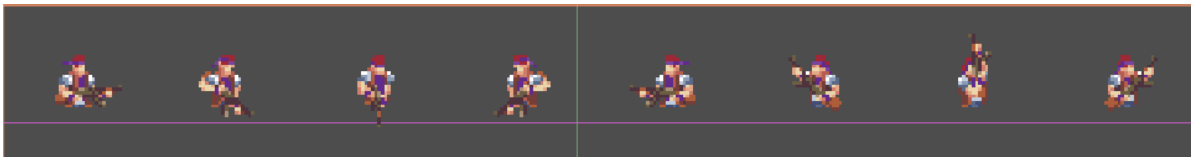
最后就简单了，添加一个jump动画，对应第17帧，启用“动画循环”，随后修改animation方法，使用内置的is_on_floor（）方法判断角色是否在地面，如下。

```
func animation():
    $Legs.flip_h = direction_x < 0
    $AnimationPlayer.current_animation = 'jump' if !is_on_floor() else 'run' if
direction_x != 0 else 'idle'
```

下半身动画已制作完毕，接下来制作上半身动画。

新建一个Sprite2D节点，命名为Torso（躯干），用来存放上半身动画，同样设置属性texture为res://graphics/characters/player.png，调整好对应参数，使用方向键微调位置使其与下半身贴图贴合。

观察躯干精灵帧素材，可以看到角色躯干共有8个朝向，我们要根据鼠标指向位置匹配对应的动画帧。



要实现这个效果没办法单纯依靠AnimationPlayer来高效实现，需要借助脚本实现对应逻辑。

首先获取鼠标相对角色的位置向量，和之前一样使用get_local_mouse_position方法。我们先在动画函数内部创建一个变量raw_dir，用于存放初始方向数据，使用normalized方法将其归一化。但我们可以看到，归一化后的方向向量数据使用起来并不方便，我们再使用round函数将其简单化处理，并存入Vector2i类型变量，赋值给新建变量adjust_dir，然后在脚本内部新建一个字典常量gun_directions，用于将八个方向与八个动画帧一一对应，最后以adjust_dir作为键对动画帧进行修改，具体如下。

```
const gun_directions = {#将8个方向与8个帧对应
    Vector2i(1,0): 0,
    Vector2i(1,1): 1,
    Vector2i(0,1): 2,
    Vector2i(-1,1): 3,
    Vector2i(-1,0): 4,
    Vector2i(-1,-1): 5,
```

```
    Vector2i(0,-1): 6,
    Vector2i(1,-1): 7,
}


func animation():
    $Legs.flip_h = direction_x < 0
    $AnimationPlayer.current_animation  = 'jump' if !is_on_floor() else 'run' if
direction_x != 0 else 'idle'
    var raw_dir = get_local_mouse_position().normalized()#获取鼠标的当前位置向量
    var adjust_dir = Vector2i(round(raw_dir.x),round(raw_dir.y))#简单化处理,使用
Vector2i类型更方便
    $Torso.frame = gun_directions[adjust_dir]#以adjust_dir作为键获取字典中对应帧值来修
改帧
```

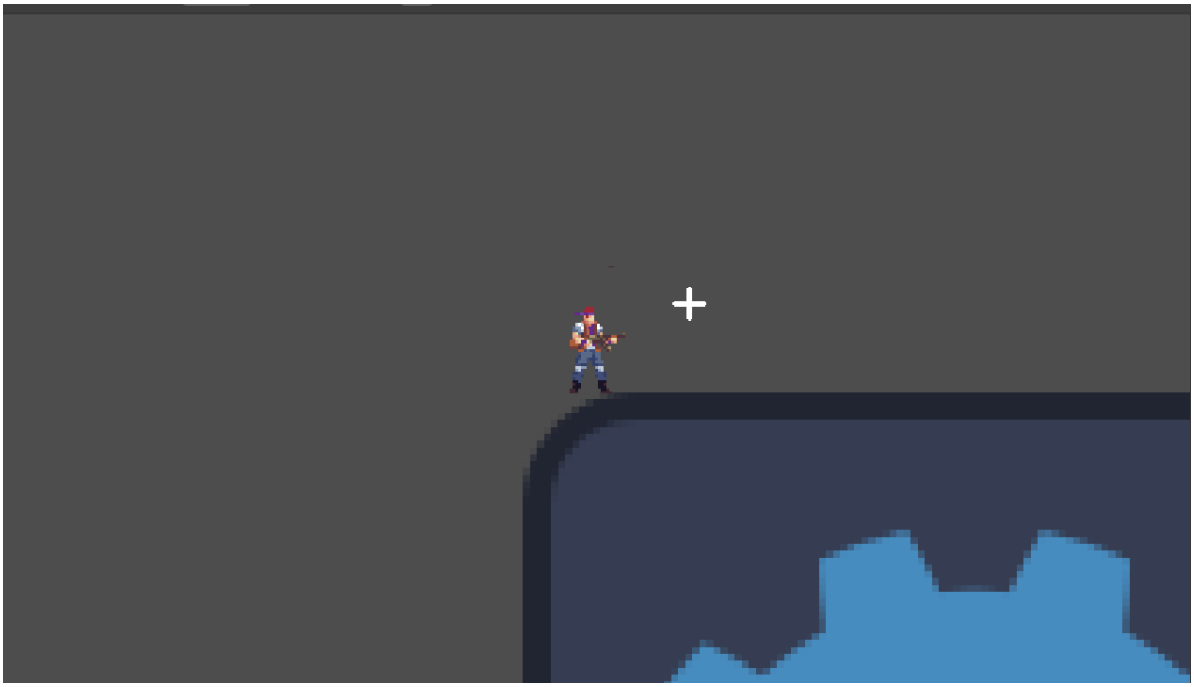
5.差值动画

接下来添加一个marker标记，帮助玩家看清射击的方向。这个标记也要做动画，但因为太过简单，不需要使用AnimationPlayer，而是使用另一种动画制作方案——Tween补间动画。

在Player场景下新建一个Sprite2D节点，命名为Marker，texture属性设置为res://graphics/ui/crosshair.png，Scale属性x、y同步设置为0.5，新建一个update_marker函数用于设置其位置，如下。

```
func update_marker():
    $Marker.position = get_local_mouse_position().normalized() * 40
```

完成后marker效果如图。



随后我们要实现一个简单的动画，即按下开火后，标记先是缩小，然后恢复原来大小。这个动画可以用AnimationPlayer来实现，但有些大材小用，我们使用Tween来实现。

在get_input方法中的开火操作后添加语句，先创建tween变量，随后使用tween的tween_property方法操作标记变化的动画，tween_property的参数前后顺序分别是：要操作的对象、要操作的对象的属性、属性最终值、间隔时间。

我们先操作标记缩小，随后恢复正常大小，Tween会根据语句顺序来执行。实际使用如下。

```
func get_input():
    direction_x = Input.get_axis("left","right")
    if Input.is_action_just_pressed("jump"):
        velocity.y = -jump_strength
    if Input.is_action_just_pressed("shoot") and $ReloadTimer.time_left == 0:
        shoot.emit(position,get_local_mouse_position().normalized())
        $ReloadTimer.start()
        var tween = get_tree().create_tween()#创建Tween对象
        tween.tween_property($Marker,"scale",Vector2(0.1,0.1),0.2)#标记先缩小
        tween.tween_property($Marker,"scale",Vector2(0.5,0.5),0.2)#标记恢复
```

至此，标记的动画效果已制作完成，我们还可以用Tween为子弹添加动画，使子弹初始大小为0，发射一段时间后大小变为1。

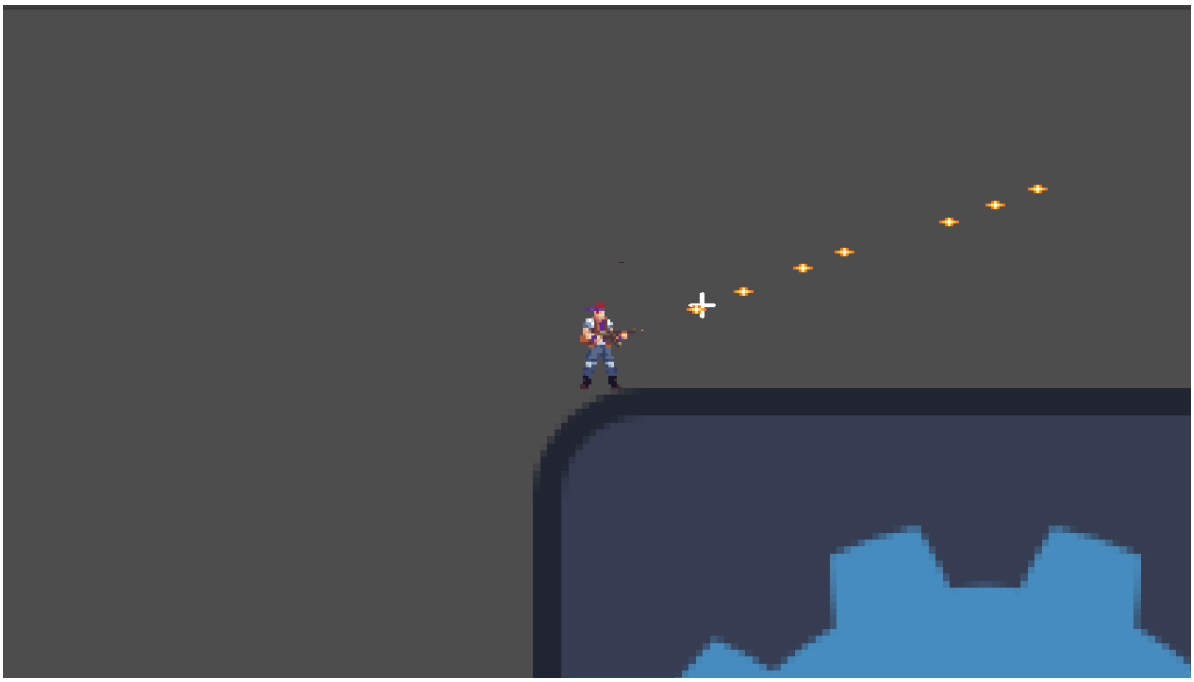
该动画效果应在bullet脚本的_ready方法中实现，有两种使用Tween的实现方法。

第一种,先设置bullet的Sprite2D节点的scale属性初始值为Vector2.ZERO，随后使用Tween将其修改为Vector2.ONE，这也是基础的写法。

第二种，使用Tween的from方法为属性设置初始值，如下。

```
func _ready() -> void:
    var tween = get_tree().create_tween()
    tween.tween_property($Sprite2D,"scale",Vector2.ONE,0.5).from(Vector2.ZERO)
```

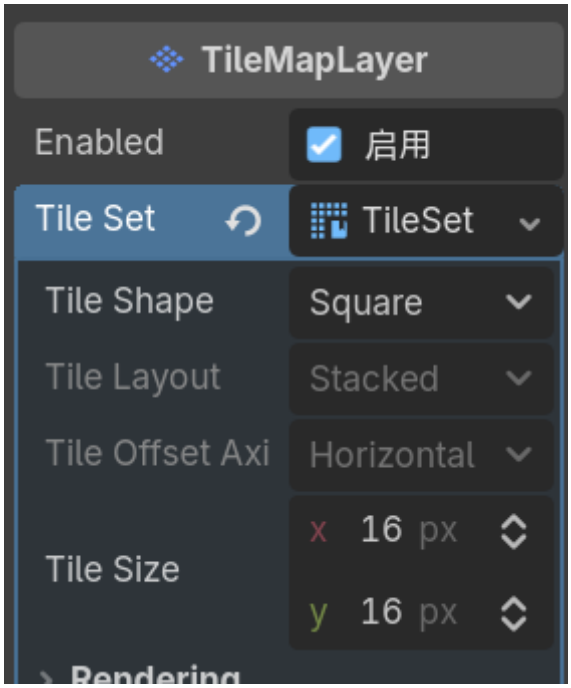
运行游戏测试，发现效果完美实现。



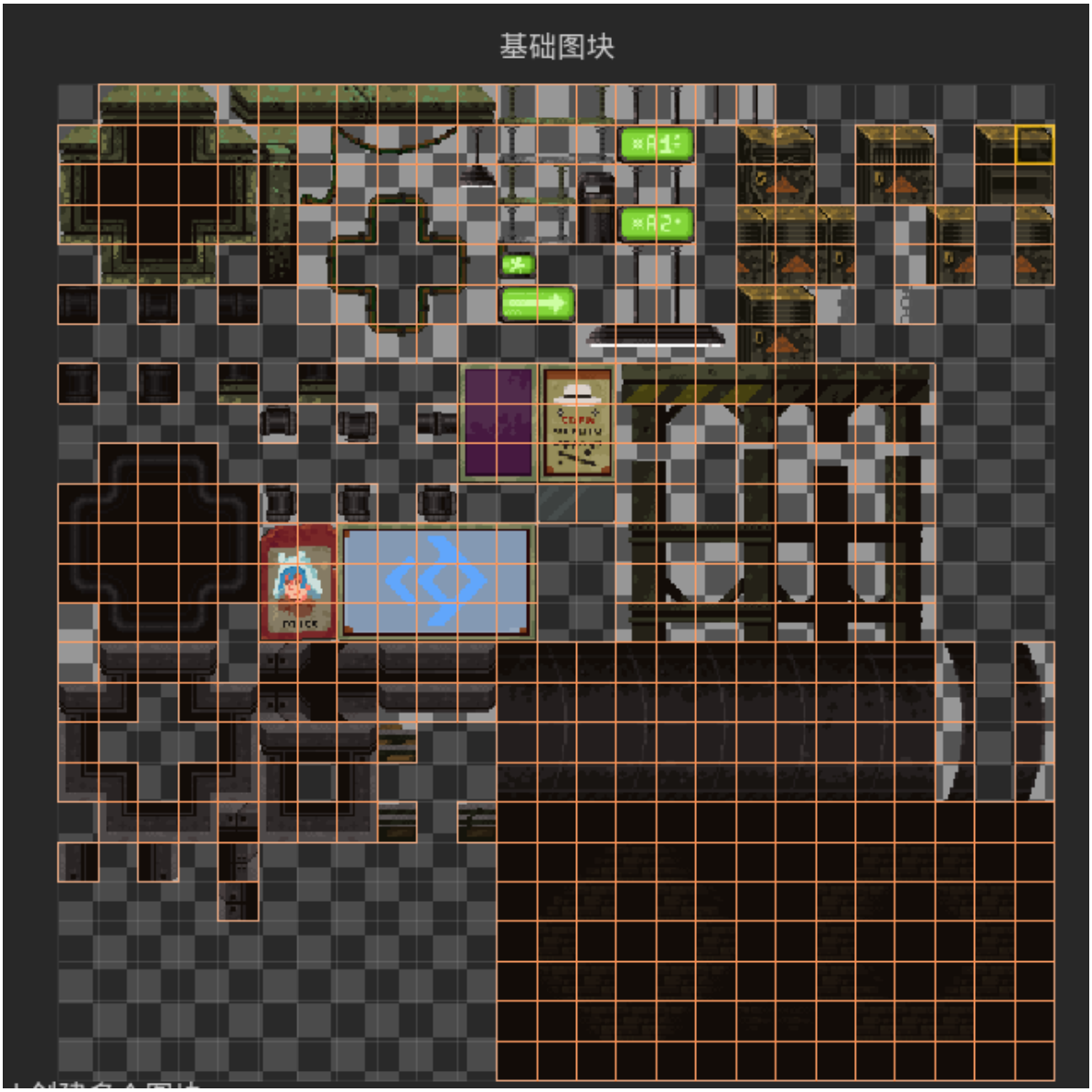
6.瓦片地图

要制作丰富的场景，就要使用TileMapLayer节点，打开Level场景，将原本的StaticBody2D节点整个删除，添加TileMapLayer节点。

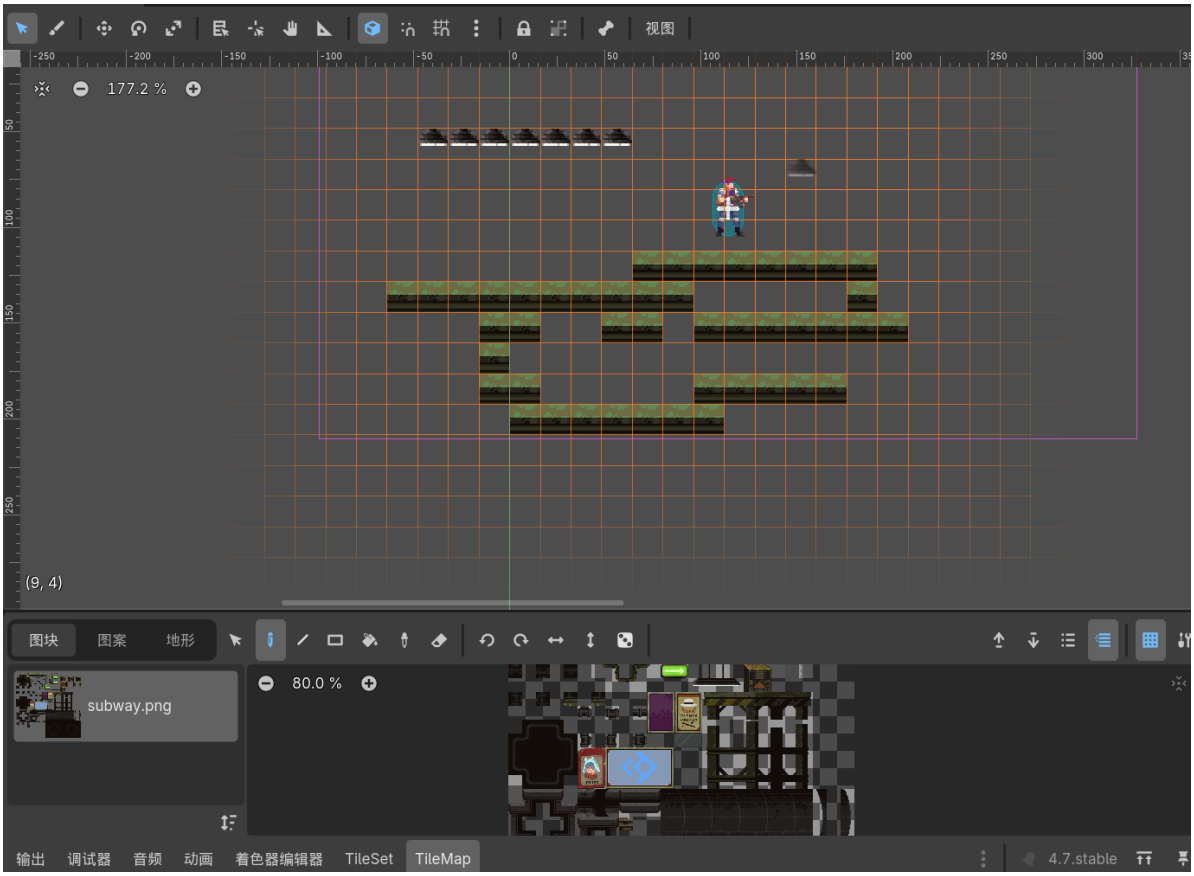
新添加的TileMapLayer节点还无法使用，在检查器中的Tile Set属性中选择新建一个TileSet，随后在底部可以看到新增了TileMap和TileSet面板。在检查器面板点击TileSet，会打开很多配置选项，我们保持默认即可。



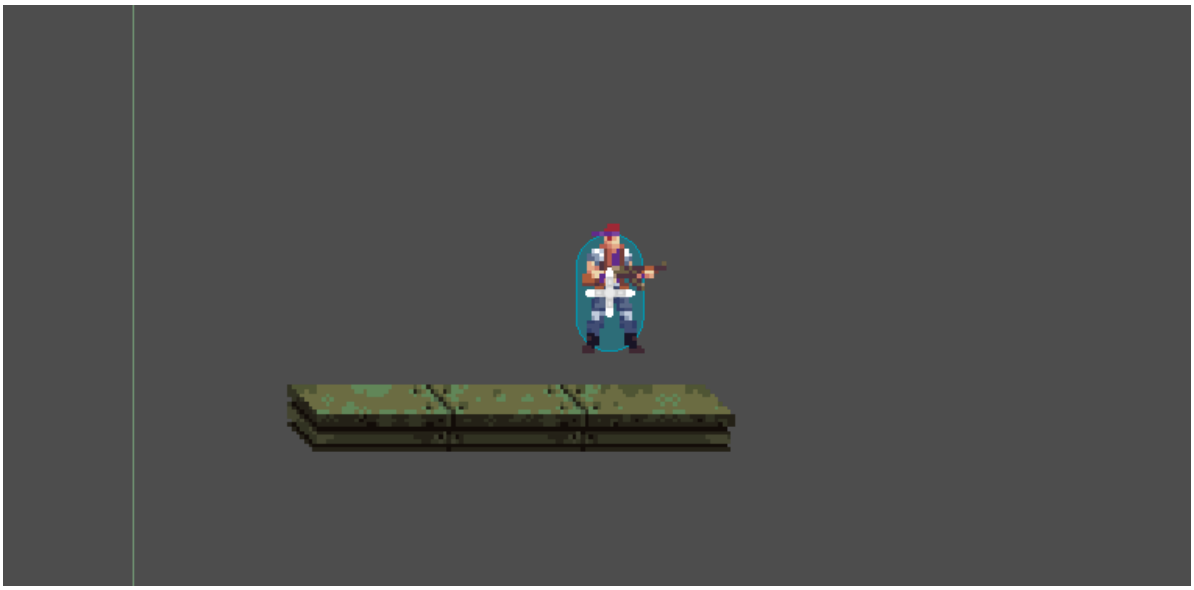
点击底部的TileSet面板，将res://graphics/tilesets/subway.png拖入到图块源中，弹窗选择确认。可以看到，引擎借助TileMapLayer节点将添加的图片分割为16*16的像素块。



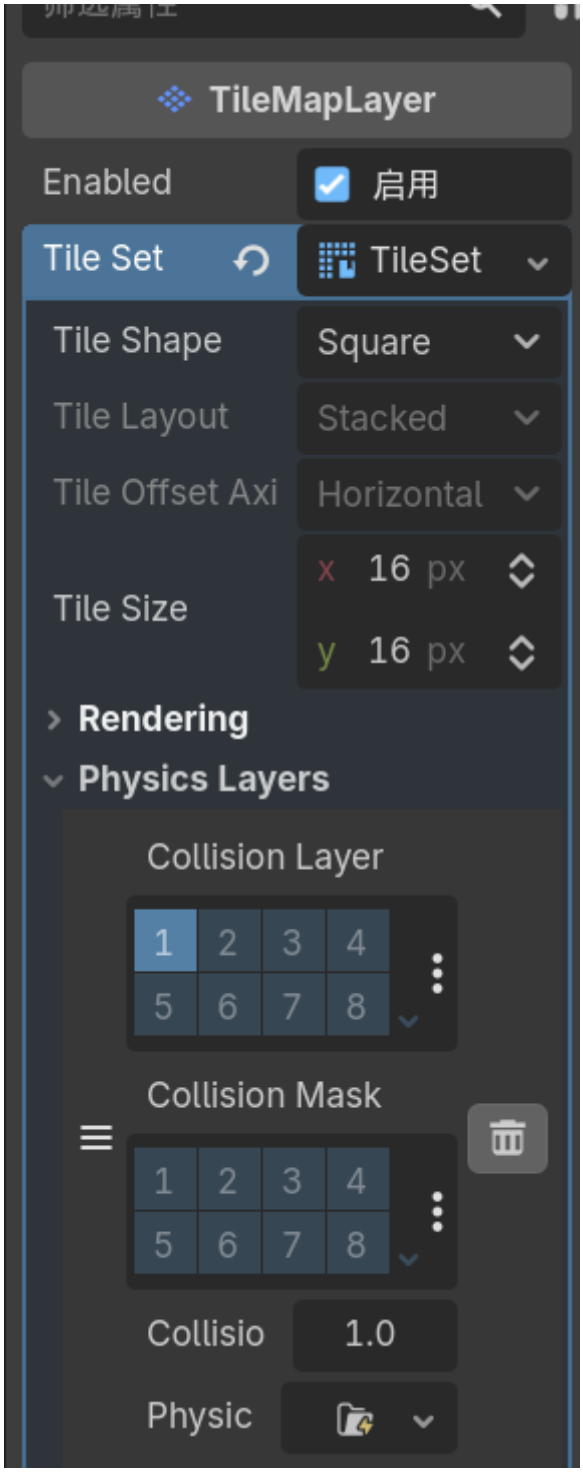
完成上述操作后，可以在TileMap面板中选中任意一块瓦片，选中铅笔按钮，自由绘制瓦片将其添加到场景中，也可以选择矩形工具按区域绘制，如图。



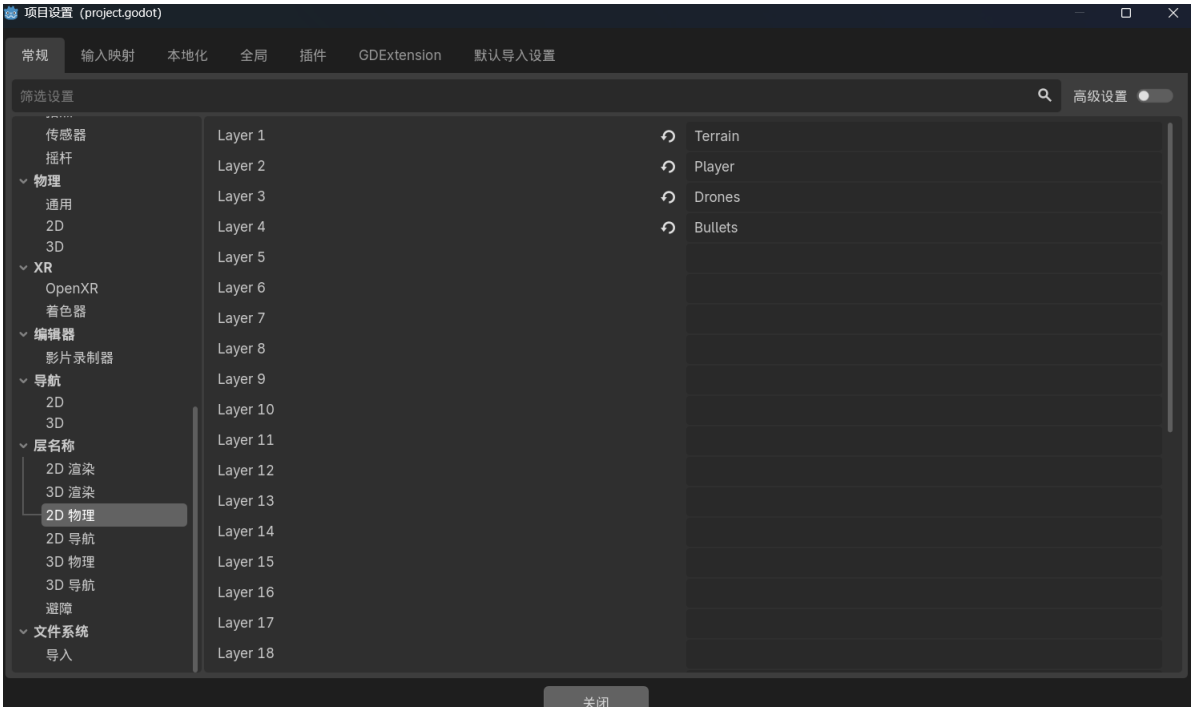
一般情况下还是逐个瓦片绘制，当然也可以同时选择多个瓦片然后批量放置到场景中。



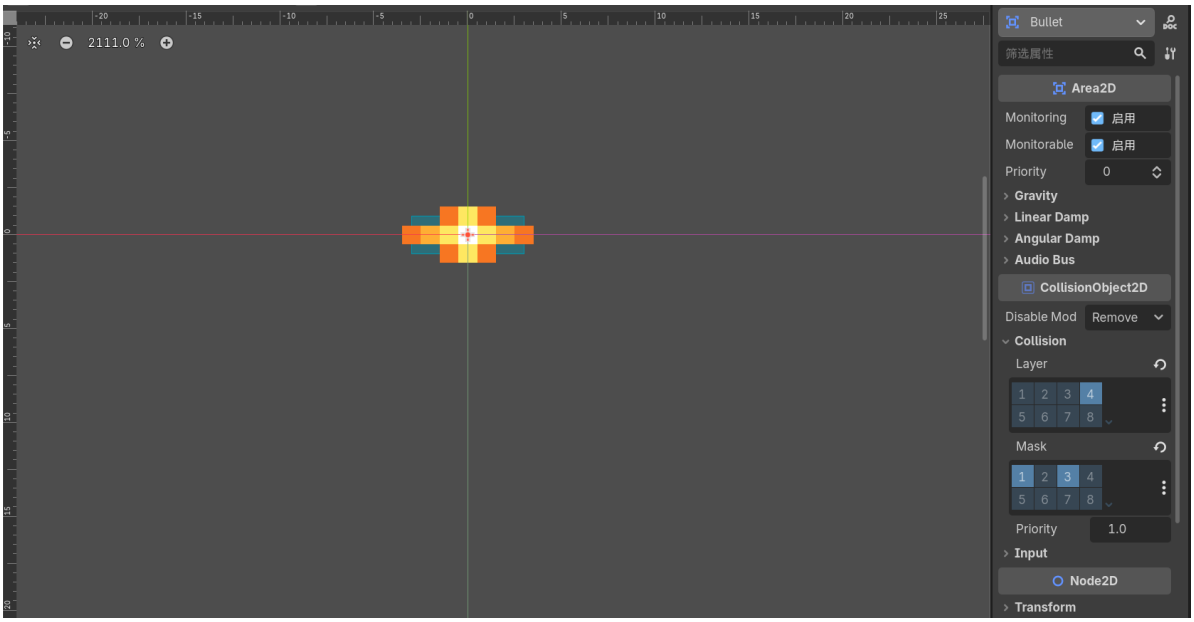
当然，目前添加的瓦片地图没有碰撞区域，要为瓦片地图添加物理碰撞，可以先选中TileSet，在弹窗中编辑Physics Layer属性，新建一个物理层。



同样的，在项目设置中设置游戏物理层的名称，做好区分，分别是地形、玩家、无人机、子弹共四层。



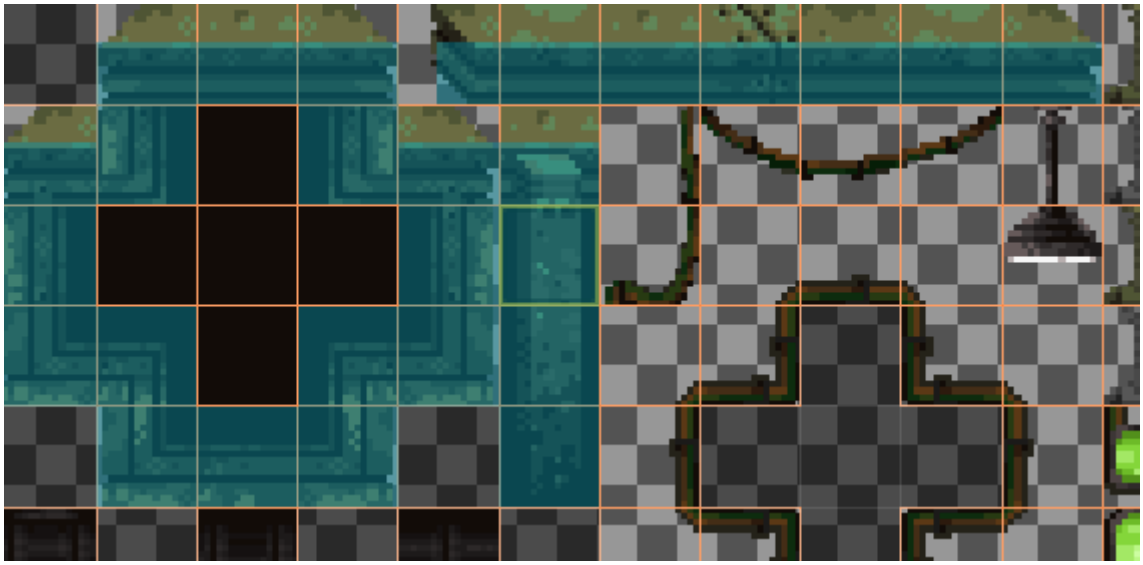
为每个场景编辑其对应所属的物理层和能检测到的物理层。玩家为第2层，能检测到第1和第3层，子弹为第4层，能检测到第1和第3层。



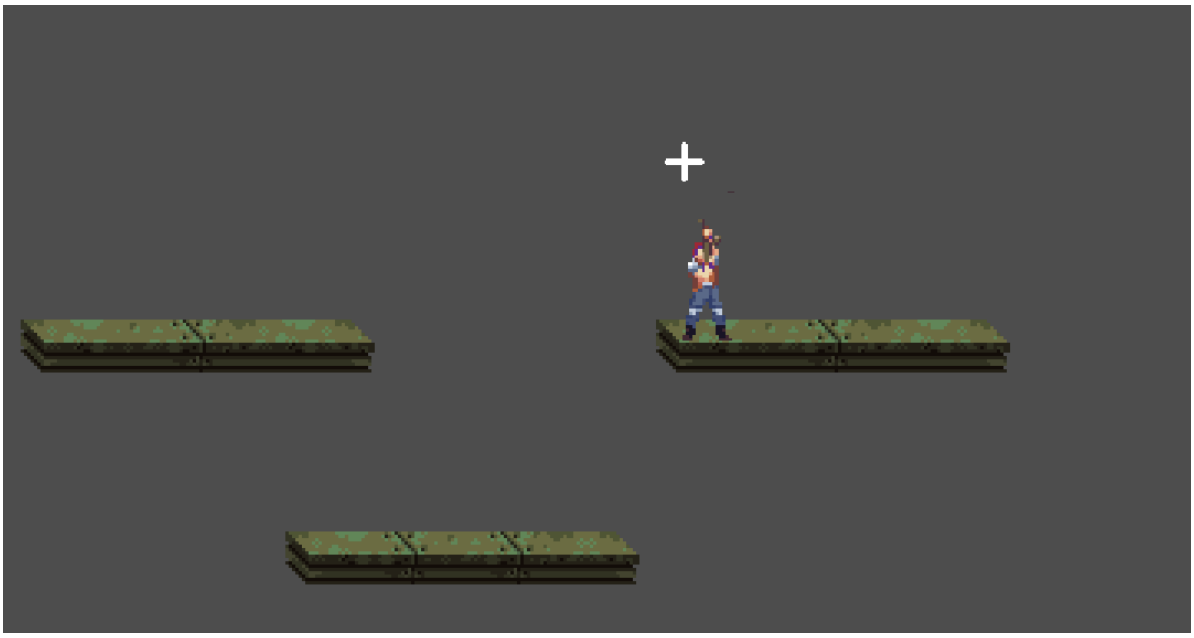
为地形配置好物理层后，还需要为场景中的每块瓦片都添加碰撞外形。

碰撞外形配置在TileSet的绘制标签页中，在绘制属性中选择物理层，这里我们只为TileMapLayer节点添加了一个物理层，因此只有物理层0可选，也可以添加多个物理层，不过一般游戏一个物理层来实现地图碰撞足矣。

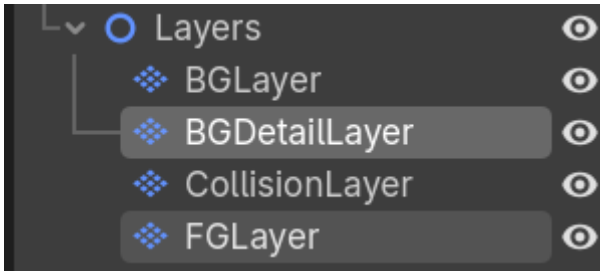
选中该分层后，界面出现碰撞外形工具，就可以对任意瓦片的碰撞外形进行编辑和绑定。我们这里先对游戏里用到的所有瓦片进行碰撞外形编辑。



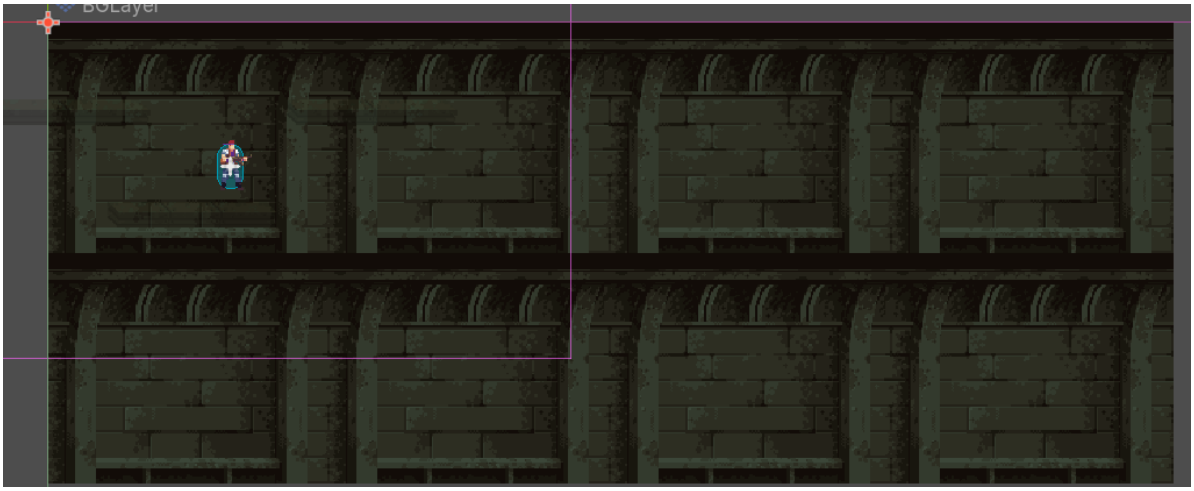
完成以上设置后，将TileMapLayer拖动到Player场景的上方，这样画面观感更佳。运行游戏测试，效果正常。



游戏中，为了丰富画面细节，地图场景除了碰撞层还有背景层和背景装饰层、前景层，我们在Level中新建一个Node2D节点命名为Layers，用于存储所有地图层，将刚才编辑的层命名为CollisionLayer层，再新建另外三个层，按如图顺序。



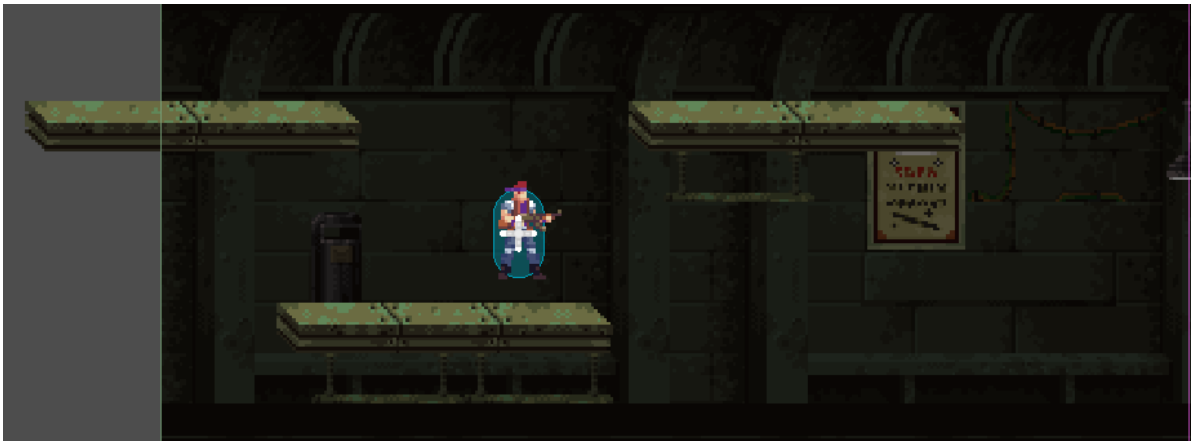
BGLayer用于存储背景，如下。



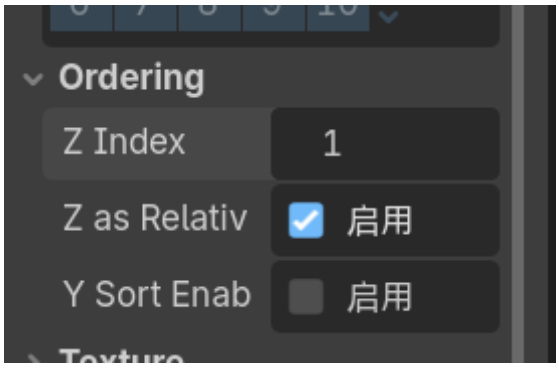
BGDetailLayer用于存放一些背景细节，如图。



CollisionLayer存放先前制作的平台等可交互物体，如图。

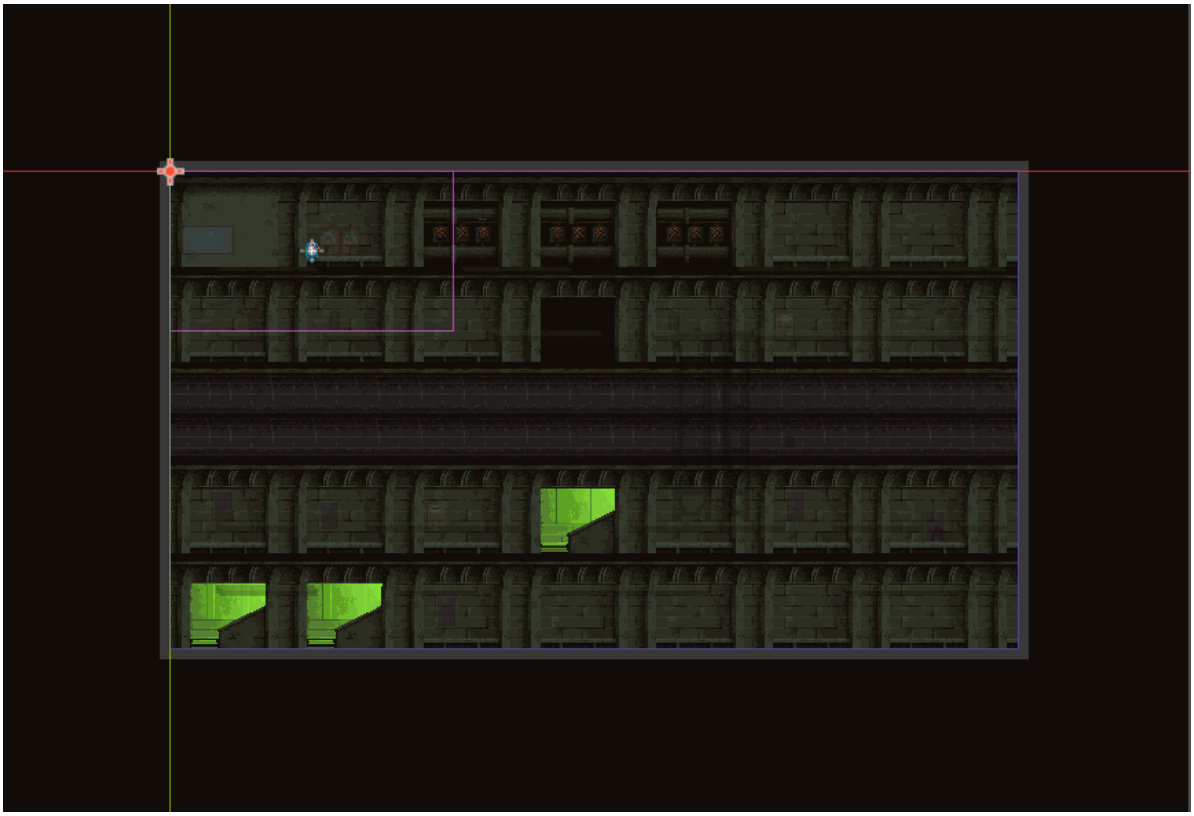


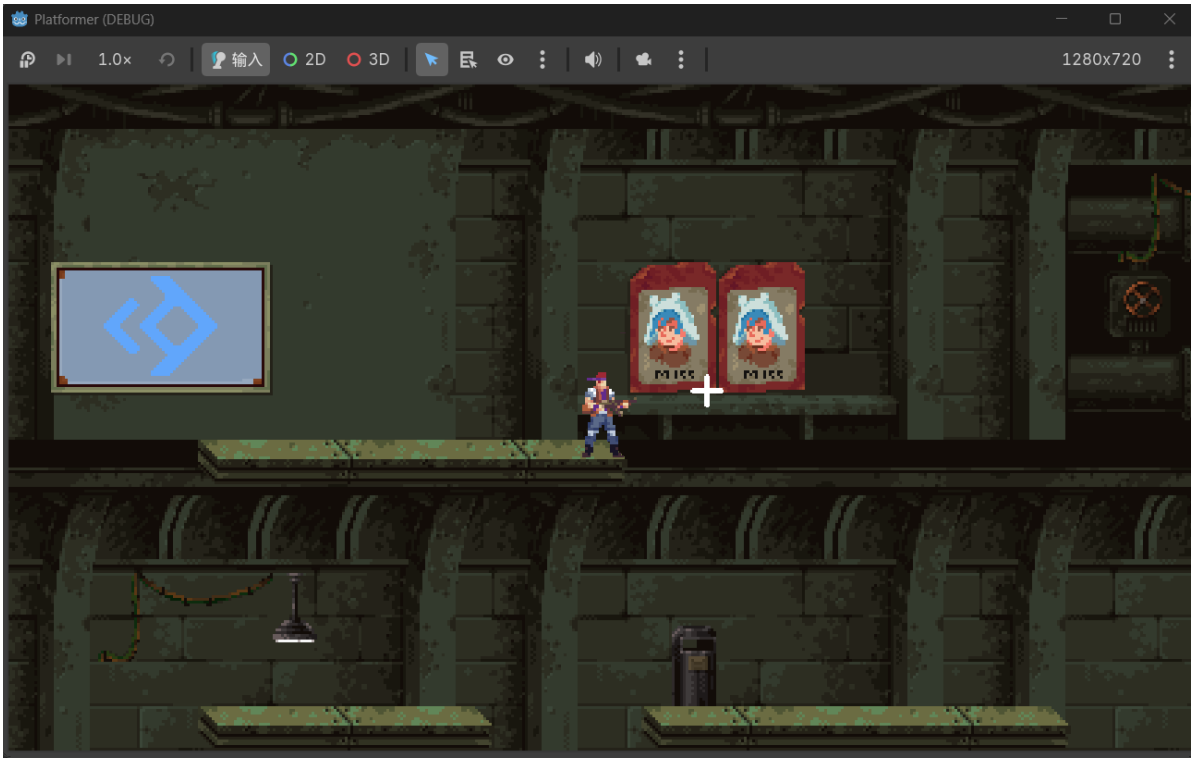
FGLayer用于存放显示在玩家之前的一些物体，如指示牌等，为了让其始终显示在画面最顶层，要将该层的Ordering属性的Z Index属性设置为1，而其他图层默认都是0，最终显示效果如图。





以上各层各司其职，即可搭建出完整的一个关卡，我制作了一个简陋的关卡作为参照，如图。





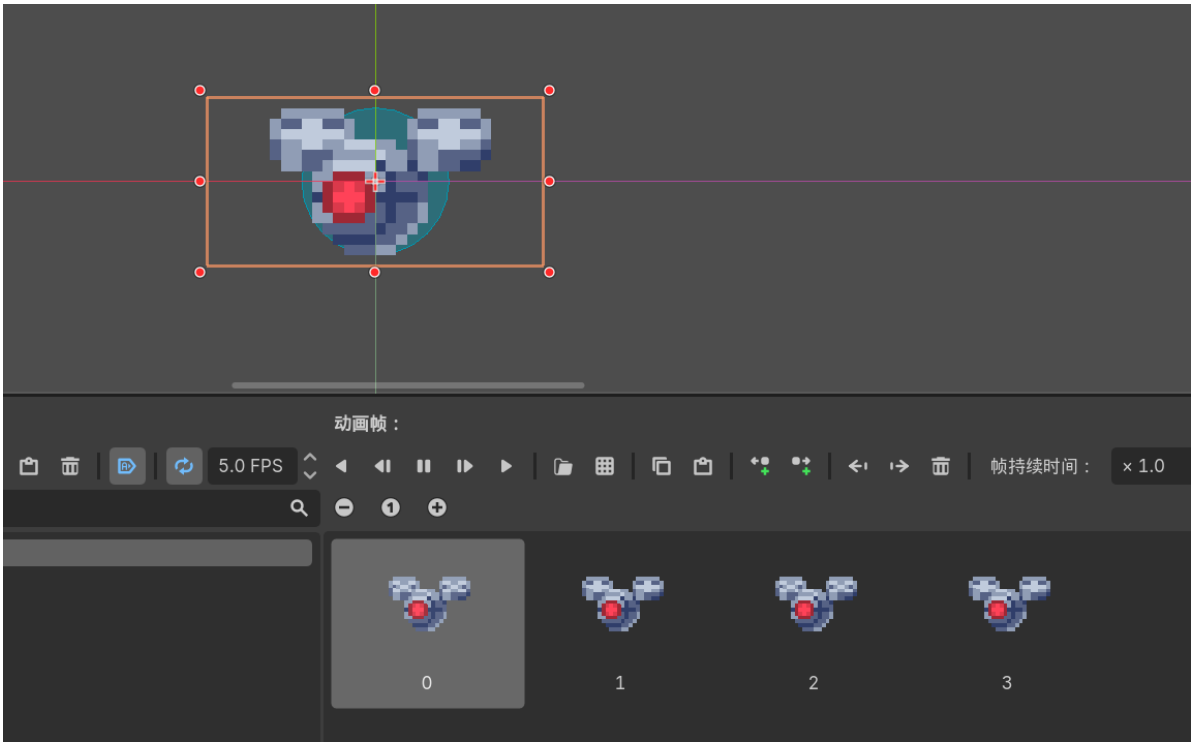
可以对Player的Camera2D节点进行Limit属性设置，防止画面显示出地图外的空白区域。

下

7.无人机敌人AI

这个部分要实现无人机敌人的行动逻辑和动画，当玩家靠近无人机后，无人机会跟着玩家，靠近后爆炸。玩家远离一定距离后，无人机停止移动。子弹击中无人机三次后无人机也爆炸。

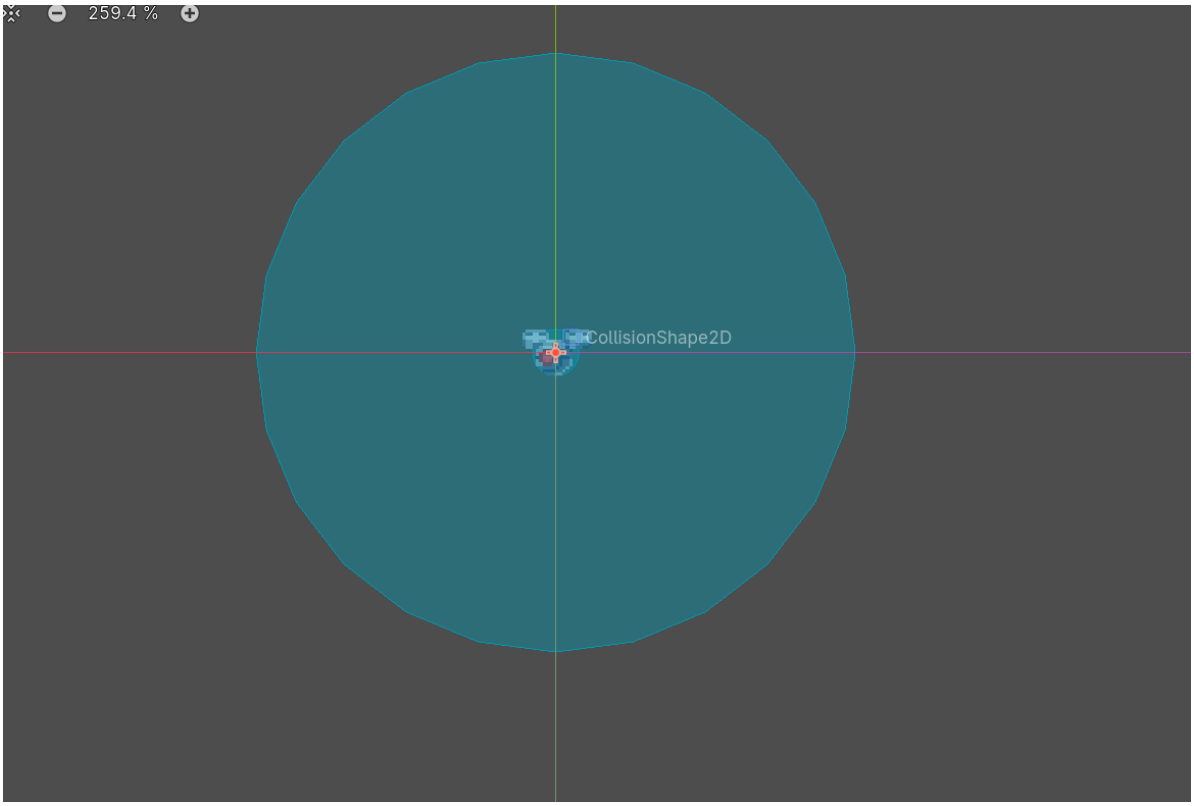
先新建一个CharacterBody2D类型场景Drone，为其添加AnimatedSprite2D节点用于实现动画效果，Sprite Frame中配置精灵帧资源，随后在资源面板点击“从精灵表中添加帧”，选择res://graphics/characters/drone.png，横向4张，纵向1张，选中全部贴图后开启自动播放。点击播放按钮预览效果，如图。



为Drone添加CollisionShape2D节点，形状选择圆形，覆盖住无人机主要部分，Drone的Collsion属性中Layer为3，Mask选择1、2、4。在Level场景中实例化一个无人机查看效果。



基础模型制作完成后，我们新建一个Area2D节点用作检测范围，将该节点重命名为DetectionArea，为其添加CollisionShape2D节点，形状为圆形，调整为合适大小，设置Collsion属性，Layer不选中任何一层，Mask选择2，意为其仅用于检测Player是否进入范围。



接下来为Drone场景挂载脚本以实现逻辑。

选中DetectionArea节点，找到body_entered信号，将其连接到Drone场景，因为on_detection_area_2d_body_entered方法仅检测玩家类型，修改其参数为player_body，类型为CharacterBody2D。在脚本中定义三个变量，direction：vector2、speed：int 和 player：CharacterBody2D。on_detection_area_2d_body_entered方法的内容为将传入的player_body的值赋给player

```
var direction: Vector2
var speed := 50
var player: CharacterBody2D
var health := 3


func _on_detection_area_2d_body_entered(player_body: CharacterBody2D) -> void:
    player = player_body
```

在_physics_process方法中实现其移动逻辑：若player不为空，使用玩家位置减去无人机当前位置来计算无人机的移动方向，随后使用move_and_slide移动，如下。

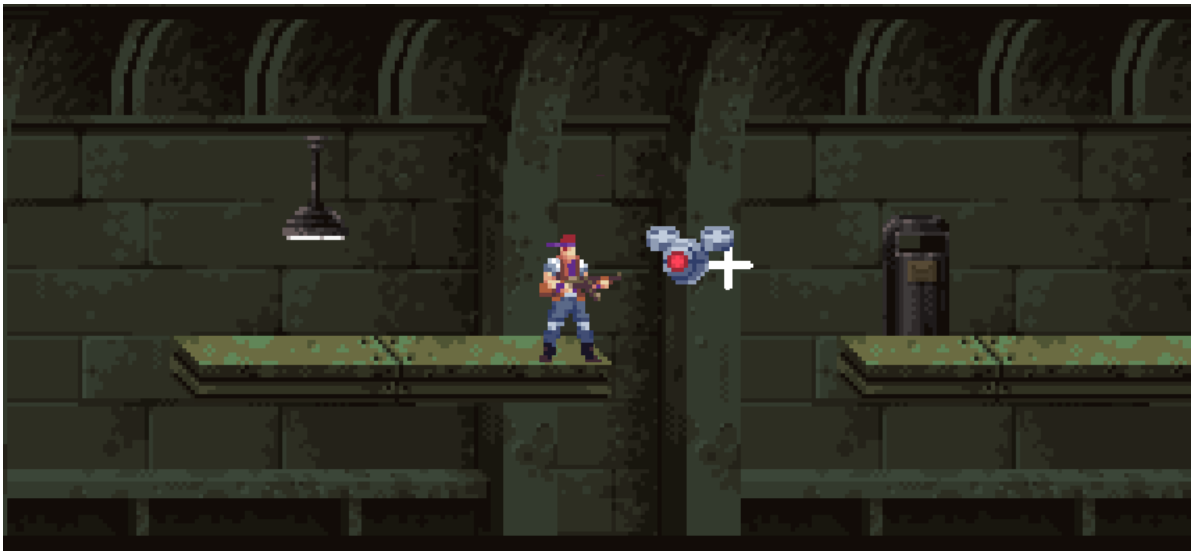
```
func _physics_process(_delta: float) -> void:
    if player:
        var dir = (player.position - position).normalized()
        velocity = dir * speed
        move_and_slide()
```

玩家进入检测范围后无人机跟踪玩家的功能已实现，需要实现玩家远离一定范围后无人机停止的功能，只需要玩家离开范围后将player置空，使_physics_process中的条件不满足即可。

选中DetectionArea节点，选择body_exited信号连接到Drone场景，on_detection_area_2d_body_exited方法中将player赋值为null，这个方法的参数同样是player_body：CharacterBody2D。

```
func _on_detection_area_2d_body_exited(_player_body: CharacterBody2D) -> void:
    player = null
```

运行查看，无人机跟踪功能正常（为了方便测试、提升手感，可以在player脚本中将速度提高到120），如图。



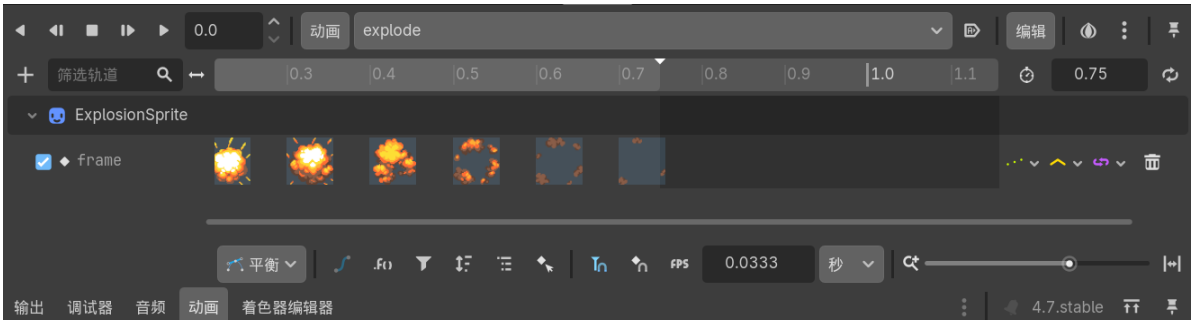
还要实现靠近玩家后停止并爆炸、被射击三次后停止并爆炸的功能。

Drone场景下新建一个Sprite2D类型节点ExplosionSprite，Texture中拖入res://graphics/fire/explosion.png，切换可见性为隐藏，只在触发爆炸动画时显示。再新建一个AnimationPlayer节点进行动画制作。如图。

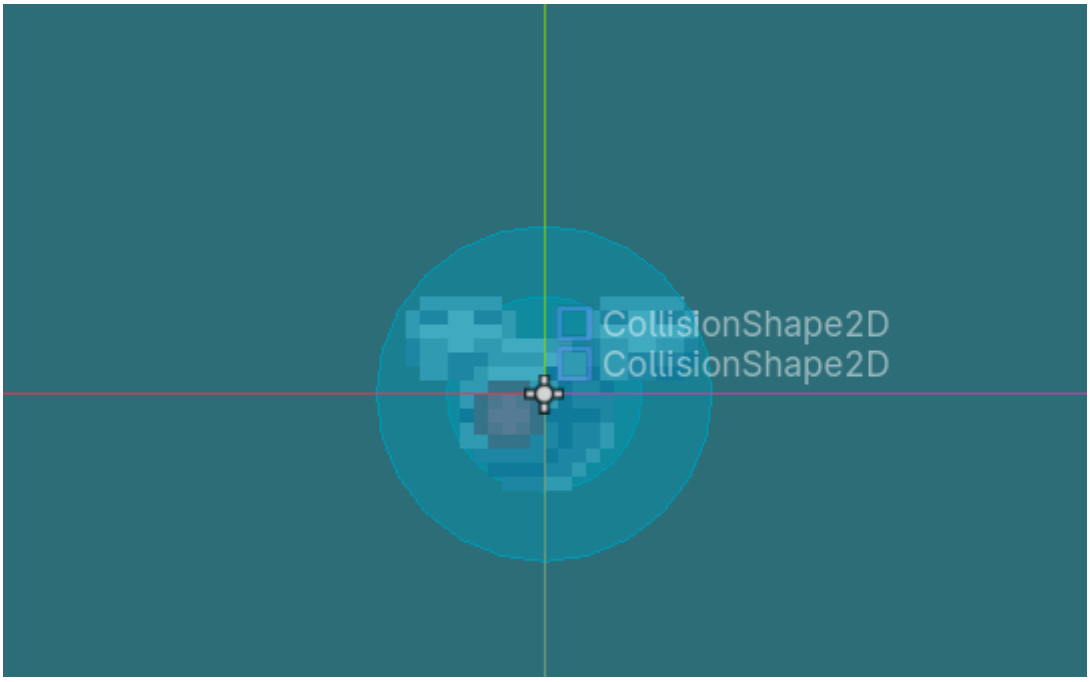


在底部的动画页中新增动画“explode”，添加属性轨道，选择ExplosionSprite的frame属性。

每隔0.1秒添加关键帧，动画总时长0.75s左右，如图。



接下来为Drone新增一个Area2D节点CollisionArea，形状为圆形，需要能够完全覆盖住无人机的贴图，设置collision不勾选任何碰撞图层，仅检测player对象，如图。



选择CollisionArea节点，选择body_entered信号连接到drone脚本。

因为都是停止并爆炸，最后销毁无人机节点，因此可以将爆事件单独封装成一个方法，根据需要调用。我们创建explode方法，后面再编写其内容，先在_on_collision_area_body_entered方法中调用explode方法。

```
func _on_collision_area_body_entered(_body: Node2D) -> void:
    explode()
```

受到攻击后爆炸更复杂一些，在drone脚本中定义一个变量health：int，初始值为3，新建一个方法“hit”，每当调用hit方法，health减1，并判断，若health小于等于0，即调用explode方法。

```
func hit():
    health -= 1
    if health <= 0:
        explode()
```

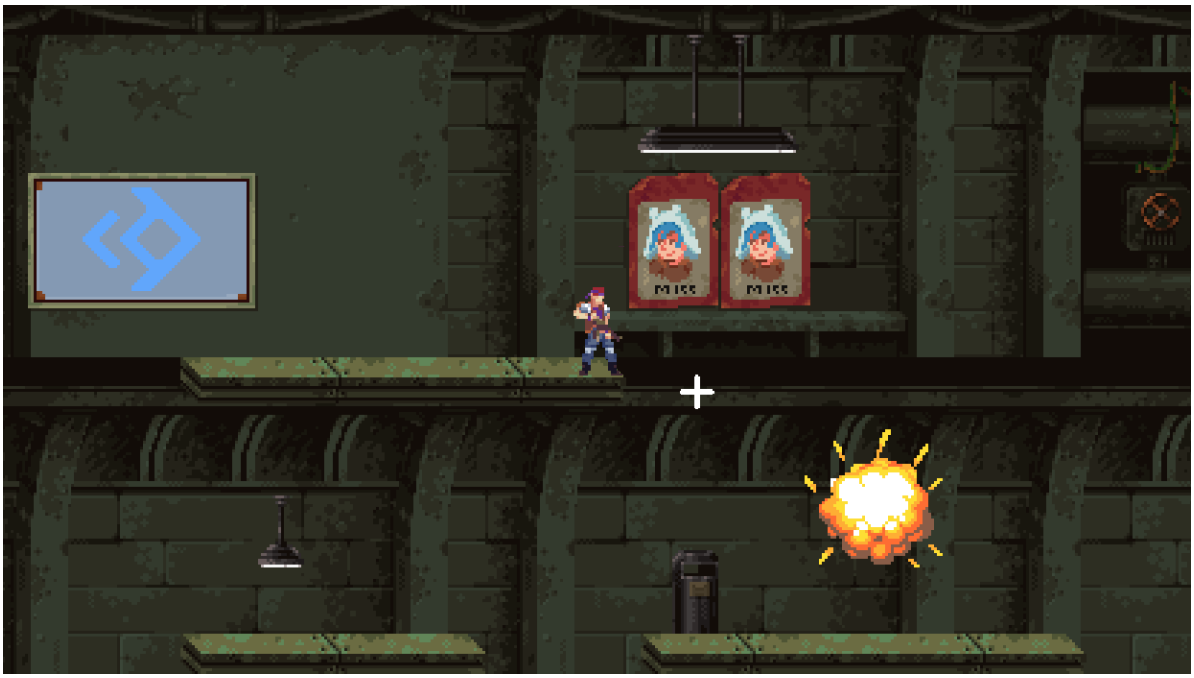
选中bullet场景，将body_entered信号连接到bullet脚本，因为bullet可以物理层配置中可以检测到地形和无人机，如果碰到地形，直接销毁子弹对象即可。只当触发对象是无人机时才调用hit方法，因此可以使用Godot内置的has_method方法，来判断传入的对象是否存在“hit”方法。若存在即调用“hit”方法再销毁。

```
func _on_body_entered(body: Node2D) -> void:
    if body.has_method("hit"):
        body.hit()
    queue_free()
```

最后编写explode方法。回到drone脚本，调用explode方法时，无人机速度赋值为0，并隐藏无人机贴图精灵，显示爆炸动画精灵并播放“explode”动画，使用await等待动画播放完毕后销毁无人机对象。详细代码如下。

```
func explode():
    speed = 0
    $AnimatedSprite2D.hide()
    $ExplosionSprite.show()
    $AnimationPlayer.play("explode")
    await $AnimationPlayer.animation_finished#等待动画播放完毕再销毁动画
    queue_free()
```

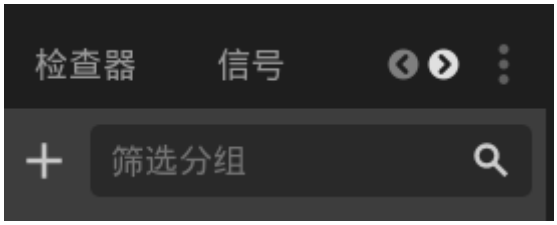
运行游戏测试，效果符合预期，如图。



8.节点分组管理

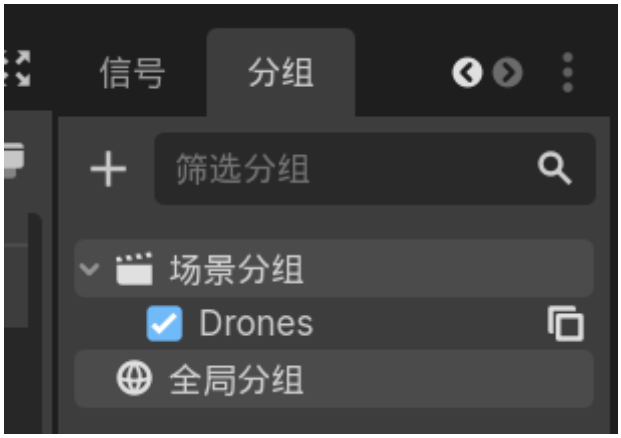
接下来要实现一台无人机爆炸,将对一定范围内所有实体造成伤害的功能，这就需要使用节点分组。

编辑器的右上方点击翻页键，可以看到分组功能页，如图。



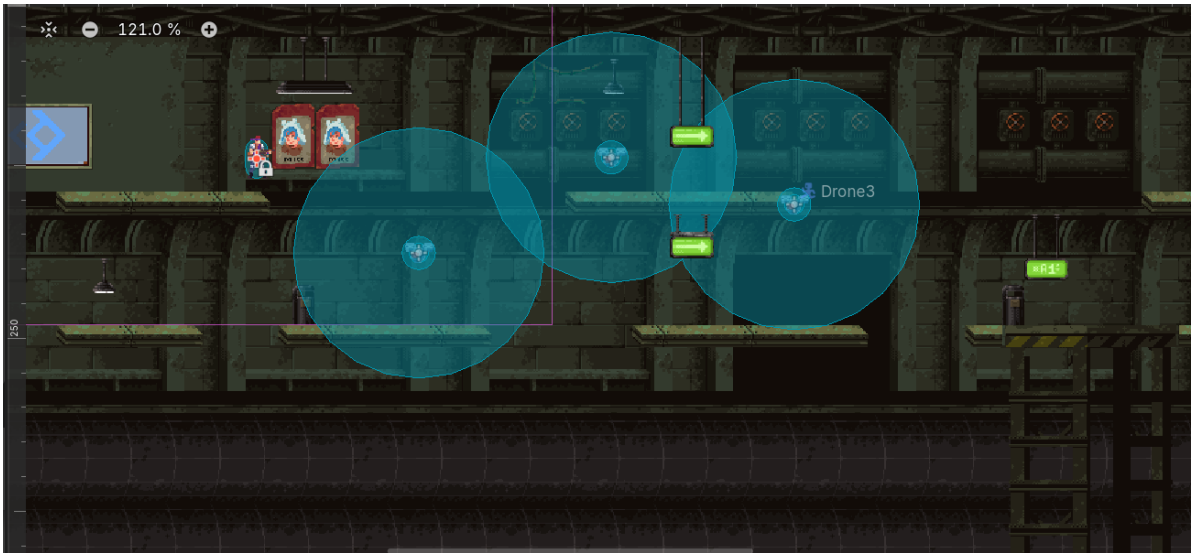


在Drone场景中选中根节点后，点击分组页的加号新增一个分组“Drones”，如图。



随后打开level脚本，编辑Ready方法，使用get_nodes_in_group方法打印场景树内指定分组的所有节点信息，可以在level场景中复制多个无人机，使效果更直观。

```
func _ready() -> void:
    print(get_tree().get_nodes_in_group('Drones'))
```



控制台输出如图。

```
Godot Engine v4.7.stable.official.5b4e0cb0f - https://godotengine.org
Vulkan 1.4.325 - Forward+ - Using Device #0: AMD - AMD Radeon 760M Graphics

[Drone:<CharacterBody2D#101955146951>, Drone2:<CharacterBody2D#102106141904>, Drone3:<CharacterBody2D#102257136857>]
```

接下来实现无人机连锁爆炸功能，我们在drone脚本的explode方法中，在爆炸动画执行完成后，使用分组功能获取场景树中其他无人机节点，并进行位置判断并执行爆炸，代码如下。

```
func explode():
    speed = 0
    $AnimatedSprite2D.hide()
    $ExplosionSprite.show()
    $AnimationPlayer.play("explode")
    await $AnimationPlayer.animation_finished #等待动画播放完毕再销毁动画
    for drone:CharacterBody2D in get_tree().get_nodes_in_group('Drones'): #遍历Drones分组中对象
        if position.distance_to(drone.position) < 20: #无人机距离小于20像素就触发连锁爆炸
            drone.explode()
    queue_free()
```

在level场景中调整无人机位置，方便体现效果，如图。



运行游戏测试，射击最下方无人机三次后，该无人机爆炸并触发中间无人机爆炸，最后连锁导致上方无人机也爆炸。

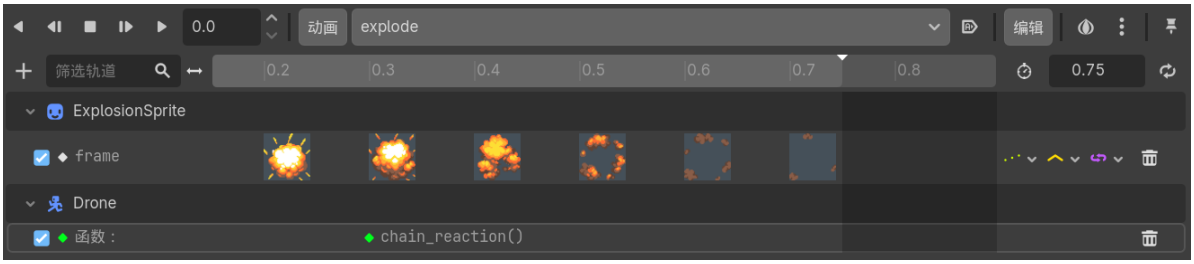
但有一个问题，无人机连锁爆炸效果是在第一个无人机爆炸动画结束后触发，画面效果并不流畅，我们希望可以在爆炸动画进行到一半时触发连锁效果，可以使用AnimationPlayer的属性轨道来实现。

先将连锁爆炸效果的语句单独提取为一个方法“chain_reaction”，方便动画调用，该方法无参，也不需要在脚本中进行调用，而是被AnimationPlayer调用。

```
func explode():
    speed = 0
    $AnimatedSprite2D.hide()
    $ExplosionSprite.show()
    $AnimationPlayer.play("explode")
    await $AnimationPlayer.animation_finished#等待动画播放完毕再销毁动画
    queue_free()

func chain_reaction():
    for drone:CharacterBody2D in get_tree().get_nodes_in_group('Drones'):
        if position.distance_to(drone.position) < 20:
            drone.explode()
```

选中Drone中的AnimationPlayer，选择explode动画后新建一个调用方法轨道，选择Drone场景，随后在轨道中间插入关键帧，选择chain_reaction方法插入，如图。



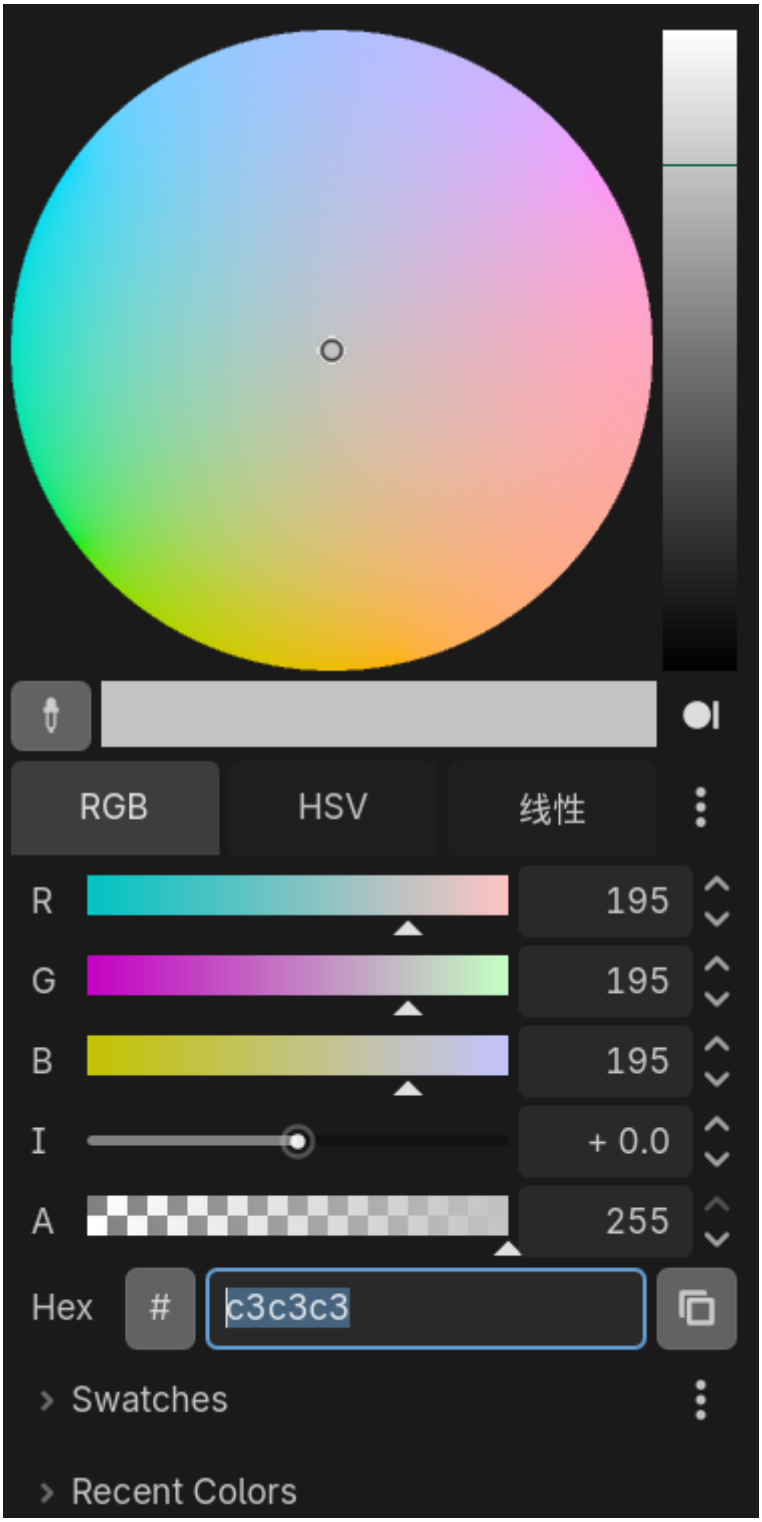
再进行测试可以看到连锁爆炸效果显著更加流畅。

至此无人机的所有功能已开发完成。

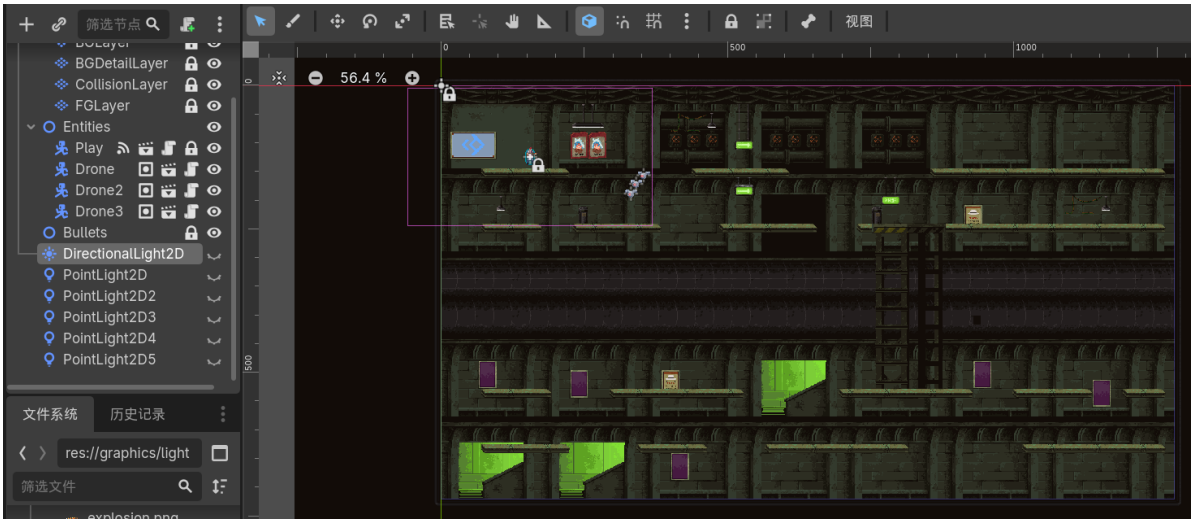
9.光源光照系统

接下来为游戏添加灯光效果。

在Level场景中添加一个DirectionalLight2D节点，该节点模拟覆盖整个场景的无数平行光线，设置其color属性，略微将光线调暗增加氛围感，将Blend M（混合模式）设置为Mix。

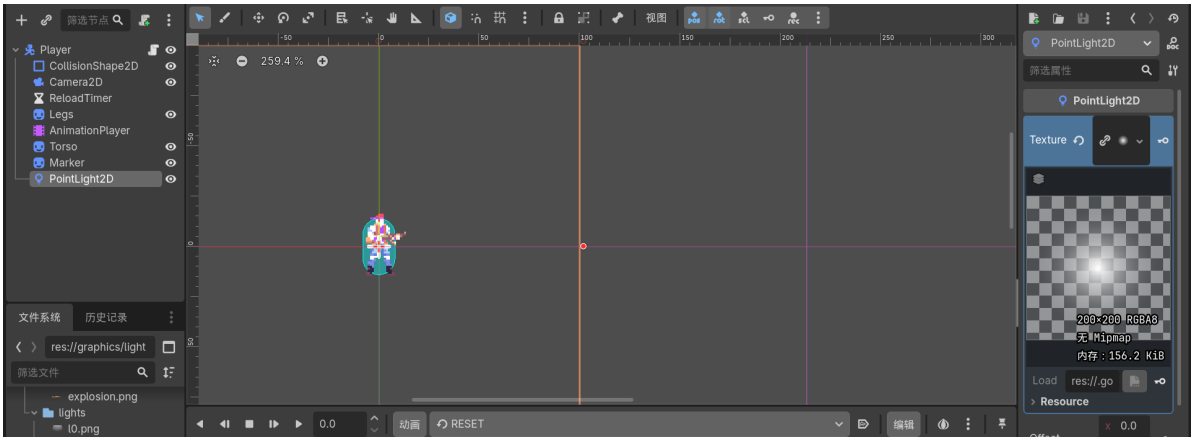


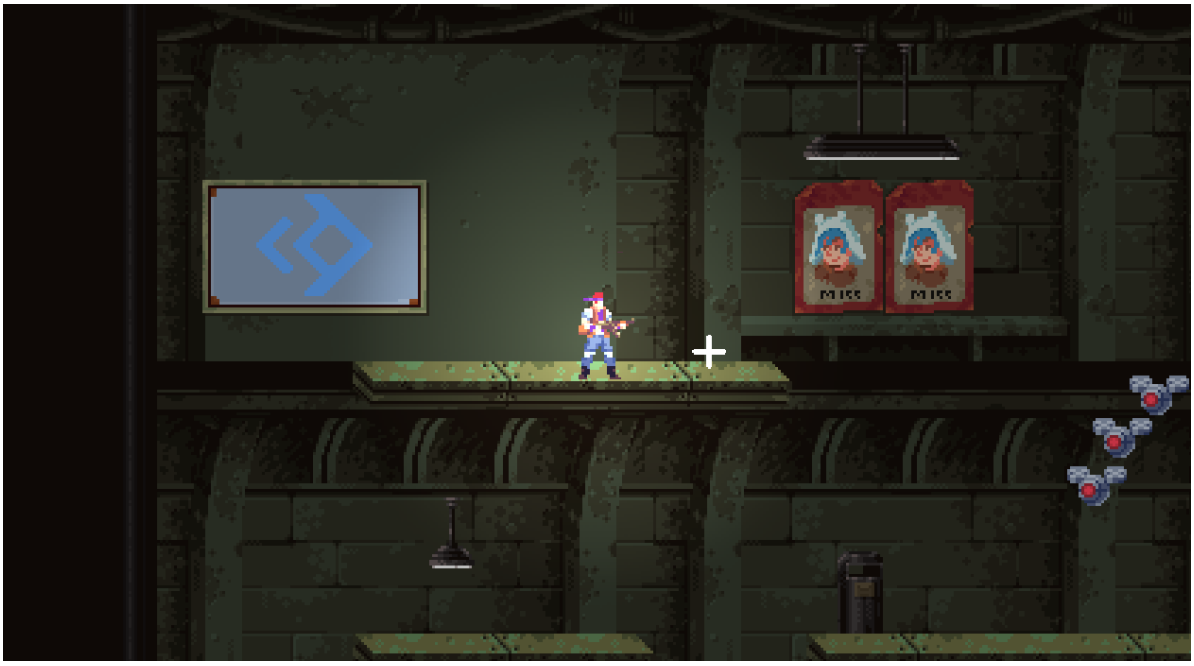
完成后切换该节点的可见性，可以看到区别明显，切换为可见后整个场景画面更暗，更有质感。





在Player场景中为角色挂载一个PointLight2D节点，Texture选用res://graphics/lights/main.png，即可为角色周围添加灯光效果，如图。

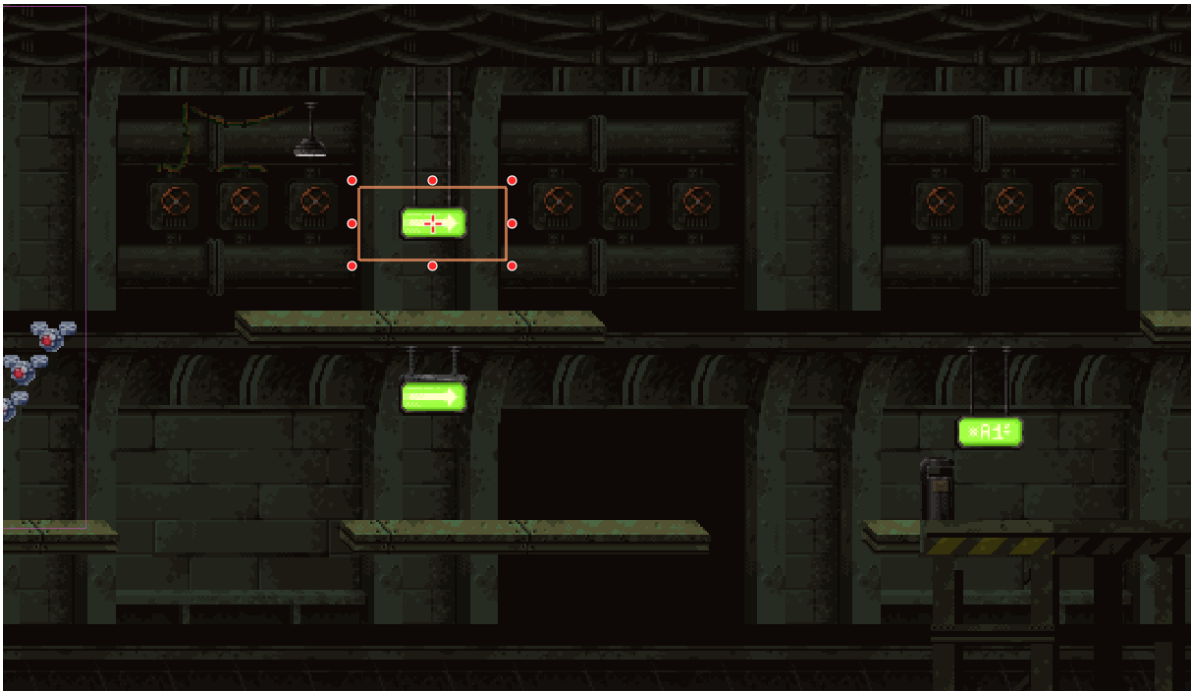




PointLight2D节点还可以在Godot内快速自定义，例如选择GradientTexture2D，直接在Godot内编辑外观作为门外的光线效果，如图。



再为灯光和指示牌添加灯光效果，选用res://graphics/lights/l3.png，并将节点拖到指示牌上覆盖住，如图。



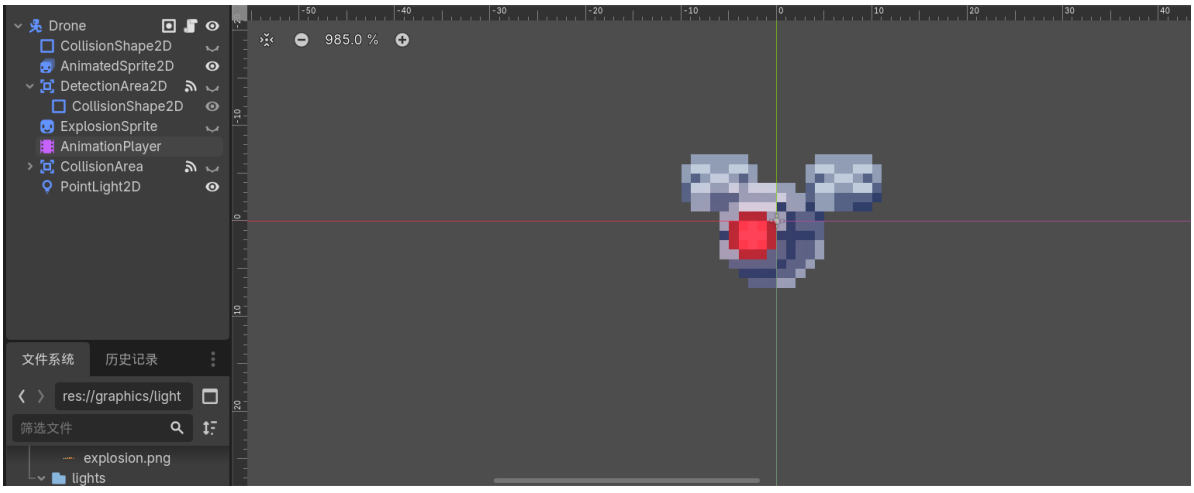
为了让指示牌的灯光效果更加精致，可以使用脚本实现其闪烁效果。

编辑level脚本中的_ready方法，如下，使用Tween来实现闪烁效果。

```
func _ready() -> void:
    var light_tween = get_tree().create_tween()
    light_tween.set_loops()#设置补间动画循环，无参数即一直循环
    light_tween.tween_property($Layers/PointLight2D3,"energy",1.5,2)
    light_tween.tween_property($Layers/PointLight2D3,"energy",0.5,2)
```

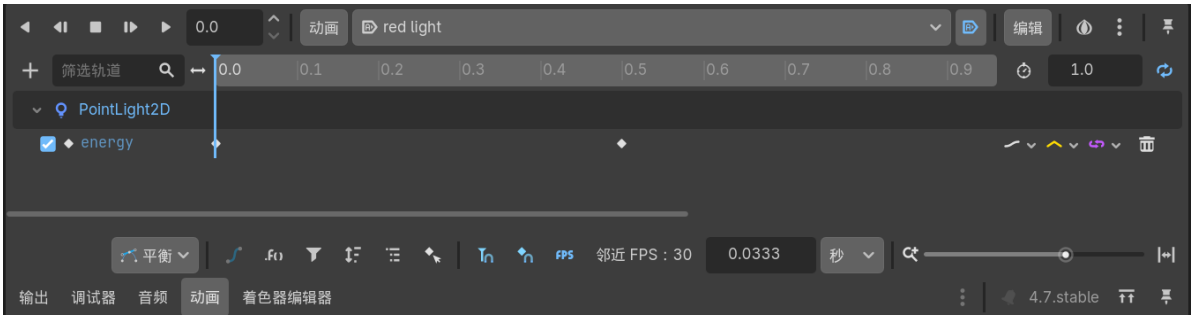
接下来为无人机添加红光闪烁效果。

先为Drone场景挂载一个PointLight2D节点，为了方便调整，将Drone下所有Area2D节点设置为不可见，画面瞬间简洁不少，如图。



调整PointLight2D的Color属性为红色，Texture为res://graphics/lights/main.png，我们使用AnimationPlayer来操作该节点的Energy属性实现闪烁效果。

选中AnimationPlayer节点，新建动画“red light”，新建属性轨道，选择PointLight2D节点的Energy，动画总时长1s，分别在0.0和0.5处插入值为0.3和1.2的关键帧，开启循环播放和自动播放，如图。

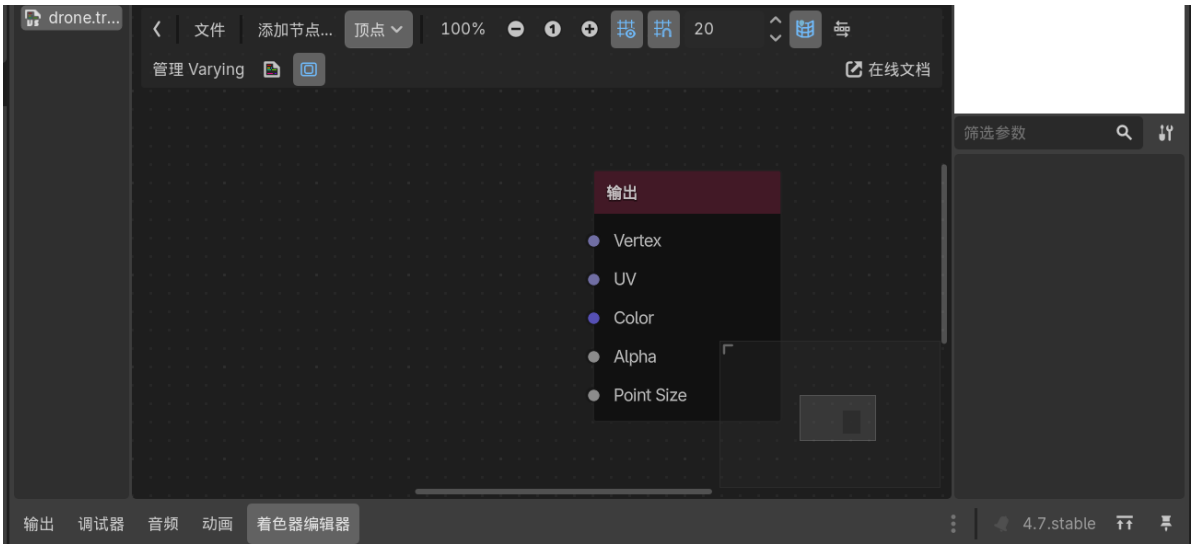


自此，已完成所有关于灯光的设置。

10.着色器特效

接下来我们希望实现无人机被击中后闪烁发光的特效,这需要学习一个新的知识点——shaders（着色器），借助shader，可以做出各种特效。

选择Drone场景，选中AnimatedSprite2D，在其Material属性中点击新建ShaderMaterial，随后在弹出的窗口中选择新建着色器，类型选择VisualShader，模式为Canvas Item，在scenes文件夹下新建一个shaders文件夹用于专门存放shader文件。点击创建后可以看到弹出的着色器编辑器窗口，如图，在窗口的顶部选项中将“顶点”改为“片段”。

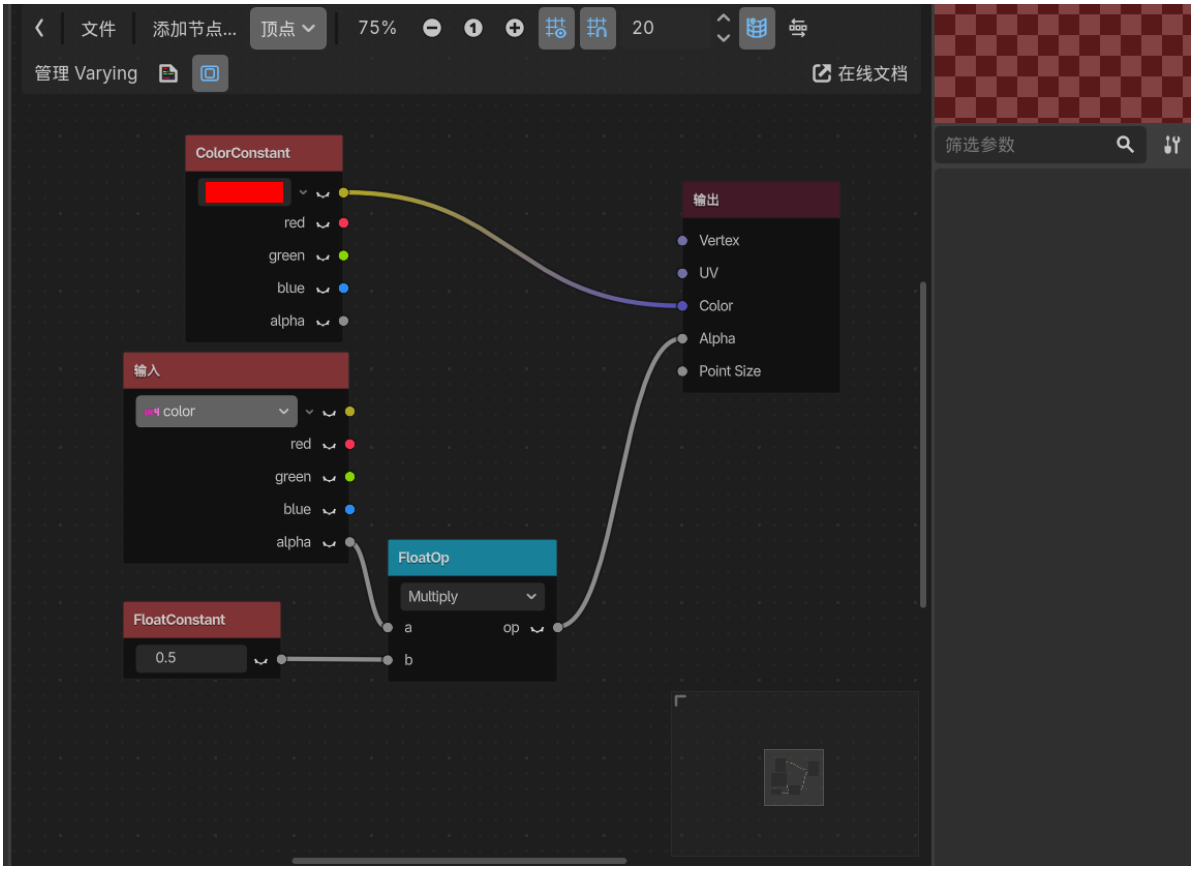


一个简单的例子，我们利用着色器让无人机变成红色。右键空白区域可以新建着色器节点。

我们分别新建Color、ColorConstant和FloatConstant、FloatOp节点，这些节点有着各自的作用。

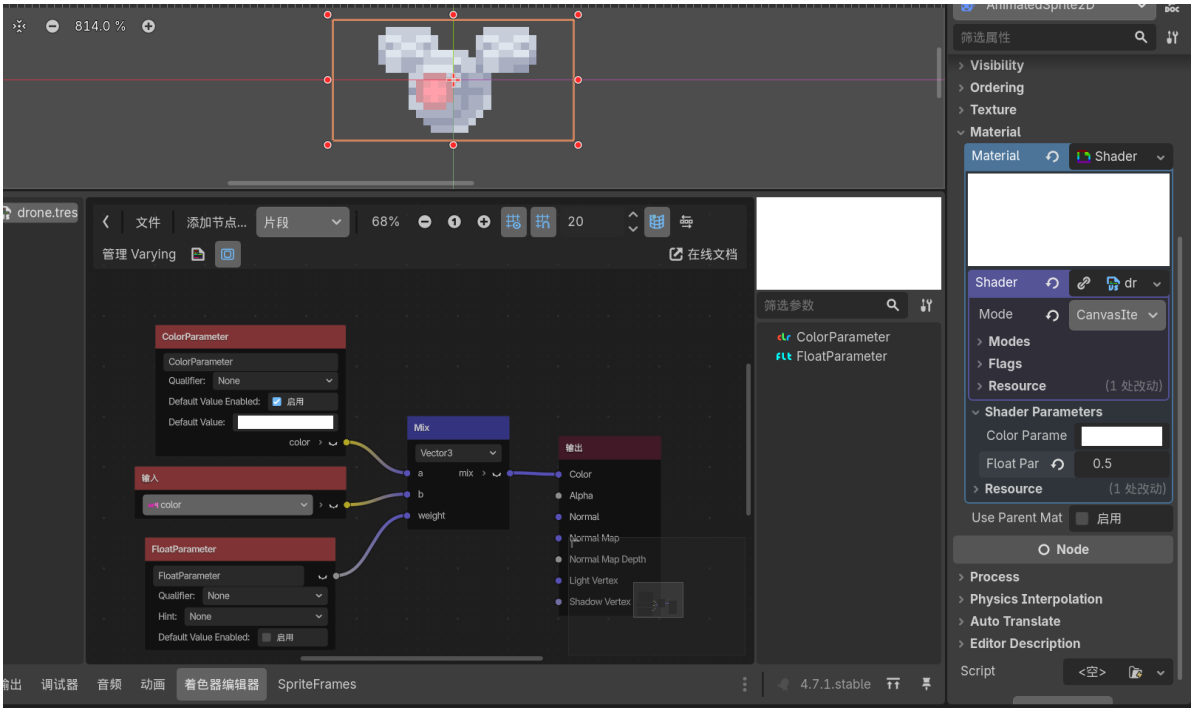
ColorConstant节点负责输出一个不可修改的颜色常量，Color输出选中的对象的颜色信息，FloatConstnat输出设置好的浮点型数据，FloatOp将两个输入的数据相乘并输出浮点型数据。

我们将各个节点进行如图上设置并连接后即可将无人机设置为红色且透明度适中。



以上只是一个例子，接下来我们回到需求，要让无人机实现被击中后闪烁。我们删除刚才的shader逻辑，新增一个ColorParameter颜色参数节点，勾选启用默认值，默认为白色，不用更改。再添加一个FloatParameter和Color节点。注意Parameter类型和Constant类型区别的在于——Parameter类型节点的值可以在检查器中修改。

添加一个Mix（Vector3）节点，将ColorParameter和Color连接到a、b接口，FloatParameter连接到Weight接口，这样就可以通过调节FloatParameter的值来调整a接口和b接口颜色的混合比重，若设置为1则完全是b接口的颜色，设置为0则完全为a接口颜色，我们这里设置为0.5，如图。我们可以重新命名FloatParamater为Progress，使其在检查器中作用更直观。



接下来我们通过脚本来修改Color Parameter和Progress两个参数。

使用补间动画来实现，在drone脚本的hit方法中如下修改。

```
func hit():
    health -= 1
    if health <= 0:
        explode()
    var tween = get_tree().create_tween()

tween.tween_property($AnimatedSprite2D.material,'shader_parameter/Progress',0.0,
0.3)

tween.tween_property($AnimatedSprite2D.material,'shader_parameter/Progress',1.0,
0.5)
```

运行后测试发现一个问题，击中一个无人机后，场景中其他无人机也会闪烁，这是因为所有无人机都被同一个Material赋值，我们修改的是这个共享的资源，为了解决这个问题，可以在ready方法中为每个无人机设置一个Material副本，如下。

```
func _ready() -> void:
    var unique_material = $AnimatedSprite2D.material.duplicate()
    $AnimatedSprite2D.material = unique_material
    unique_material.set_shader_parameter("Progress", 1.0)
```

使用duplicate创建单独的副本后，每个无人机调用hit方法后修改的material都是独自的副本，初始的material始终不变，再次测试发现效果符合预期。

11.项目收尾完善

最后,我们分别为子弹发射和无人机爆炸加上音效，使用AudioStreamPlayer2D来实现。

为Bullet场景添加AudioStreamPlayer2D，Stream属性中选用res://audio/laser.wav，开启Autoplay或在bullet脚本的_ready中使用$AudioStreamPlayer2D.play()皆可。

Drone场景中同样添加AudioStreamPlayer2D，选用res://audio/explosion.wav，但更复杂的是，我们如果直接在explode方法中调用音频播放，会发现音频播放了两次，这是因为在动画过程中节点没有进行爆炸条件判断，导致若是在爆炸过程中再次被击中，爆炸动画将再次从头播放，为了解决这个问题，我们可以新增一个布尔类型字段is_explode，初始值为false，具体判断逻辑如下。

```
func hit():
    if is_explode:#若已爆炸，则返回，避免多次触发
        return
    health -= 1
    var tween = get_tree().create_tween()

tween.tween_property($AnimatedSprite2D.material,'shader_parameter/Progress',0.0,
0.1)

tween.tween_property($AnimatedSprite2D.material,'shader_parameter/Progress',1.0,
0.5)
    if health <= 0:
        explode()

func explode():
    if is_explode:#若已爆炸，则返回，避免多次触发
        return
    is_explode = true #已爆炸
    speed = 0
    $AudioStreamPlayer2D.play()
```

```
    $AnimationPlayer.stop()
    $AnimatedSprite2D.hide()
    $ExplosionSprite.show()
    $AnimationPlayer.play("explode")
    await $AnimationPlayer.animation_finished#等待动画播放完毕再销毁动画
    chain_reaction()
    queue_free()
```

再次运行测试发现，爆炸音频已正常。

至此，横板闯关类游戏已制作完成，只需要增加无人机数量、丰富场景、地形，即可增加游戏的可玩性。

3 农场经营类游戏

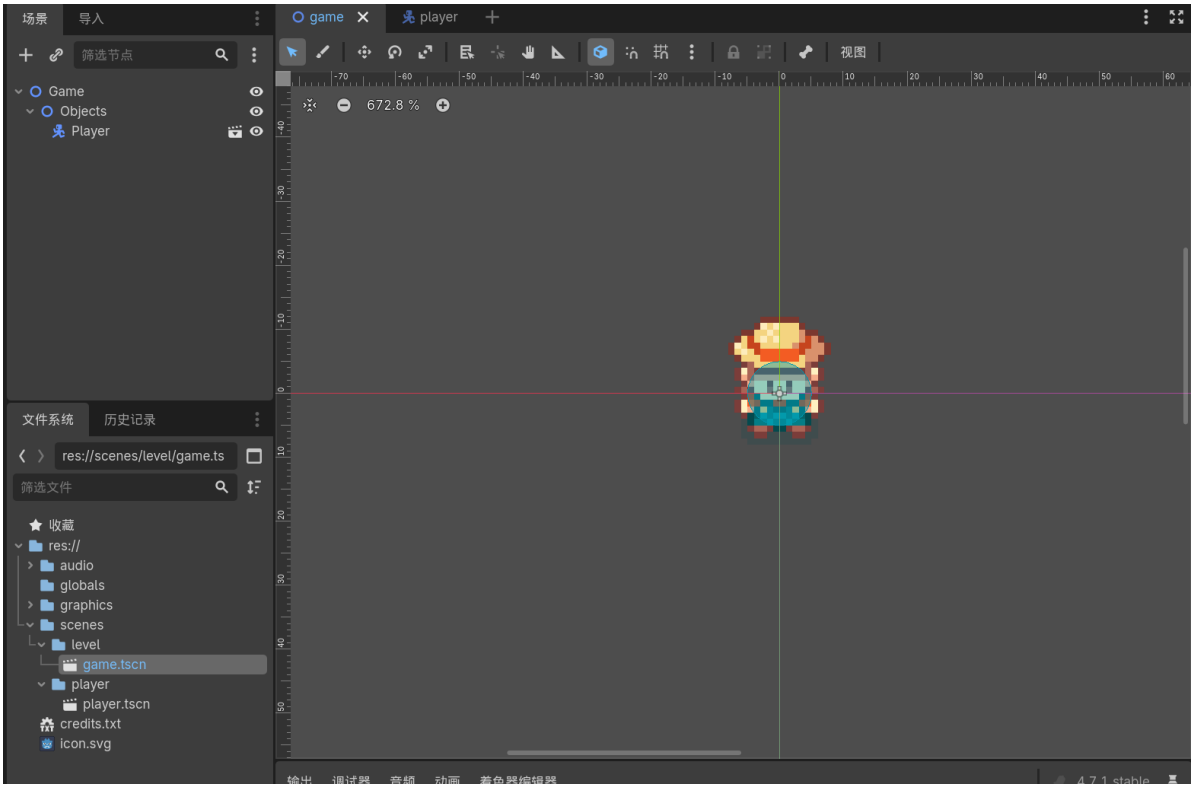
上

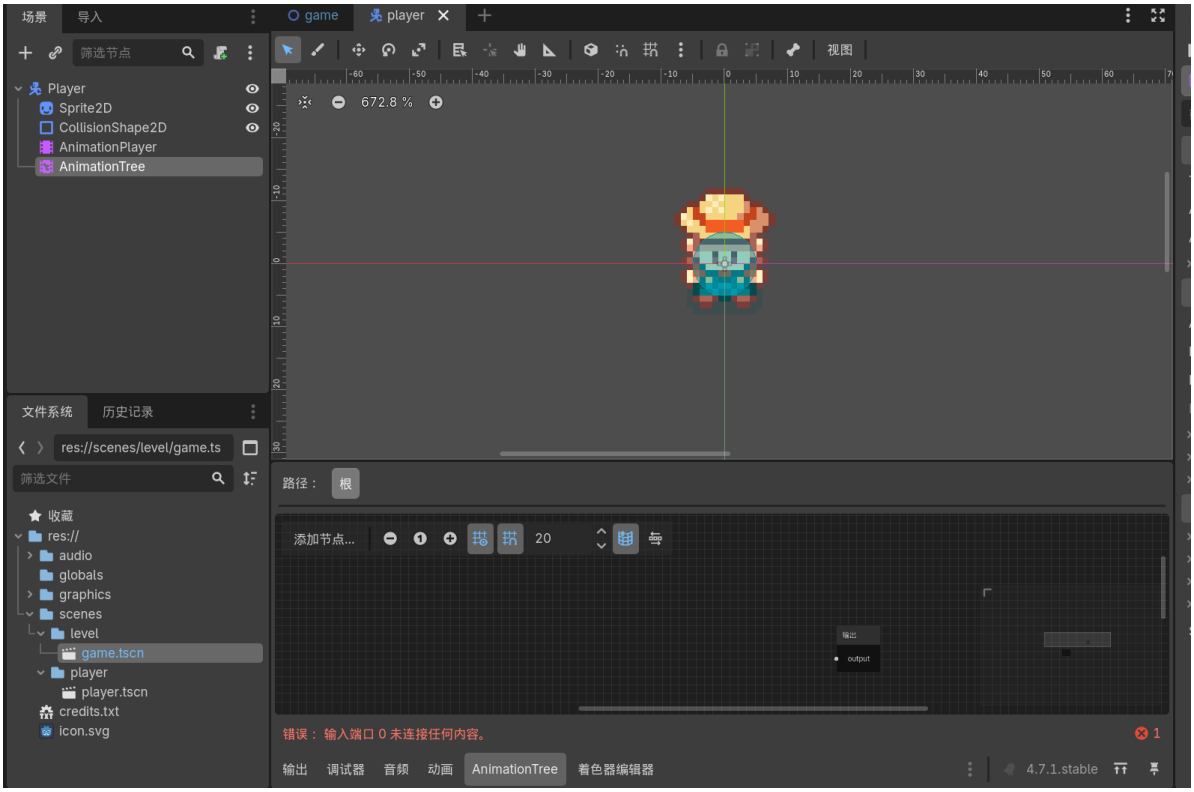
1.课程简介

我们现在开始制作一款类似"星露谷物语"的农场经营类游戏,这款游戏依旧是2D类型,不过会涉及到一些更进阶的游戏制作机制——一.复杂动画结构管理，对于这个游戏的主角，有着大量游戏动画需要规整管理。针对这个需求，Godot提供了AnimationTree，借助它我们能够轻松的切换各类动画，完整管控这套动画逻辑。二，这个项目会大量使用TileMap自动铺瓦功能，借助它快速生成各类地形，并在游戏运行时修改地形，以此来实现农作物的耕地功能。三，本教程将涉及到一个重要的功能——枚举，借助枚举，我们能清晰区分不同的状态，如玩家当前手持的工具。我们将逐步进行学习。

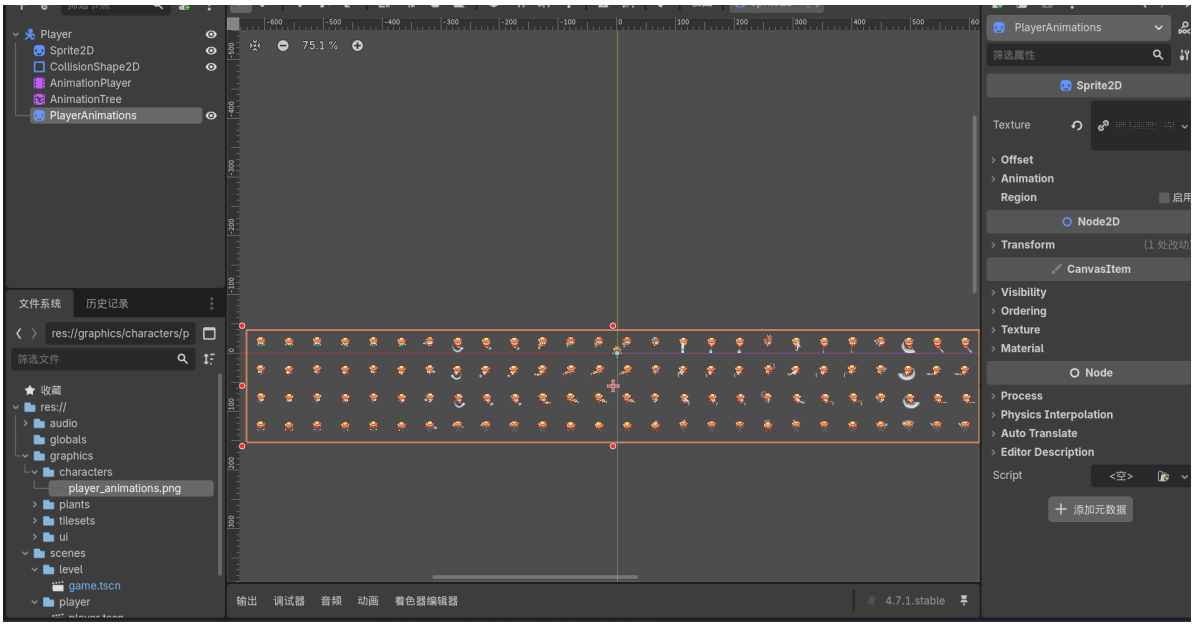
首先处理玩家角色，现阶段我们先实现玩家角色自由移动，再搭配不同的动作动画。

打开项目，可以看到项目已预先准备好两个场景，分别是level文件夹下的Game游戏场景和player下的Player玩家场景。结构如图。

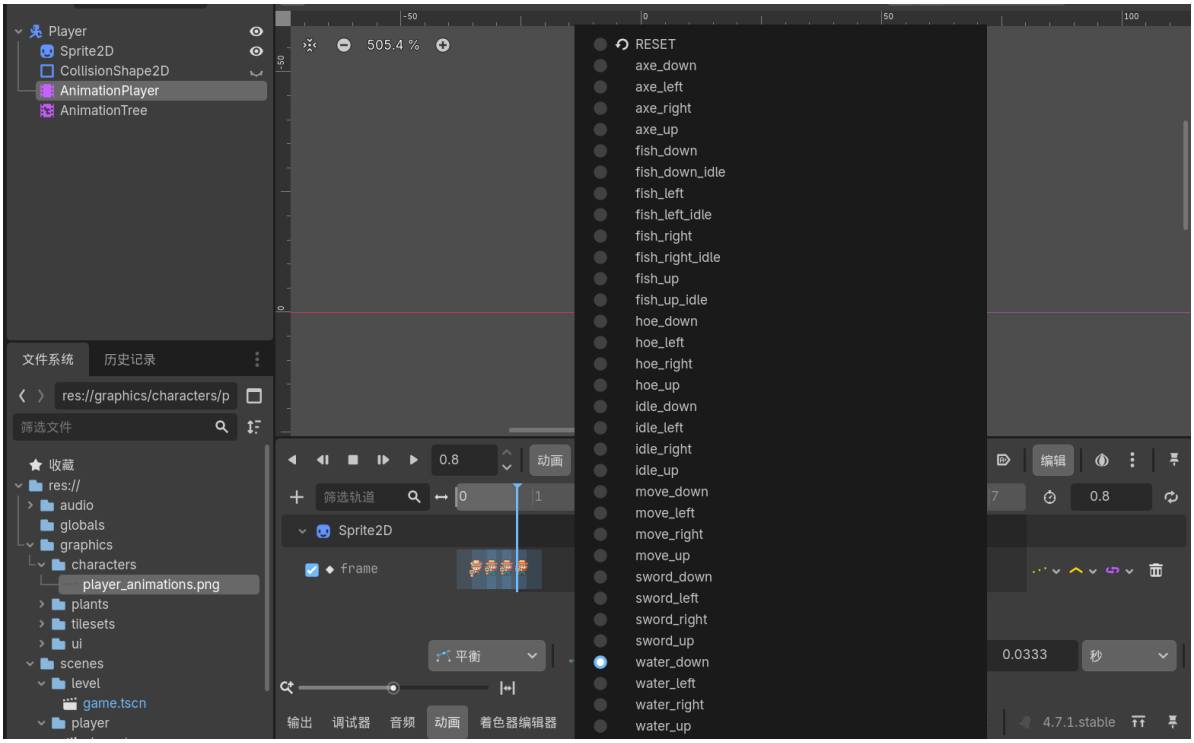




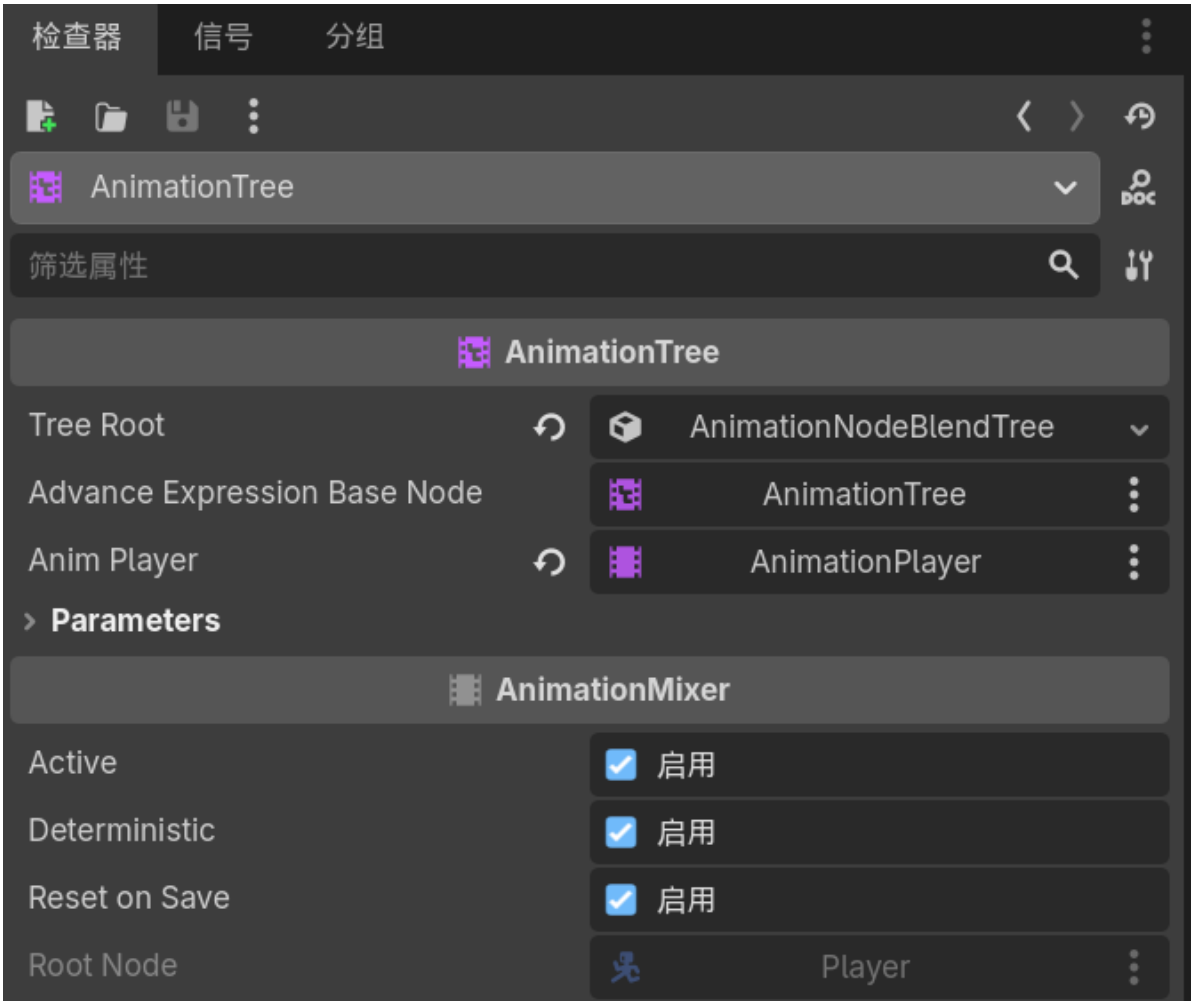
可以看到Player场景下的Sprite2D使用res://graphics/characters/player_animations.png，26列4行可以组成四个方向的多种不同动作动画，我们将在游戏中对这些动画进行管控。

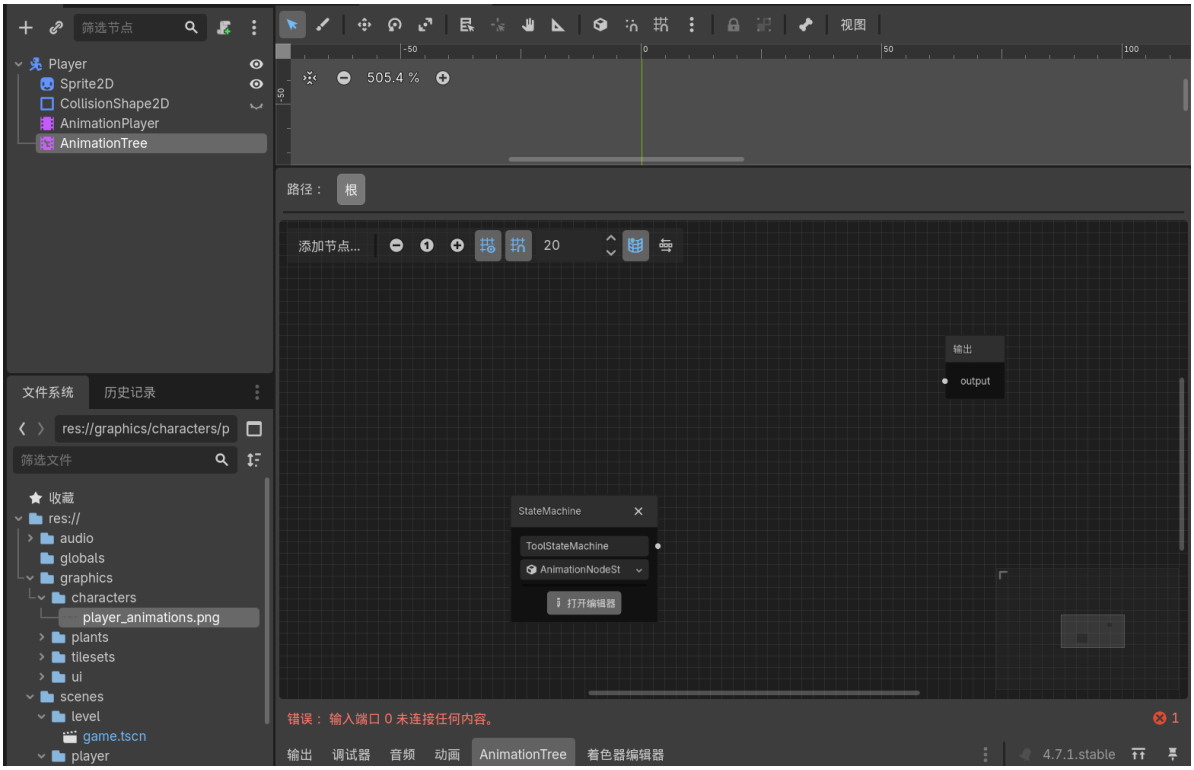


打开AnimationPlayer节点，点击动画可以看到已预先配置好所有动作的动画效果，如图。

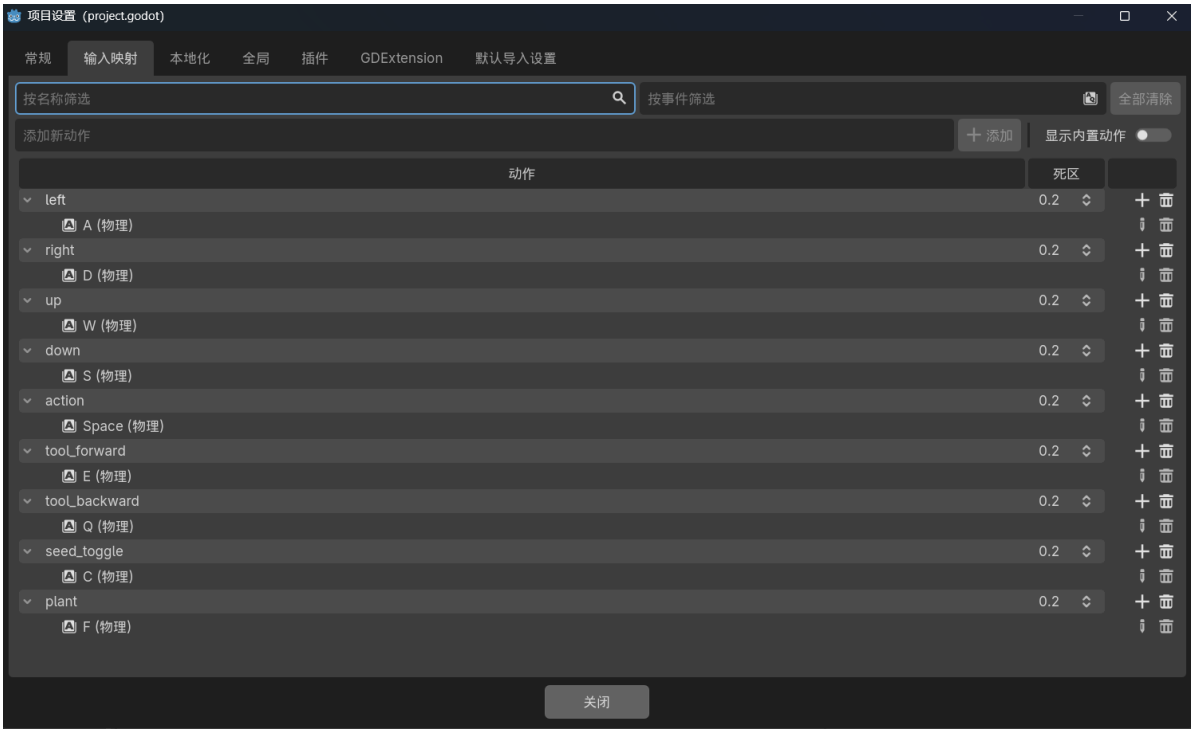


最后，点击AnimationTree节点，可以看到AnimationTree面板，该节点创建时根节点为AnimationNodeBlendTree，绑定了AnimationPlayer，已经创建好了一个状态机节点，如图，具体的状态机配置比较繁琐，后面再详细讲解。





打开项目设置查看输入映射，可以看到已经预设好基础按键，如图。



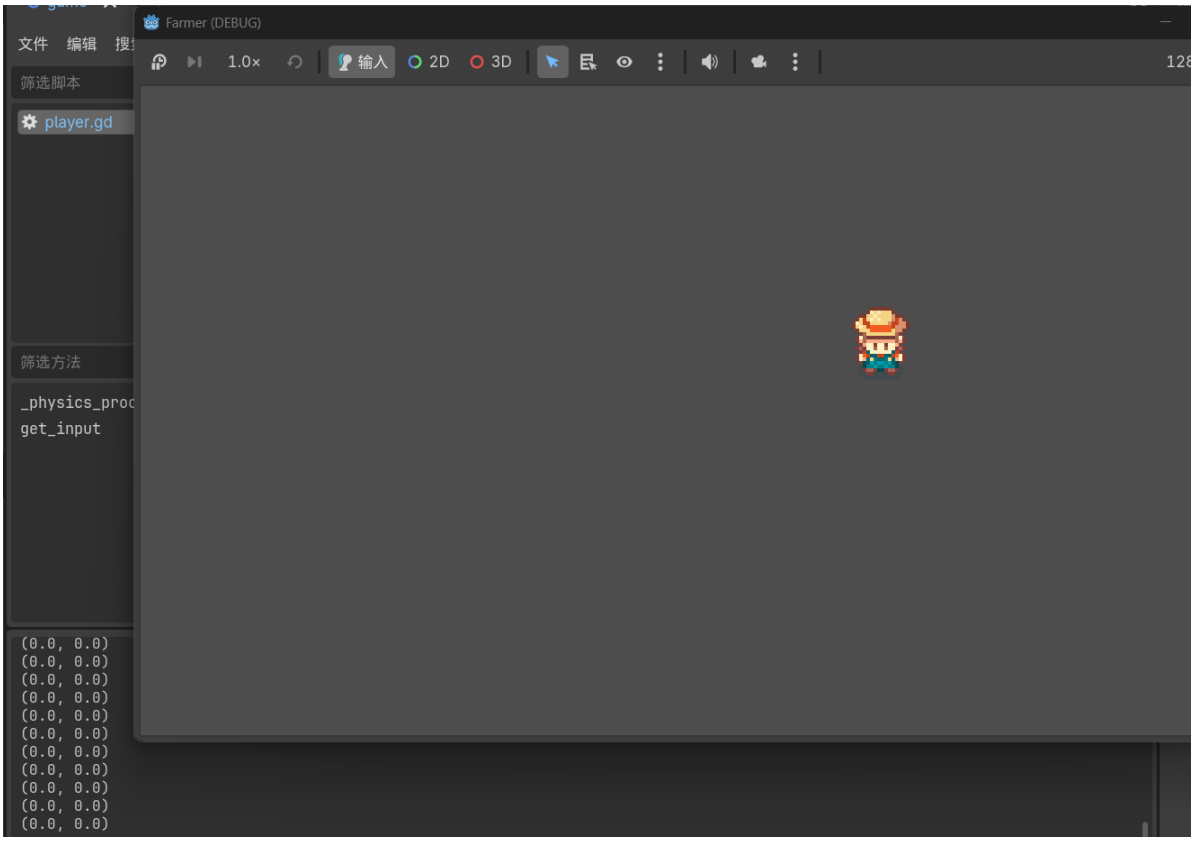
现阶段只需要使用上下左右的移动按键，选中Player场景根节点，挂载脚本。脚本内容如下。

```
var direction: Vector2
var speed := 300

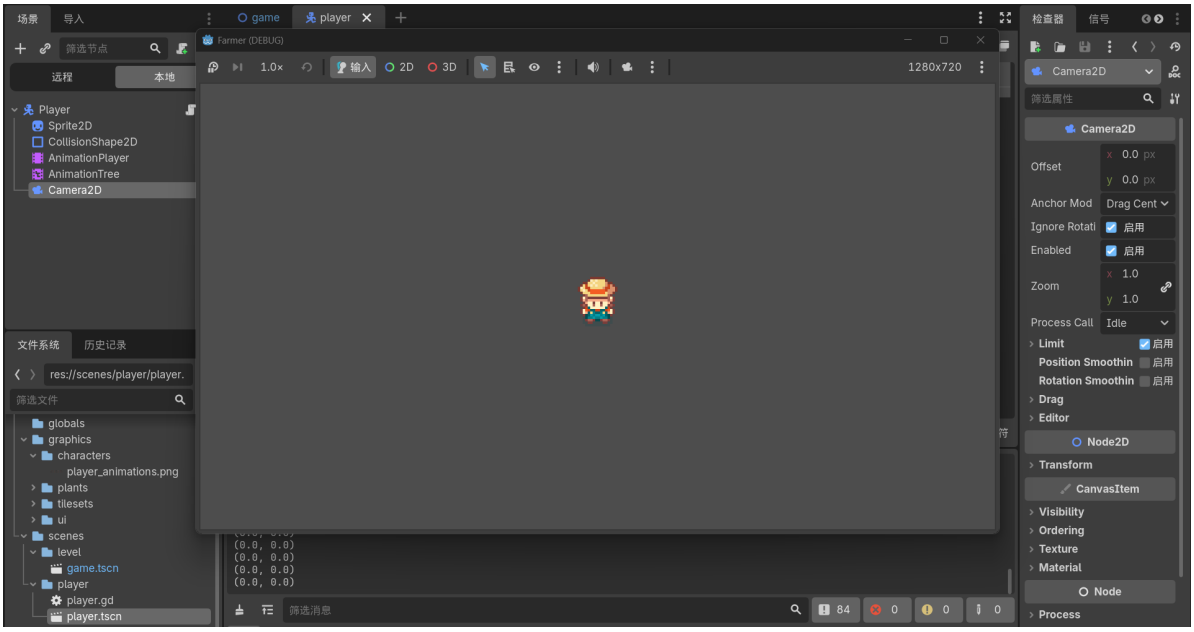
func _physics_process(_delta: float) -> void:
    get_input()
    velocity = direction * speed
    move_and_slide()

func get_input():
    direction = Input.get_vector("left","right","up","down")
    print(direction)
```

运行后可正常移动角色，且控制台打印direction



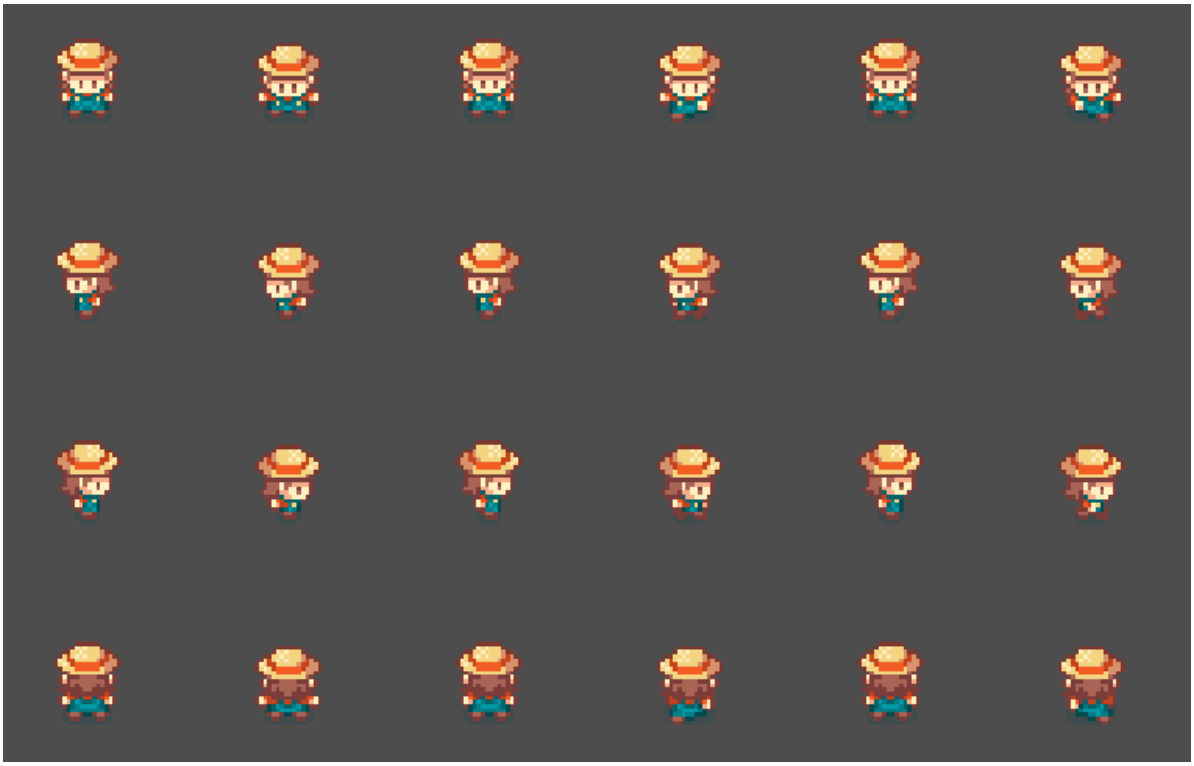
接下来添加Camera2D，直接保持默认缩放比例即可。



至此，游戏初期阶段已完成。

2.玩家角色动画

接下来先为游戏添加待机和行走的动画。观察res://graphics/characters/player_animations.png，前两列是四个方向的待机动画，后四列是四个方向的行走动画。即共八段动画。这些动画已全在AnimationPlayer中设置完成，但若要靠代码来实现这些动画的播放，工程量非常巨大，因此我们将完全依靠AnimationTree动画状树节点来完成。



AnimationTree的核心功能就是从AnimationPlayer中调取动画并播放，内部包含三种实现方式：

一、BlendTree混合树，用于动画间的融合过渡；

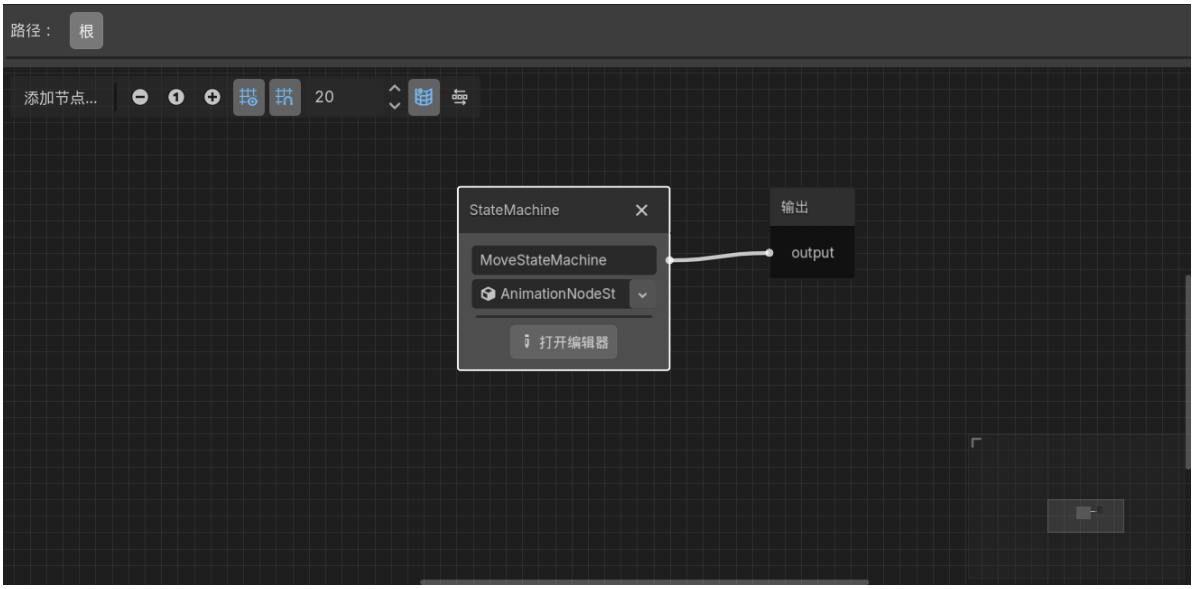
二、StateMachine状态机，用于切换不同动画状态；

三、BlendSpace混合空间，用于根据标记点的位置切换对应动画

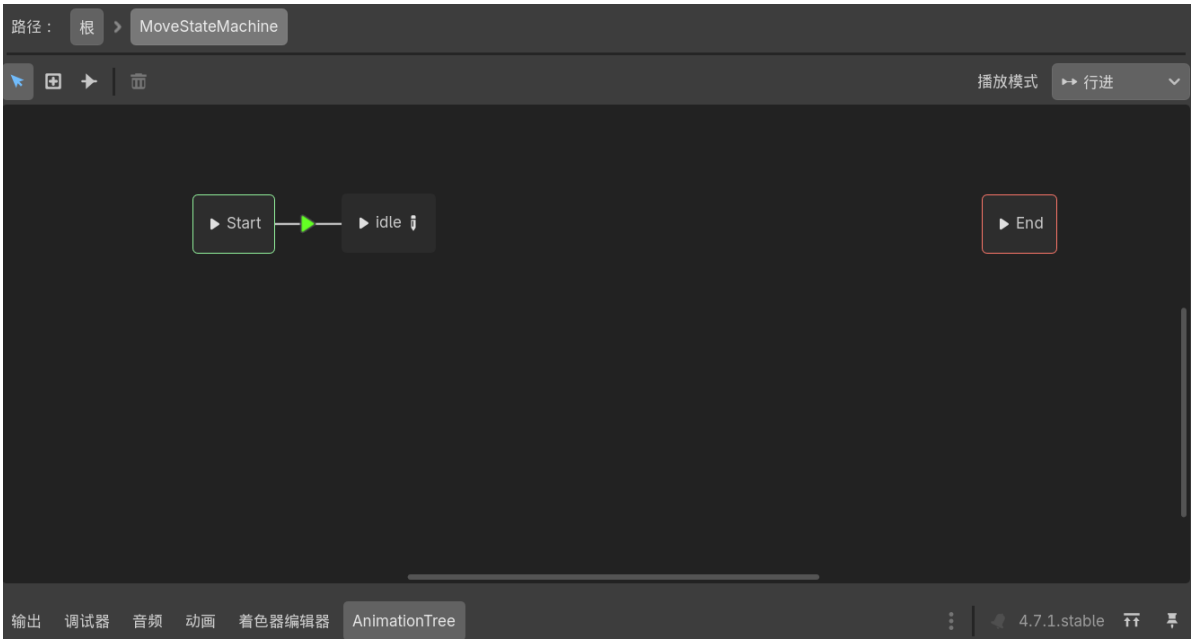
三种方案实际功能有重合之处，但各自有专属适用场景，且三者可以组合使用、互相嵌套。

基本理论讲完，在Godot咱们就直接练起来！

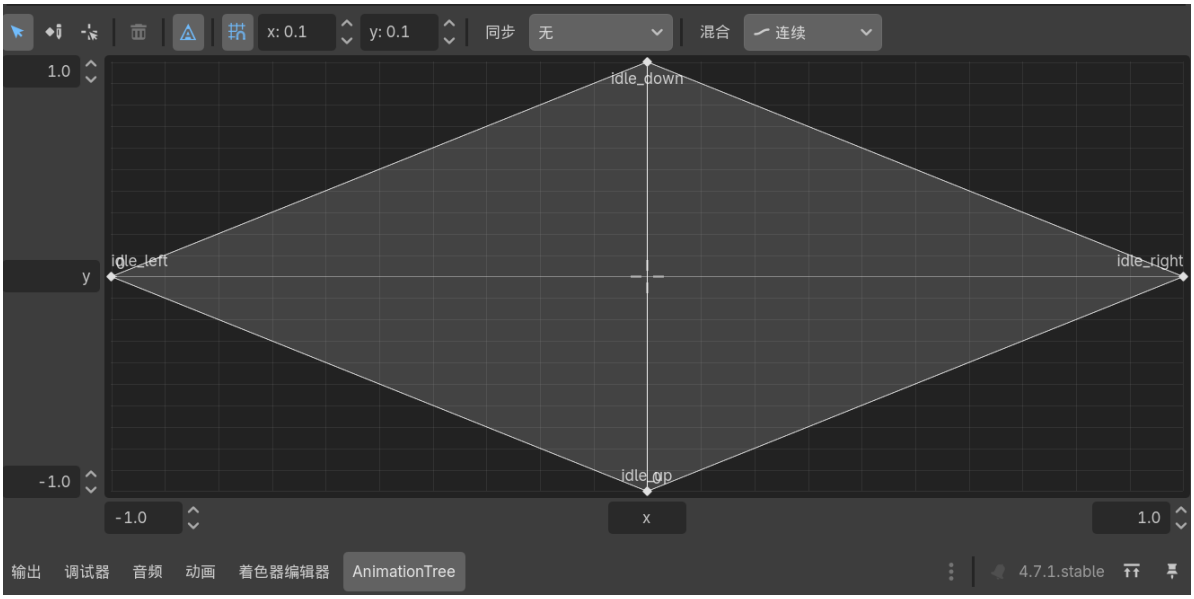
打开AnimationTree面板，右键空白区域新建一个StateMachine节点，命名为MoveStateMachine，将它和输出节点连接。



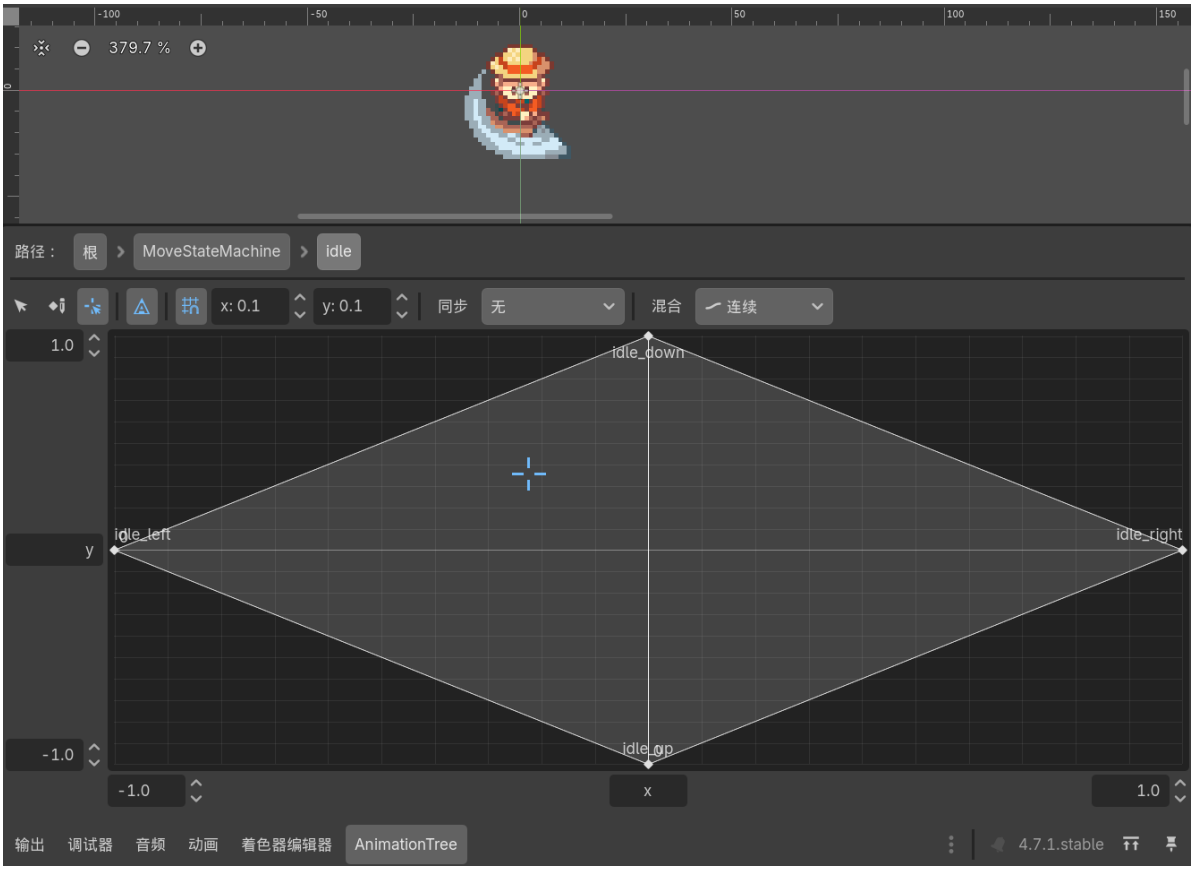
打开MoveStateMachine的编辑器，右键添加一个BlendSpace2D节点，命名为idle，专门负责处理待机动画。将它和Start连接。



点击idle右侧的铅笔图标进入编辑页面，在上下左右分别添加对应的待机动画，需要注意的是，该编辑页面中的Y轴是和其它模块相反的，也就是说idle_down在最上面，idle_up在最下面。



BlendSpace2D节点会根据混合位置也就是十字光标的不同自动切换到不同的动画，也就是说我们只需要在脚本中修改混合位置就可以快速切换不同的动画，如图。



打开player脚本进行编辑，内容如下。

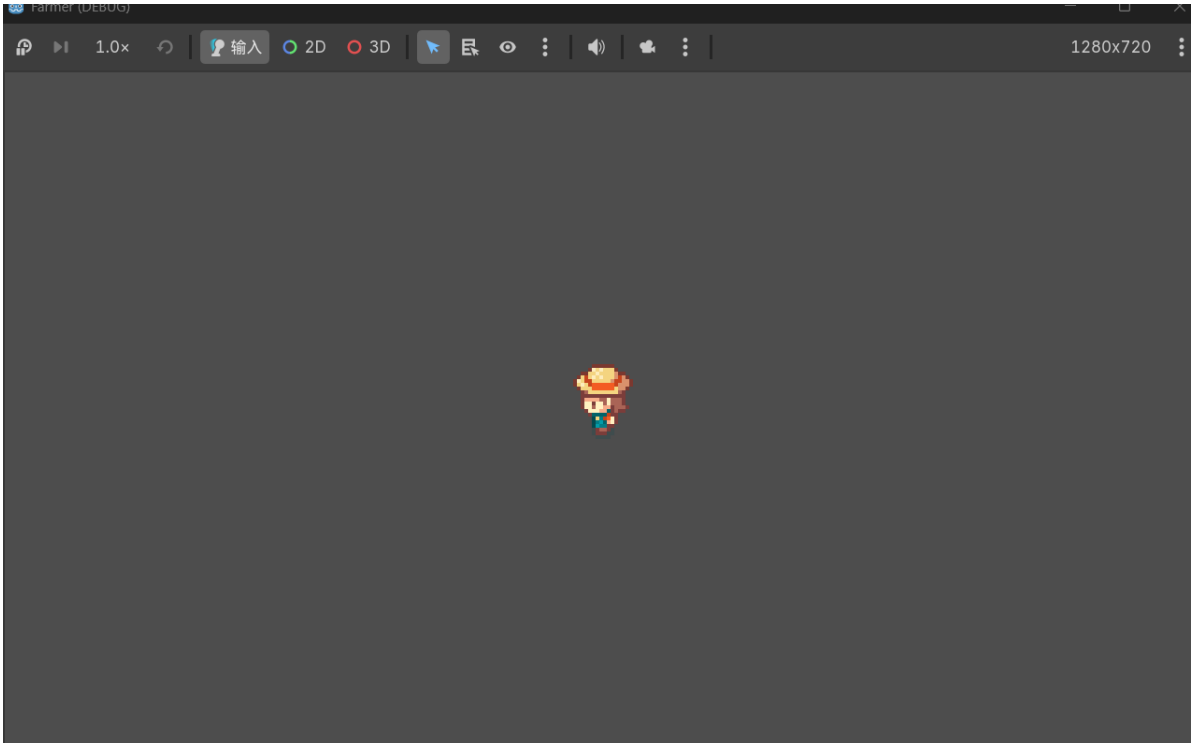
```
var direction: Vector2
var speed := 300

func _physics_process(_delta: float) -> void:
    get_input()
    velocity = direction * speed
    move_and_slide()
    animation()#对动画方法进行调用

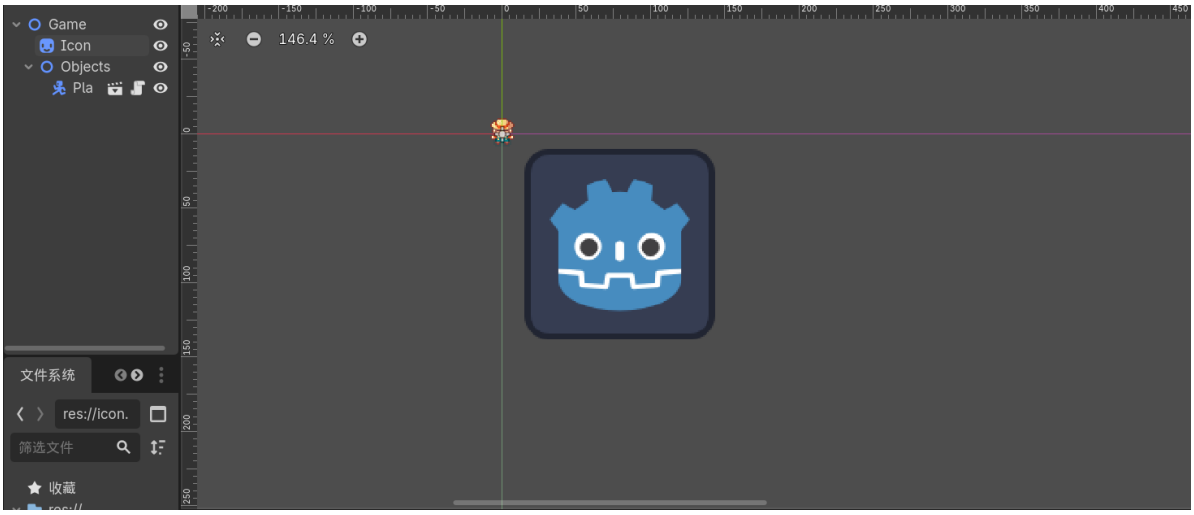
func get_input():
    direction = Input.get_vector("left" , "right" , "up" , "down")

func animation():
    if direction:#设置混合位置为获取的玩家输入
        $AnimationTree.set("parameters/MoveStateMachine/idle/blend_position",
direction)
```

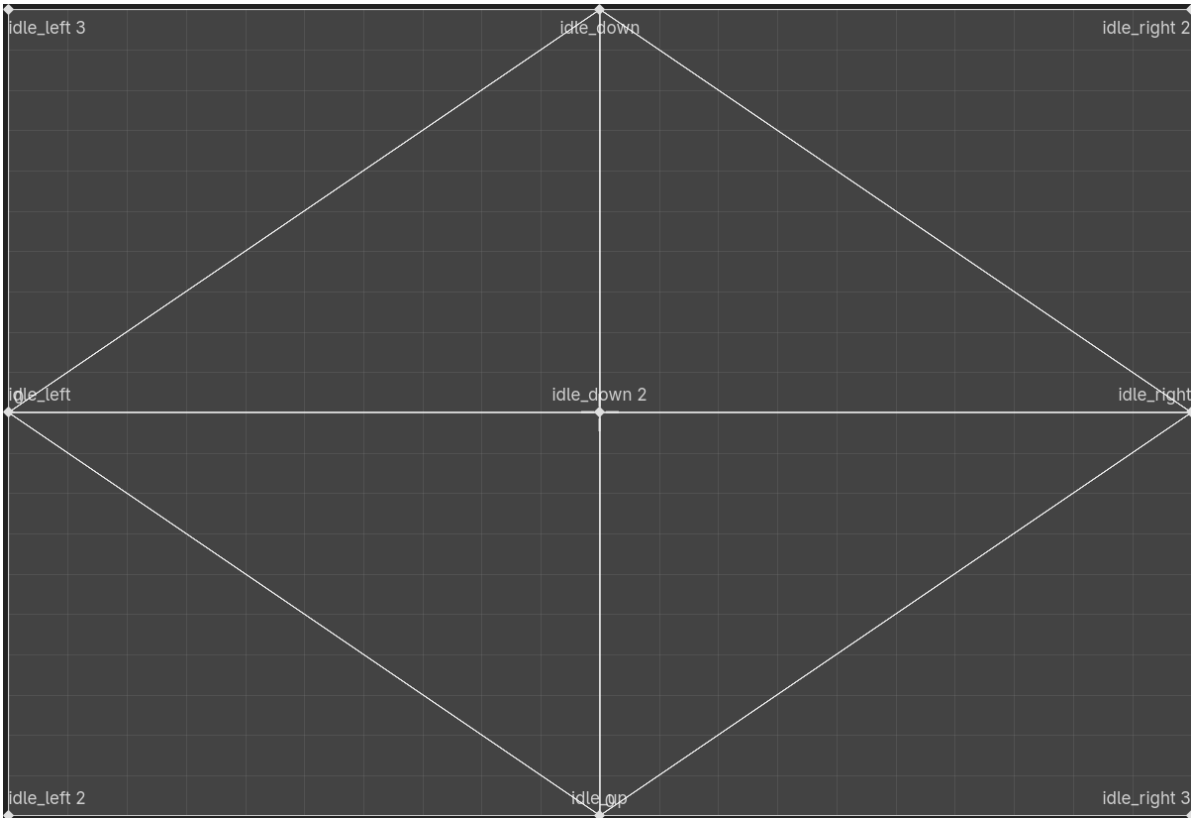
运行测试看到游戏可以正常根据玩家输入切换到不同的待机动画，如图。



在Game场景中添加一个贴图作为参照物方便观察角色移动状态。



运行游戏后发现角色移动后正常播放上下左右四个方向的待机动画，但斜向移动时的动画会出现错乱。这是因为AnimationTree中未添加斜向动画，重新打开idle节点的编辑器，添加如图节点，还可以在中心点添加idle_down动画节点，使角色停止后自动切换到idle_down动画。



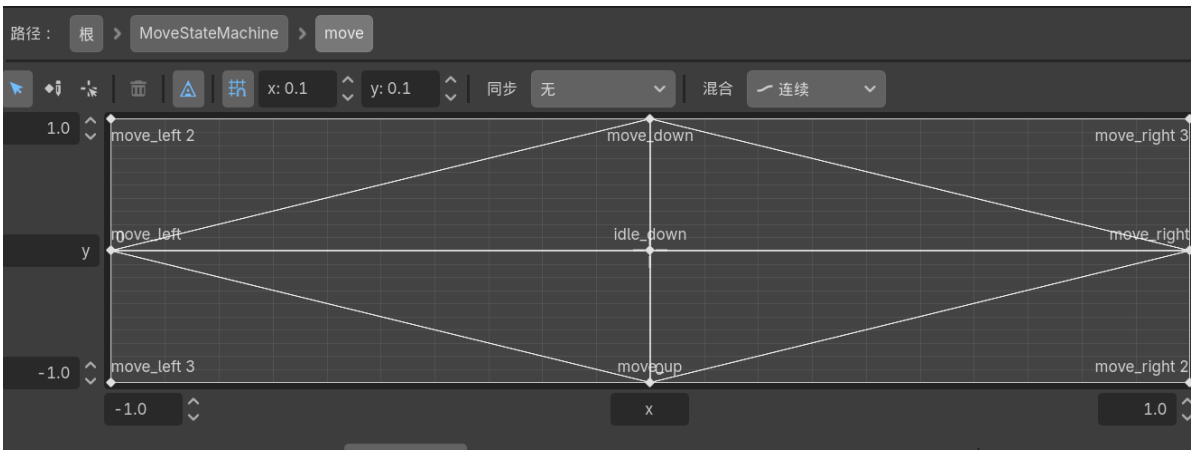
另外可以在脚本中对方向向量进行归一化，避免输出小数。

```
func animation():
    if direction:
        var target_vector: Vector2 =
Vector2(round(direction.x),round(direction.y))#归一化
        $AnimationTree.set("parameters/MoveStateMachine/idle/blend_position",
target_vector)
```

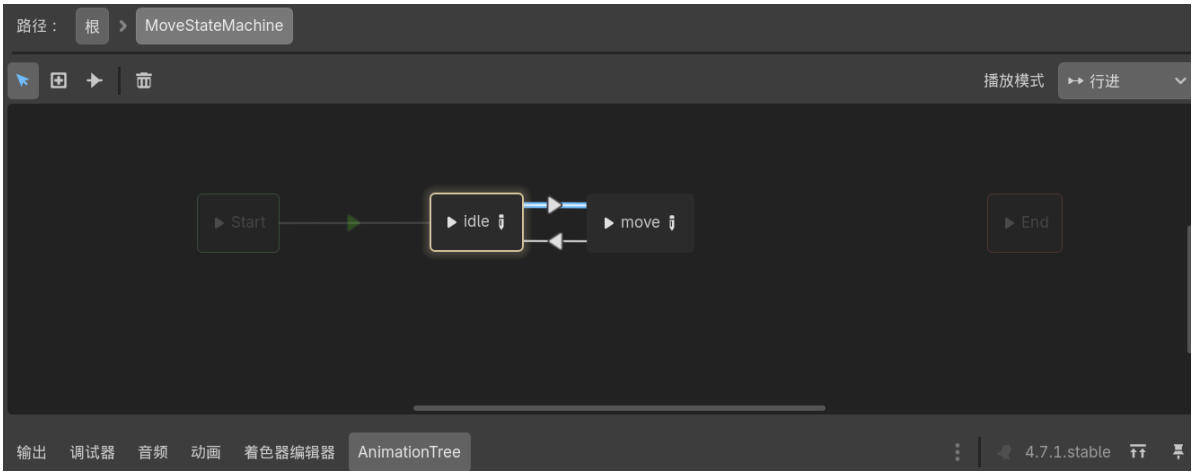
3.移动状态动画

接下来要对动画效果进行拓展，也就是添加行走动画，并添加切换动画的逻辑，使动画在行走和待机间切换，以此实现完整的行走循环动画。

首先在MoveStateMachine中新增一个BlendSpace2D节点“move”，同样为其添加对应动画，如图。



其次，将move节点和idle节点连接，前进模式改为启用而不是自动，防止自动且切换，而是使用脚本切换。



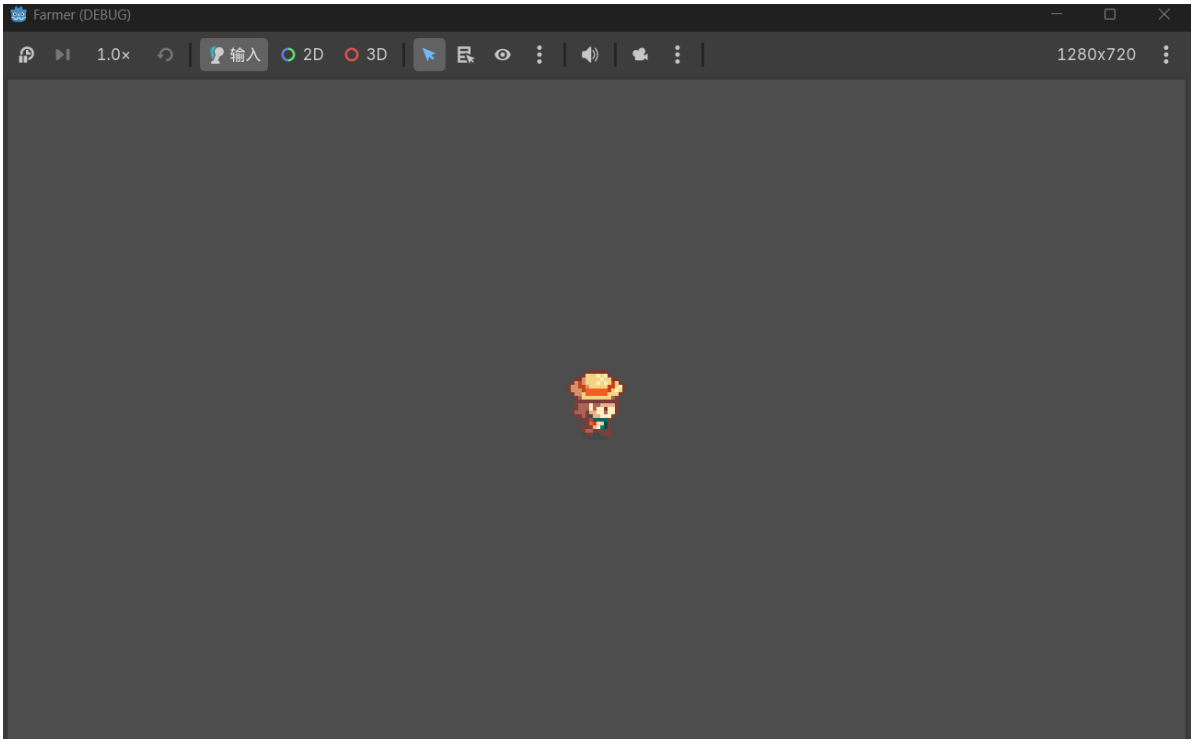
在player脚本中创建AnimationNodeStateMachinePlayback型变量move_state_machine，使用@onready注解将其标记，值为$AnimationTree.get("parameters/MoveStateMachine/playback")，如下。

```
@onready var move_state_machine:AnimationNodeStateMachinePlayback =
$AnimationTree.get("parameters/MoveStateMachine/playback")
```

在animation方法中添加逻辑，若有输入，即将状态机过渡到move，并更新两个状态的混合位置为target_vector，否则将状态机过度到idle，如下。

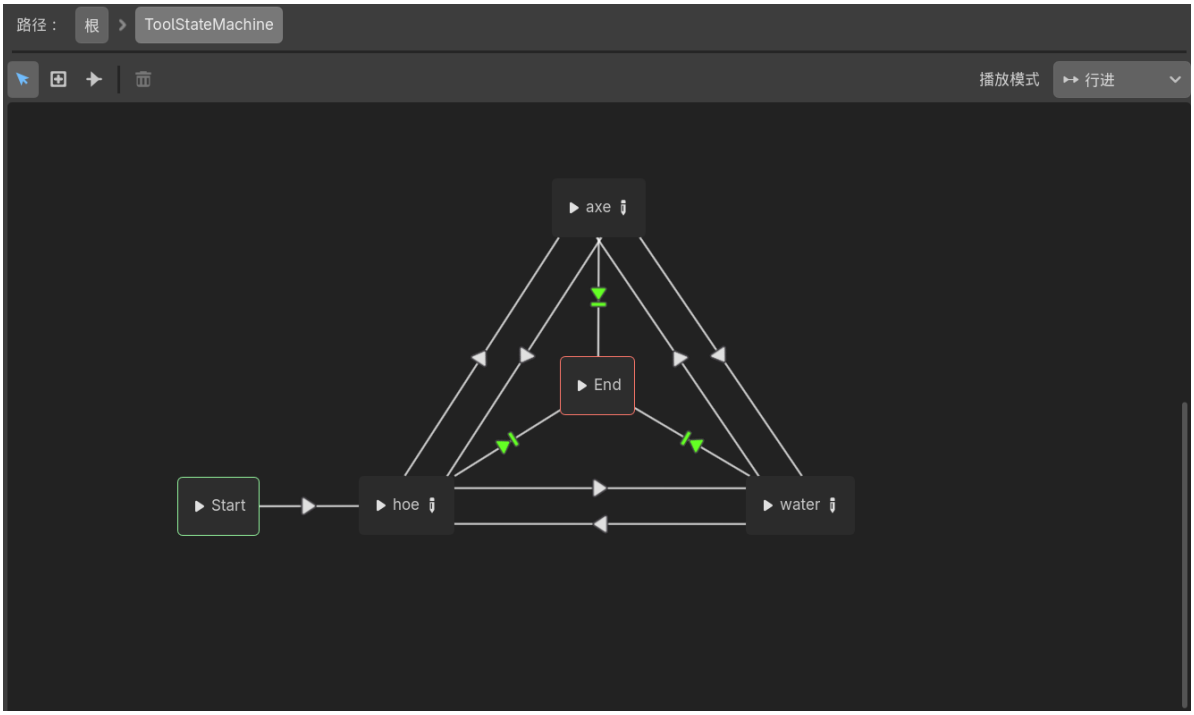
```
func animation():
    if direction:
        move_state_machine.travel('move')
        var target_vector: Vector2 =
Vector2(round(direction.x),round(direction.y))
        $AnimationTree.set("parameters/MoveStateMachine/move/blend_position",
target_vector)
        $AnimationTree.set("parameters/MoveStateMachine/idle/blend_position",
target_vector)
    else:
        move_state_machine.travel('idle')
```

运行测试可以看到，玩家有输入时，播放对应的移动动画，无输入时切换到对应方向的待机动画。

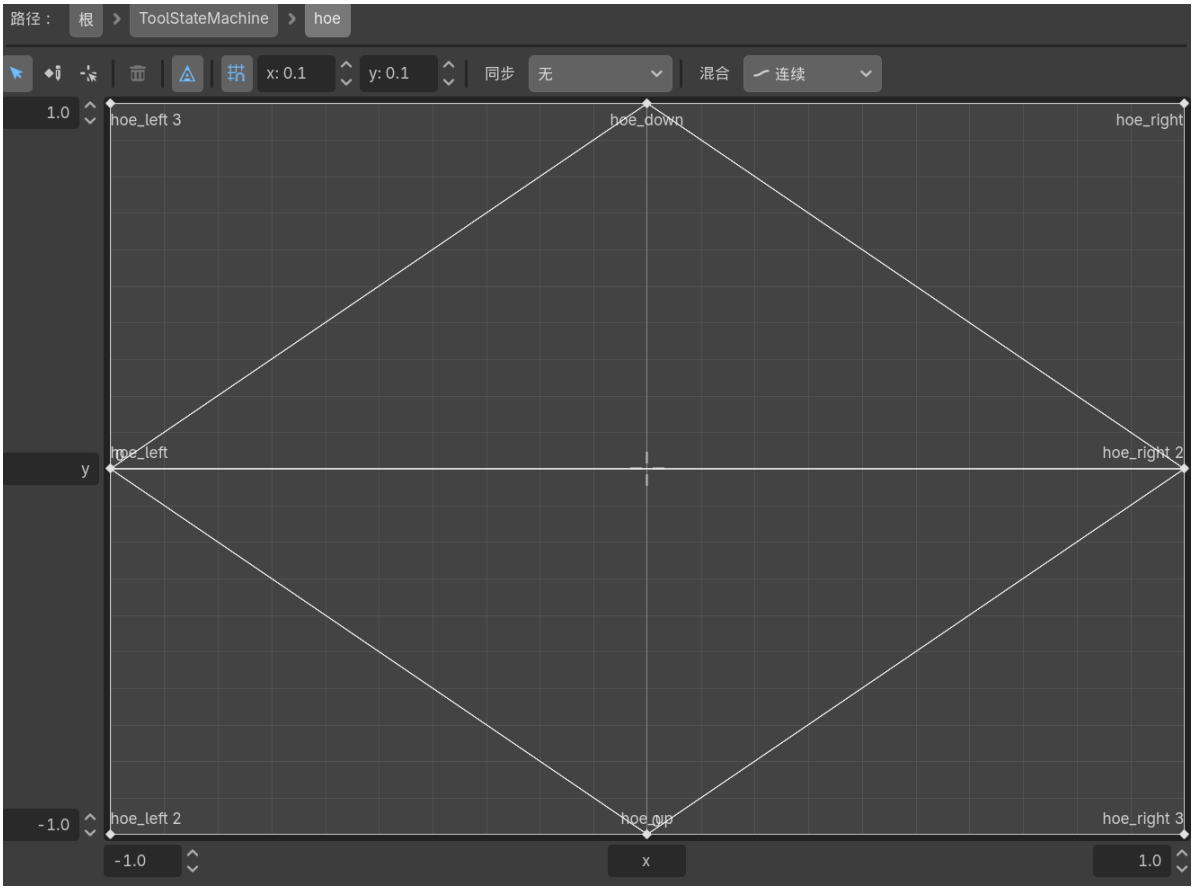


接下来，还剩下工具的动画逻辑需要实现。

在AnimationTree中新增StateMachine节点，命名为ToolStateMachine，打开其编辑器，如图编辑。



对每一个工具的BlendShape2D节点进行对应的动画配置，如图。



后续只需要使用脚本实现动画切换逻辑，即可实现整套动画。

4.枚举类型

在实现动画切换逻辑前，要先制作各类工具，这些工具可以和游戏场景产生交互，且每种工具有专属动画。在实现这些工具之前，我们要先在Player类内部记录当前使用的工具（Axe、Hoe、Water can），针对这三种工具，我们需要记录其当前状态，并实现工具切换功能，让玩家可以自由选择当前使用工具。

若直接使用一个变量来记录，容易存在各种问题，因此这里应该使用枚举类型（Enums）

枚举是一种用于定义一组命名整数常量的数据类型。它能让代码更具可读性、可维护性，并避免使用难以理解的数字。

枚举中定义的元素会自动从0开始递增赋值，也可以自定义其值，如下，HOE值为0、AXE值为1、WATER值为2.

```
enum Tools {HOE,AXE,WATER}
```

我们刚好可以借助这套数值机制，只需要对当前数值加一减一即可实现工具切换。

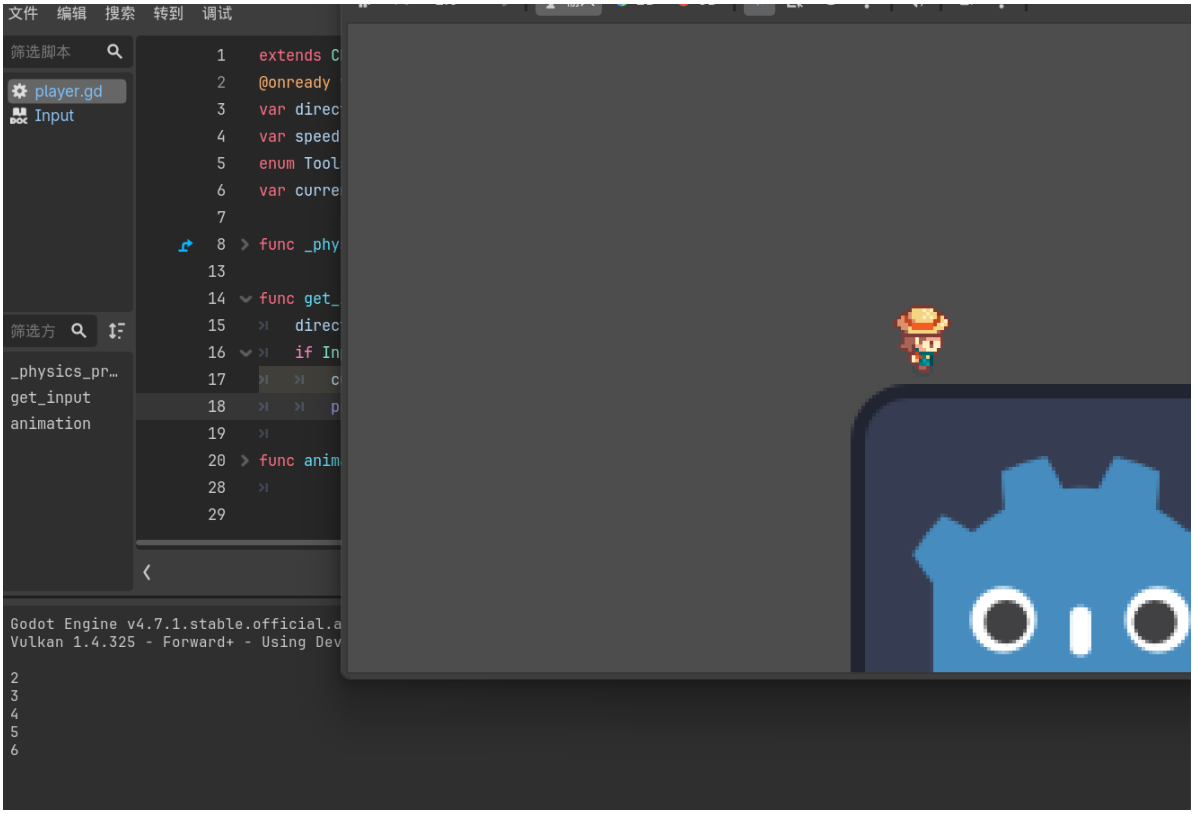
在player中定义枚举类型Tools用于存放三个工具，并定义current_tool记录当前工具，默认值为AXE。

```
enum Tools {HOE,AXE,WATER}
var current_tool: Tools = Tools.AXE
```

在get_input方法中检测，若按下tool_forward按键，current_tool加一并打印，如下。

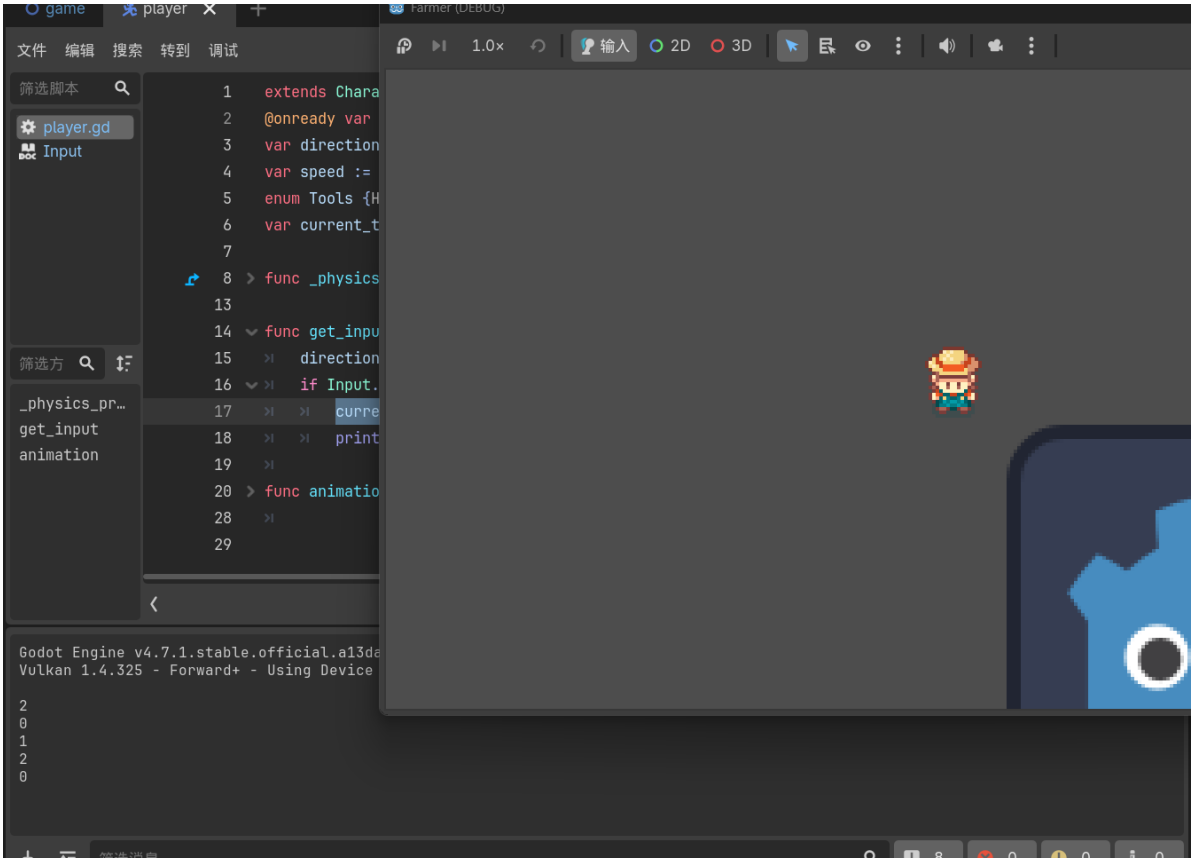
```
func get_input():
    direction = Input.get_vector("left" , "right" , "up" , "down")
    if Input.is_action_just_pressed("tool_forward"):
        current_tool+=1
```

运行后按下E键，输出如图。



但这种无限递增的效果不符合我们的要求，因此我们需要用到posmod方法对该值取余并保证其是正整数，如下，这样current_tool将会在0、1、2间循环，如图。

```
current_tool = posmod(current_tool + 1,Tools.size()) as Tools
```



另外记得将current_tool的类型使用as关键字强制转换为Tools型，否则Godot会由于类型不匹配触发警告。

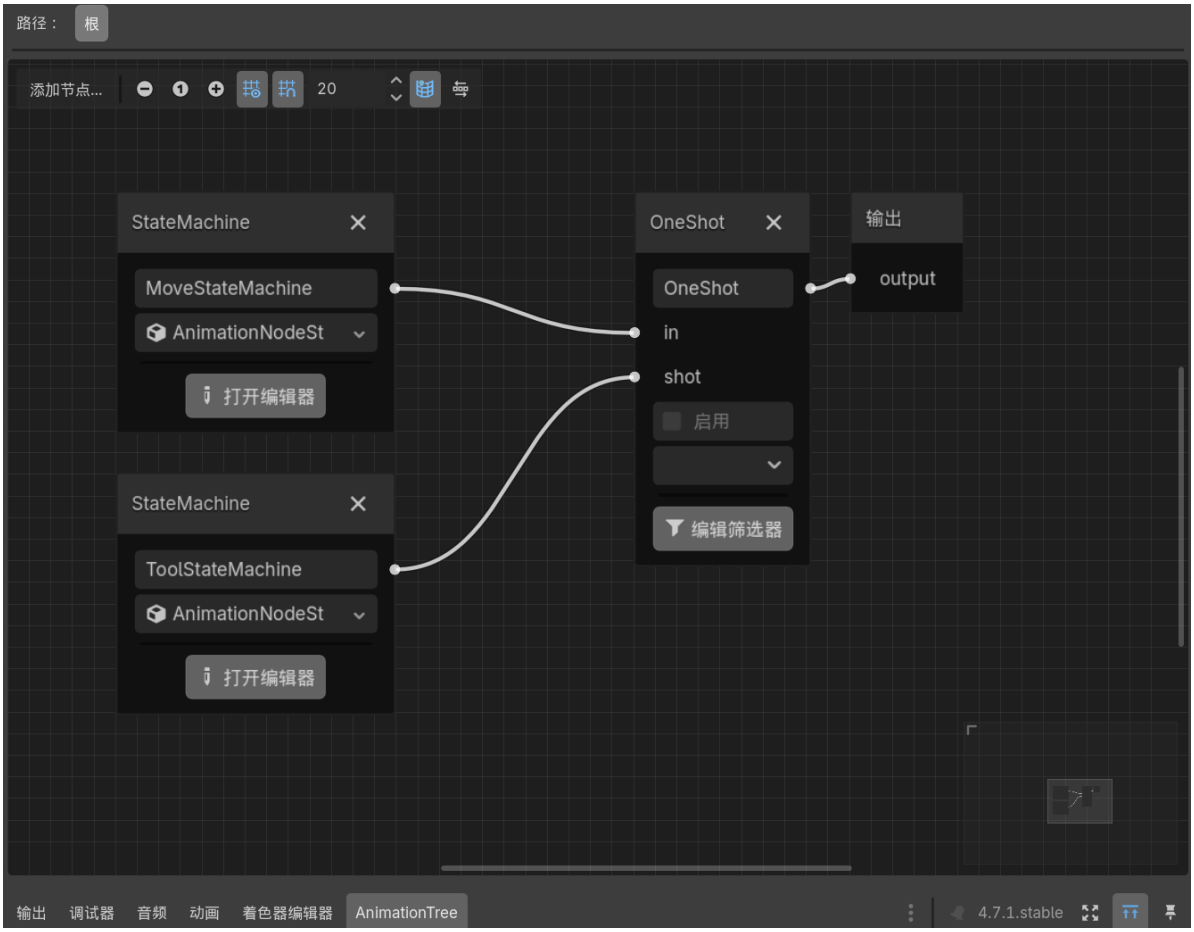
正向切换实现后，还需要添加反向切换操作，可以新建一个变量tool_direction，若按下tool_backward即赋值为-1，按下tool_forward即赋值为1，current_tool不再减1，而是减去变量tool_direction，如下。

```
if Input.is_action_just_pressed("tool_forward") or
Input.is_action_just_pressed("tool_backward"):
        var tool_direction = Input.get_axis("tool_backward","tool_forward") as
int
        current_tool = posmod(current_tool + tool_direction,Tools.size()) as
Tools
```

至此已完成工具切换逻辑，还需要为动作绑定对应的动画。

5.工具动画

要实现按下action按键后播放对应工具动画，需要让AnimationTree可以自动在MoveStateMachine和ToolStateMachine间切换，我们需要在AnimationTree中添加一个OneShot节点，in端连接MoveStateMachine，shot端连接ToolStateMachine，意思是触发时播放shot端动画，其余时间播放in端动画，如图。



接下来在player脚本中实现动画切换播放逻辑。

先创建一个字典型常量tool_connection，将枚举值和对应动画名称对应，能够大量减少重复代码，如下。

```
const tool_connection = {
    Tools.HOE: 'hoe',
    Tools.AXE: 'axe',
    Tools.WATER: 'water'
}
```

get_input方法中检测按键“action”，按下即切换工具状态机动画为对应工具，并请求播放连接到shot端的动画，如下。

```
if Input.is_action_just_pressed("action"):
        tool_state_machine.travel(tool_connection[current_tool])

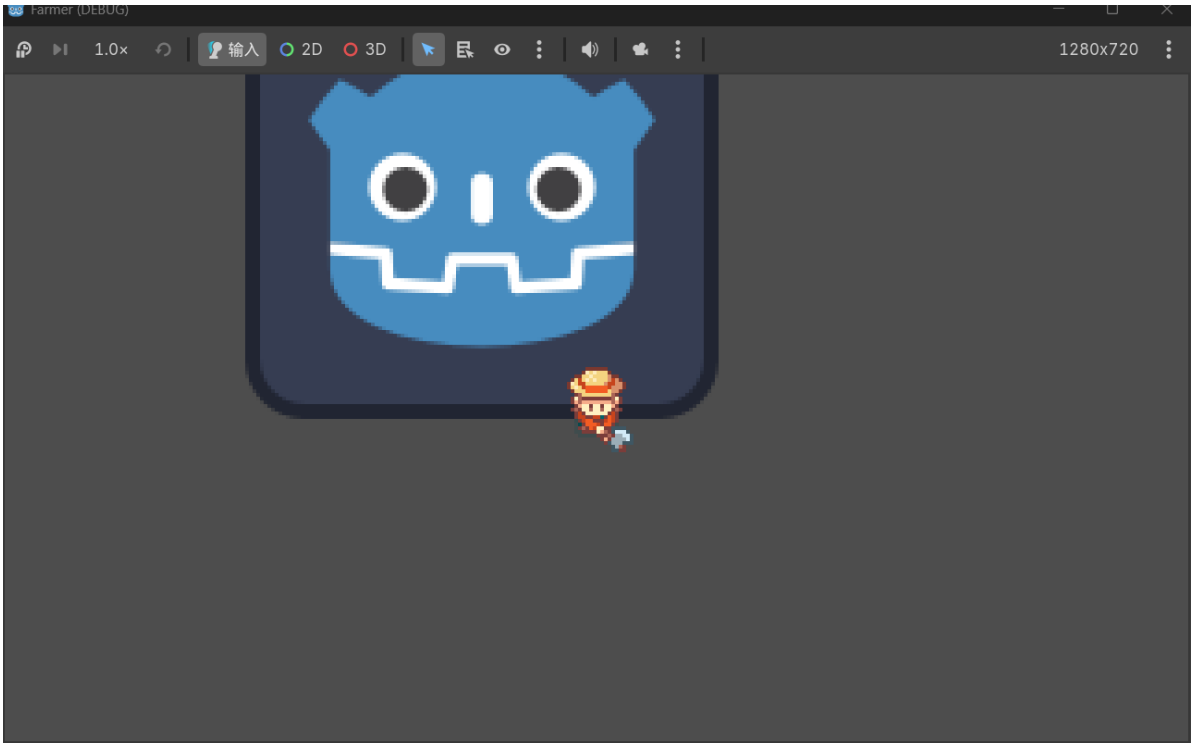
$AnimationTree.set("parameters/OneShot/request",AnimationNodeOneShot.ONE_SHOT_REQUEST_FIRE)
```

另外还需要在animation中修改各工具的BlendShape2D的混合位置，可以使用for循环遍历加字符串拼接来实现，如下。

```
for state in tool_connection.values():

$AnimationTree.set("parameters/ToolStateMachine/"+state+"/blend_position",target_vector)
```

运行测试，按下Q和E键可以切换工具，按下空格正常播放工具动画。



这里要注意，ToolStateMachine中各个工具都连向End节点，且Switch Mode是At End，Mode是Auto，表示该工具动画在触发播放后，不会循环，而是会自动结束，无需手动控制。这是一个很稳妥的容错设计，可以避免动画异常循环。

接下来我们实现工具使用时禁止移动的逻辑，避免播放工具动画时角色仍能移动，观感很怪异。

在player脚本中新建一个布尔类型变量can_move，初始值为true，在_physics_process添加条件判定：只有can_move为true时才读取输入，如下。

```
func _physics_process(_delta: float) -> void:
    if can_move:
        get_input()
    velocity = direction * speed * int(can_move)
    move_and_slide()
    animation()
```

在get_input中设置，一旦读取到“action”按键触发，立马将can_move设置为false，如下。

```
if Input.is_action_just_pressed("action"):
        tool_state_machine.travel(tool_connection[current_tool])

$AnimationTree.set("parameters/OneShot/request",AnimationNodeOneShot.ONE_SHOT_RE
QUEST_FIRE)
        can_move = false
```

在Player场景中选中AnimationTree，查看其信号。选中animation_finished信号将其绑定到player上，在_on_animation_tree_animation_finished方法中仅需要将can_move设置为true。

```
func _on_animation_tree_animation_finished(_anim_name: StringName) -> void:
    can_move = true
```

需要注意，animation_finished信号仅对不设置循环的动画生效，如工具类触发后仅播放一次的动画，而如idle和move这种循环动画无法触发animation_finished信号。

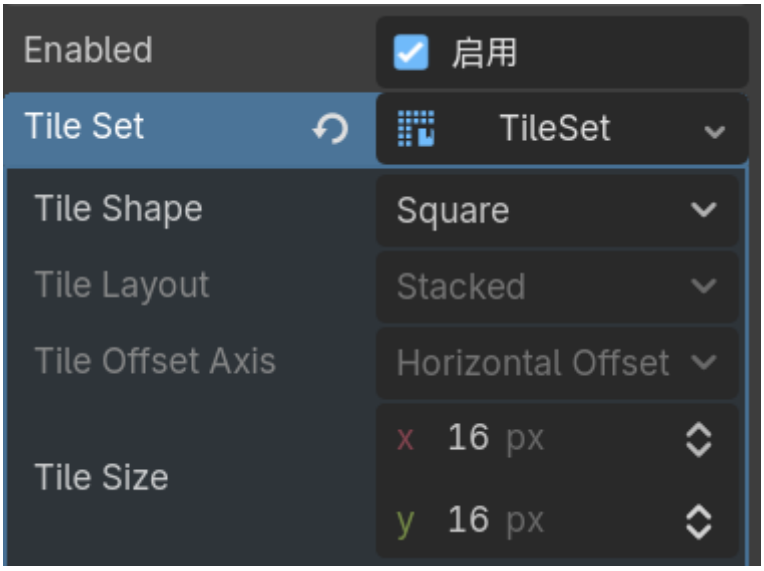
运行游戏测试可以发现，现在在工具动画播放过程中，无法移动且无法再次除非工具使用动画。效果完美符合预期。

6.瓦片自动拼接

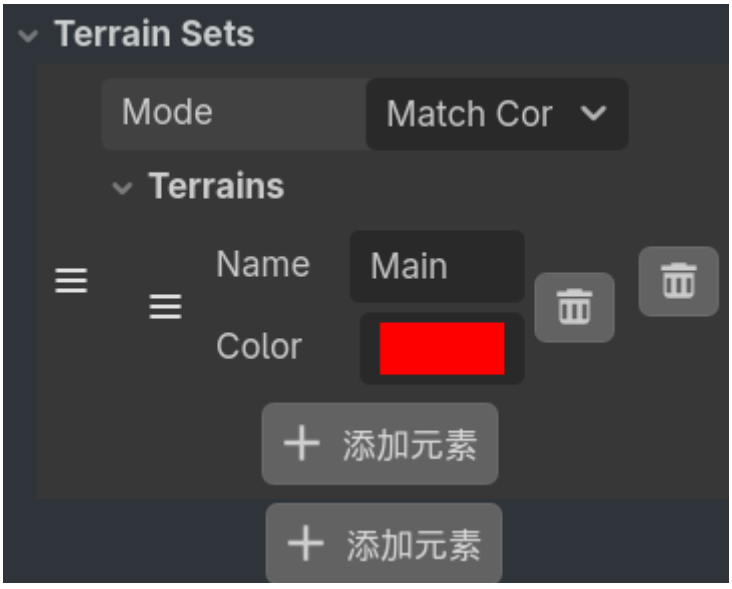
至此，角色相关基础内容已制作完毕，现在可以开始制作关卡本身。

这里涉及到一个功能——“autotiling（自动瓦片拼接）”，借助这个功能，可以使绘制关卡的工作量大大降低，同时效果极佳。制作大型关卡时，这个功能不可或缺。

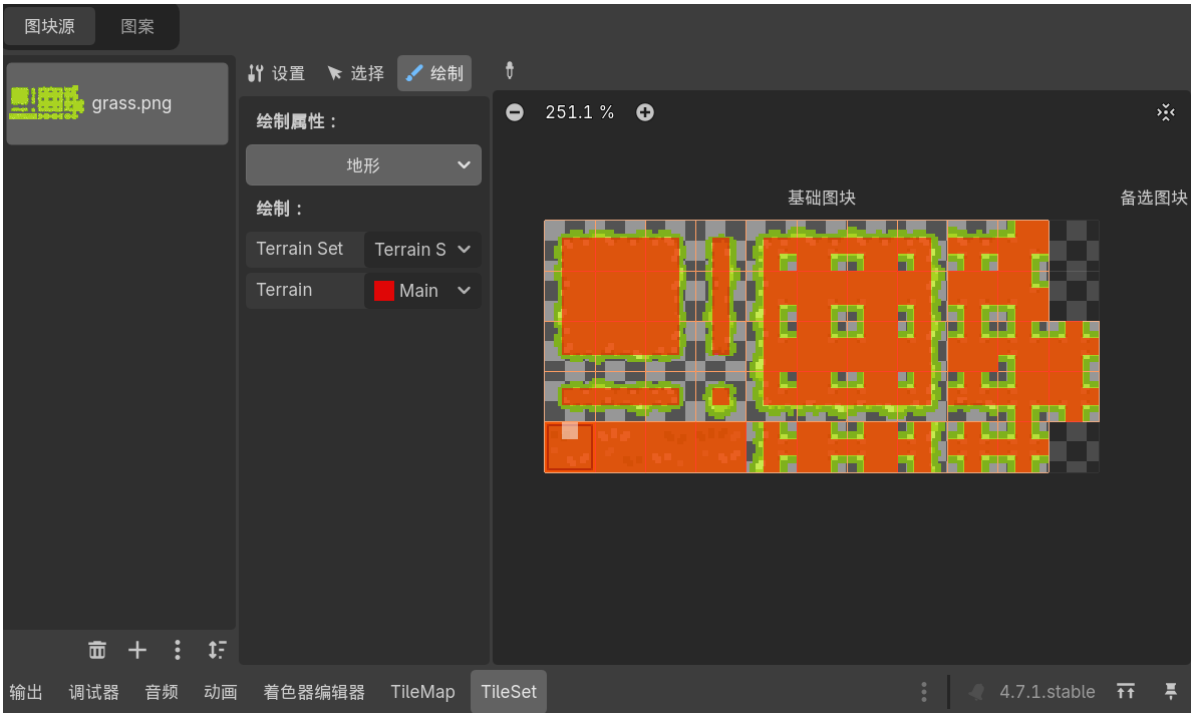
删除Game场景中的Sprite2D节点，新增一个Node2D节点Layers放置于Objects节点上，用于存放地图资源，确保地图比角色先渲染。Layers内新增一个TileMapLayer节点，在检查器面板新建TileSet，将res://graphics/tilesets/grass.png拖入TileMap面板的图块中，Tile Size默认为16*16，如图。



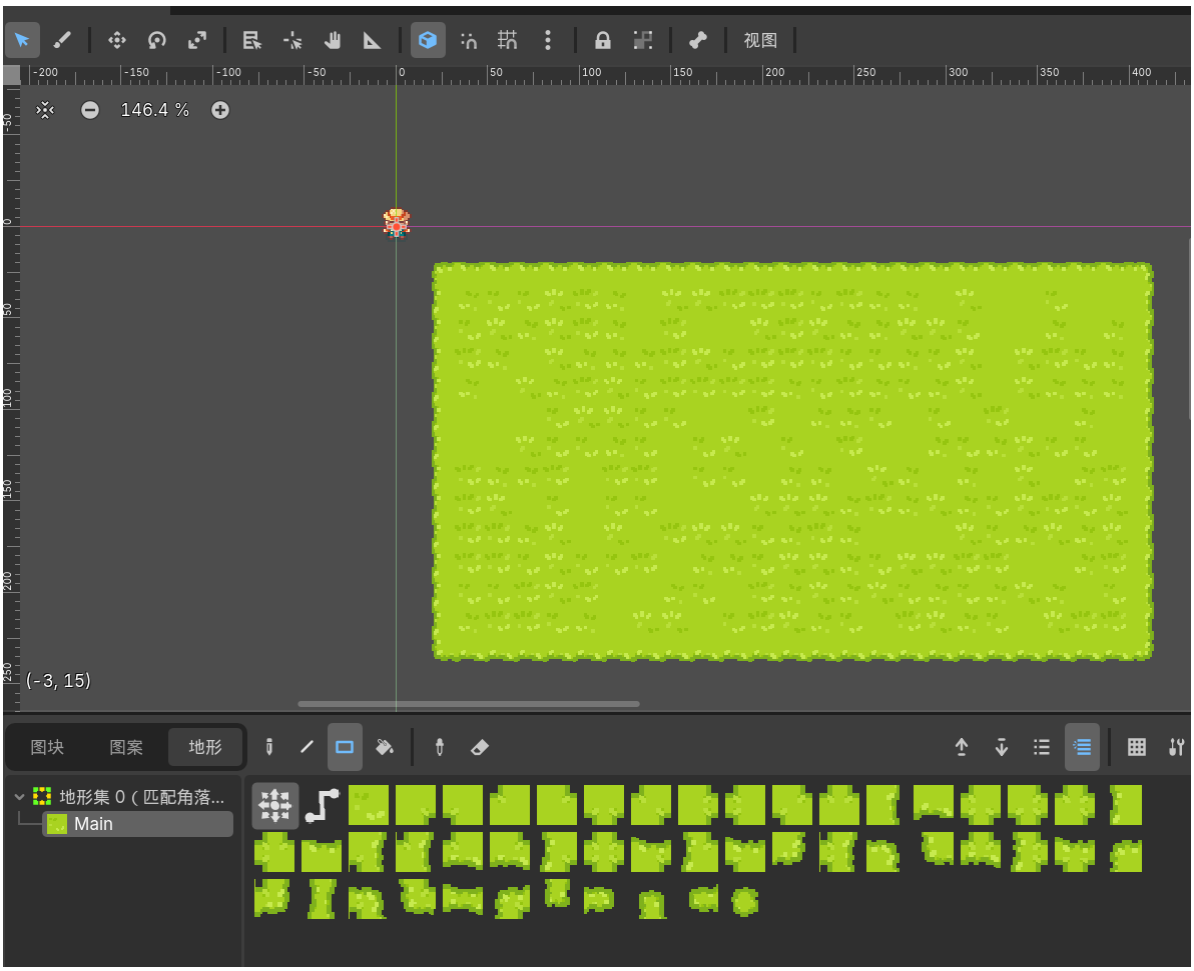
为了让Godot可以自动拼接瓦片，需要创建Terrain Sets，Mode为Match Corners and Sides，点击添加元素，Name设置为Main，颜色选择饱和度高的红色，便于区分。



在TileSet面板中绘制属性中选择地形，选择我们刚刚创建的Terrain Set 0，Terrain选择Main，随后对地形的拼接规则进行绘制，即将瓦片可以互相拼接的部分涂选，如图。



随后切换到TileMap面板的地形页，选择地形集0，选择Main，选择第一个Connnect Mode，在画面上进行绘制，Godot就会自动进行拼接，如图。



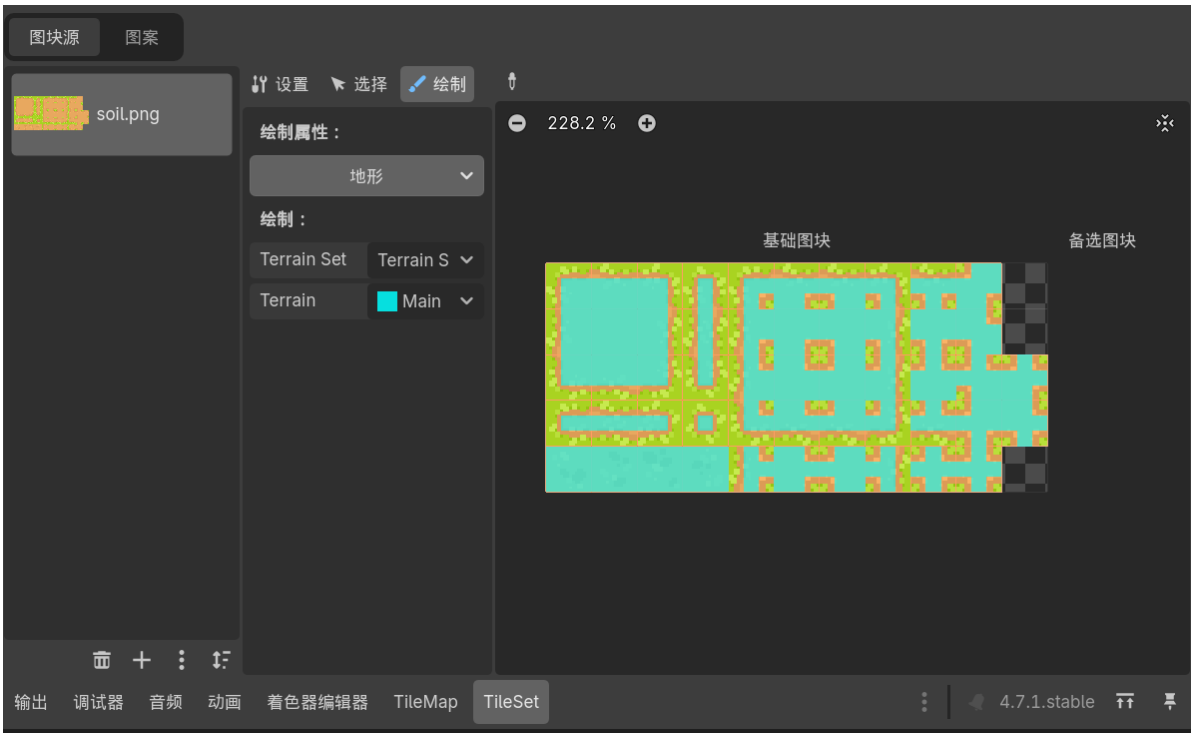
这样就无需手动切换各种瓦片进行拼接，大大加快了效率。

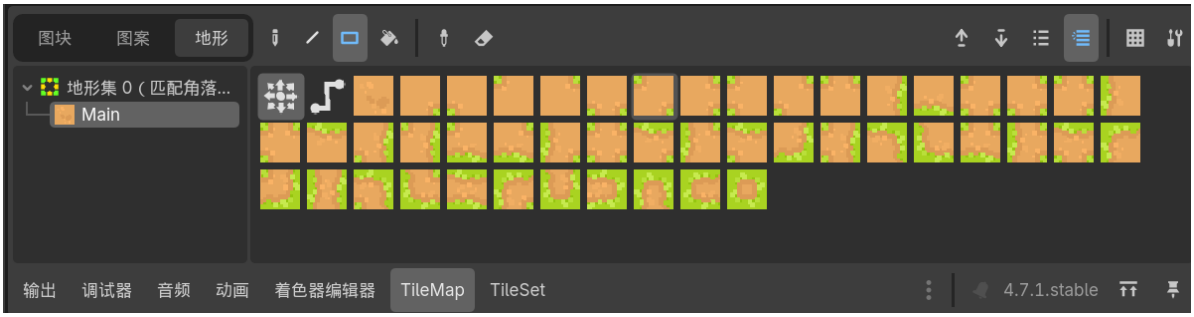
将Layers下的TileMapLayer命名为GrassLayer，方便区分。

接下来再新建一层TileMapLayer，命名为SoilLayer（泥土层），这一层中无提前绘制内容，而是游戏中玩家使用锄头后，依靠脚本自动生成对应的地形瓦片。

当然，我们仍需提前配置好泥土层对应的地形数据。

同样的，为泥土层配置地形集，如图。



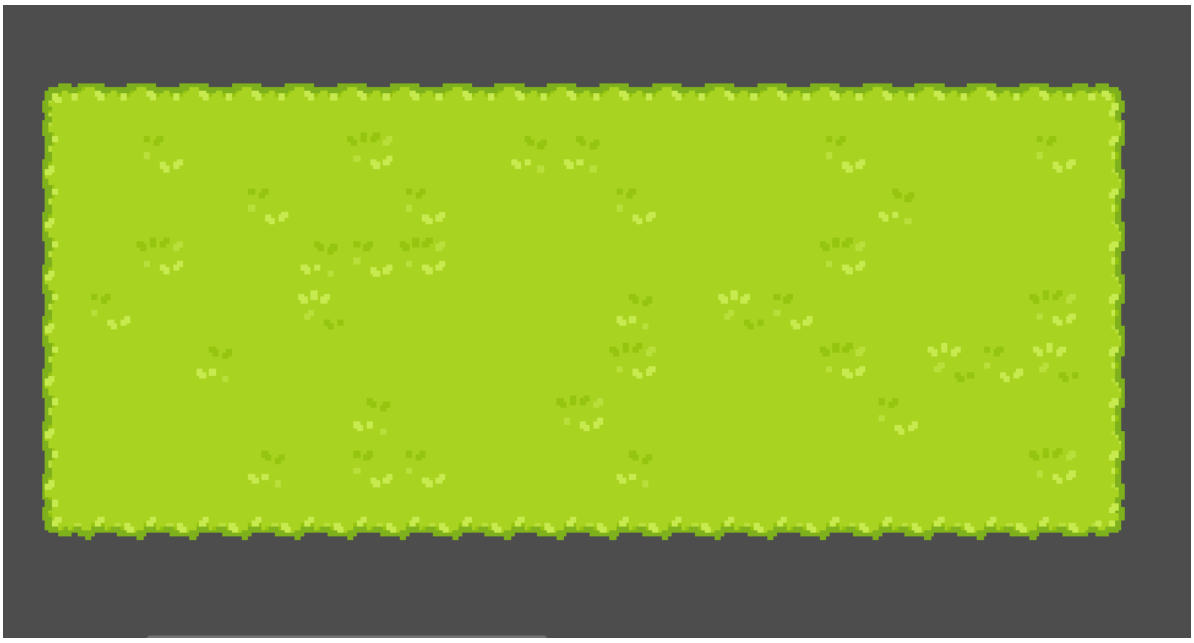


另外有一个小细节，草地层和泥土层的左下角都有着中心瓦片的变体，Godot可以从中随机选取进行生成，通常每个瓦片都是同概率，但我们也可以自定义其概率。

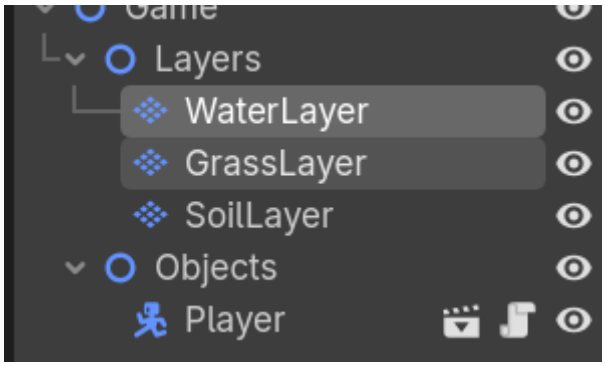
在TileSet中选择绘制属性为概率，可以看到每块瓦片都是1.0的默认概率，先设置想要的概率，再选中需要修改的瓦片进行概率修改。



将四个替换瓦片概率都设置为0.1后，再进行绘制，可以看到观感协调很多。

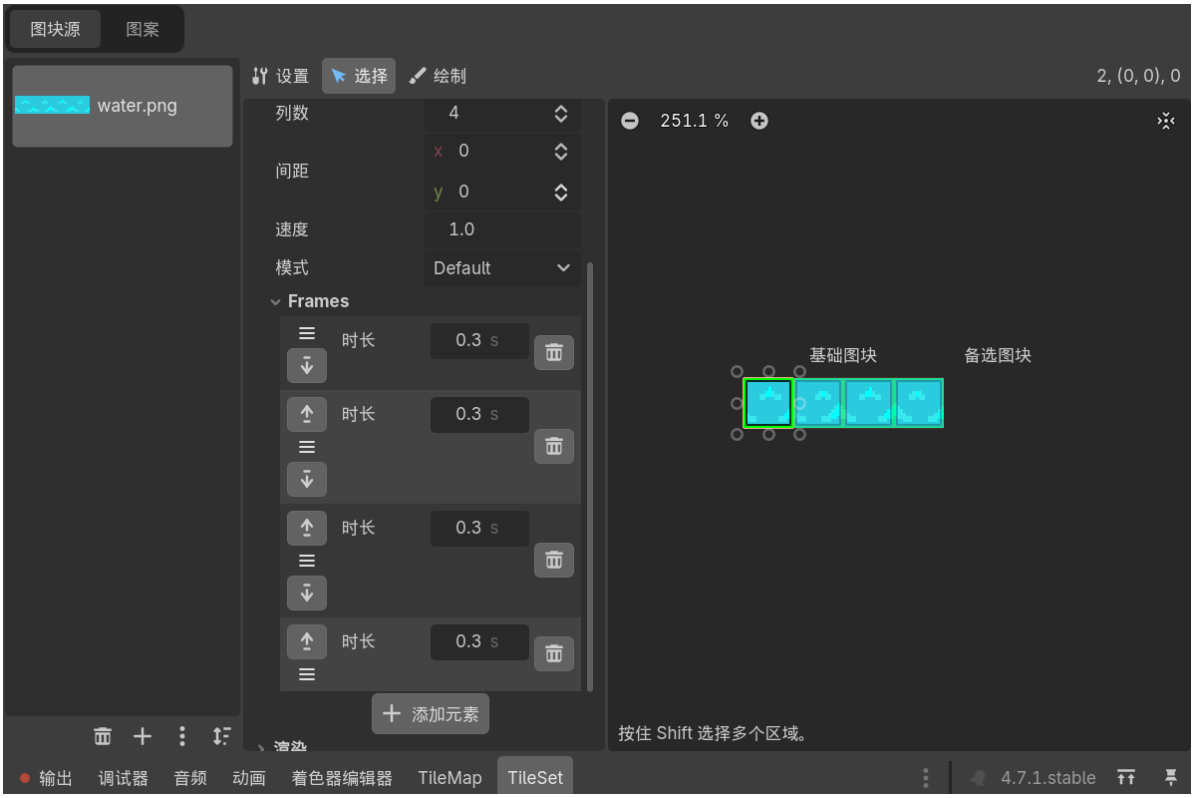


接下来新建一个TileMapLayer图层WaterLayer，用来放置water水面素材，水面层要放在其他层上面，保证画面中水面在画面最底层，如图。

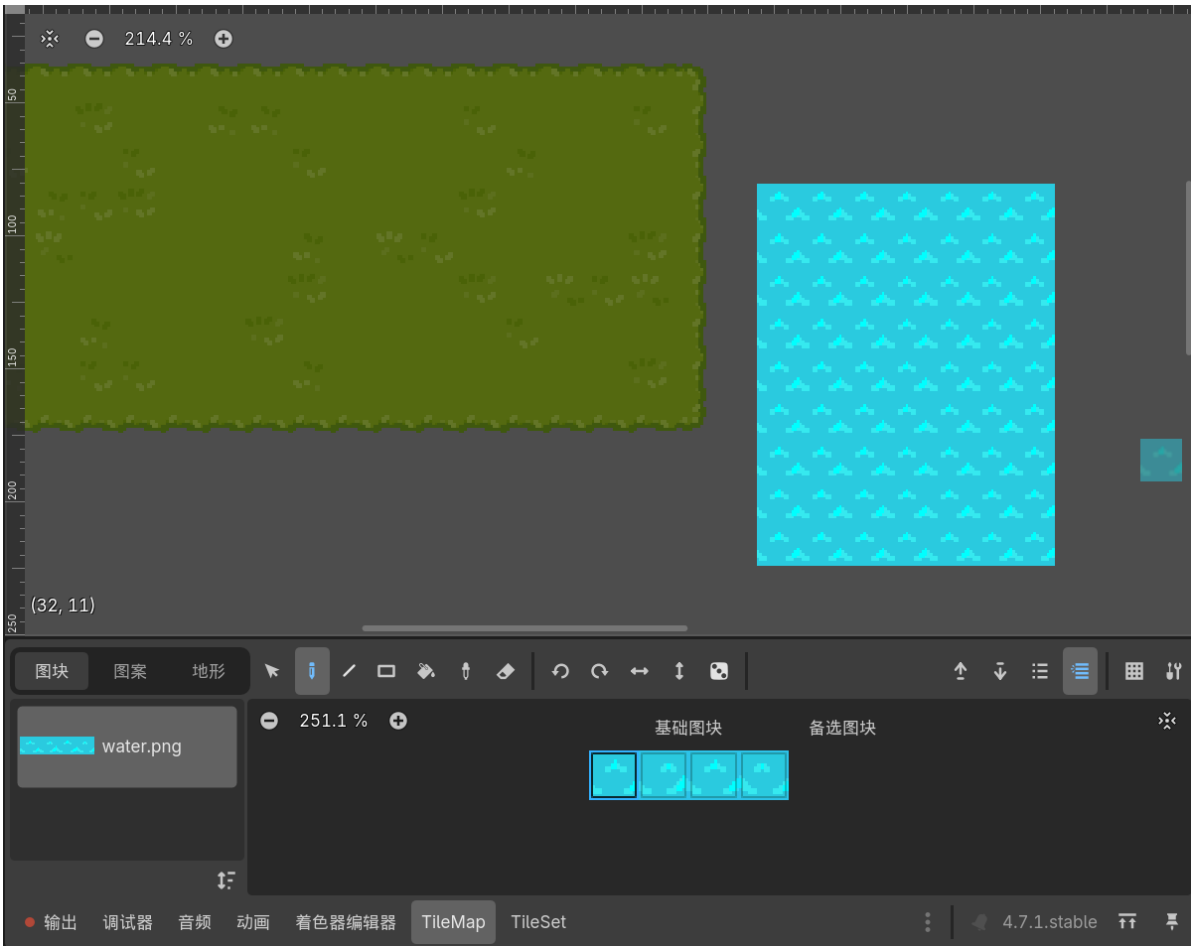


创建TileSet，保持默认设置，随后在TileSet面板中拖入res://graphics/tilesets/water.png，但不要选择创建图块。

选中第一个水面图块，可以看到其它三个图块保持灰色，随后切换到选择页，选中第一个图块，在动画选项中设置列数为4，在Frames中添加帧，将每一帧的时长设置为0.3s。



切换到TileMap页，选中图块在画面中进行绘制，可以看到绘制的水面具有流动效果。

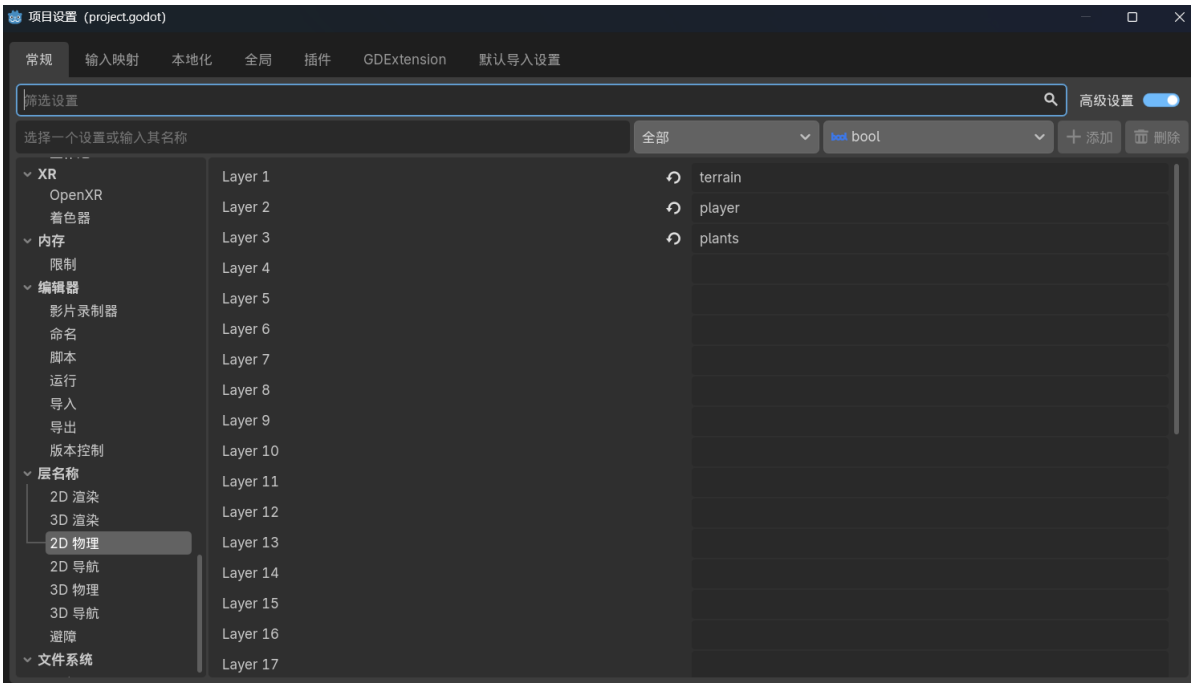


下

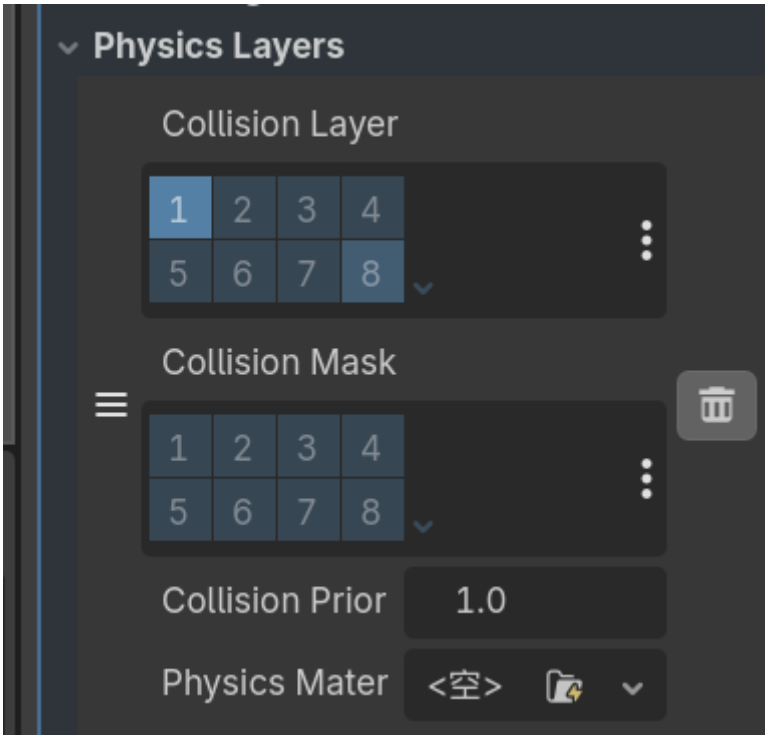
7.关卡制作

接下来先为地形设置碰撞箱。可以直接将碰撞添加到草地的边缘，水面不添加任何碰撞。

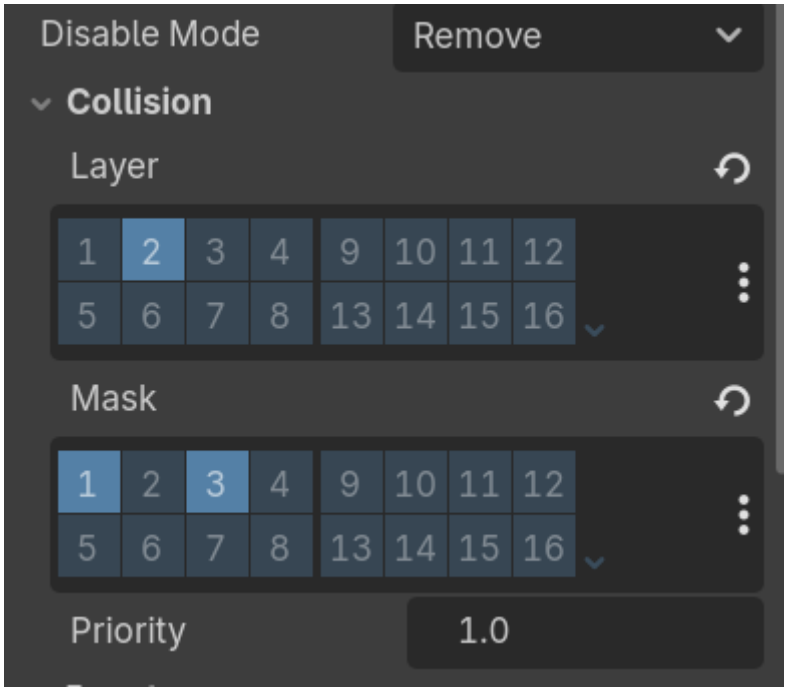
打开项目设置，设置三个碰撞层：terrain、player、plants。



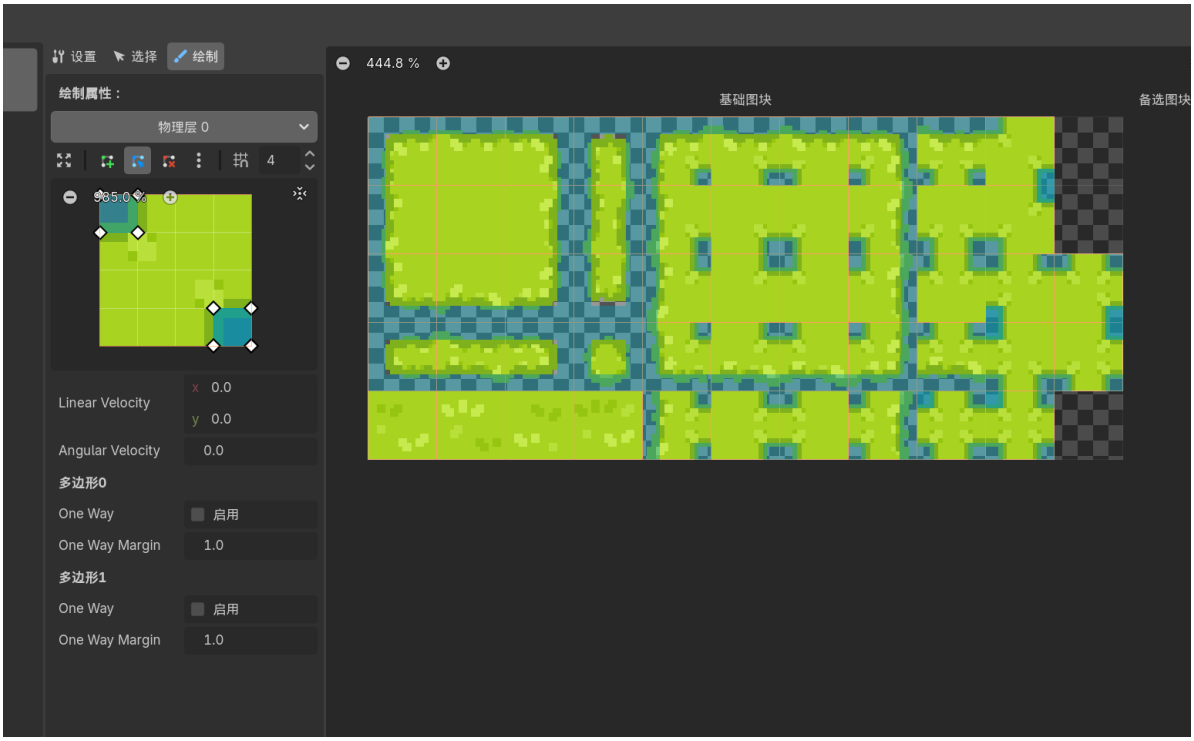
选中GrassLayer层，选中Tile Set后在弹出的窗口中找到Physics Layers，将其Collision Layer设置为1，Collision Mask无需勾选。



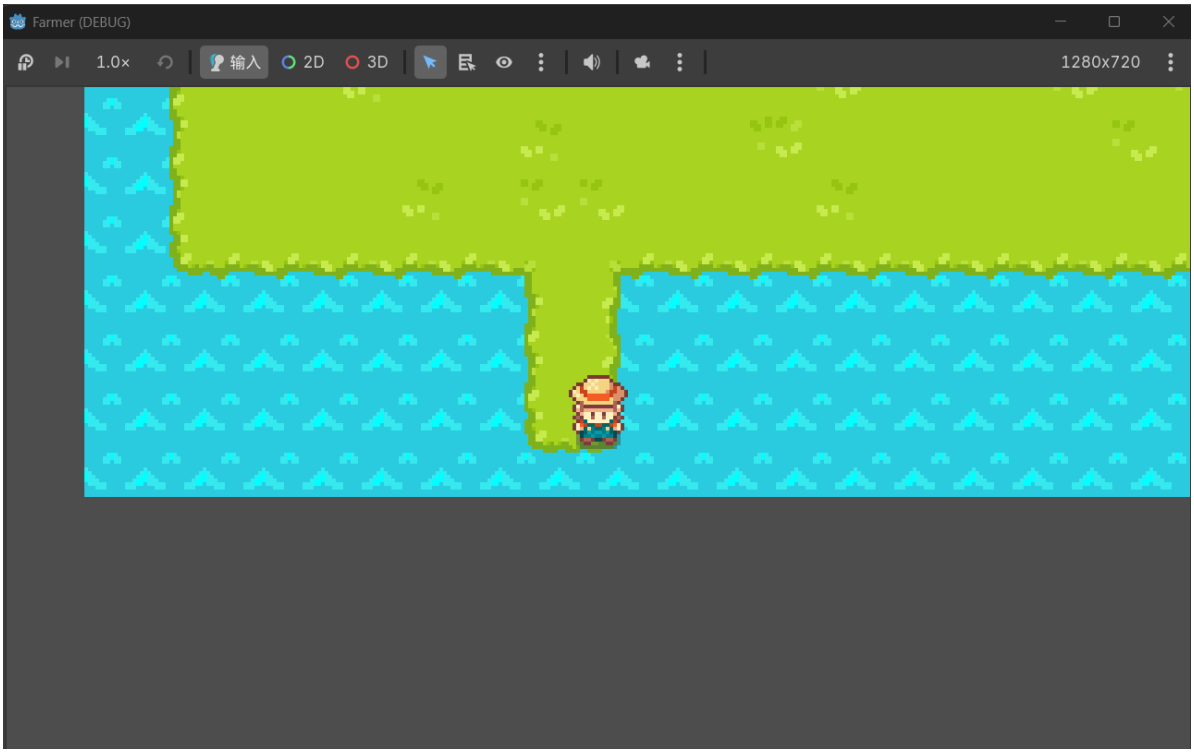
另外，玩家层的Collision Layer设置为2，需要检测地形和植被，因此Collision Mask勾选1和3。



设置好碰撞层后，选中GrassLayer，在TileSet面板中选择绘制物理层0，对每个草地的瓦片进行绘制，确保碰撞效果精准，如图。

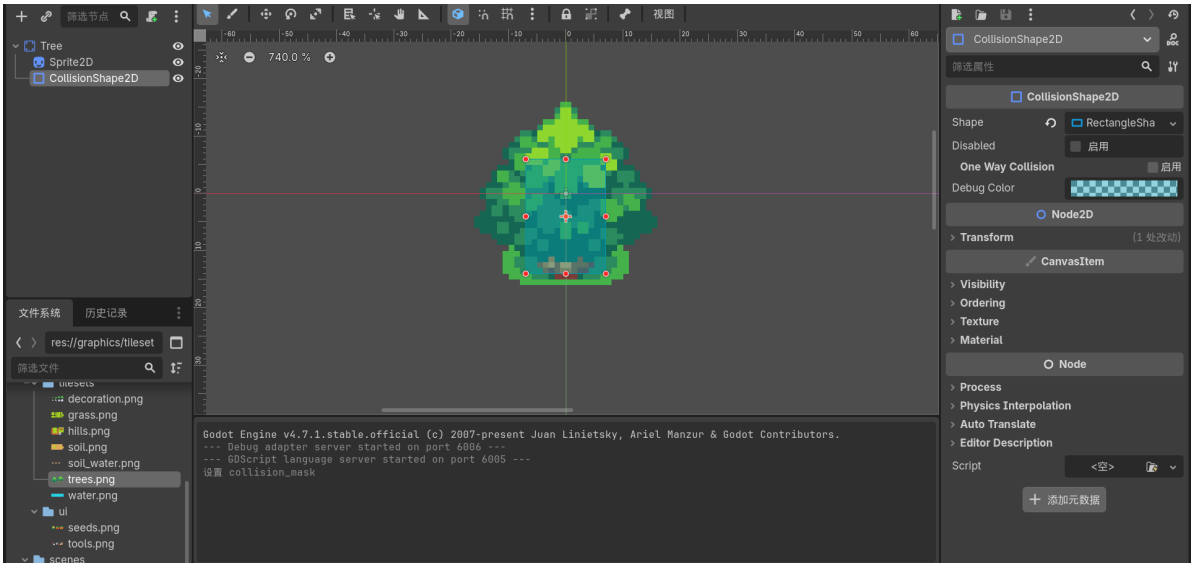


绘制完成后再进行测试，碰撞箱正常运行，角色无法走出草地，如图。



接下来完善关卡内容，还需要添加树木和山丘。

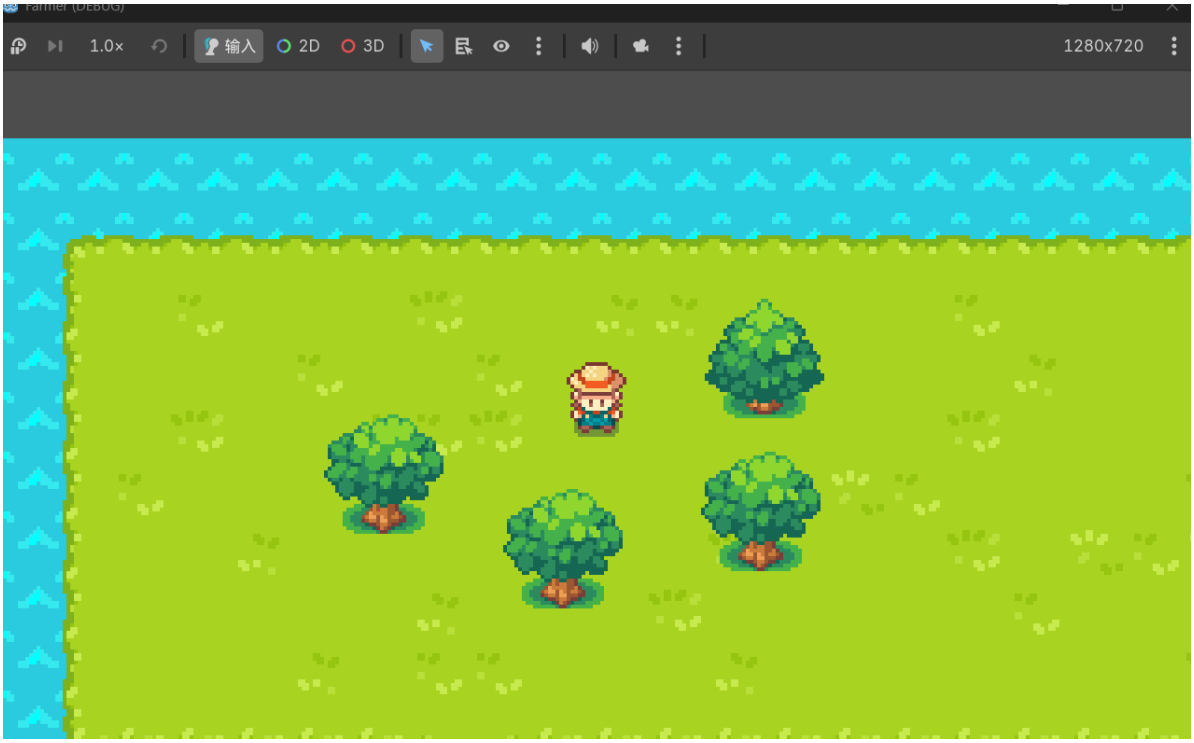
新建以StaticBody2D为根节点的场景，命名为Tree，为其新增Sprite2D，将res://graphics/tilesets/trees.png拖入其Texture中，在Sprite2D的Animation属性中将Hframe设置为2，这样后续使用脚本就可以实现随机生成两种树木样式其中一种的效果。同时为其添加CollisionShape2D节点，选用矩形碰撞形状，调整到合适大小，如图。



接下来编写脚本，为Tree场景挂载脚本，在_ready方法中编写其随机生成规则，如下。

```
func _ready() -> void:
    $Sprite2D.frame = [0,1].pick_random()
```

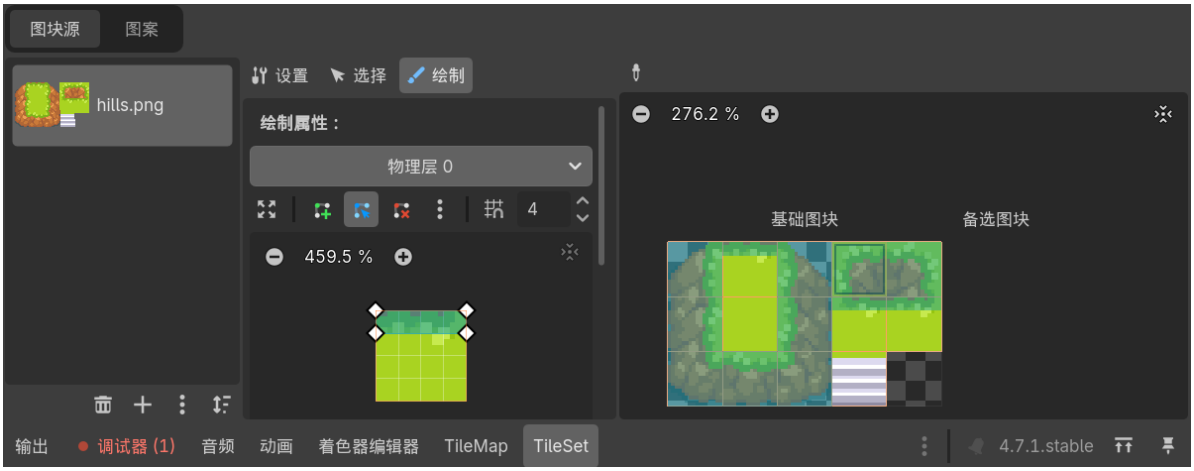
在Game场景中多复制几棵树，运行测试效果，可以看到树木样式随机为两种中的一种，如图。



接下来在Layers中创建山丘层，和之前步骤一致，新建一个TileMapLayer，命名为HillsLayer，新建TileSet保持默认参数。将res://graphics/tilesets/hills.png拖入TileSet面板的图块源，弹窗选择是，如图。



接下来为瓦片添加碰撞区域。先在检查器中的TileSet中选择碰撞层，归属于地形，不检测任何层。随后在TileSet面板的绘制页选择物理层0，进行绘制，如图。



接下来就可以在场景中进行山体绘制，可惜山丘瓦片不支持自动拼接，需要手动摆放拼接造型，简单摆放后运行测试如图。



接下来一定程度的扩充场景，方便接下来的功能实现和测试。

8.自定义编辑工具

接下来要实现斧头、锄头、洒水壶三种工具的功能，将它们与信号绑定，与场景做出交互。

先在player脚本中自定义一个信号tool_use，参数为tool：Tools和pos：Vector2，如下。

```
signal tool_use(tool: Tools,pos: Vector2)
```

在get_input方法中检测到”action“输入后发送该信号，如下。

```
    if Input.is_action_just_pressed("action"):
        tool_state_machine.travel(tool_connection[current_tool])

$AnimationTree.set("parameters/OneShot/request",AnimationNodeOneShot.ONE_SHOT_RE
QUEST_FIRE)
        can_move = false
        tool_use.emit(current_tool,position)#发送信号
```

在Game场景中新建脚本，将tool_use信号挂载到Game场景，先让信号打印两个参数，运行游戏测试，为了方便后续代码编写，可以提前引入Player对象。运行游戏测试可以看到参数正常输出，如下。

```
extends Node2D
@onready var player = $Objects/Player

func _on_player_tool_use(tool: int, pos: Vector2) -> void:
    print(tool)
    print(pos)
```



接下来分别处理各工具逻辑。

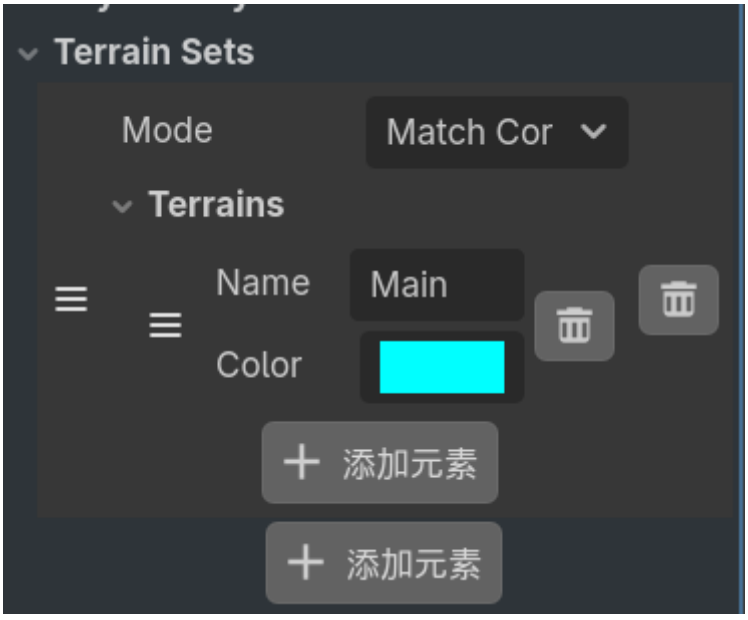
先处理锄头的逻辑，判断若玩家当前工具为锄头，则使用脚本生成土壤的贴图。但这里有一个问题，玩家的position使用的是像素坐标，但瓦片贴图使用的是网格坐标，因此我们需要将像素坐标先转换为网格坐标，为此我们新建一个grid_pos网格坐标变量，赋值为Vector2i(pos.x / 16，pos.y / 16)，因为设置的一个网格默认为16像素，经过这样换算后，grid_pos即是玩家位置对应的网格坐标。

另外为防止Godot弹出类型转换警告，可以将换算结果使用int强制转换包裹，如下。

```
extends Node2D
@onready var player = $Objects/Player

func _on_player_tool_use(tool: int, pos: Vector2) -> void:
    var grid_pos = Vector2i(int(pos.x / 16),int(pos.y / 16))
    if tool == player.Tools.HOE:
        pass
```

接下来编写核心功能，我们要通过代码修改SoilLayer土地图层，使用set_cells_terrain_connect方法来实现，第一个参数是将要修改的网格坐标存放在数组中再传入，第二个参数是地形集的索引，第三个参数是地形集内的地形的索引，因为SoilLayer中只定义了一个地形集且只有一个地形，因此第二个参数和第三个参数都为0，如图。



代码具体如下。

```
func _on_player_tool_use(tool: int, pos: Vector2) -> void:
    var grid_pos = Vector2i(int(pos.x / 16),int(pos.y / 16))
    if tool == player.Tools.HOE:
        $Layers/SoilLayer.set_cells_terrain_connect([grid_pos],0,0)
```

运行测试发现略有问题，土壤生成的位置应该符合锄头的落点，且应该等待锄头动画播放完毕后才生成土壤，如图。



在player脚本中直接修改信号发送的位置参数，但要注意，direction的值在没有输入时为Vector2.ZERO，因此我们需要新建一个Vector2变量last_direction存放上一次的direction的值，在_physics_process方法中角色移动时更新该值。

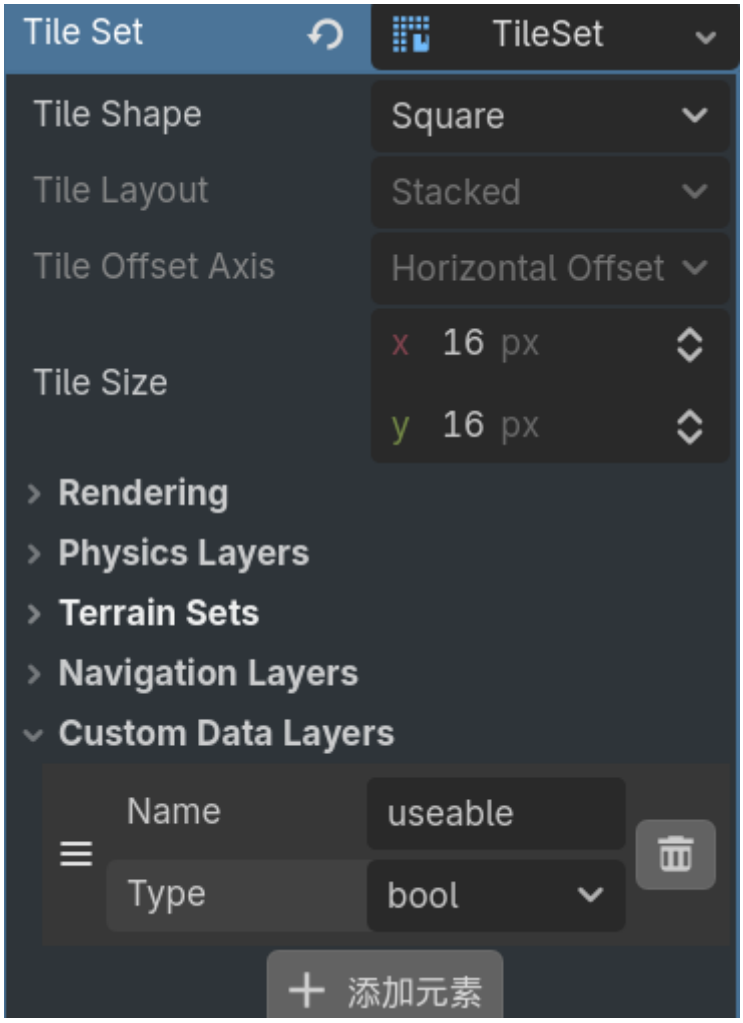
```
func _physics_process(_delta: float) -> void:
    if can_move:
        get_input()
    if direction:
        last_direction = direction#记录工具触发时上一次的direction值
    velocity = direction * speed * int(can_move)
    move_and_slide()
    animation()
```

再新建一个属性tool_direction_offset，用于计算工具的偏移值，可以使用@export注解，方便调试，实测最终值为14较为合适。另外可以注意到土壤生成的位置偏上，这是因为位置是以角色的中心点为原点，我们可以新建一个tool_y_offset变量作为Y轴偏移值，最终信号发送的位置是当前角色位置加上一次方向和工具偏移值的乘积，再加上Y轴偏移值。如下。

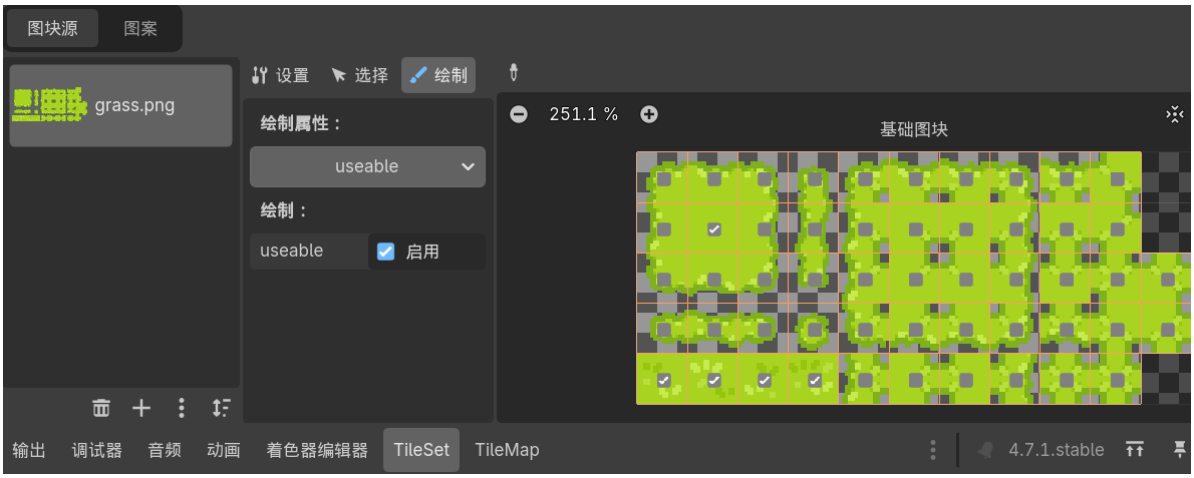
```
@export var tool_direction_offset: int = 14
@export var tool_y_offset :int = 4
#get_input方法中判断输入“action”后执行
if current_tool == Tools.HOE:
            await $AnimationTree.animation_finished
        tool_use.emit(current_tool,position + last_direction *
tool_direction_offset +Vector2(0,tool_y_offset))
```

最后，还有一个问题，当玩家在草地边缘挥动锄头时，会在水面生成一块土壤，非常违和。要修复这一点，需要为草地添加自定义标识。

先选中Game场景中的GrassLayer，在检查器的Tile Set中的Custom Data Layers中添加自定义标识useable，类型为布尔型，如图。



随后在TileSet面板中选择绘制useable属性，启用后将所有完整草地图块勾选，如图。



接下来在代码中读取该标识进行判断，game脚本中的具体逻辑如下。

```
extends Node2D
@onready var player = $Objects/Player

func _on_player_tool_use(tool: int, pos: Vector2) -> void:
    var grid_pos = Vector2i(int(pos.x / 16),int(pos.y / 16))
    if tool == player.Tools.HOE :
        var cell = $Layers/GrassLayer.get_cell_tile_data(grid_pos) as TileData
        if cell and cell.get_custom_data('useable'):#当瓦片为True且useable为True时才执行
            $Layers/SoilLayer.set_cells_terrain_connect([grid_pos],0,0)
```

接下来的洒水壶功能也类似。

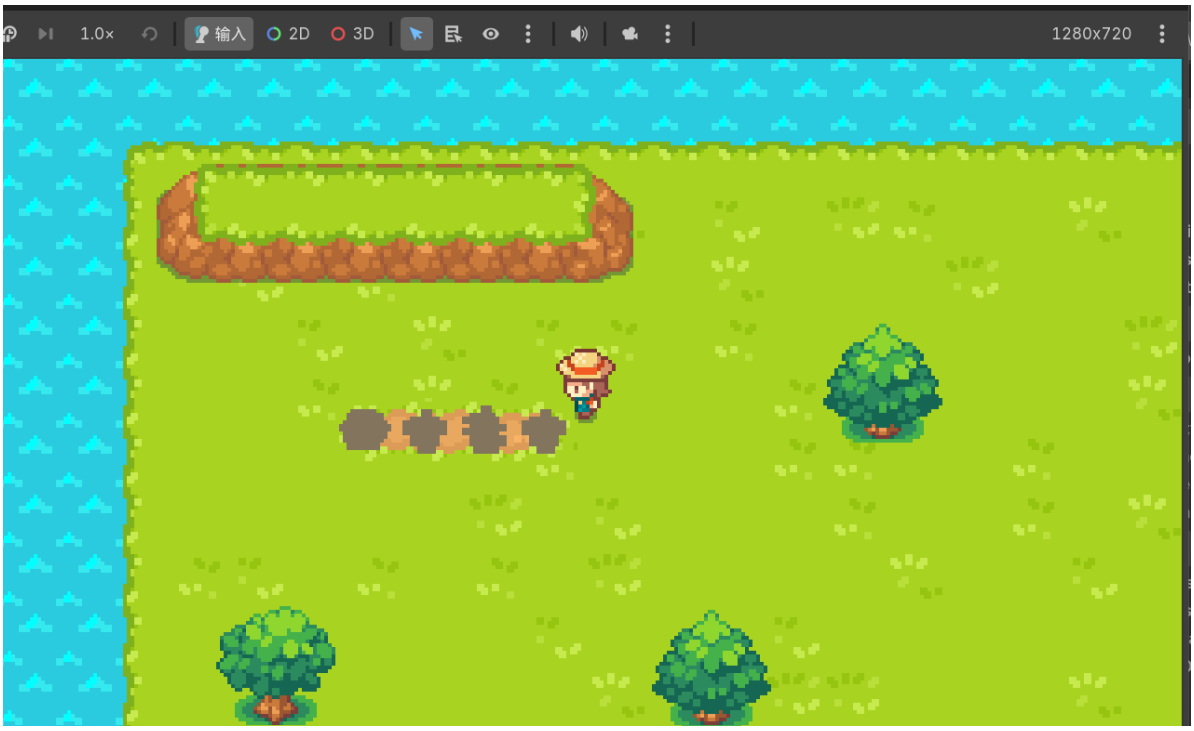
先修改player中的get_input方法中的判断条件，如下。

```
if current_tool in [Tools.HOE,Tools.WATER] :#锄头或水壶都符合条件
            await $AnimationTree.animation_finished
        tool_use.emit(current_tool,position + last_direction *
tool_direction_offset +Vector2(0,tool_y_offset))
```

随后在game脚本中添加逻辑，如下。

```
if tool == player.Tools.WATER:
        if $Layers/SoilLayer.get_cell_tile_data(grid_pos) :#若指定坐标的土壤瓦片不为
空
            #set_cell参数列表为：指定坐标、图集ID、图集中瓦片ID
            $Layers/SoilWaterLayer.set_cell(grid_pos, 0,
Vector2i(randi_range(0,2), 0))
```

运行测试，先使用锄头，再浇水，正常生成湿润土地贴图，如图。

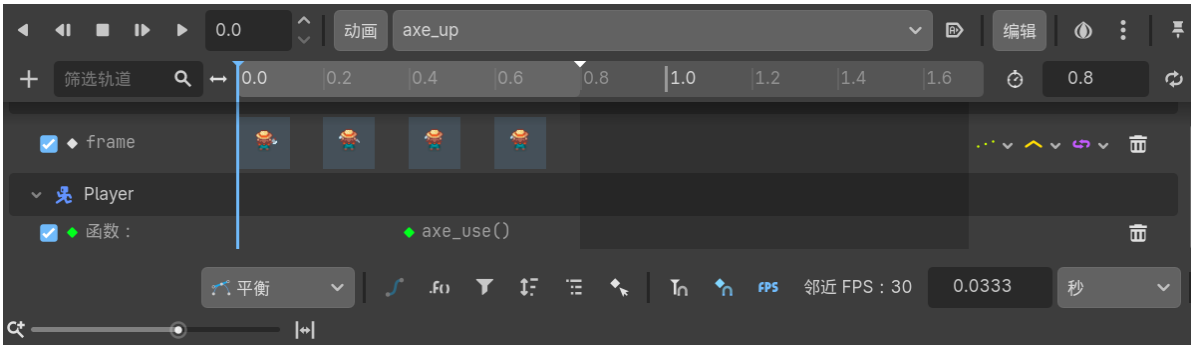


接下来是斧头，斧头的交互逻辑和前两种有所区别，锄头和洒水壶都是等待动画完成再交互，而斧头不能这么处理。打开一个斧头操作动画查看，可以看到动画总长约0.8s，我们最好在0.4s时触发逻辑，这样不会显得拖沓。

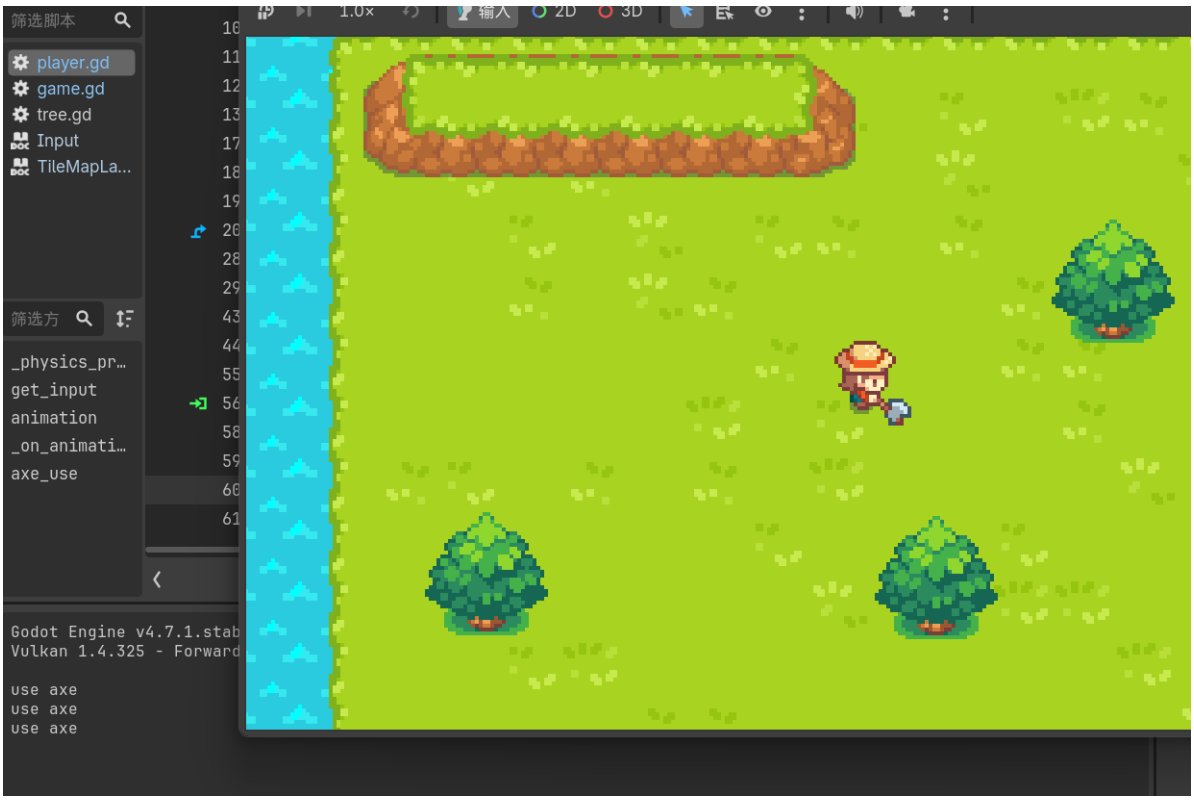
我们可以为动画添加调用方法轨道来实现，首先在player脚本中新建一个axe_use方法，先让其打印“use axe”进行测试。

```
func axe_use():
    print('use axe')
```

在axe的四个方向的动画中添加调用方法轨道，在0.4s时添加axe_use方法的关键帧，如图。



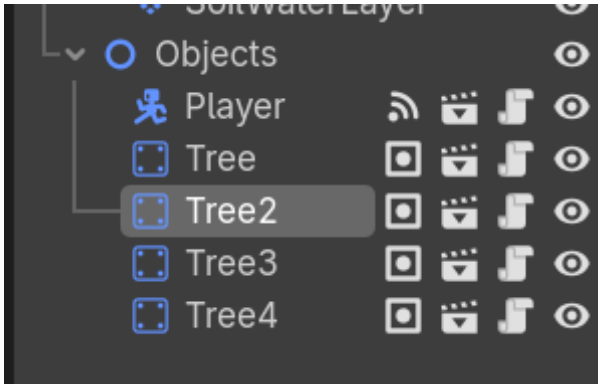
运行游戏测试，挥动斧头后控制台正常输出，如图。



接下来制作斧头和树木的交互，需要判断玩家和树木的距离。首先，在player脚本中修改axe_use方法，同样是发送信号，且需要计算偏移值，可以直接复制先前的代码，如下。

```
func axe_use():
    tool_use.emit(current_tool,position + last_direction * tool_direction_offset
+vector2(0,tool_y_offset))
```

为了方便判断Tree与玩家位置，在Tree场景中新建一个场景分组Trees，之后主场景中所有Tree对象都会归类到分组中，如图。



随后是game脚本中添加AXE工具的判断，对分组中的tree节点进行遍历，若距离小于10px，则调用受击方法，如下。

```
if tool == player.Tools.AXE:
        for tree in get_tree().get_nodes_in_group('Trees'):
            if tree.position.distance_to(pos) < 10:
                tree.hit()
```

接下来编写hit受击方法，我们希望制作树木受击后的闪光特效。

选择Tree场景，在检查器中的Material中新建ShaderMaterial，新建Shader，命名为falsh，为每棵树单独触发特效，勾选Resource中的Local to Scen，如图。



编写Shader脚本，如下。

```
shader_type canvas_item;

uniform float progress: hint_range(0.0, 1.0);

void fragment() {
    COLOR.rgb = mix(COLOR.rgb,vec3(1.0),progress);
}
```

着色器配置完成后，在tree脚本中的hit方法使用Tween操作着色器，如下。

```
func hit():
    var tween = create_tween()
    tween.tween_property($Sprite2D.material,'shader_parameter/progress',1.0,0.2)
    tween.tween_property($Sprite2D.material,'shader_parameter/progress',0.0,0.4)
```

运行游戏测试，树木受击后正常闪烁。

9.作物基础配置

接下来是作物的种植逻辑，有玉米、番茄等各类作物，这些作物都要支持播种、生长以及采收，接下来我们实现这些操作。

首先我们在player中定于枚举类型Seeds，存放三种种子，新建变量current_seed记录当前种子。

```
enum Seeds {CORN,TOMATO,PUMPKIN}
var current_seed: Seeds = Seeds.CORN
```

在项目设置的输入映射中可以看到seed_toggle（种子切换）绑定了C键，我们在get_input方法中监听seed_toggle，若按下C键即使当前的种子值加一，使用posmod对其取模，使其仅在0到2间循环。输出当前种子值方便测试，如下。

```
if Input.is_action_just_pressed("seed_toggle"):
        current_seed = posmod(current_seed + 1,Seeds.size()) as Seeds
        print(current_seed)
```

运行后按C键进行测试，控制台正常输出种子切换值。

除此之外，还需要监听plant（播种）的按键F，触发后执行播种作物的逻辑，需要新建一个自定义信号seed_use，参考tool_use。另外游戏没有配置播种的动画，可以设置按下F键后0.5秒内无法移动，并将direction置0，让行走动画停止，如下。

```
if Input.is_action_just_pressed("plant"):
        can_move = false
         direction = Vector2.ZERO
        seed_use.emit(current_seed,position + last_direction *
tool_direction_offset +Vector2(0,tool_y_offset))
        await get_tree().create_timer(0.5).timeout#创建一个0.5s的定时器，监听
        can_move = true
```

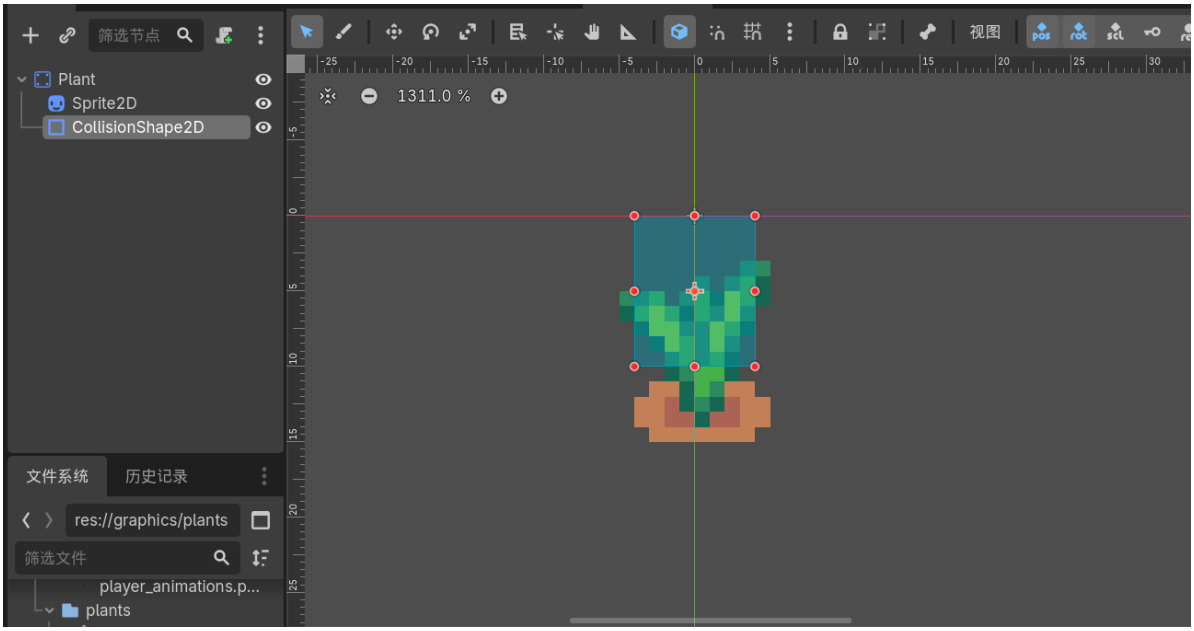
玩家端的逻辑至此已完成，接下来编写game脚本逻辑。

将player的seed_use信号绑定到game脚本，在_on_player_seed_use中编写逻辑，同样先获取网格坐标，随后判断坐标是否为土壤，条件符合后，这里先输出“seed”进行测试，运行测试，结果正常输出。完整代码如下。

```
func _on_player_seed_use(seed: int, pos: Vector2) -> void:
    var grid_pos = Vector2i(int(pos.x / 16),int(pos.y / 16))
    var cell = $Layers/SoilLayer.get_cell_tile_data(grid_pos) as TileData
    if cell:
        print('seed')
```

接下来再实现具体操作，在指定坐标生成作物实体。作物需要单独新建一个场景文件。

新建场景Plant，根节点为StaticBody2D，添加Sprite2D和CollisionShape2D，贴图先选择res://graphics/plants/corn.png，横向帧数设置为4，方便后续通过控制帧数来切换生长状态。碰撞形状为矩形，如图。



回到game脚本，发现Godot警告_on_player_seed_use的参数'seed'和内置关键字相同，我们将参数名改为seed_enum避免冲突。

接下来在游戏场景中实例化作物场景，生成作物实体。

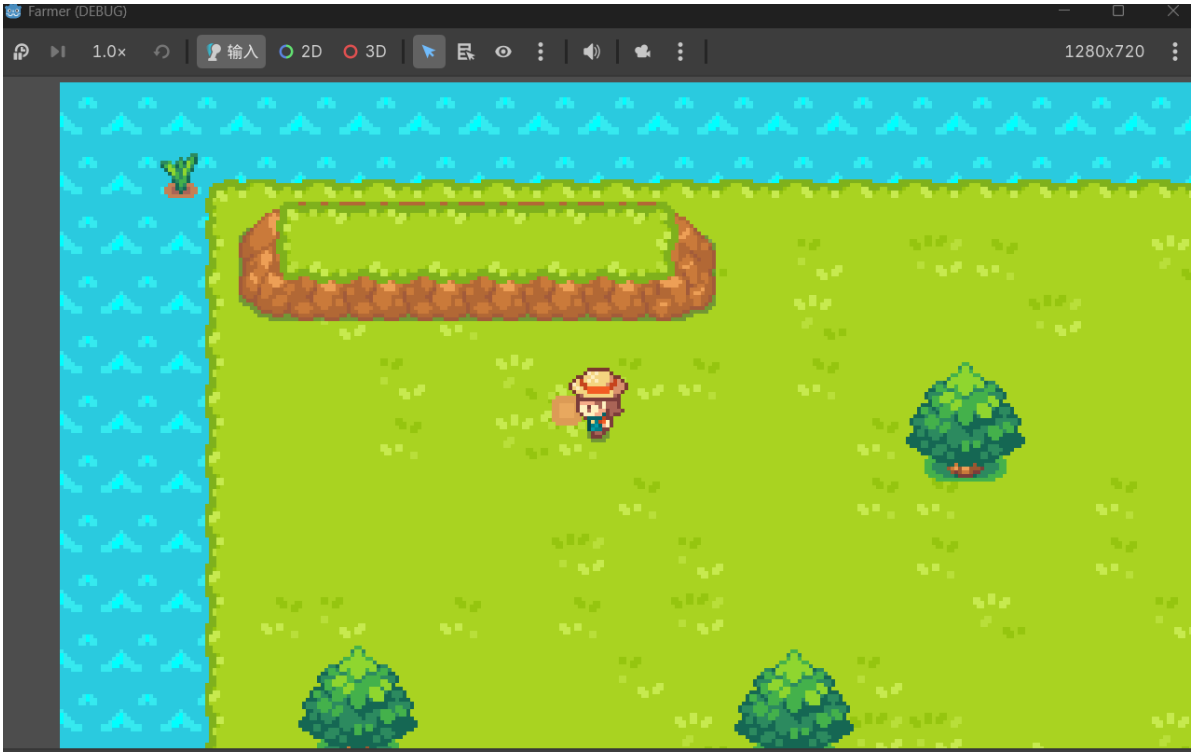
在game脚本中创建变量plant_scene，用于预加载作物场景，如下。

```
var plant_scene:PackedScene = preload("res://scenes/level/plant.tscn")
```

随后编写_on_player_seed_use方法，将作物实例化并添加到Game场景的Object节点中，如下。

```
func _on_player_seed_use(seed_enum: int, pos: Vector2) -> void:
    var grid_pos = Vector2i(int(pos.x / 16),int(pos.y / 16))
    var cell = $Layers/SoilLayer.get_cell_tile_data(grid_pos) as TileData
    if cell:
        var plant = plant_scene.instantiate()#实例化
        $Objects.add_child(plant)#添加到Object节点下
```

运行游戏测试，发现作物可以正常生成，但位置有问题，会生成在场景原点坐标处，如图。



因此我们还需要通过grid_pos换算像素坐标，而不是使用pos坐标，使用前者可使作物位置和网格中心点完美对齐。

新建变量plant_position，根据grid_position进行换算，直接乘16后还是有些许偏差，x轴和y轴需要再各自微调，如下。

```
var plant_position = Vector2(grid_pos.x*16+8,grid_pos.y*16-4)
```

将plant_position的值赋给plant.position，再次运行游戏测试，效果完美，如图。



接下来要配置作物的信息，方便处理作物的状态。为Plant场景挂载脚本，在脚本中新建setup方法，参数为seed_num: Seeds和grid_position: Vector2i。但这里有一个问题：Seeds是在player脚本中定义的，在plant脚本中无法访问到。为了解决这个问题，我们将Seeds改为全局变量。

在项目的globals文件夹下新建global.gd脚本，将player中定义的Seeds迁移到其中，如下。

```
extends Node

enum Seeds {CORN,TOMATO,PUMPKIN}
```

在项目设置的全局页中将global脚本添加为全局变量。随后将其他脚本中所有Seeds改为Global.Seeds，例如下。

```
var current_seed: Global.Seeds = Global.Seeds.CORN #所有Seeds都改为Global.Seeds
signal tool_use(tool: Tools, pos: Vector2)
signal seed_use(seed: Global.Seeds, pos: Vector2)
```

回到plant脚本，在setup方法中设置输出两个参数，运行游戏测试，正常输出两个参数，证明全局变量生效。

```
func setup(seed_enum: Global.Seeds,grid_position: Vector2i):
    print(seed_enum)
    print(grid_pos)
```



要在游戏中种下对应种子后显示对应贴图，并且生长速度等数据也是对应的作物数据，我们需要在plant中新建对应的变量：var grid_pos: Vector2（作物位置）、var max_age: int（生长最大时长）、var grow_speed: float（生长速度），为了后续方便取用这些数据，还需要新建字典常量plant_data，字典内嵌套字典，存储对应数据，如下。

```
const plant_data = {
    Global.Seeds.CORN: {'texture': preload('res://graphics/plants/corn.png'),
'max age': 3, 'grow speed': 0.6},
    Global.Seeds.TOMATO: {'texture':
preload('res://graphics/plants/tomatoes.png'), 'max age': 3, 'grow speed': 0.8},
    Global.Seeds.PUMPKIN: {'texture':
preload('res://graphics/plants/pumpkin.png'), 'max age': 3, 'grow speed': 0.5}
}
```

在setup方法中对这些数据进行取用，如下。

```
func setup(seed_enum: Global.Seeds,grid_position: Vector2i):
    max_age = plant_data[seed_enum]['max age']
    grow_speed = plant_data[seed_enum]['grow speed']
    grid_pos = grid_position
    $Sprite2D.texture = plant_data[seed_enum]['texture']
```

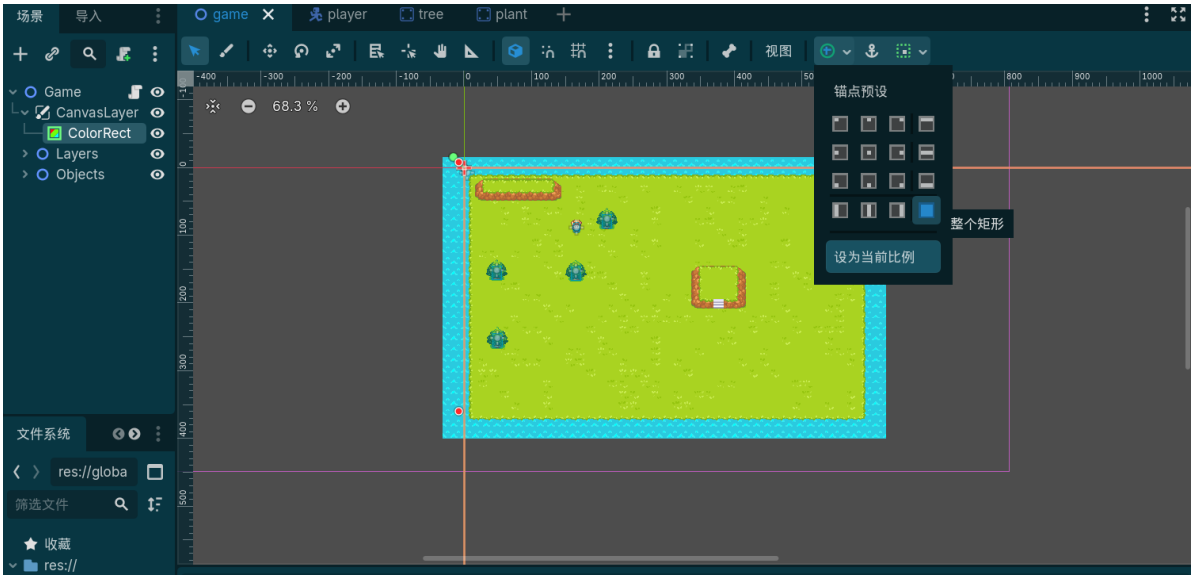
运行游戏测试，分别种下不同的种子，贴图正常显示为对应的作物，如图。



10.作物生长周期逻辑

接下来我们需要完成昼夜循环系统，并让作物根据昼夜循环和生长速度生长，当成熟后玩家靠近就可采摘。

在Game场景中添加CanvasLayer节点，其下添加ColorRect节点，将其锚点模式设置为整个矩形，如图。



我们希望的昼夜切换效果是屏幕渐变为黑色，随后再渐变恢复，这可以使用Tween操作ColorRect的Modulate来完成。

先将ColorRect的color设置为黑色，随后将CanvasItem下的Modulate中的a属性即透明度设置为0，该属性将影响其所有子节点的可见性，这样常态下的ColorRect将不会显示。



在game脚本中新建方法day_switch，具体如下。

```
func day_switch():
    var tween = create_tween()
    tween.tween_property($CanvasLayer/ColorRect, 'modulate:a', 1.0, 1.0)
    tween.tween_interval(1)#Tween的专属延迟方法，延迟1s再执行下一项
    tween.tween_property($CanvasLayer/ColorRect, 'modulate:a', 0.0, 1.0)
```

在_ready方法中对其进行调用，运行游戏测试，可以看到屏幕渐变为黑色，1s后渐变恢复，效果符合预期。

接下来要选择什么时候调用该函数，删除ready方法中的调用，改为process中进行。

process方法中判断，若按下了内置的按键映射“ui_focus_next”（即Tab），即执行day_switch方法，如下。

```
unc _process(_delta: float) -> void:
    if Input.is_action_just_pressed("ui_focus_next"):
        day_switch()
```

运行游戏测试，按下Tab后正常进行昼夜循环，当然，正式游戏中一般会选择更加合理的昼夜切换机制，比如角色上床睡觉、计时器触发等，这里是为了方便我们测试时手动控制，进行功能测试。

接下来，我们要在昼夜切换时完成对整个场景的更新，如浇过水的植物生长到下一个阶段，这部分操作我们放在day_switch方法的补间动画切换中间进行。

我们新建一个level_reset方法，用来实现更新操作，在day_switch中使用补间回调方法执行它，如下。

```
func day_switch():
    var tween = create_tween()
    tween.tween_property($CanvasLayer/ColorRect, 'modulate:a', 1.0, 1.0)
    tween.tween_callback(level_reset) #进行场景更新
    tween.tween_interval(1.0)
    tween.tween_property($CanvasLayer/ColorRect, 'modulate:a', 0.0, 1.0)
```

接下来实现level_reset方法，先选中Plant场景，创建一个新的场景分组Plants，在level_reset方法中循环获取分组中的对象并进行更新，如下。

```
func level_reset():
    for plant in get_tree().get_nodes_in_group('Plants'):
        plant.grow()
```

其中使用的grow方法还未创建，在plant脚本中新建grow方法，需要传入一个布尔类型参数watered，判断作物是否浇过水，这里先让其打印参数，方便测试。

```
func grow(watered: bool):
    print(watered)
```

回到game脚本中的level_reset方法中，为了判断指定plant的位置所在土壤是否浇过水，我们分别获取场景中所有的湿润土壤和植物位置，进行判断。由于plant对象自带grid_pos属性，这里直接使用即可，先输出判断结果，如下。

```
func level_reset():
    for plant in get_tree().get_nodes_in_group('Plants'):
        print(plant.grid_pos in $Layers/SoilWaterLayer.get_used_cells())
```

运行游戏后种植一株植物，按下Tab，输出false，给它浇水后再按下Tab，输出false。



将这个判断的布尔值传给grow方法，再次测试，输出符合预期，效果完美。

```
func level_reset():
    for plant in get_tree().get_nodes_in_group('Plants'):
        plant.grow(plant.grid_pos in $Layers/SoilWaterLayer.get_used_cells())
```

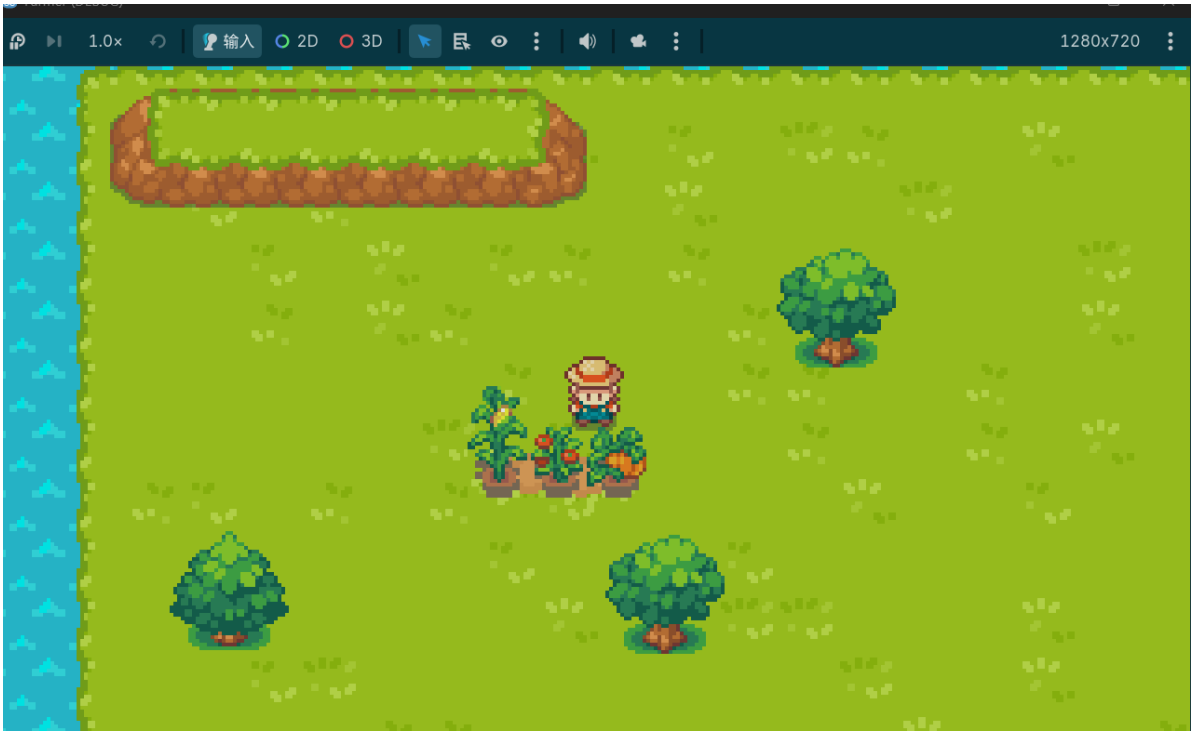
接下来编写grow中的作物生长逻辑代码，在plant脚本中新增age值，用于记录生长进度。若作物浇过水，age增加grow_speed值，并且作物的贴图将切换到对应的阶段帧，帧的索引可以由age强制转换为整型得到。

```
func grow(watered: bool):
    if watered:
        age += grow_speed
        $Sprite2D.frame = int(age)
```

但这里有一个问题，贴图中的生长阶段帧只有4帧，也就是从0到3，当age值大于等于4时将会触发报错，因此我们需要限制age值，如下。

```
func grow(watered: bool):
    if watered:
        age = min(age + grow_speed, max_age) #若age大于max_age，始终选择max_age
        $Sprite2D.frame = int(age)
```

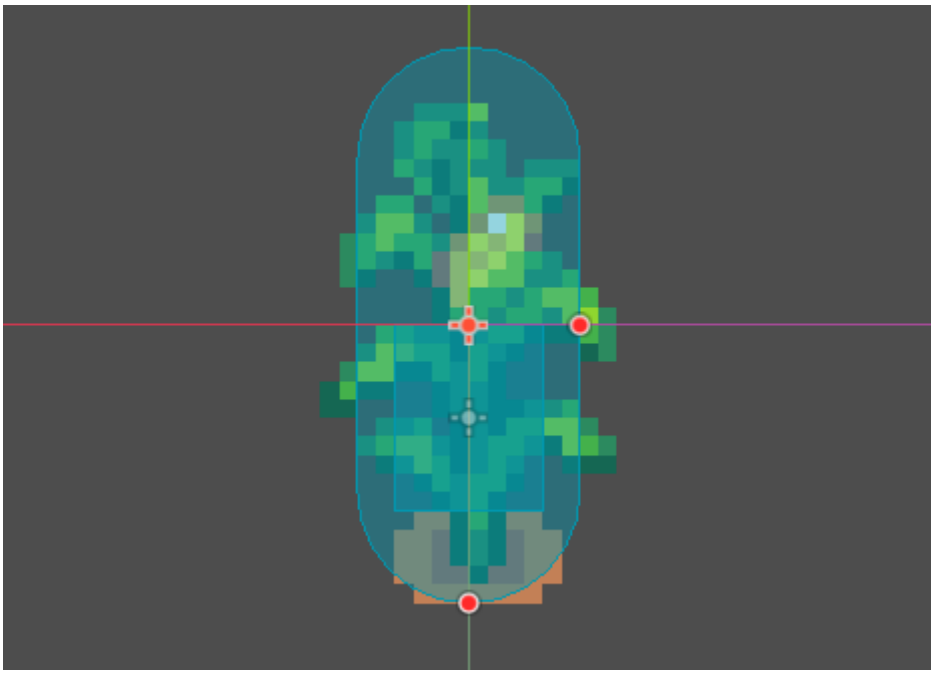
这样，age的值始终不会大于3，作物生长功能也能正常运行，如图。



另外，在level_reset方法中新增将湿润土壤全部清除的语句，符合作物每天都要浇水的规则，如下。

```
func level_reset():
    for plant in get_tree().get_nodes_in_group('Plants'):
        plant.grow(plant.grid_pos in $Layers/SoilWaterLayer.get_used_cells())
    $Layers/SoilWaterLayer.clear() #获取湿润土壤图层，调用clear方法将其清除
```

到这里，整个基础农场系统就搭建完毕了。接下来处理作物成熟后的简易交互逻辑。为Plant场景添加Area2D节点，其下添加CollisionShape2D节点，形状选择椭圆，调整到合适大小，如图。



将Area2D的Collision属性中层级改为植物层，仅检测玩家，随后选择body_entered信号进行绑定。

在plant脚本中，编辑_on_area_2d_body_entered方法，若age大于等于max_age，即直接销毁作物，如下。

```
func _on_area_2d_body_entered(_body: Node2D) -> void:
    if age >= max_age:
        queue_free()
```

运行游戏测试，当作物成熟后，角色靠近作物后作物将自动消失，效果完美。



至此，这个农场经营类游戏的简易雏形和全部基础功能就实现完成了。当然想要做成完整可玩的游戏，还需要完善资源存储、消耗系统、背包系统等功能

11.项目收尾

最后，为了完善项目，我们还需要添加游戏音效，并制作光照随时间变化的效果。

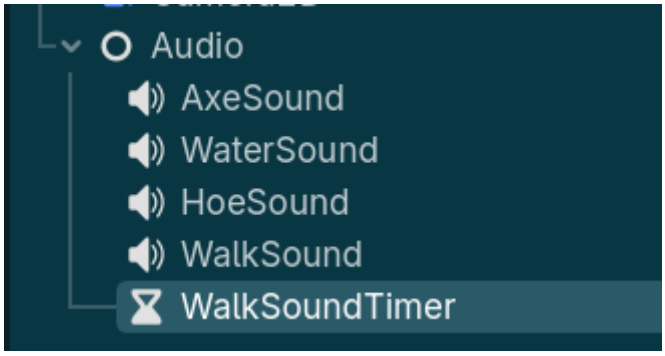
为Player场景添加一个Node节点Audio，用于存放工具音效，其下创建AudioStreamPlayer节点，命名为AxeSound，Stream属性选择res://audio/axe.wav，随后在player脚本的axe_use方法中添加对音频的调用，如下。

```
func axe_use():
    tool_use.emit(current_tool,position + last_direction * tool_direction_offset
+Vector2(0,tool_y_offset))
    $Audio/AxeSound.play()
```

这样，斧头工具的音效就完成了，其他两个也大同小异，Audio下再新建两个AudioStreamPlayer，分别命名为WaterSound和HoeSound，并绑定对应音效，在get_input方法中发送工具使用信号后添加判断逻辑，如下。

```
if current_tool in [Tools.HOE,Tools.WATER] :
            await $AnimationTree.animation_finished
            tool_use.emit(current_tool,position + last_direction *
tool_direction_offset +Vector2(0,tool_y_offset))
            if current_tool == Tools.HOE:
                $Audio/HoeSound.play()
            else:
                $Audio/WaterSound.play()
```

工具音效就处理完了，下一步是添加行走音效，同样添加一个AudioStreamPlayer节点，绑定res://audio/step.mp3，还需要新增一个计时器WalkSoundTimer，防止行走音效不间断播放，导致音效异常，如图。



设置WalkSoundTimer的等待时间为0.5s，将timeout信号绑定到player脚本。编辑_on_walk_sound_timer_timeout方法，如下。

```
func _on_walk_sound_timer_timeout() -> void:
    $Audio/WalkSound.play()
```

为了让音效跟随移动状态播放，我们在_physics_process方法中新增调用逻辑，如下。

```
if direction:
        last_direction = direction
        if not $Audio/WalkSoundTimer.time_left:#若计时器剩余时间为0
            $Audio/WalkSoundTimer.start()
    else:
        $Audio/WalkSoundTimer.stop()
```

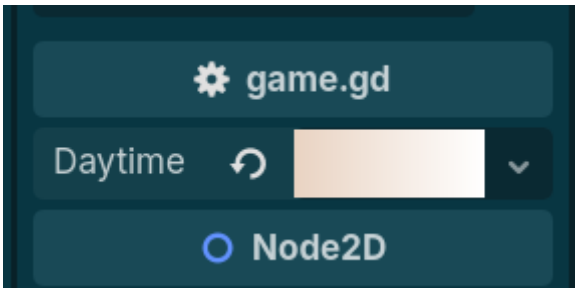
接下来还需要新增背景音效，在Game场景新建AudioStreamPlayer节点Music，stream属性绑定res://audio/music.mp3，开启Autoplay。

运行运行测试，可以听到所有音效都正常生效。

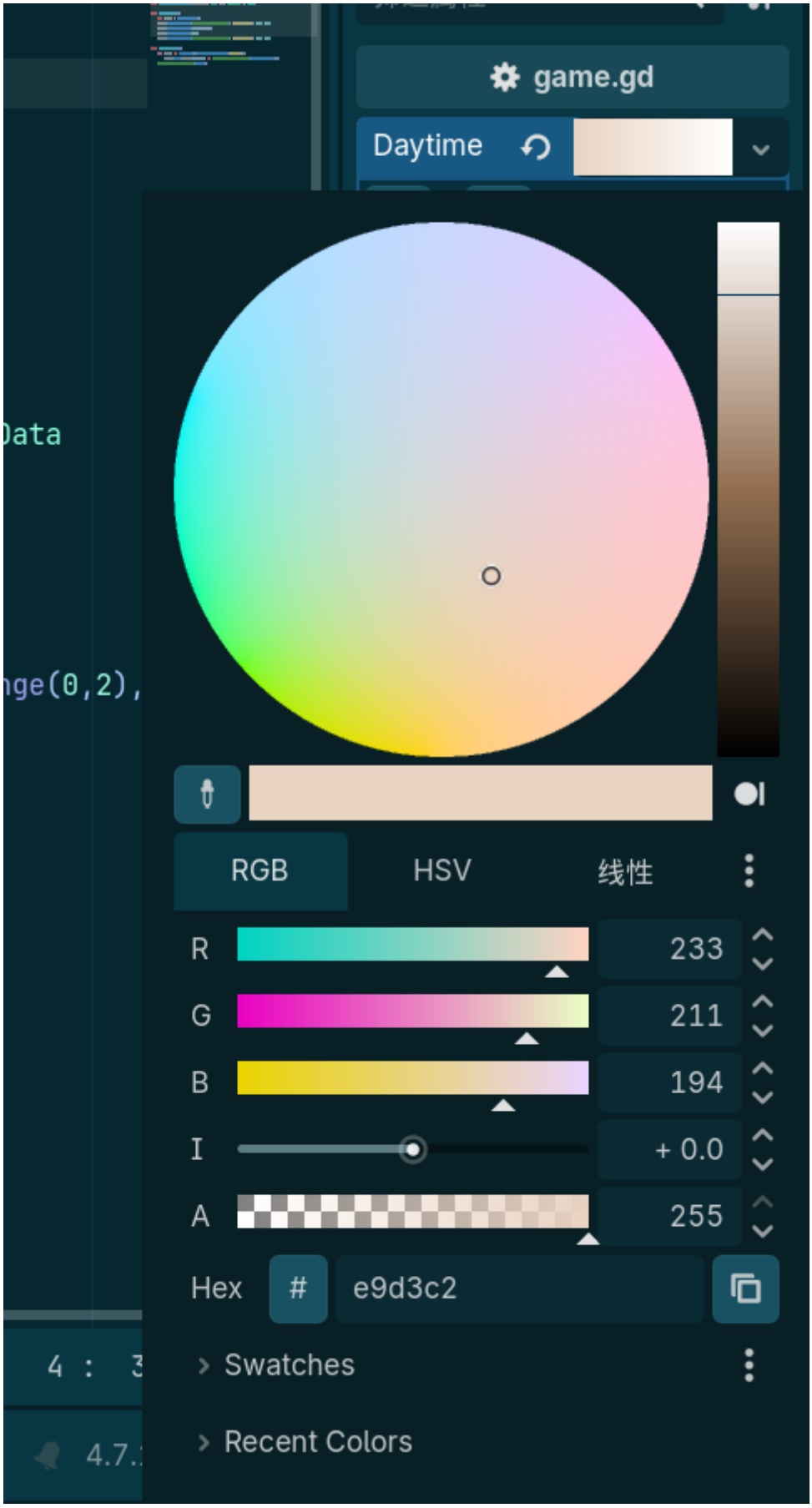
最后是实现画面色彩偏移效果，用来体现游戏内时间流逝，时间越往后推移，色调越暗并往蓝色偏移，Godot中内置了专门实现该效果的节点——CanvasModulate。

Game场景中新增CanvasModulate，同时新增一个计时器DayTimer，等待时间设置为300s，即游戏内一天时间，同时开启One Shot和Autostart。

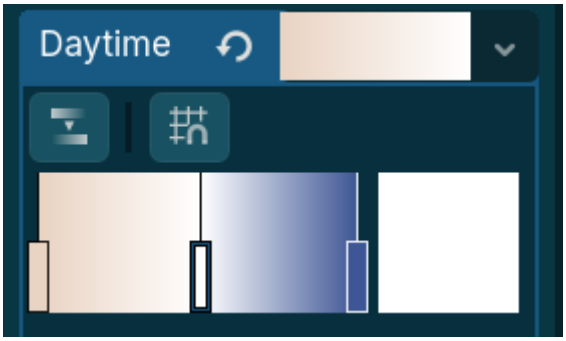
在game脚本新建导出变量daytime_gradient，类型为Gradient，随后选中Game场景打开检查器，可以看到daytime_gradient属性，选择新建Gradient，如图。



打开Gradient面板，可以看到渐变条上有两个色标，编辑色标颜色，起始色标颜色设置为e9d3c2，是一种暖黄色，用来体现清晨，如图。



末尾色标色值设置为3e5695，是一种深蓝色，用来体现夜晚，点击渐变条中间位置可以新增色标，中间点设置纯白色，对应游戏默认的画面观感。当游戏昼夜流逝时，我们将从这条渐变中实时采样颜色，并赋值给CanvasModulate节点的颜色属性，如图。



这段采样逻辑写在process方法内，先新建daytime_point变量，值为计时器剩余时间除以总时间，再用1减去这个值，得到的就是0到1范围内递增的计时器时间进度。

```
var daytime_point: float = 1.0 - ($DayTimer.time_left / $DayTimer.wait_time)
```

得到这个值后就可以使用sample方法截取daytime_gradient渐变条内对应的颜色采样，如下。

```
$CanvasModulate.color = daytime_gradient.sample(daytime_point)
```

运行游戏测试，可以将DayTimer的等待时间缩短，方便观察效果，游戏内颜色随着时间流逝根据预设的渐变条渐变，开局呈现偏黄色调，随后逐渐明亮，到最后变得昏暗，效果符合预期。

最后是当调用level_reset方法重置关卡时，重启DayTimer计时器，即昼夜切换后游戏内颜色重置，如下。

```
func level_reset():
    for plant in get_tree().get_nodes_in_group('Plants'):
        plant.grow(plant.grid_pos in $Layers/SoilWaterLayer.get_used_cells())
    $Layers/SoilWaterLayer.clear()
    $DayTimer.start() #重启计时器
```

进入游戏测试，当时间为夜晚时，按下Tab键切换昼夜，色调又变为暖黄色。

至此，游戏主要内容已全部制作完毕。

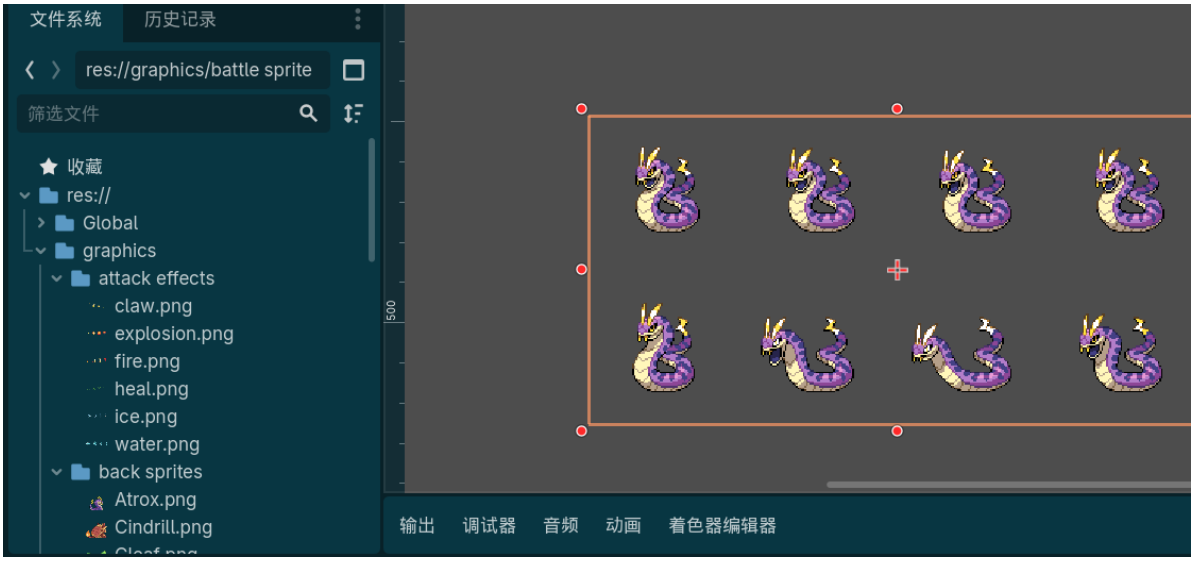
4 精灵对战类游戏

上

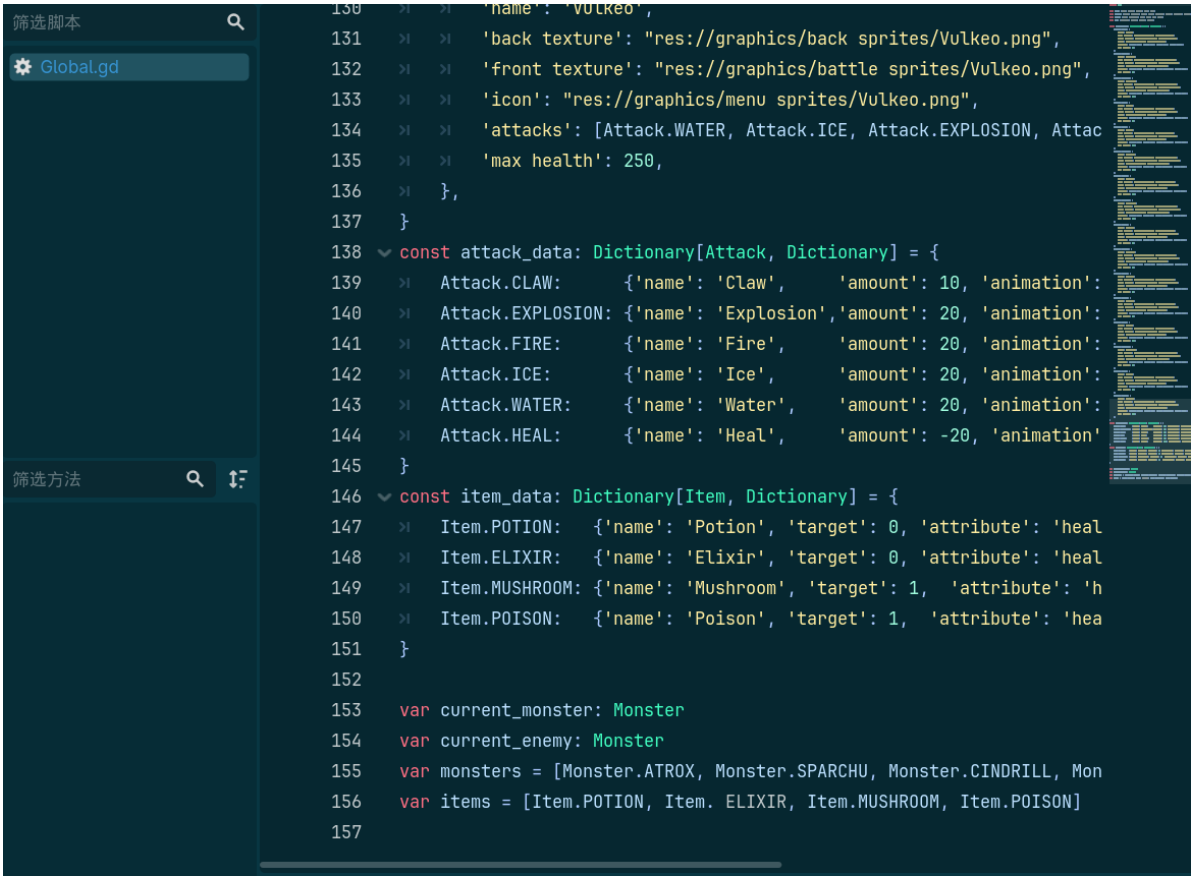
1.工程初始化配置

接下来我们将要做一款类宝可梦风格的回合对战游戏，这个项目最核心的内容是UI元素，几乎全程都在讲这方面内容。之前第一款游戏里简单提过布局节点，在本项目中，我们将会深入讲解。

首先，导入项目，项目中已提前准备好不少资源，graphics中存放了大量贴图素材，如怪物攻击特效贴图和怪物正/背面的Sprite2D素材，我们后续将使用这些素材进行开发，如图。



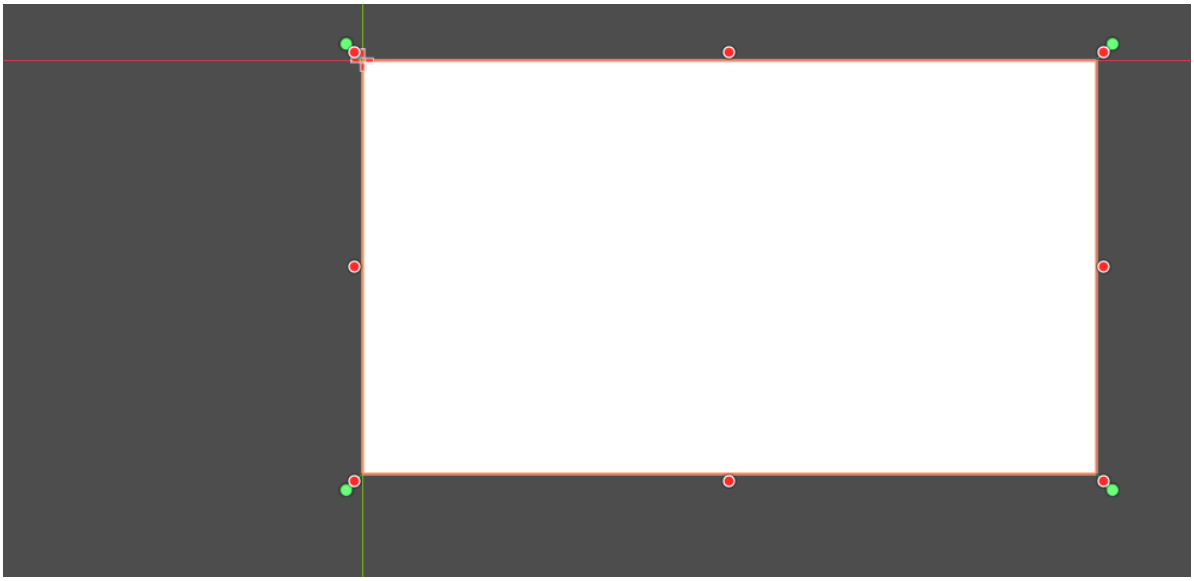
除贴图资源外，项目中只有Global文件夹下的global.gd脚本，这个脚本已被设置为全局脚本，其中存放了大量游戏数据，如图。



脚本中定义了大量枚举，包括4种菜单状态、游戏内全部怪物、所有攻击技能、可使用道具，配套这些枚举，还有怪物具体数据，每一只怪物都包含多种属性如名称、贴图、专属攻击技能和最大生命值。技能数据和物品数据也类似。这些数据现阶段不用深入了解，到使用时再具体讲解。

接下来先新建一个用户界面场景，命名为Test，接下来将使用它来讲解UI元素。

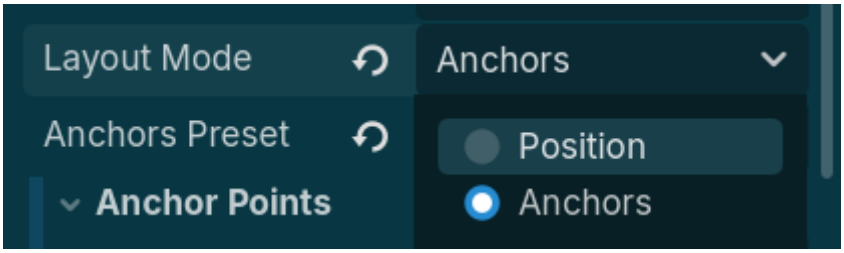
讲解UI元素时，最适合入门学习，使用频率最高的就是ColorRect，这个节点本质就是一块可以随意修改颜色的矩形面板，在Test场景中新增一个ColorRect节点，在顶部菜单中“控制节点的锚点和偏移值的预设”选项中设置为“整个矩形”，如图。



可以看到画面四角的绿色点，那就是锚点，通过调整锚点，就可以控制布局控件随窗口缩放的方式。打开检查器面板找到Layout，其中就有锚点相关设置项，将Anchor Preset修改为Custom就可以自定义锚点数值参数，通过修改它们来修改锚点位置。

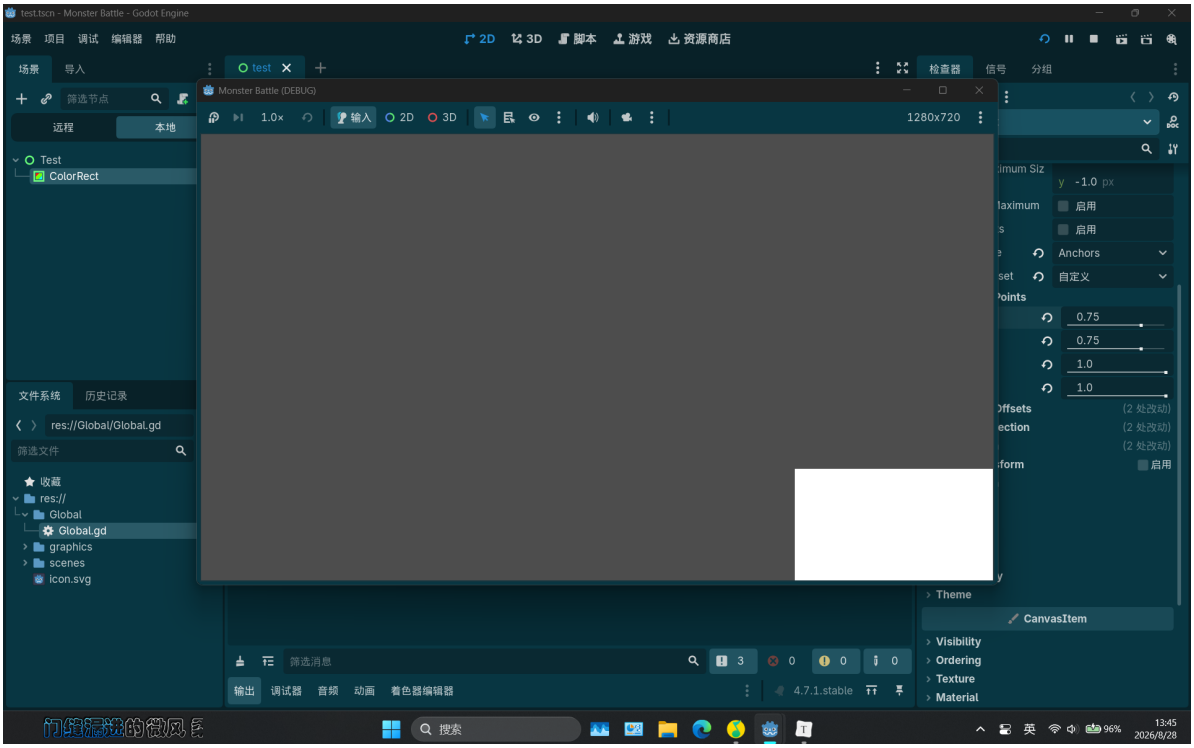


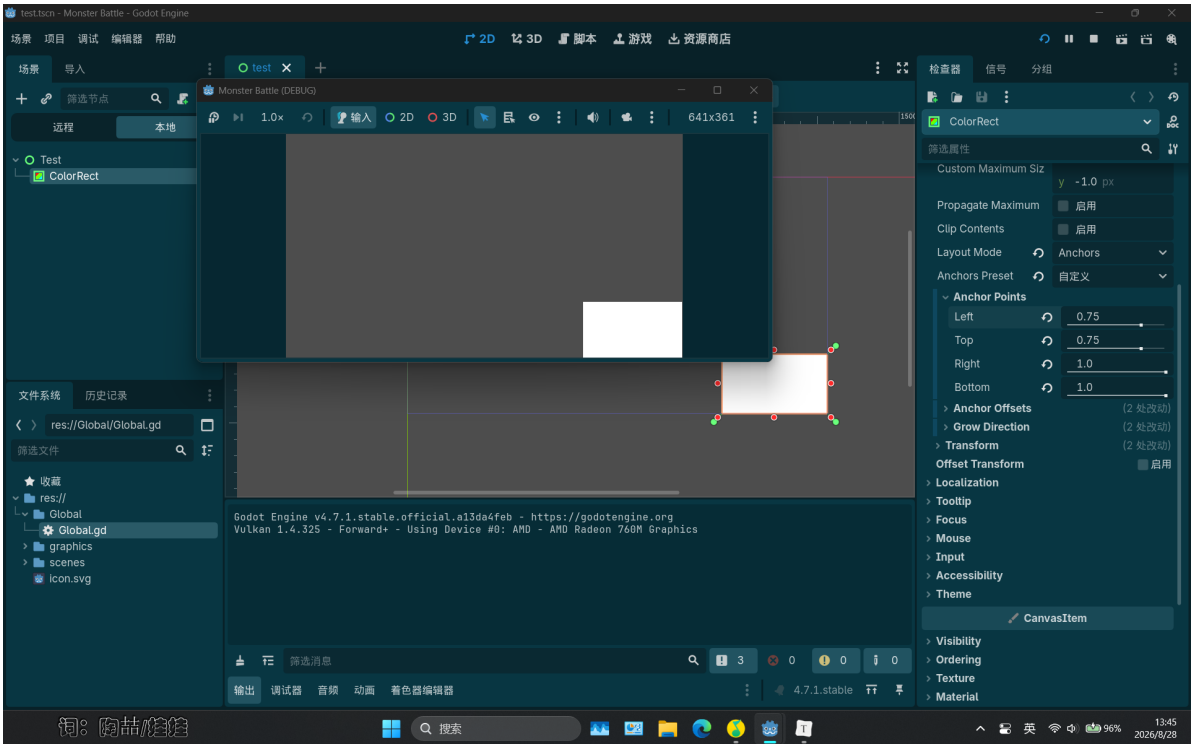
当然，布局模式除了使用锚点，也可以使用像素坐标模式，如图。



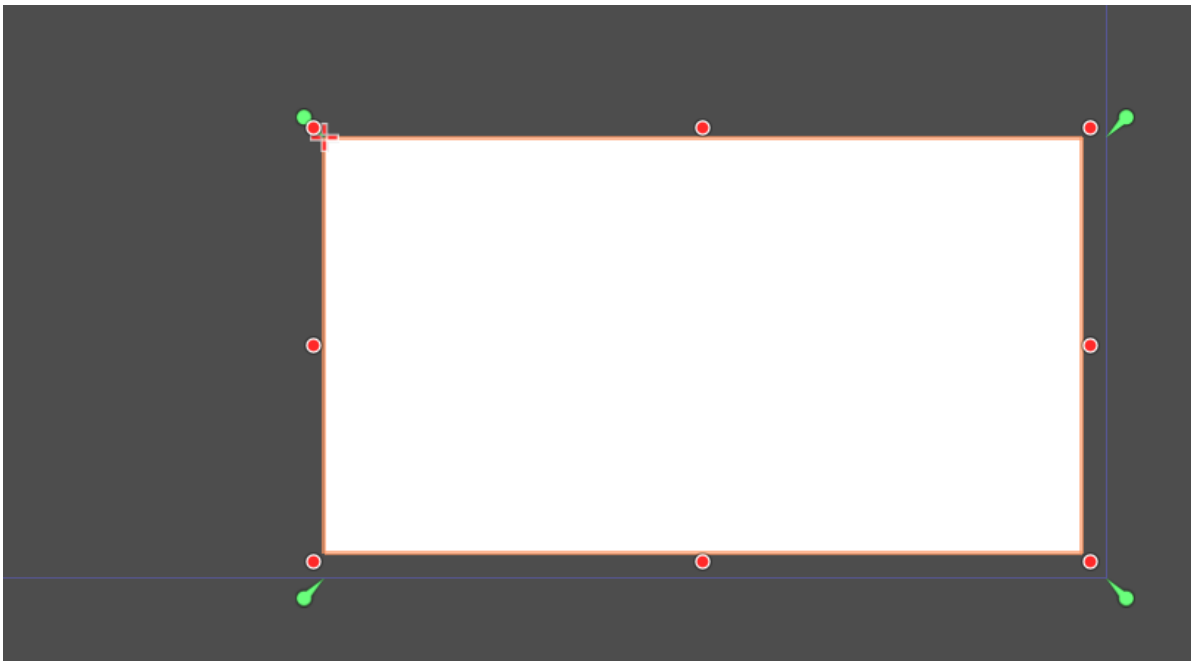
但使用坐标模式有一个缺陷，就是不会随着窗口同步缩放，会一直固定在原来的坐标。因此只适用于部分场景，绝大多数UI元素还是使用锚点模式，例如制作永远显示在界面左上角的状态条。核心需求就是UI元素保持对窗口的相对位置。

设置锚点为Left：0.75、Top：0.75、Right：1.0、Bottom1.0后，运行游戏，调整窗口大小，矩形始终在窗口右下角，如图。





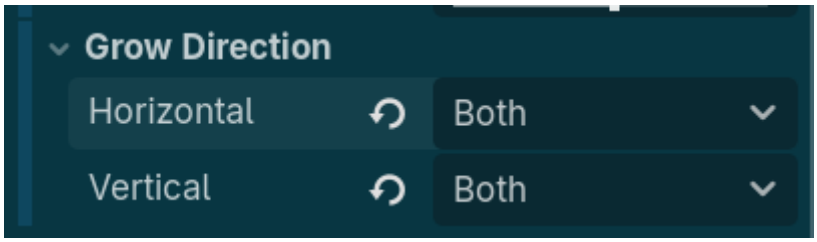
理解锚点，是掌握Godot布局系统的核心。除了锚点设置，还有锚点偏移值，可以额外设置UI对锚点的距离，如将Right和Bottom的锚点偏移设置为-10，UI就会离屏幕下边界和右边界有10像素距离，如图。



此外，还可以限制UI的大小边界，如图。

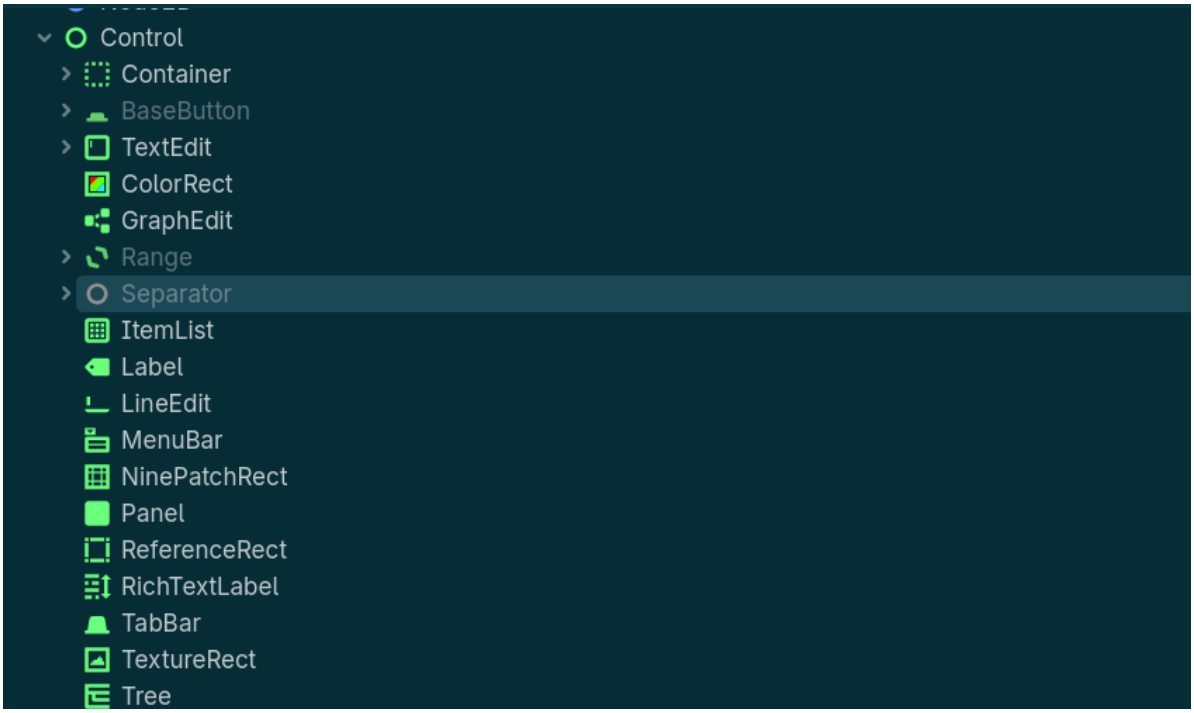


当设置了UI的最小大小后，调整窗口大小有可能导致UI溢出屏幕外，此时可以设置UI的拓展方向，若设置右下角的UI拓展方向为上方和左方，就不会导致UI溢出屏幕，如图。



依靠上述四项设置，我们可以将UI摆放到屏幕的任意位置。

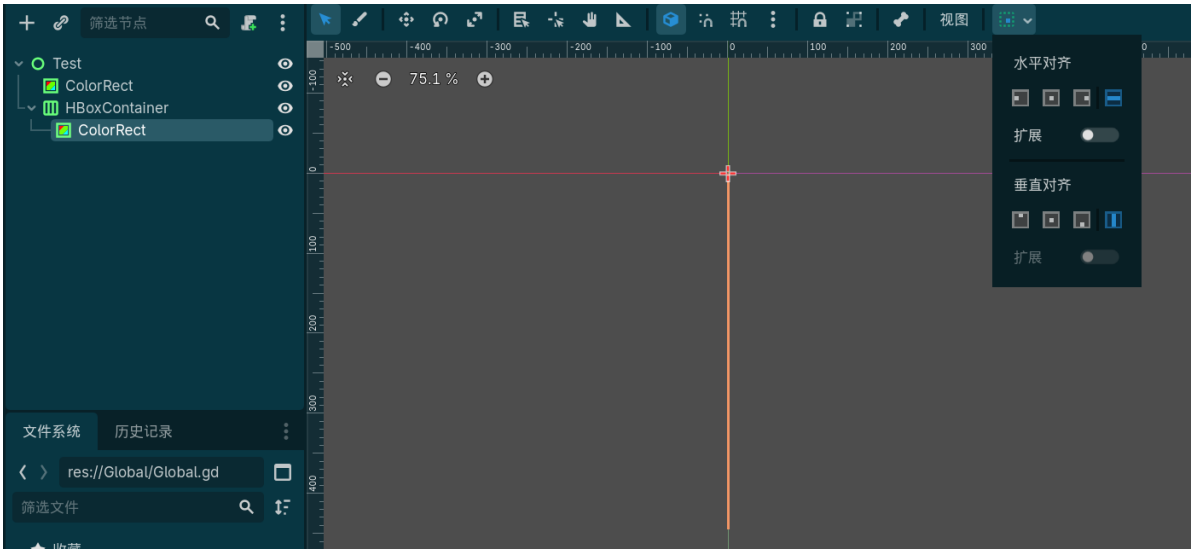
除了以上内容，我们来看到Control节点分类下的其他容器节点，如图。



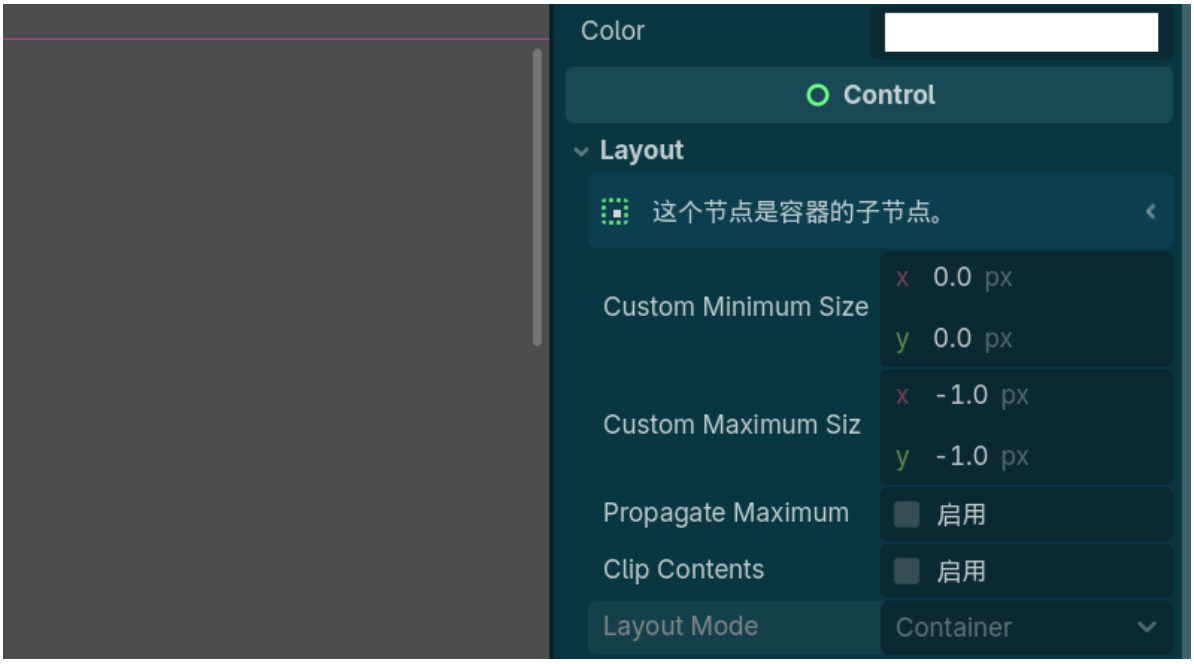
各种容器节点可能和上述的锚点设置有所不同，我们用一个使用频率极高的容器节点举例——BoxContainer。

BoxContainer分为VBoxContainer垂直容器和HBoxContainer水平容器，我们这里先讲解HBoxContainer。

这类容器的标准使用方法，是往里面添加子节点，我们王容器内新建一个ColorRect纯色容器，选中这个容器后，编辑器顶部会切换为全新的功能菜单，如图。

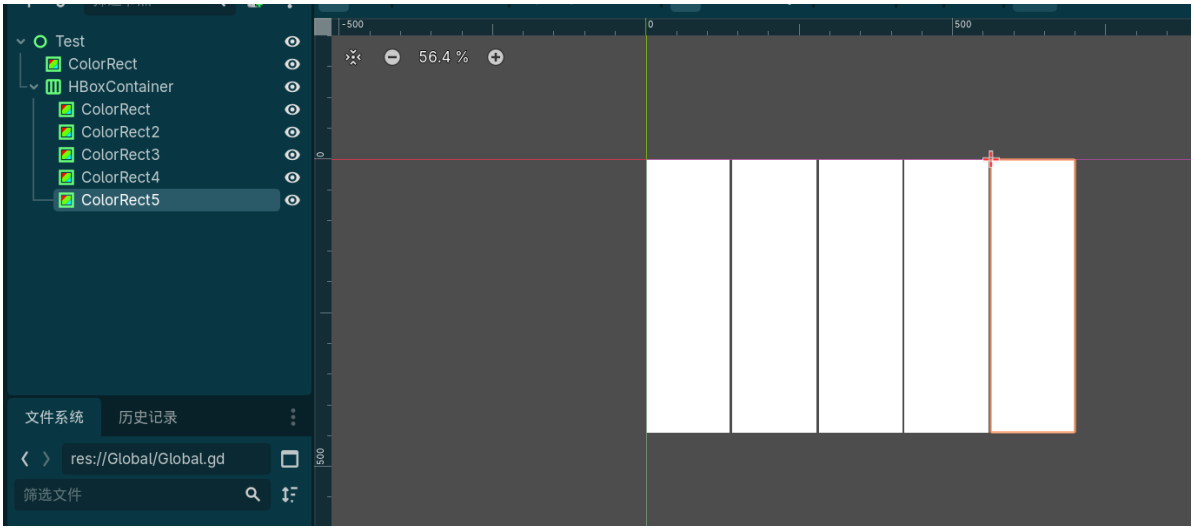


检查器中也会提示这个节点是容器的子节点，使用容器专属参数来修改位置，且Layout Mode固定为容器模式。

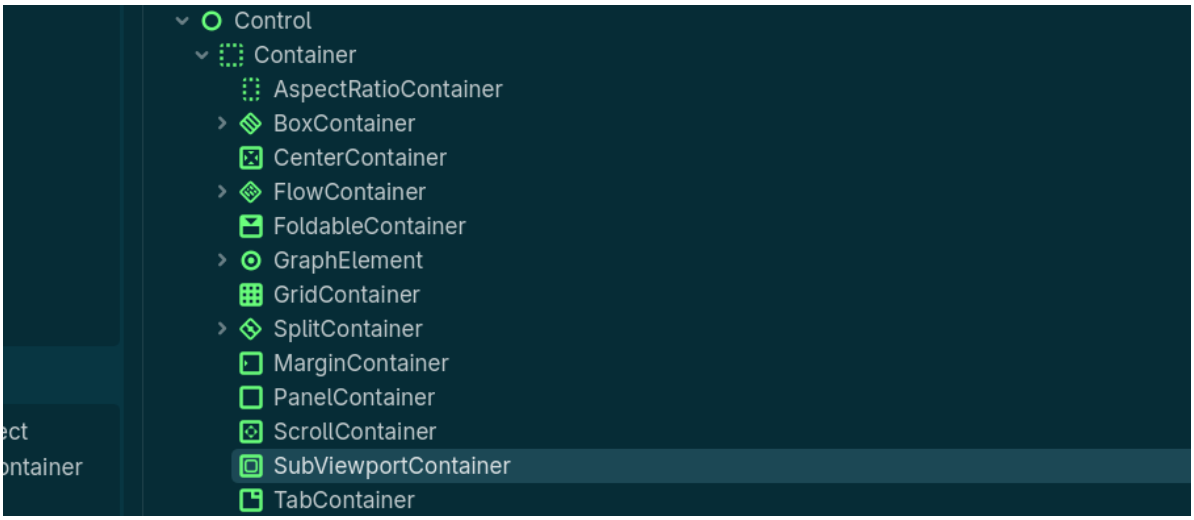


原因是，对于这个ColorRect，它的位置排布完全由父节点HBoxContainer来接管。实际运行逻辑是，容器会把所有子节点沿着水平轴线依次排列。举个例子，我们开启顶部布局选项的“拓展”，ColorRect会自动填满整个容器，将其多复制几份，可以看到它们沿水平方向整齐排列。





这里可以理解，HBoxContainer的核心作用就是按照预设好的规则沿水平轴线排列全部子节点。引擎内还有很多其他容器，打开Control下的容器分类就能看到，如图。



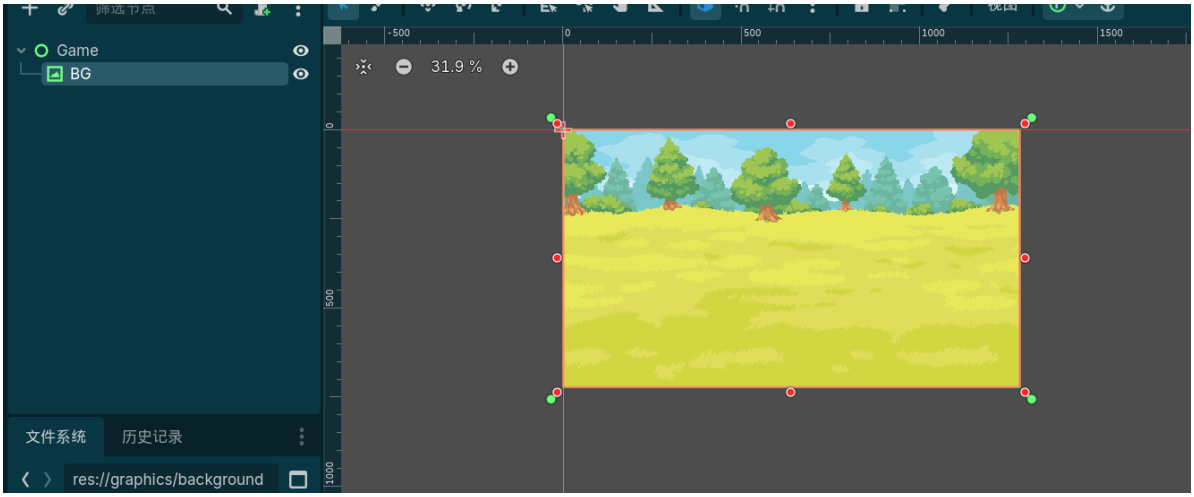
后面具体使用时，我们再讲解这些容器。

2.基础核心知识

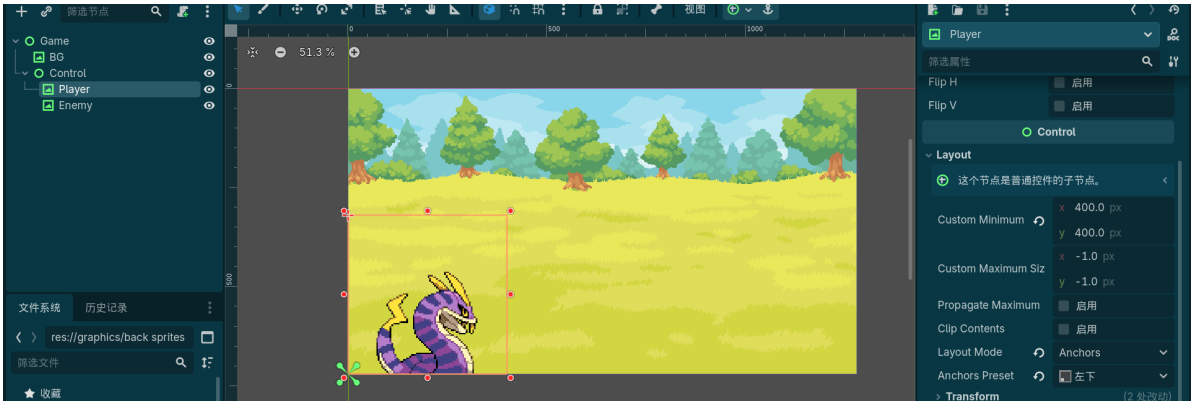
正式制作游戏时，我们先搭建基础内容。我准备放上两只怪物，还有一套基础的按钮选择界面，同时展示状态信息，也就是名称和血条。

删除Test场景，新建一个用户界面Game，保存后打开项目设置，在应用的运行设置中将其设置为主场景。

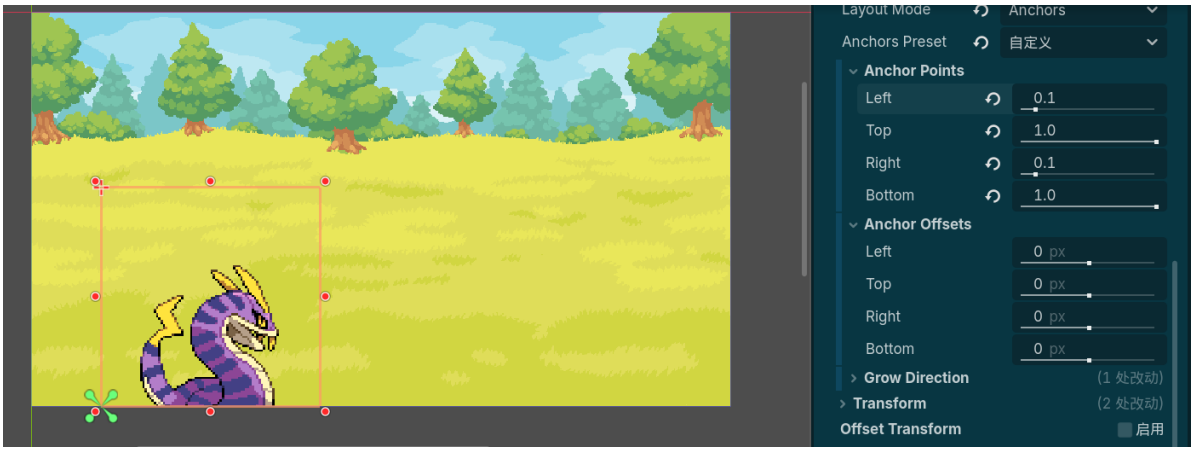
为其添加一张背景图，选择TextureRect，命名为BG，选择res://graphics/backgrounds/background1.png拖入Texture，锚点预设设置为铺满矩形，检查器中Stretch Mode设置为Scale，如图。



完成背景后，再添加一个Control节点Monster，用于存放怪物素材，预设为铺满，其下新建两个TextureRect，一个Player，一个Enemy，Player贴图使用res://graphics/back sprites/中任意一张贴图，随后预设为左下。此时贴图尺寸偏小，我们通过设置custom_minimum_size来修正，尺寸设置为400*400，如图。

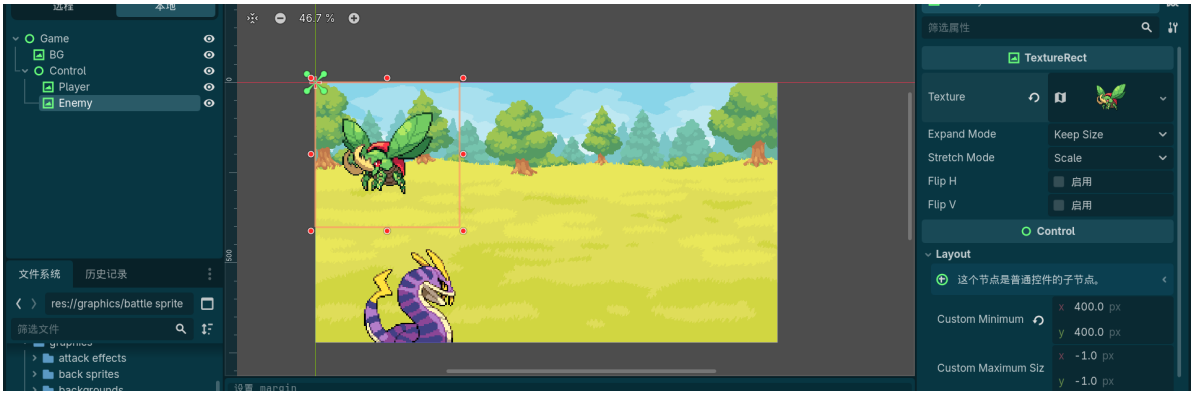


除此之外，我们还可以打开Anchors Preset，选择Custom自定义，随后实时微调锚点位置和偏移量，随后运行查看效果，如图。



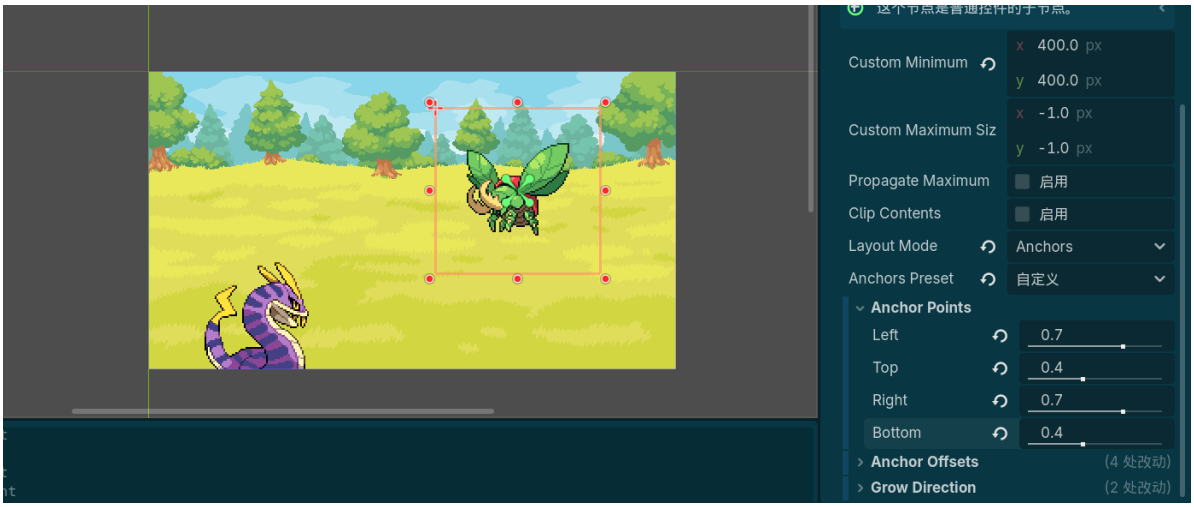


接下来处理敌方怪物，敌方怪物的贴图选择res://graphics/battle sprites/中任意一张，但由于其中的资源自带动画，而TextureRect节点无法处理动画资源，因此不能直接将其添加到TextureRect类型节点中，而是在Texture中新建AtlasTexture资源，在Altas中选择res://graphics/battle sprites/中资源，并设置其Region中的w和h为96，因为res://graphics/battle sprites/中的图片的一帧大小为96像素。这样设置后，正常情况下TextureRect节点就只会显示其一帧的画面，最后在TextureRect的Layout属性中同样设置其自定义最小大小为400*400，如图。



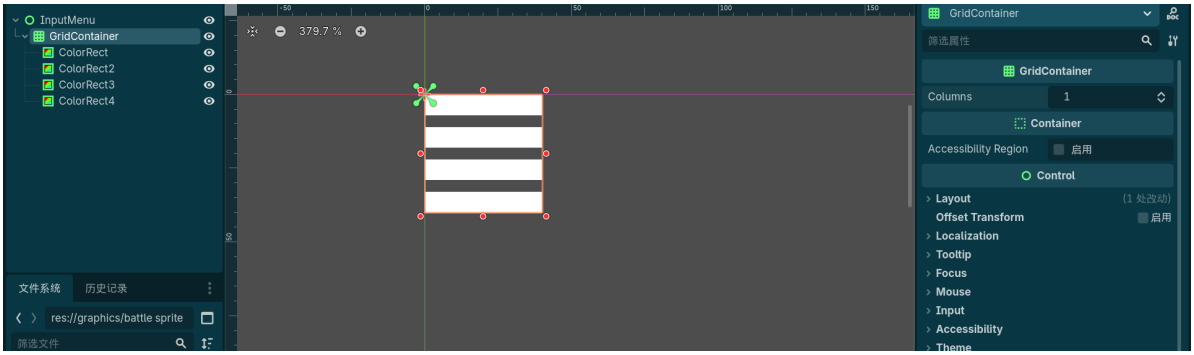
接下来通过设置其锚点修改其位置，先设置其预设为居中，使锚点可以包裹住整个贴图，防止贴图出现异常拉伸效果。

随后修改其锚点位置，如图。

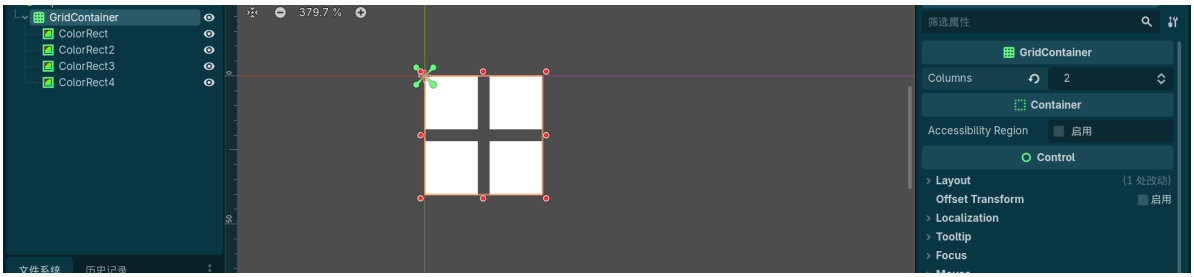


接下来我们制作操作面板。现阶段面板只展示4个按钮，面板需要单独新建一个场景，也是用户界面类型，命名为InputMenu，预设设置铺满，其下添加GridContainer，这是一个可以将子控件按网格布局排列的容器，核心属性是列数。

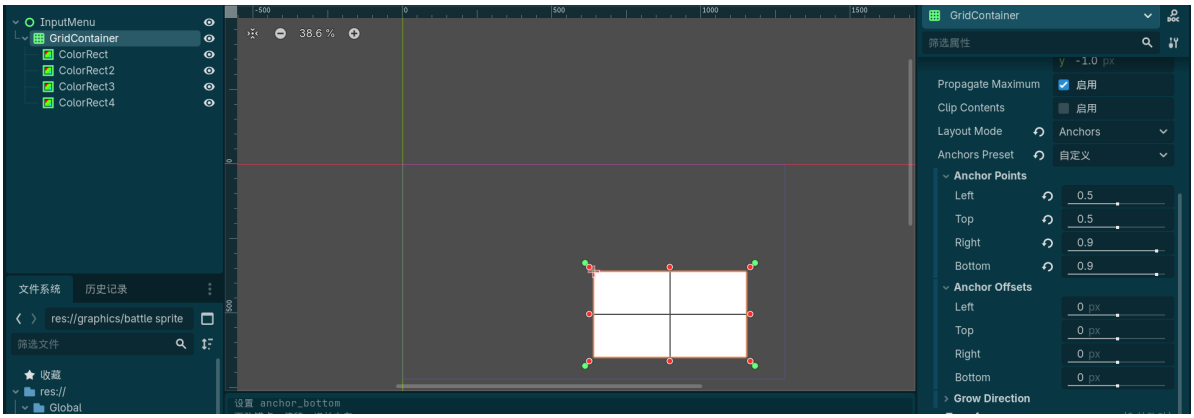
我们在GridContainer下添加ColorRect，开启其水平拓展和垂直拓展后，复制几个ColorRect，如图。



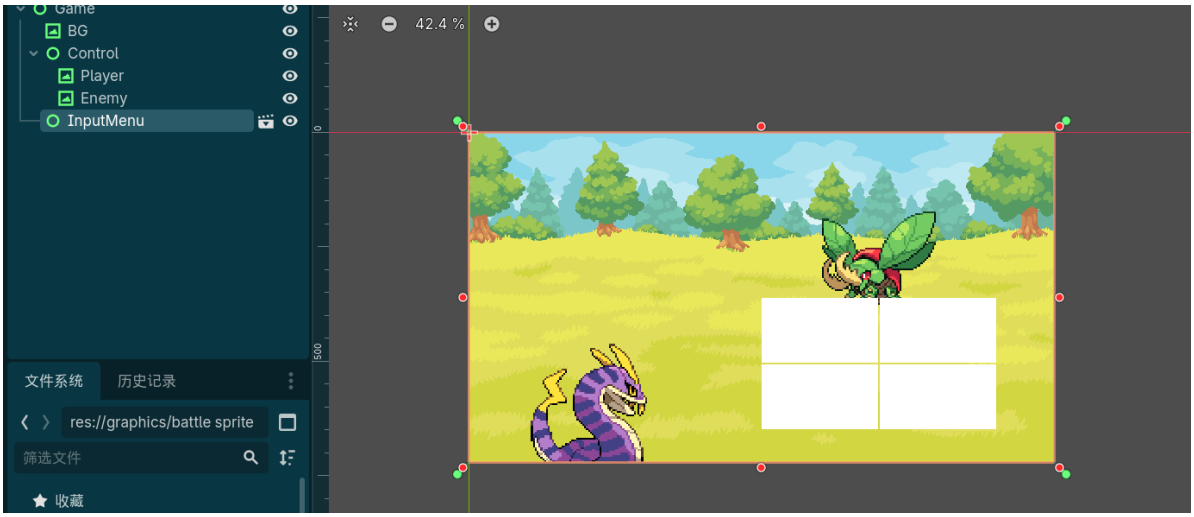
修改Columns为2，如图。



这样就便于我们后续添加实际按钮控件。在Game场景实例化input_menu，随后在Input_menu场景调整GridContainer位置，如图。



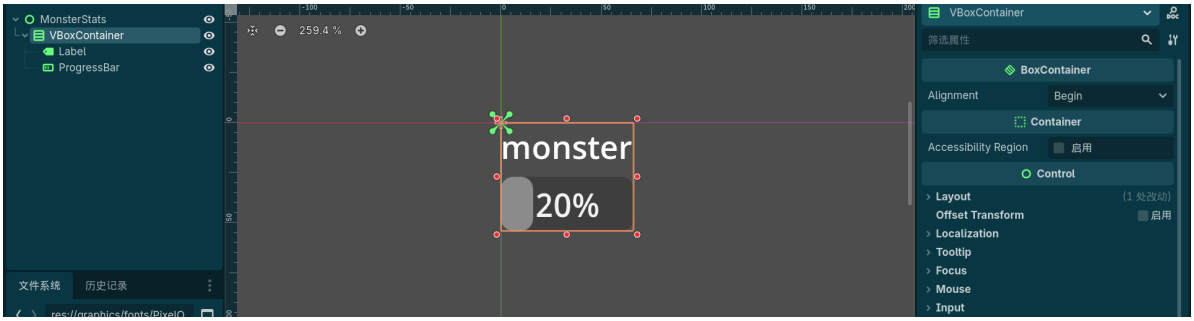
回到Game场景，查看效果，如图。



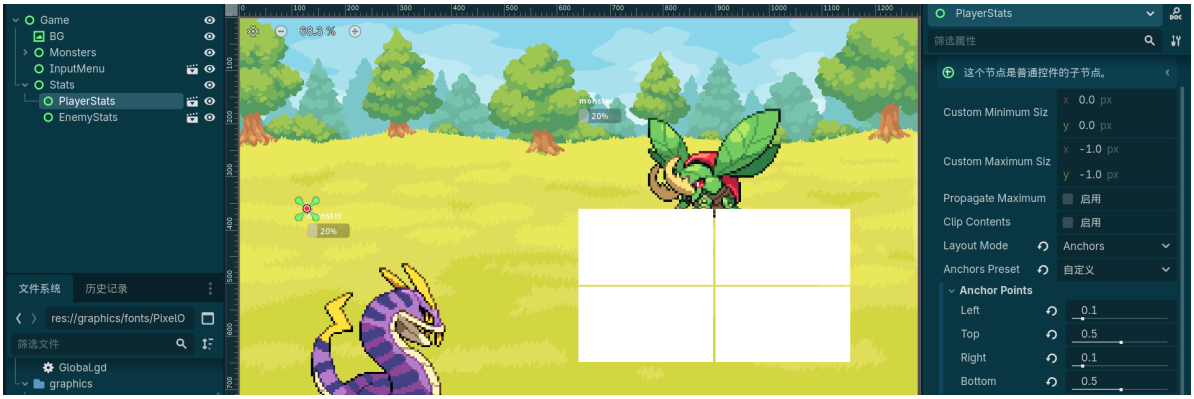
至此，按钮面板的布局就调整好了，整体效果不错。

还差最后一块内容——在怪物上方添加状态信息显示区域，展示名称与血量，玩家怪物和敌方怪物都需要配置状态显示区域。

新建用户界面场景MonsterStats，其下添加一个显示怪物名称的Label节点和一个显示血量的ProgressBar节点，为了让二者规整排布可以将其添加到VBoxContainer下，VBoxContainer会自动将二者纵向排列，如图。



为VBoxContainer设置最小大小为80*40，随后保存该场景，回到Game场景，新建一个Container类型父节点Stats专门用于存放状态面板，布局预设为铺。在其下将MonsterStats实例化，命名为PlayerStats，调整其位置到合适位置。随后复制一份PlayerStats，改名为EnemyStats，同样修改锚点位置，最终效果如图。



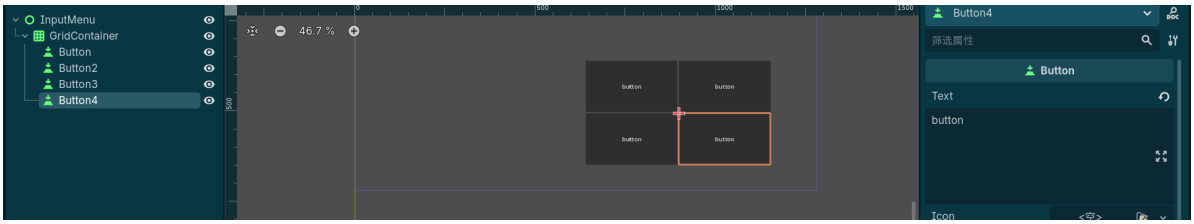
目前效果虽然简陋，但已够用，后续我们再完善其视觉效果。至此，这套基础对战界面就搭建完成了。

3.网格布局按钮

接下来我们可以开始制作基础按钮，现阶段先不做任何样式美化，后续再处理。

另外，这些按钮需要做成动态的，举个例子，主功能按钮和攻击按钮需要使用同一个GridContainer容器，但两个按钮需要执行不同的逻辑，这一点非常重要。

现在选择InputMenu场景，我们只有四个ColorRect，我们先将这些纯色矩形删除，往界面里添加Button，暂时设置其Text为‘button’，预设中开启横向拓展和纵向拓展，复制三个如图。

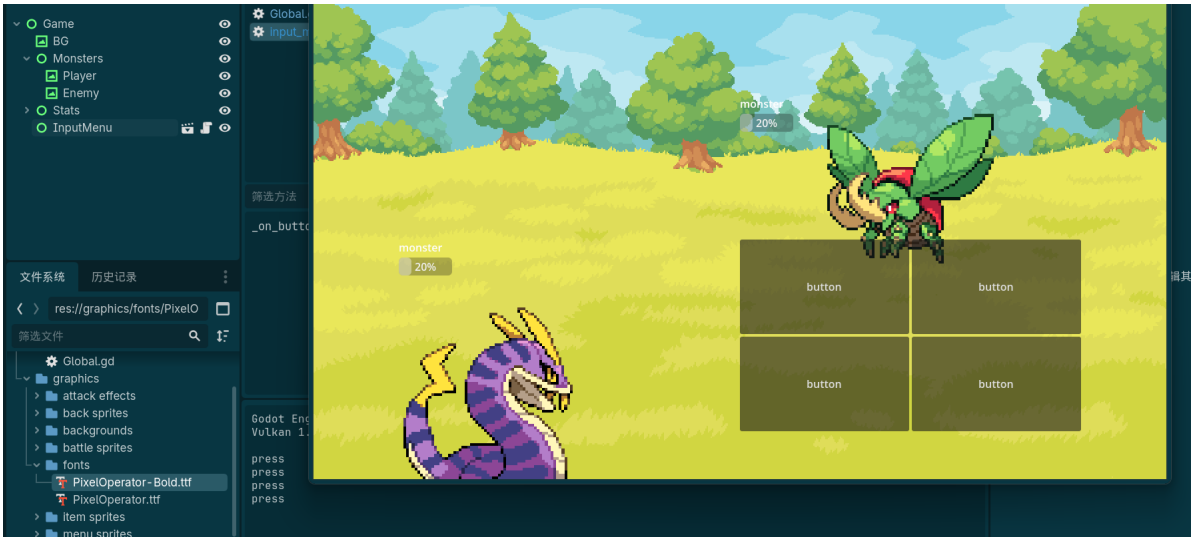


为InputMenu场景添加脚本。选中第一个按钮，切换到信号面板，选择press信号，将其绑定到InputMenu脚本，编写_on_button_pressed方法，让其输出‘press’，如图。



完成后运行游戏，点击按钮却并未输出任何语句，这是因为在Game场景中Stats和InputMenu都预设为铺满屏幕，因此实际上我们点击在了Stats上，鼠标点击无法穿透到下层的输入菜单，并未真正和按钮交互。

最简单的修复方式是将InputMenu场景移动到Game子节点的最底部，再次运行游戏，已能正常交互并输出语句。

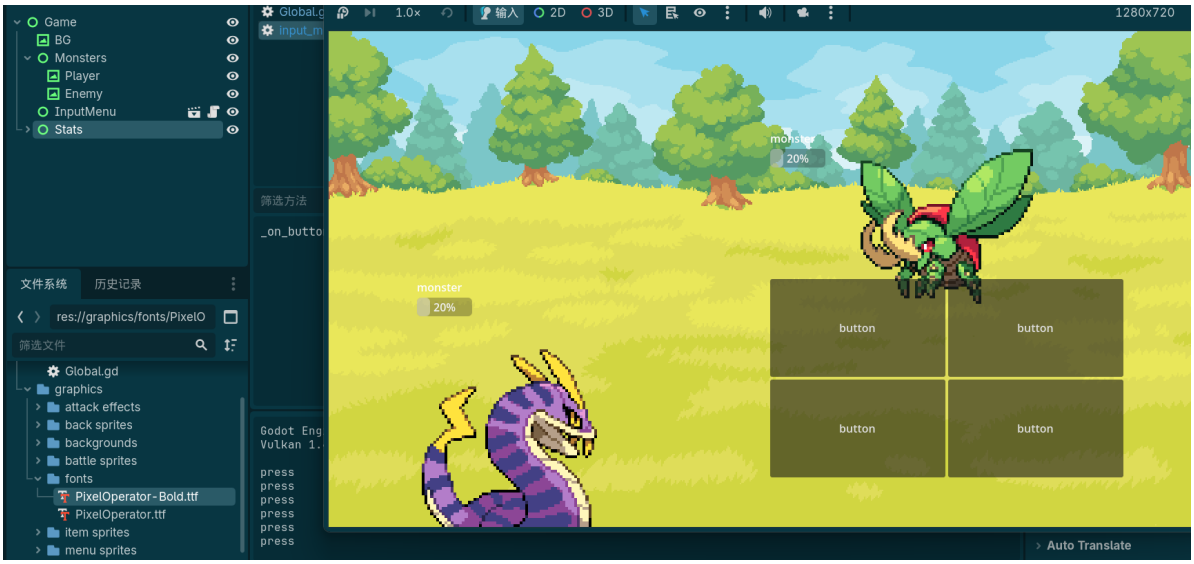


简单来说，Godot处理鼠标点击时会从上到下依次遍历所有节点，最先遮挡鼠标的节点会捕获鼠标输入，如果某个节点铺满全部屏幕，它就会拦截所有鼠标输入。

打开检查器面板，找到Mouse设置，这里可以设置Filter属性从而让鼠标穿过或直接忽略该层，我们选中Stats状态面版节点，将其Mouse的Filter设置为ignore或Pass，即可无需调节节点次序就正常与InputMenu层交互。



我们将InputMenu移回原先位置，再次运行游戏测试，按钮仍然能正常交互，如图。



如果有某个UI节点无法交互，大概率是有某个别的节点压在它的上层将信号遮挡了。

目前界面中已有可交互的按钮，但这些按钮都是静态被写死的，这不符合我们的要求，我们要换一种实现方式。

我们直接新建一个以Button为根节点的场景，命名为GridButton，Text暂时设置为‘button’，并为其挂载脚本。

切换到信号面板，选择press信号挂载到脚本，现阶段同样让其打印语句，如图。



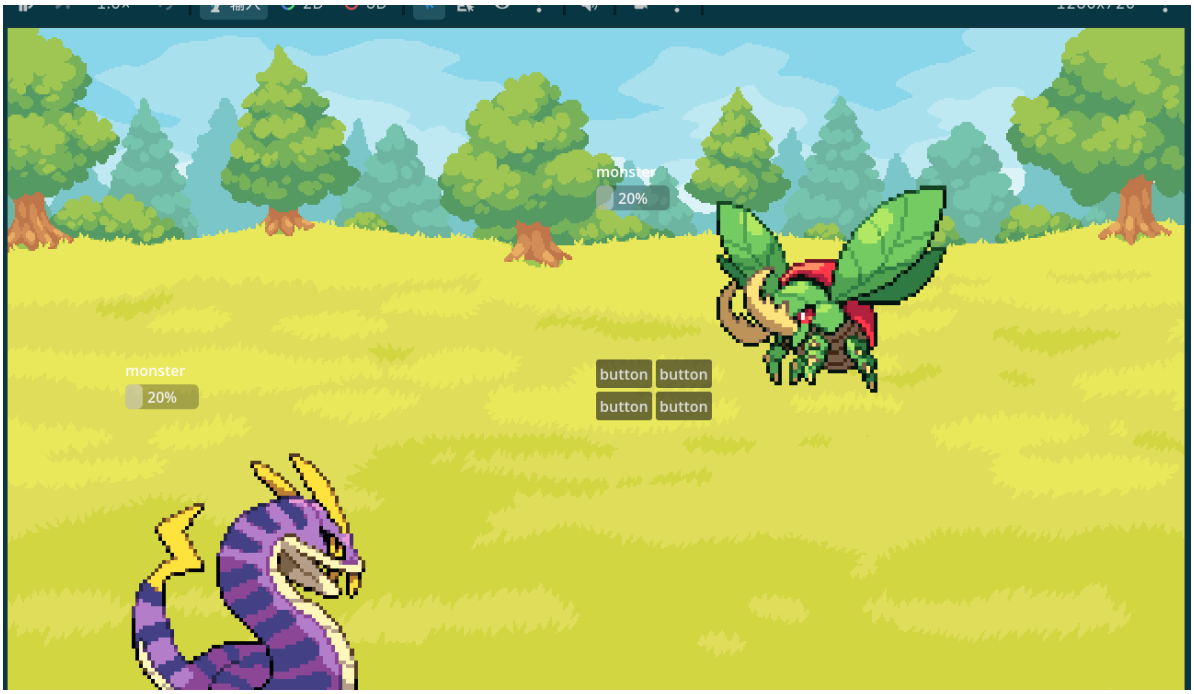
接下来，我们要动态生成GridButton实例，在InputMenu创建GridContainer时生成按钮。换个说法，我们将彻底删掉先前手动创建的的Button，等到输入菜单加载完成后再动态生成。

删除4个原先按钮后，在grid_button脚本中编写，先新建一个变量grid_button_scene用于预载按钮场景，随后循环实例化4个按钮并调用GridCotainer的add_child接口，将实例化的按钮添加为其子节点，如下。

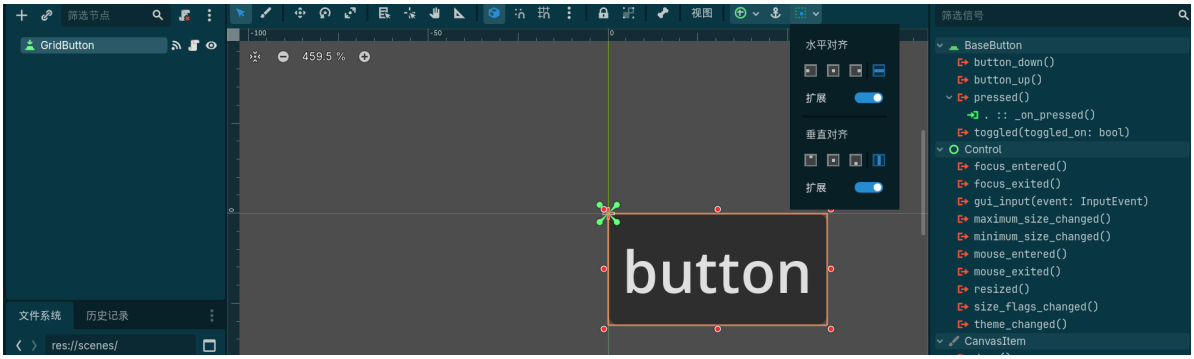
```
var grid_button_scene = preload("res://scenes/grid_button.tscn")

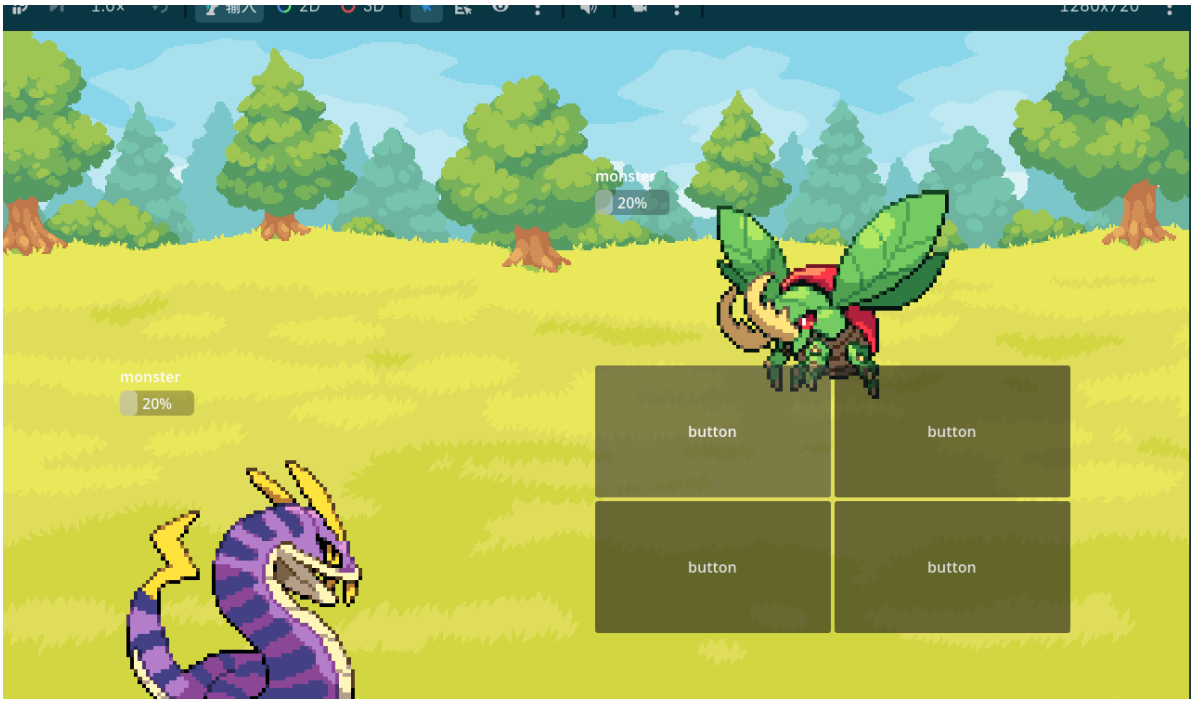
func _ready() -> void:
    for i in 4:
        var grid_button = grid_button_scene.instantiate()
        $GridContainer.add_child(grid_button)
```

运行游戏测试发现，按钮虽然创建成功，但大小有问题，按钮没有填满GridContainer容器，如图。



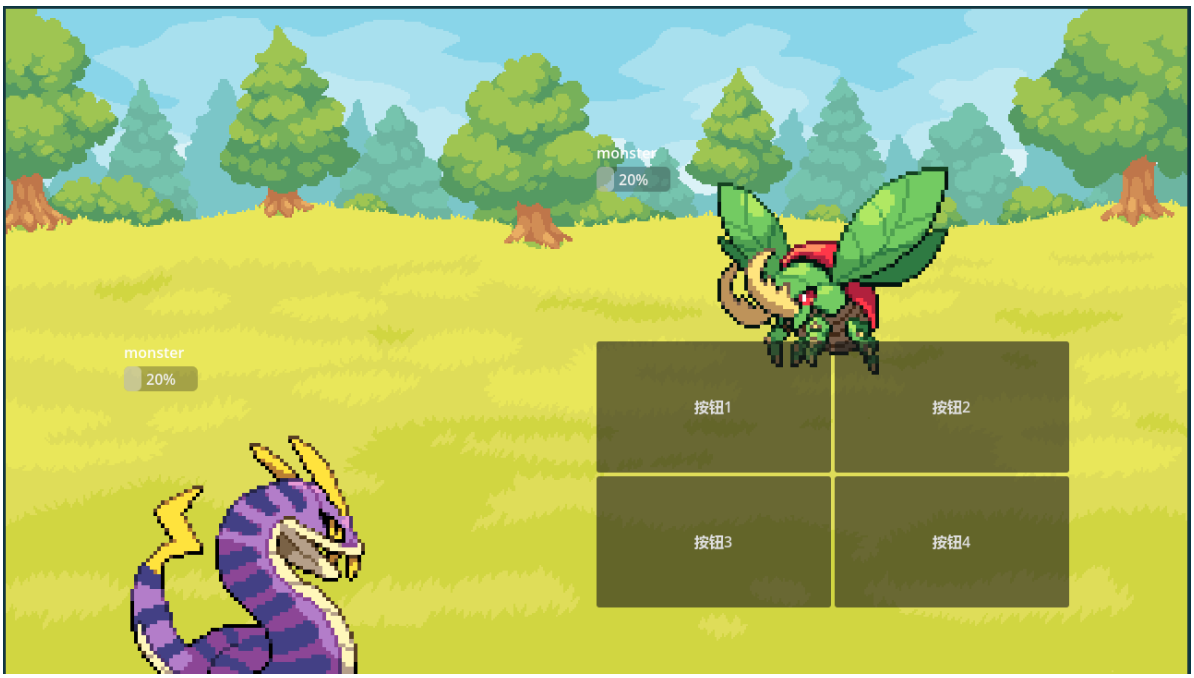
处理方法也很简单，选择GridButton场景后，在预设中开启其水平拓展和横向拓展，这样实例化后的按钮将会自动填满GridContainer容器。运行测试，效果符合预期，如图。





此外，我们还可以修改按钮的文字，如下。

```
func _ready() -> void:
    for i in 4:
        var grid_button = grid_button_scene.instantiate() as Button
        grid_button.text = '按钮'+ str(i+1)
        $GridContainer.add_child(grid_button)
```



当前，整套逻辑还需要提升灵活度，我们可以新建一个字典常量main_buttons，其中的字典键对应全局脚本中的枚举state，state中存放了5种菜单的状态，具体如下。

```
enum state {MAIN, ATTACK, DEFEND, SWAP, ITEM}
```

因此，我们在main_buttons即主要状态时的菜单显示按钮中直接使用state的元素作为键，值为按钮显示的名称，如下。

```
const main_buttons = {
    Global.state.ATTACK: '攻击',
    Global.state.DEFEND: '防御',
    Global.state.SWAP: '切换',
    Global.state.ITEM: '道具',
}
```

为了让整套逻辑更加清晰，我们新建一个函数creat_grid_buttons，将在这个函数内处理按钮数据，这个函数有两个参数，当前菜单状态和该菜单的按钮字典，如下。

```
func creat_grid_buttons(state:Global.State, data:Dictionary):
```

将ready函数内的逻辑删除，仅留下对creat_grid_buttons的调用，使ready函数内更加简洁。

```
func _ready() -> void:
    creat_grid_buttons(Global.State.MAIN,main_buttons)
```

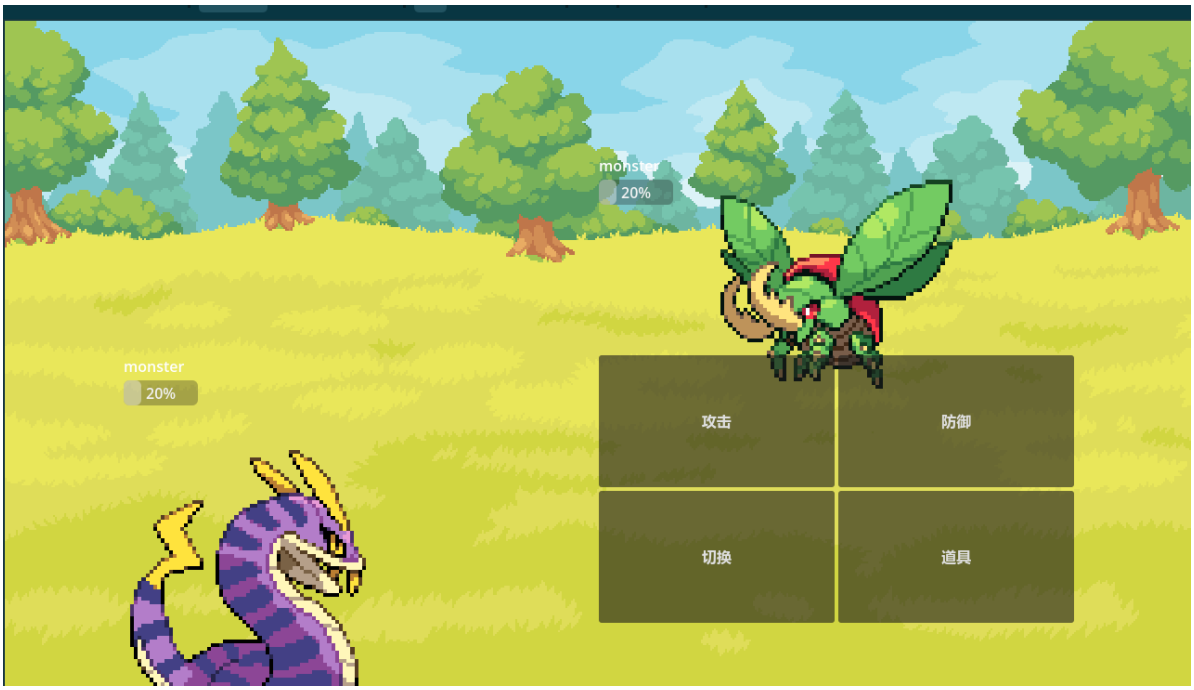
在creat_grid_buttons函数内，我们预想的效果是切换菜单状态时菜单中的按钮随之改变，因此在调用这个函数后，要先清除之前的按钮对象，随后再根据参数调用setup方法添加对应按钮，具体代码如下。

```
func creat_grid_buttons(state:Global.State, data:Dictionary):
    for i in $GridContainer.get_children(): #销毁先前按钮
        i.queue_free()
    for key in data:
        var grid_buttton = grid_button_scene.instantiate() #实例化出按钮
        grid_buttton.setup(state,key,data[key]) #参数为当前菜单状态、当前按钮类型、当前按钮显示文本
        $GridContainer.add_child(grid_buttton) #添加到场景
```

随后我们转到grid_button脚本实现setup方法，setup方法有三个参数，分别是菜单状态、按钮类型和按钮文本，先在脚本内新建两个变量state和type，随后对它们进行赋值，如下。

```
func setup(menu_state:Global.State, button_type, button_text:String):
    text = button_text #直接修改按钮文本
    state = menu_state
    type = button_type
```

运行游戏测试，按钮文本已成功修改为各自值，如图。



在脚本中新建一个自定义信号press，该信号接受两个参数，state和type。修改原先的_on_pressed方法，让它发送该信号，这样按下按钮后就会将state和type发送出去，便于处理每个按钮不同的逻辑。

```
signal press(state, type)

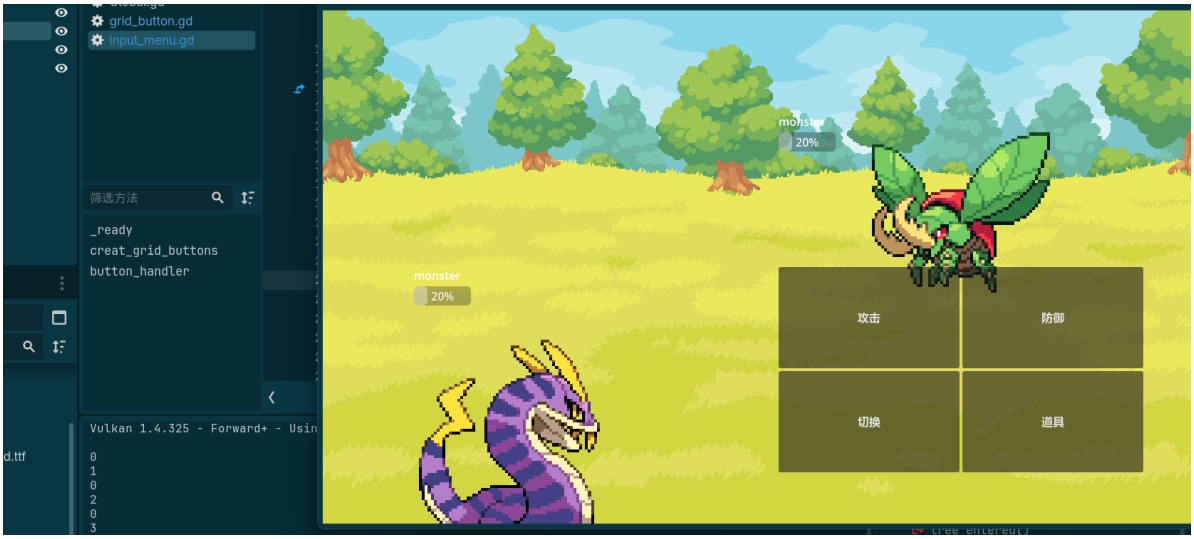
func _on_pressed() -> void:
    press.emit(state, type)
```

回到input_menu脚本，想要在input_menu脚本中使用press信号，需要先获取grid_button的实例，再使用connect方法将grid_button的press信号和button_handler方法绑定，我们后续将在button_handler中处理不同按钮触发后的操作，当前先新建该方法，让它打印两个信号传来的参数，方便测试效果。

```
func creat_grid_buttons(state:Global.State, data:Dictionary):
    for i in $GridContainer.get_children():
        i.queue_free()
    for key in data:
        var grid_buttton = grid_button_scene.instantiate()
        grid_buttton.setup(state,key,data[key])
        $GridContainer.add_child(grid_buttton)
        grid_buttton.connect('press', button_handler) #使用connect方法连接信号和方法

func button_handler(state, type):
    print(state)
    print(type)
```

运行游戏测试，按下不同的按钮，控制台将输出菜单状态和按钮各自的类型，如图。



4.基础攻击功能实现

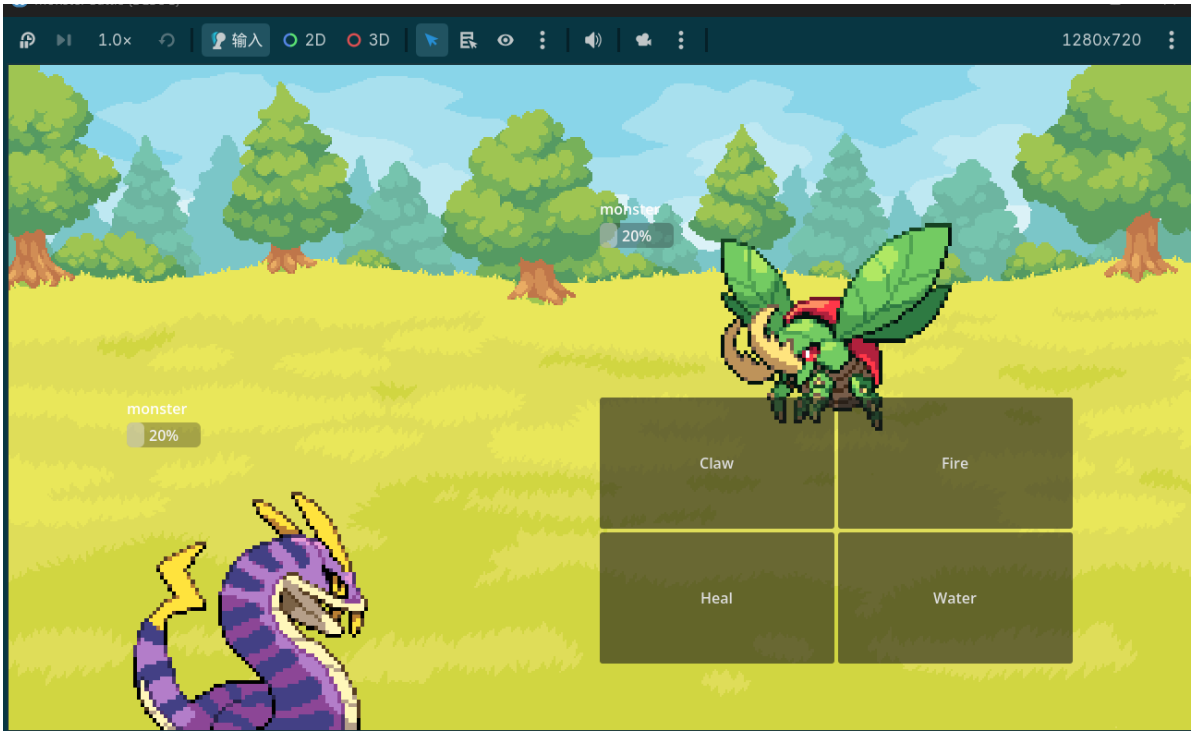
要实现基础攻击功能，我们先给game场景挂载脚本，在脚本的ready函数中，先暂时使用Global.monsters中的第一个元素为Global.current_monster赋值。我们观察Global脚本可以看到，Global.monsters存放的是所有怪物的名称，Global.current_monster是用于记录当前怪物的变量。

在游戏启动后修改了游戏当前怪物后，我们回到Input_menu脚本，让按钮菜单显示当前怪物的攻击技能。

在Input_menu脚本的ready函数中，我们先注释掉先前的语句，创建一个变量monster_attacks获取当前怪物的技能列表数组，随后调用creat_grid_buttons方法，传入第一个菜单状态参数为Global.State.ATTACK，标识了当前菜单为攻击菜单。第二个字典类型参数为当前怪物的各个攻击名称，该字典键为monster_attacks，值为使用monster_attacks在全局变量attack_data中获取的对应名称，具体代码如下。

```
func _ready() -> void:
    #creat_grid_buttons(Global.State.MAIN,main_buttons)
    var monster_attacks = Global.monster_data[Global.current_monster]['attacks']
    print(Global.monster_data[Global.current_monster]['name'])
    creat_grid_buttons(Global.State.ATTACK, {
        monster_attacks[0]: Global.attack_data[monster_attacks[0]]['name'],
        monster_attacks[1]: Global.attack_data[monster_attacks[1]]['name'],
        monster_attacks[2]: Global.attack_data[monster_attacks[2]]['name'],
        monster_attacks[3]: Global.attack_data[monster_attacks[3]]['name'],
    })
```

这样就我们就成功给每个按钮传入了它们的攻击技能名称，运行游戏测试，可以看到正常显示各技能名称，点击按钮后控制台会输出当前菜单状态和技能索引。



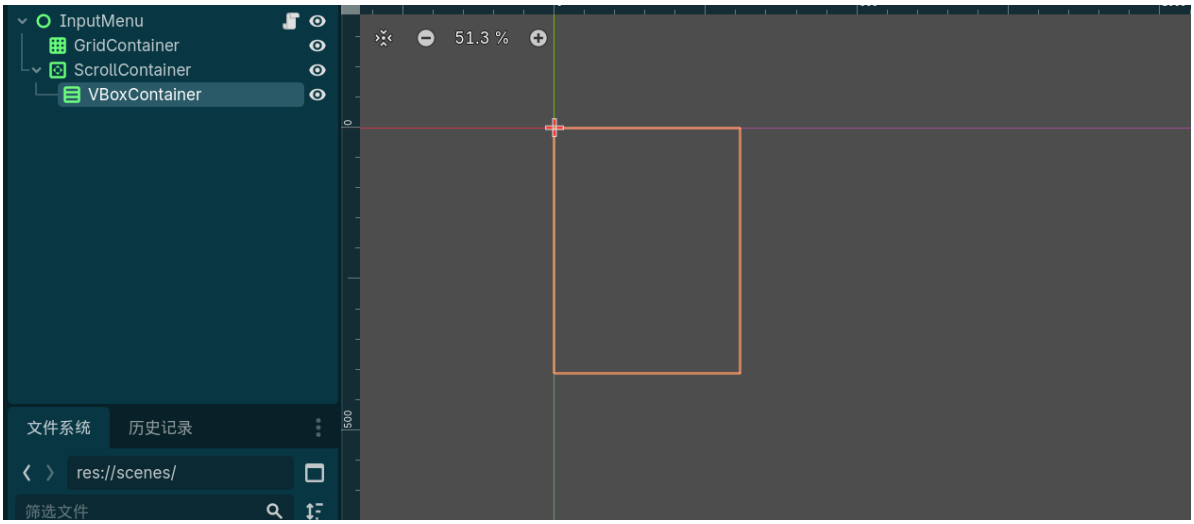
像这样提取、传递游戏内数据，流程会比较繁琐难懂，但只要一步步梳理，也不难理解。

5.列表数据容器搭建

除了网格按钮，我们还有物品列表，这个列表同样支持滚动，通过它，可以切换选中其它怪物或想要使用的道具。

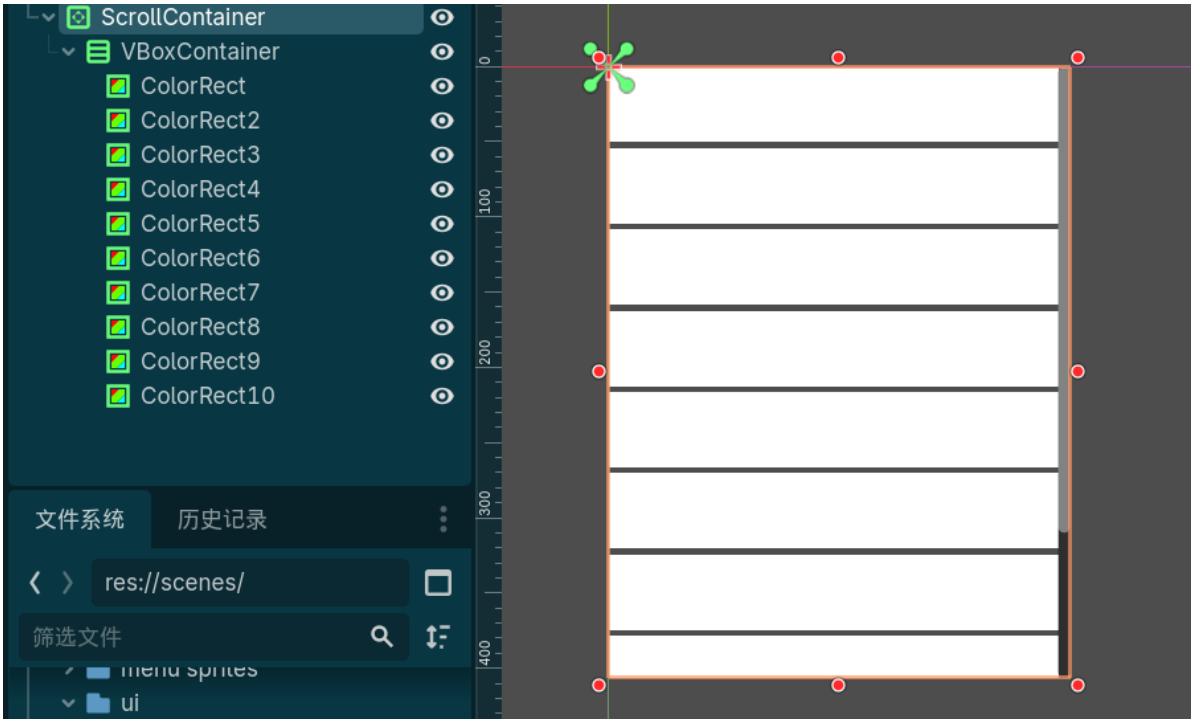
在InputMenu场景中新建ScrollContainer容器，创建完成后，Godot会提示该节点一般于VBox、HBox或Grid容器一起使用，这三种是最常用的容器，我们这里选择使用VBoxContainer作为其子节点。

拖动ScrollContainer到合适的大小，随后设置VBoxContainer横向和水平扩展，确保其填满整个容器。

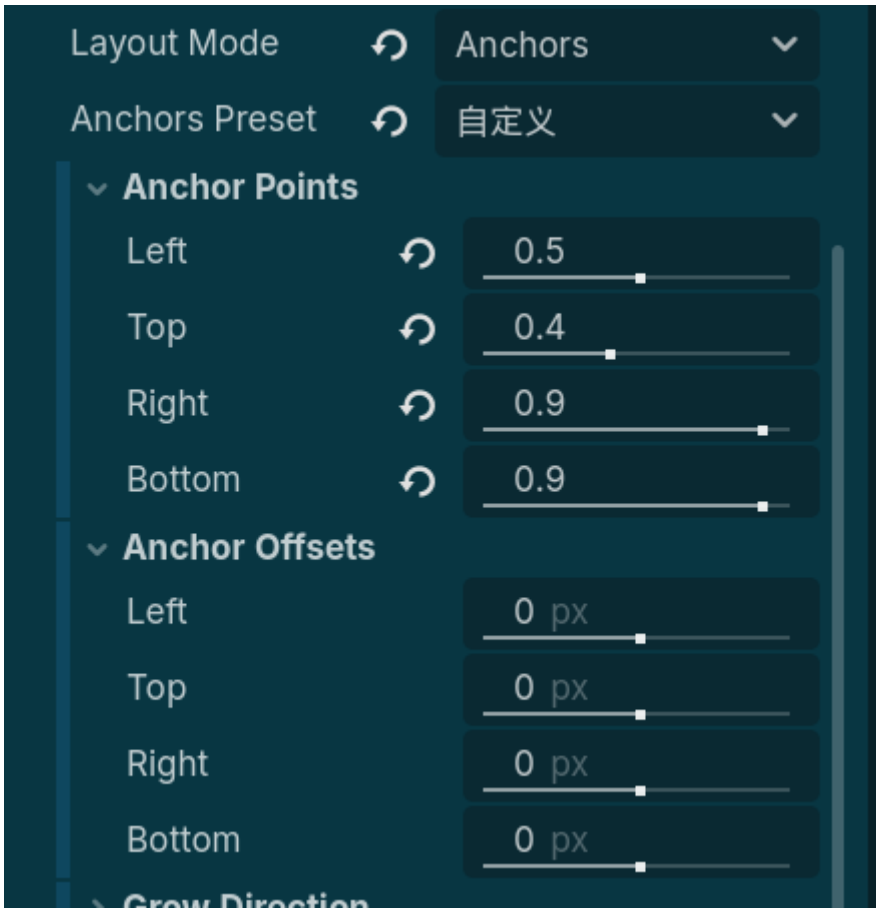


接下来为ScrollContainer添加展示用的元素，我们这里先使用ColorRect作为演示即可。

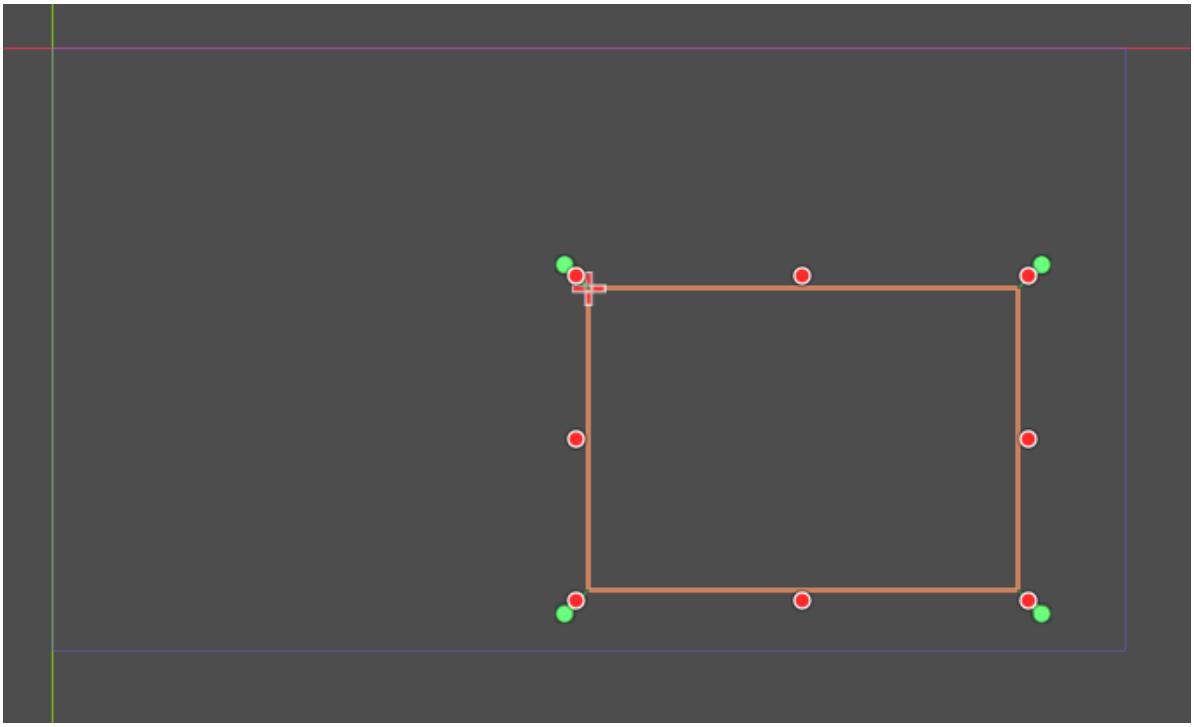
使用ColorRect时要注意，使用可滚动容器时不要为它开启水平扩展，而是手动为其设置自定义最小尺寸。我这里设置y轴最小尺寸为50。多次复制这个矩形节点，达到一定数量以后，界面就会出现滚动条，如图。



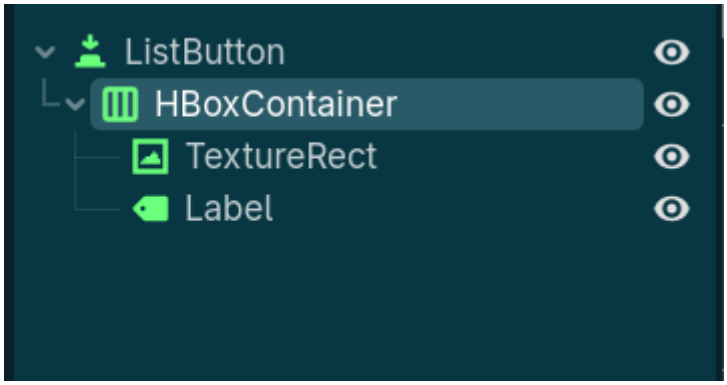
接下来将ScrollContainer中的矩形节点改为按钮，先将所有矩形节点删掉，随后选中ScrollContainer节点，定位模式设置为锚点，清除所有偏移量，锚点位置调整如图。



这样，ScrollContainer就会大致在右下角位置，如图。



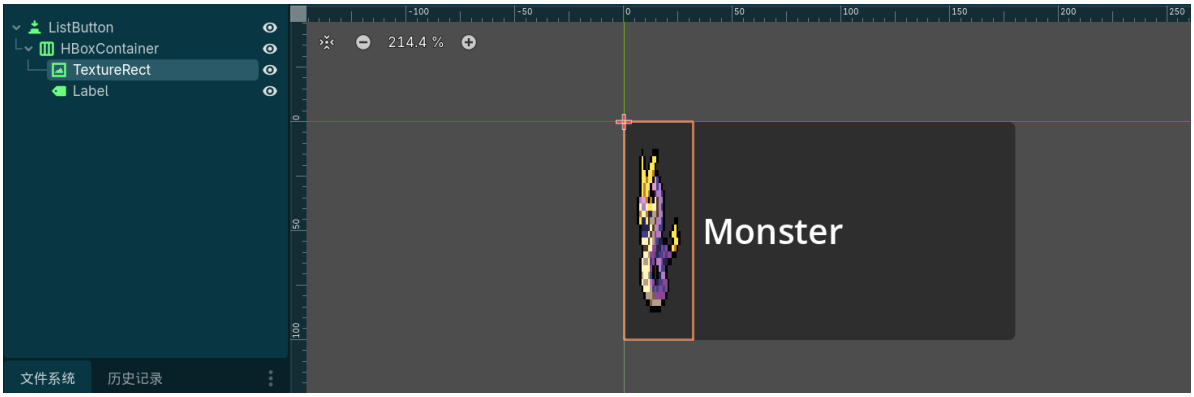
现在新建一个场景，根节点为Button，命名为ListButto。ScrollContainer内的按钮样式包含怪物的贴图和名称，因此在场景内新增一个HBoxContainer，其下新增TextureRect和Label。



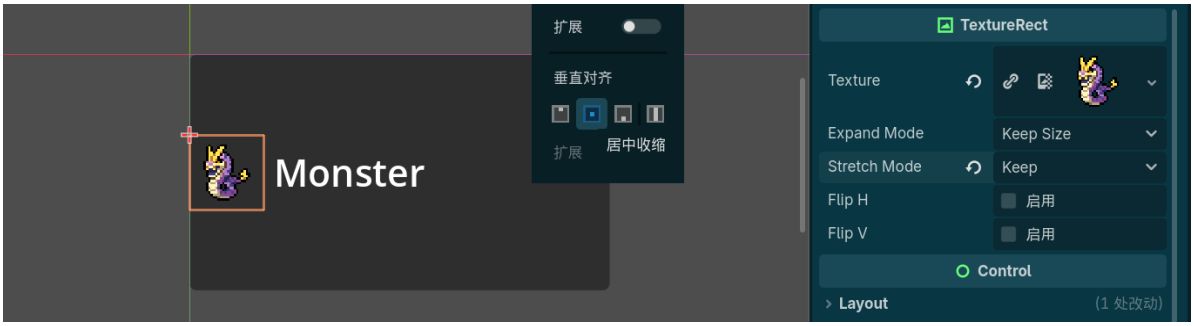
为了观感更好，设置ListButton的最小大小，如图。



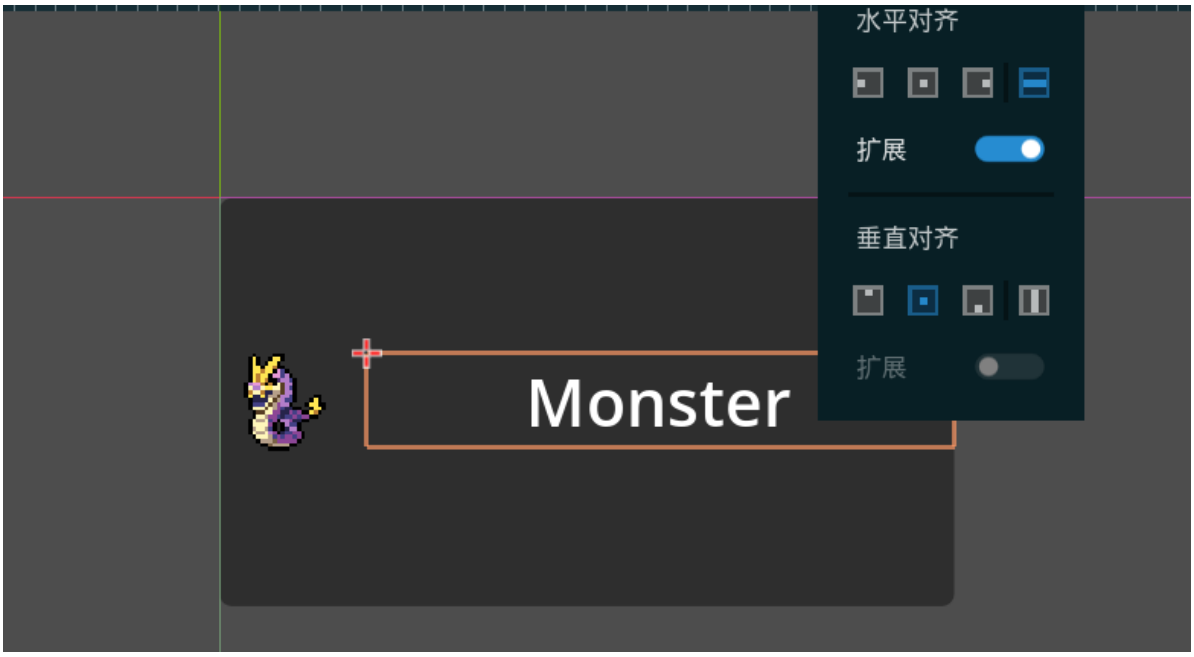
HBoxContainer的预设模式设置为铺满，确保其填满整个ListButton。为TextureRect添加临时贴图，这里选择res://graphics/menu sprites/Atrox.png，Label文字暂时设置为Monster，如图。



目前贴图的观感不好，我们选中TextureRect，检查器中设置其Stretch Mode为keep（保持图片比例），并设置垂直对齐模式为居中，如图。



至于Label，只需要设置其Horizontal Alignm（文字水平对齐模式）为居中，目前还看不出效果，是因为Monster占满了整个Label区域，我们将Label的水平拓展开启即可看出区别。



至此，我们制作完成了基础按钮，现在只需要将其生成到ScrollContainer内部的VBoxContainer即可。为ListButton挂载脚本。

list_button的逻辑和grid_button类似，区别主要在setup方法的内容。setup只需要两个参数menu_state和button_type，而文本只需要使用button_type作为参数，在全局变量monster_data内获取。

我们观察全局变量的monster_data，如下。

```
const monster_data: Dictionary[Monster, Dictionary] = {
    Monster.ATROX: {
        'name': 'Atrox',
        'back texture': "res://graphics/back sprites/Atrox.png",
        'front texture': "res://graphics/battle sprites/Atrox.png",
        'icon': "res://graphics/menu sprites/Atrox.png",
        'attacks': [Attack.CLAW, Attack.FIRE, Attack.HEAL, Attack.WATER],
        'max health': 60,
    },
    ......
```

而在我们设计中，列表按钮按下后传入的button_type应该是对应的怪物名称，我们可以利用button_type直接获取到'name'和'icon'，分别作为按钮文本和贴图，脚本内容如下。

```
var state
var type
signal press(state, type)

func setup(menu_state, button_type):
    var  data = Global.monster_data
    $HBoxContainer/Label.text = data[button_type]['name']
    $HBoxContainer/TextureRect.texture = load(data[button_type]['icon'])
    state = menu_state
    type = button_type

func _on_pressed() -> void:
    press.emit(state, type)
```

这段逻辑虽然有些复杂，但不算太难理解，本质就是做数据传输操作。

想要直观看到最终效果，我们依旧需要使用GDScript来批量生成按钮，我们在input_menu中新建一个函数create_list_buttons，在其中实现。

create_list_buttons有两个参数，一个是和create_grid_buttons一样的state:Global.State，用于传入菜单状态，另一个是data，我们打开Global脚本查看，这里的参数要么是怪物数据，要么是物品数据，这两类数据都是数组型，因此data只能是Array类型，如下。

```
    func create_list_buttons(state:Global.State, data:Array):
    pass
```

create_list_buttons函数中先获取$ScrollContainer/VBoxContainer的全部子节点并销毁，随后获取参数data的值并打印进行测试。

```
func create_list_buttons(state:Global.State, data:Array):
    for i in $ScrollContainer/VBoxContainer.get_children():
        i.queue_free()
    for key in data:
        print(key)
```

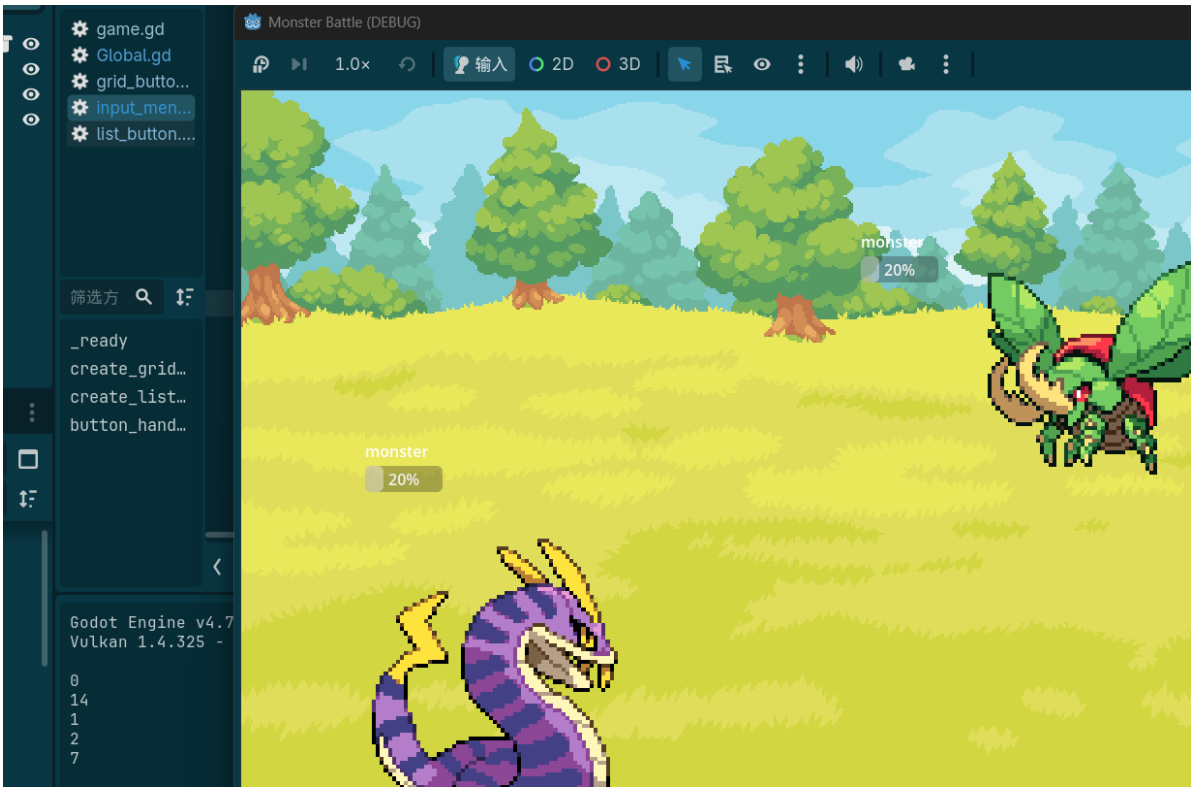
在ready函数中先将先前的语句注释，添加对create_list_buttons的调用语句，如下。

```
create_list_buttons(Global.State.SWAP,Global.monsters)
```

我们进入Global脚本查看Global.monsters的内容，它表示玩家拥有的怪物，如下。

```
var monsters = [Monster.ATROX, Monster.SPARCHU, Monster.CINDRILL, Monster.CLEAF, Monster.GULFIN]
```

运行游戏测试，正常输出了data中获取的Global.monsters的枚举值，如图。



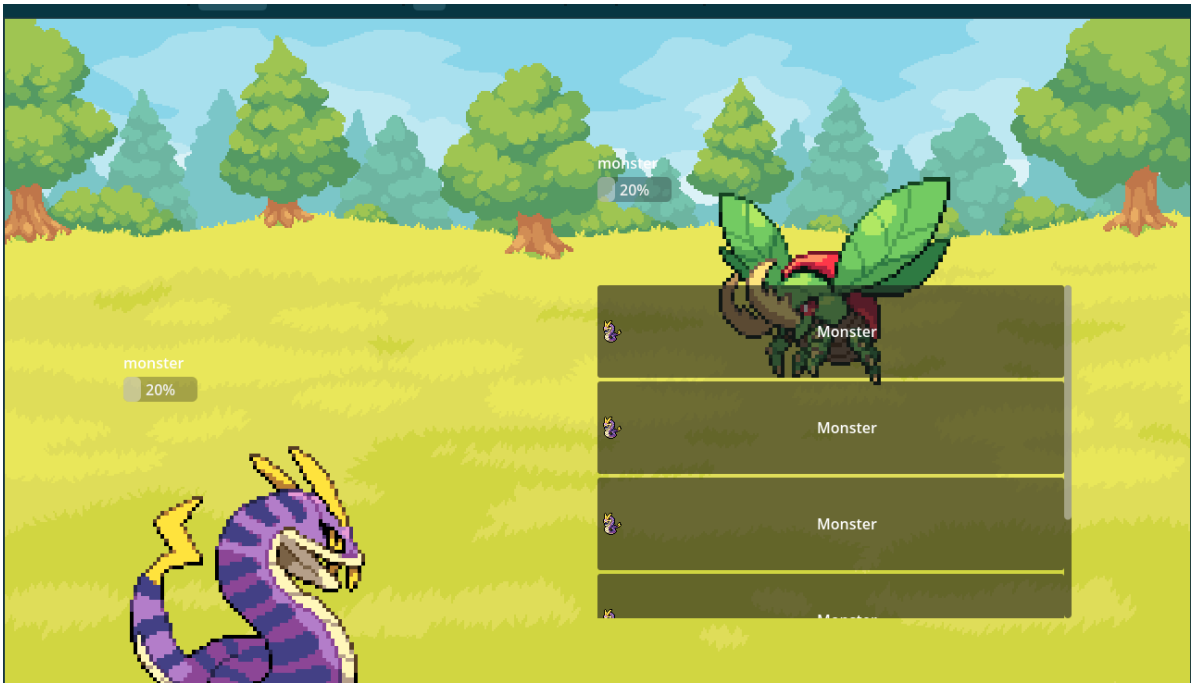
接下来要利用获取到的data数据，我们需要先在脚本内定义变量list_button_scene，值为预加载的ListButton场景，方便将其实例化到场景，如下。

```
var list_button_scene = preload("res://scenes/list_button.tscn")
```

在遍历data的循环内部，新建变量list_button，赋值为实例化的list_button_scene，将其直接添加为VBoxContainer的子节点，如下。

```
unc create_list_buttons(state:Global.State, data:Array):
    for i in $ScrollContainer/VBoxContainer.get_children():
        i.queue_free()
    for key in data:
        var list_button = list_button_scene.instantiate()
        $ScrollContainer/VBoxContainer.add_child(list_button)
```

运行游戏测试，界面正常生成对应按钮，如图。



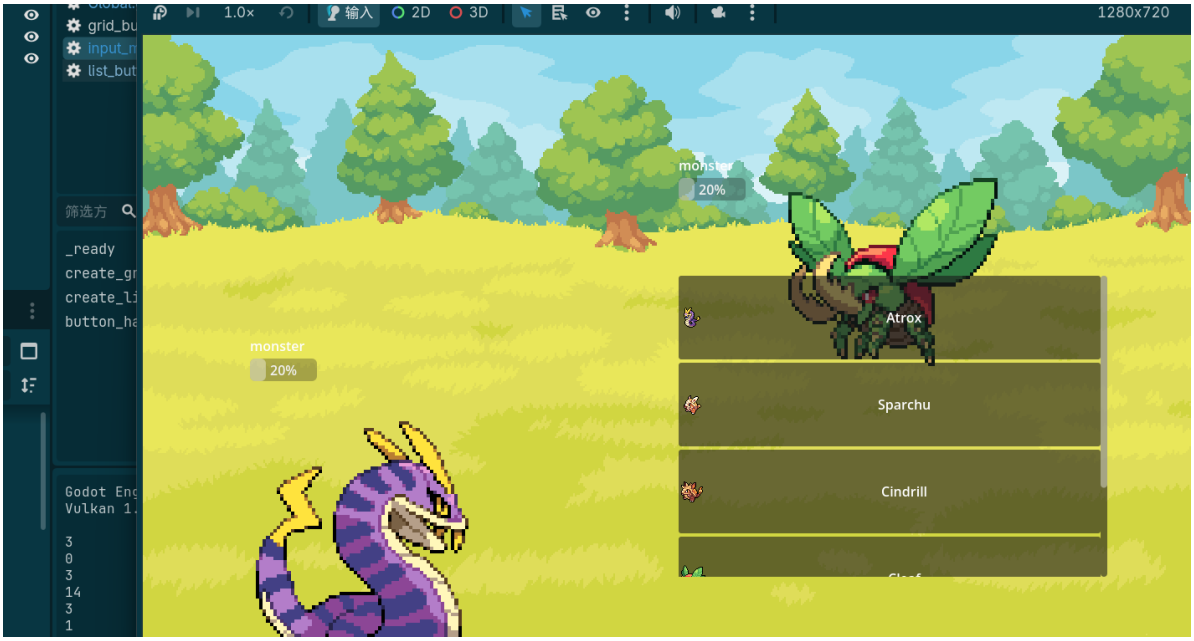
接下来我们先处理数据绑定逻辑，也就是给按钮匹配上正确的文字标签和图标贴图。调用list_button的setup函数，传入state和循环的key，也就是怪物枚举值，运行游戏测试，按钮数据显示已正常，如图。

```
list_button.setup(state, key)
```



现在每个按钮都会匹配对应怪物名称，整体逻辑运行稳定。

接下来还是在循环内，使用connect方法将list_button的press信号与button_handler函数绑。因为传入参数相同，button_handler同时适配网格按钮和列表按钮。这样，点击按钮后，就会在控制台输出按钮对应的菜单状态和怪物枚举值信息，如图。



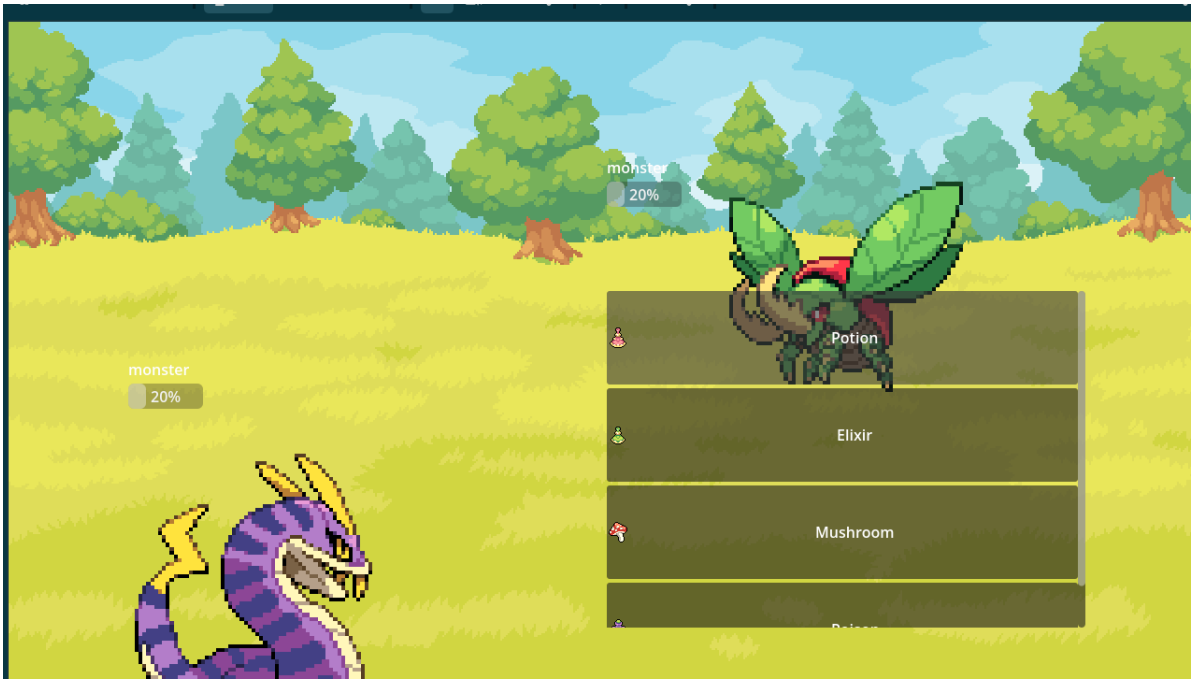
最后还需要处理一处逻辑：让列表按钮可以分别展示可用怪物或可用道具。create_list_buttons函数需要支持两种入参，SWAP状态搭配怪物集合，或是ITEM状态搭配道具集合，我们先注释ready函数中语句，添加如下语句。

```
create_list_buttons(Global.State.ITEM,Global.items)
```

处理道具属性时，list_button脚本中只需要修改一处，即data的值由Global.monster_data修改为Globa.item_data，我们这里使用条件判断来切换，如下。

```
var  data = Global.monster_data if menu_state == Global.State.SWAP else
Global.item_data
```

运行游戏测试，道具列表可以正常加载。



回到input_menu脚本，注释掉道具的create_list_buttons调用，取消怪物调用的注释，界面就会正常显示怪物的列表按钮。

6.状态管理逻辑

现在我们已经把所有按钮摆放到位，接下来就要给它们建立信号连接。只有这样，我们才能做出完整可用的菜单。

所有操作都在input_menu中完成，我们定义一个新变量current_state，用来记录当前菜单状态，这个变量值的类型为Global.State。菜单一共有如下5种状态。

```
enum State {MAIN, ATTACK, DEFEND, SWAP, ITEM}
```

一旦current_state发生变动，就要切换显示不同类型按钮，因此需要给这个变量绑定一个自定义函数，具体方法是在current_state变量后通过set关键字指定一个函数，这样current_state的值更新时，就会自动调用该函数，我们这里新建一个函数state_handler，专门用于处理状态变化。

```
var current_state: Global.State: set = state_handler
```

state_handler函数必须传入一个参数，否则Godot将会报错。原因是当current _state变化时，Godot会自动调用set关键字函数，更新变量自身数值，引擎默认不会自动更新，想要实现自动更新，就要读取current_state并将传入state_handler函数的参数赋值给current_state，如下。

```
var current_state: Global.State: set = state_handler

func state_handler(value): #更新current_state的值
    current_state = value
    print(value)
```

本质上这就是将state_handler作为变量的setter函数。

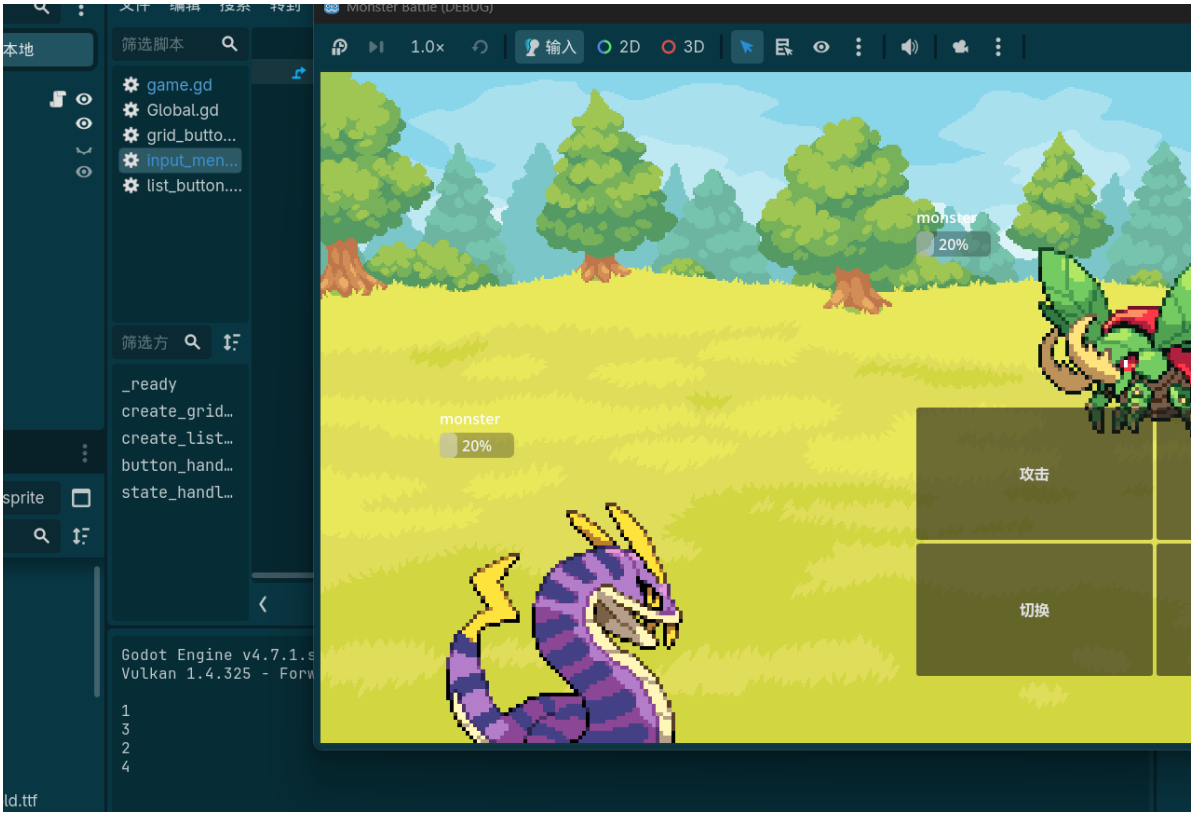
接下来我们在button_handler中给各个按钮添加实际交互逻辑，删除先前的打印语句，改为如下语句。

```
func button_handler(state, type):
    if state == Global.State.MAIN:
        current_state = type
```

想要生成这些按钮，先将InputMenu下的ScrollContainer切换为不可见，再在ready函数中调用create_grid_buttons进行生成，如下。

```
func _ready() -> void:
    create_grid_buttons(Global.State.MAIN,main_buttons)
```

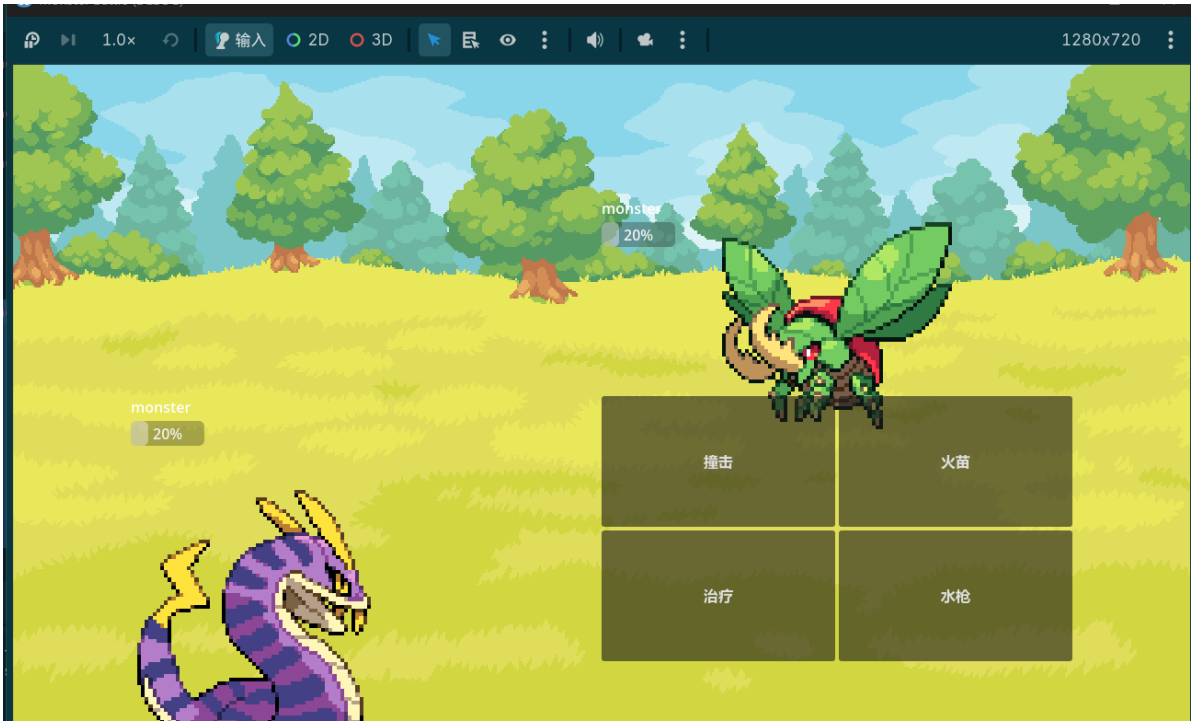
这样，运行游戏按下按钮后，控制台会输出按钮对应的状态枚举值，如图。



接下来我们还是在state_handler中处理状态，我们将先前_ready函数中注释的攻击按钮处理逻辑剪切到state_handler，当传入的value为Global.State.ATTACK时执行，如下。

```
func state_handler(value):
    current_state = value
    if value == Global.State.ATTACK:
        var monster_attacks = Global.monster_data[Global.current_monster]
['attacks']
        create_grid_buttons(Global.State.ATTACK, {
        monster_attacks[0]: Global.attack_data[monster_attacks[0]]['name'],
        monster_attacks[1]: Global.attack_data[monster_attacks[1]]['name'],
        monster_attacks[2]: Global.attack_data[monster_attacks[2]]['name'],
        monster_attacks[3]: Global.attack_data[monster_attacks[3]]['name'],
    })
```

运行测试，按下攻击按钮后，网格按钮会切换为技能，如图。



当然，此时点击技能不会有反应，接下来实现点击效果还需要补充大量if判断语句。

我们已经实现了ATTACK的切换，还有三种状态，其中SWAP和ITEM是列表按钮，因此我们还要添加对网格按钮和列表按钮的可见性处理，防止同时出现多种按钮相互叠加，如下。

```
func state_handler(value):
    current_state = value
    if value == Global.State.ATTACK:
        $GridContainer.show() #网格按钮可见
        $ScrollContainer.hide() #列表按钮不可见
        var monster_attacks = Global.monster_data[Global.current_monster]
['attacks']
        create_grid_buttons(Global.State.ATTACK, {......
```

考虑到每个状态切换都需要判断一次，这里使用if判断并不合适，我们改为使用match匹配分支，格式如下。

```
match value:
        Global.State.ATTACK:
            $GridContainer.show()#状态为ATTACK时处理逻辑
            $ScrollContainer.hide()
            var monster_attacks = Global.monster_data[Global.current_monster]
['attacks']
            create_grid_buttons(Global.State.ATTACK, {
            monster_attacks[0]: Global.attack_data[monster_attacks[0]]['name'],
            monster_attacks[1]: Global.attack_data[monster_attacks[1]]['name'],
            monster_attacks[2]: Global.attack_data[monster_attacks[2]]['name'],
            monster_attacks[3]: Global.attack_data[monster_attacks[3]]['name'],
            })
        Global.State.MAIN:
            ...#状态为MAIN时处理逻辑
        Global.State.SWAP:
            ...#状态为SWAP时处理逻辑
        Global.State.ITEM:
            ...#状态为ITEM时处理逻辑
```

像上面一样，使用match指定变量，下面是变量为不同值时的处理逻辑，配合缩进，就可以代替大量if语句进行条件判断。

类似于其他语言的switch，善用match可以使代码更加简洁易读。

接下来处理MAIN状态，和ATTACK类似，如下。

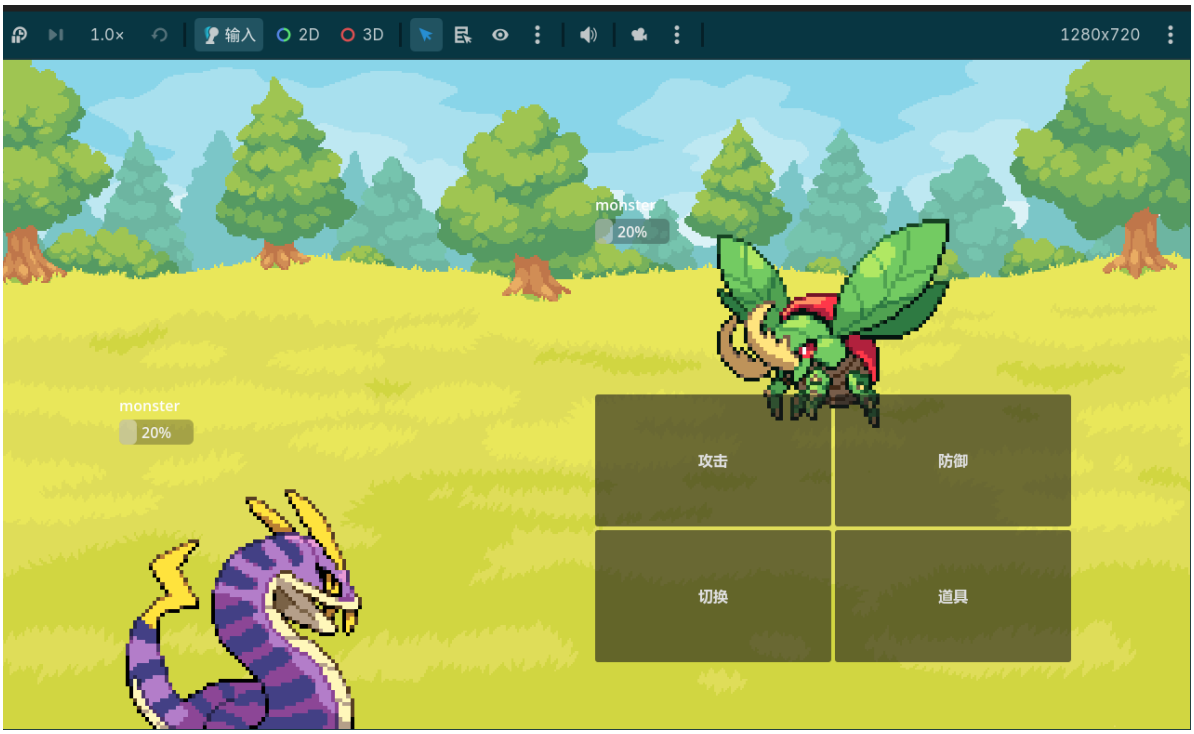
```
Global.State.MAIN:
            $GridContainer.show()
            $ScrollContainer.hide()
            create_grid_buttons(Global.State.MAIN,main_buttons)
```

SWAP和ITEM只是略有不同，如下。

```
Global.State.SWAP:
            $GridContainer.hide()
            $ScrollContainer.show()
            create_list_buttons(Global.State.SWAP,Global.monsters)

        Global.State.ITEM:
            $GridContainer.hide()
            $ScrollContainer.show()
            create_list_buttons(Global.State.ITEM,Global.items)
```

DEFEND状态仅需简单逻辑，不需要改变菜单按钮。至此，我们仅用一个函数就完成了状态管理逻辑。此时ready函数中为空白，我们在其中将current_state的初始值设置为Global.State.MAIN，确保游戏运行后初始菜单状态为MAIN，随后运行游戏测试，初始菜单正常，从主菜单切换到其他状态功能正常，如图。



还有一个缺陷，目前我们只能从主菜单切换到其他菜单，无法回到主菜单。接下来实现菜单返回功能。

我们新建_input函数，任何按键操作都会触发这个内置函数，函数会接收event事件对象，我们可以通过事件对象处理交互。这里我们暂时用不到，先添加下划线。

在input函数中仅做一个判断，若按下Escape键，就将current_state的值设置为Global.State.MAIN，Godot内置动作中已将Escape键绑定动作“ui_cancel”，代码如下。

```
func  _input(_event: InputEvent) -> void:
    if Input.is_action_just_pressed("ui_cancel"):
        current_state = Global.State.MAIN
```

运行游戏测试，可以正常从其他菜单返回MAIN菜单。

至此，状态管理逻辑的全部功能完成。

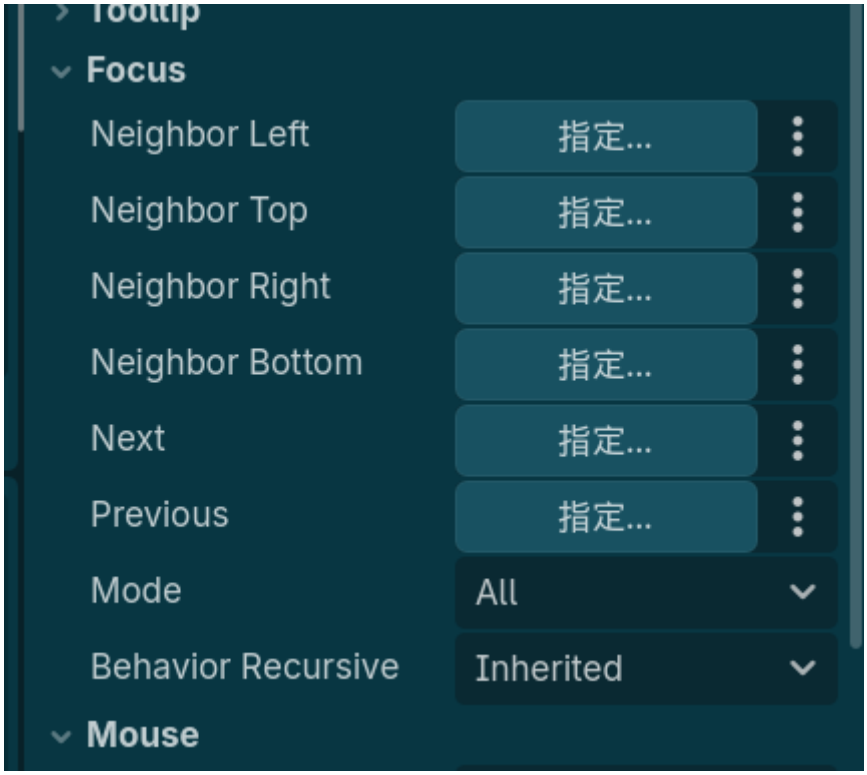
下

7.UI焦点交互

UI界面的制作已经基本完成了，但目前我们想要操作，只能通过鼠标点击按钮的方式，但显然宝可梦类游戏正常情况下是搭配手柄进行操作，再不济也该支持键盘操作。在Godot中该如何实现这类输入捕获?

我们运行游戏后，将鼠标移动到按钮上，再按下回车或空格，就可以触发按钮操作，这是因为所有按钮或可交互控件都带有一个focus属性，只要控件处于焦点选中状态，我们就能通过键盘、手柄输入来选中它，Godot会自动识别上下左右相邻的其他交互元素。

举个例子，打开Button节点对应的检查器面板，面板中就能找到focus相关设置，如图。

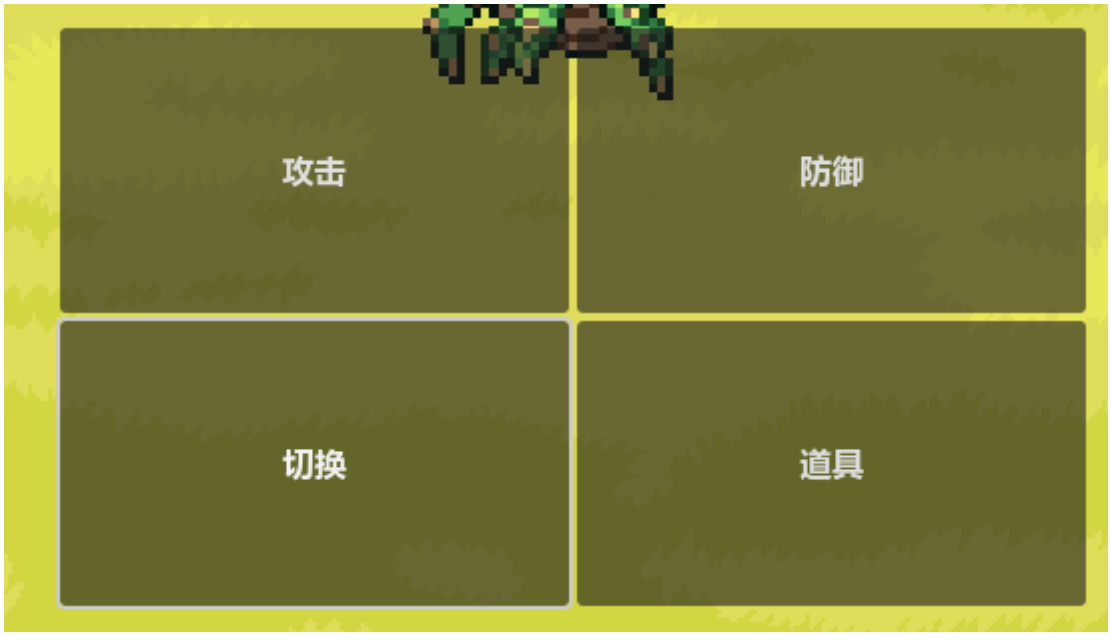


我们可以手动配置上下左右四个方向对应的相邻控件，还能设置切换上一项、下一项对应的控件。这里的下一项，就是我们按下Tab后会跳转选中的控件。

这套导航系统功能很强，但在我们的项目中目前用不上。我们真正需要实现的功能只有一个，就是在所有按钮加载完成后，自动给其中一个控件赋予焦点。举个例子，在InputMenu场景脚本中，我们在grid_buttons、list_buttons中批量生成按钮时，等待所有按钮创建完毕后，我们要获取其中一个按钮并为其设置焦点。

拿grid_buttons举例，我们只需要先拿到GridContainer节点，再获取它索引值为0的第一个子节点，并调用grab_focus函数，该按钮就会获取焦点，我们就能用键盘切换选中各个按钮，并按回车或空格触发，如下。

```
func create_grid_buttons(state:Global.State, data:Dictionary):
    for i in $GridContainer.get_children():
        i.queue_free()
    for key in data:
        var grid_buttton = grid_button_scene.instantiate()
        grid_buttton.setup(state,key,data[key])
        $GridContainer.add_child(grid_buttton)
        grid_buttton.connect('press', button_handler)
    $GridContainer.get_child(0).grab_focus() #为第一个按钮获取焦点
```



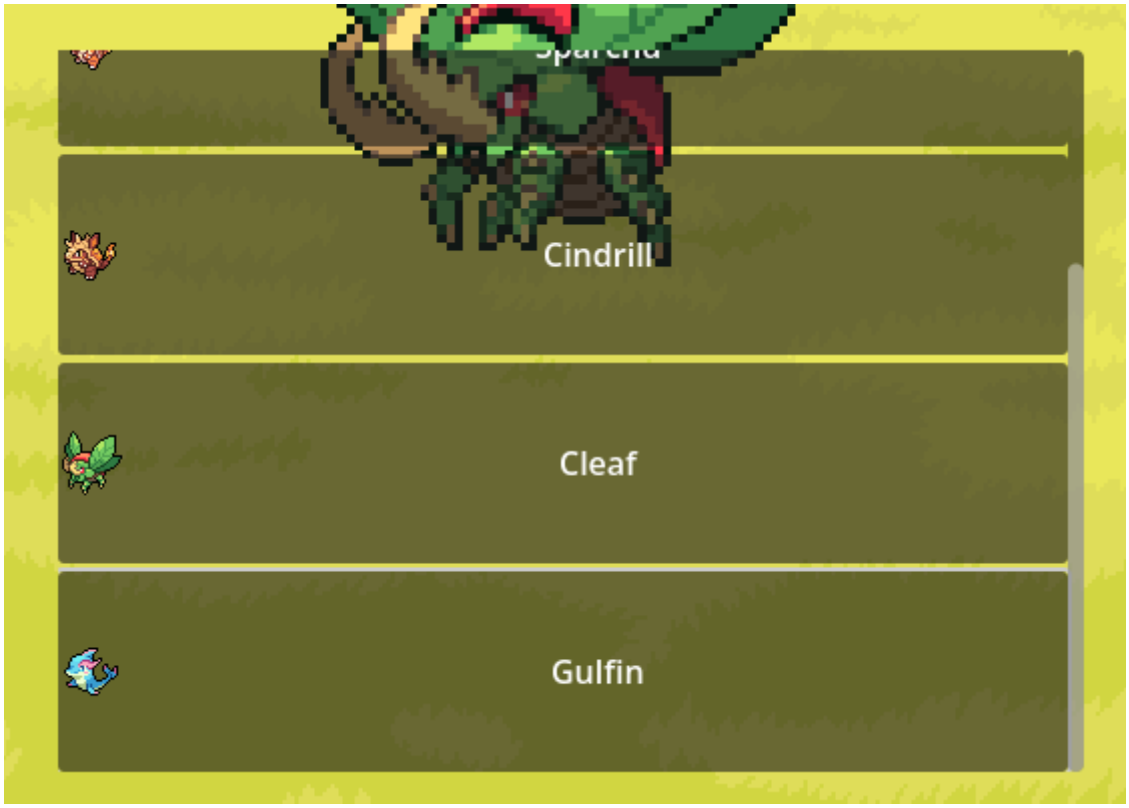
但按下回车触发按钮，跳转到下一级菜单后，此时菜单又失去了焦点，这是因为Godot生成这些按钮时，需要一小段时间完成内部处理逻辑，所以我们执行获取内部焦点的代码时，这些按钮还未完全加载生成完毕，导致焦点赋值的逻辑失效。解决这个问题的办法就是使用await等待，搭配process_frame函数，这样代码会暂停执行，等待当前帧全部渲染完成后再继续运行，如下。

```
func create_grid_buttons(state:Global.State, data:Dictionary):
    for i in $GridContainer.get_children():
        i.queue_free()
    for key in data:
        var grid_buttton = grid_button_scene.instantiate()
        grid_buttton.setup(state,key,data[key])
        $GridContainer.add_child(grid_buttton)
        grid_buttton.connect('press', button_handler)
    await get_tree().process_frame
    $GridContainer.get_child(0).grab_focus()
```

再次运行游戏，无论是切换到下级菜单，还是按下Esc返回MAIN菜单，焦点交互始终正常。针对list_buttons也要做相同处理。直接复制最后两行代码粘贴到create_list_buttons函数末尾，唯一需要修改的是将节点改为$ScrollContainer/VBoxContainer，如下。

```
func create_list_buttons(state:Global.State, data:Array):
    for i in $ScrollContainer/VBoxContainer.get_children():
        i.queue_free()
    for key in data:
        var list_button = list_button_scene.instantiate()
        $ScrollContainer/VBoxContainer.add_child(list_button)
        list_button.setup(state, key)
        list_button.connect('press', button_handler)
    await get_tree().process_frame
    $ScrollContainer/VBoxContainer.get_child(0).grab_focus()
```

但此时运行还会发现一个问题，ScrollContainer内部的滚动条不会跟随焦点移动，修复方法也很简单：选中SrollContainer节点，在检查器中勾选Follow Focus（跟随焦点）选项。再次运行游戏，滚动条已能正常跟随焦点移动。



至此关于focus的知识点已基本讲解完毕，需要注意的是，之所以可以通过键盘按键来切换焦点，是因为引擎内置了对应的输入动作，Godot就是靠这些内置动作来切换焦点选中状态，如ui_left对应三套内置输入动作，如果想要自定义焦点切换逻辑，可以自行新增修改动作绑定按键，如图。



8.攻击力逻辑判定

输入菜单基本制作完成，我们要利用这些数 据实现各类交互逻辑，在游戏内部生效。我们先从基础功能入手，接下来我们先实现攻击逻辑。

我们需要监听按钮被点击触发的事件，为此input_ menu脚本中新建一个信号selected，如下。

```
signal selected(state: Global.State, type)
```

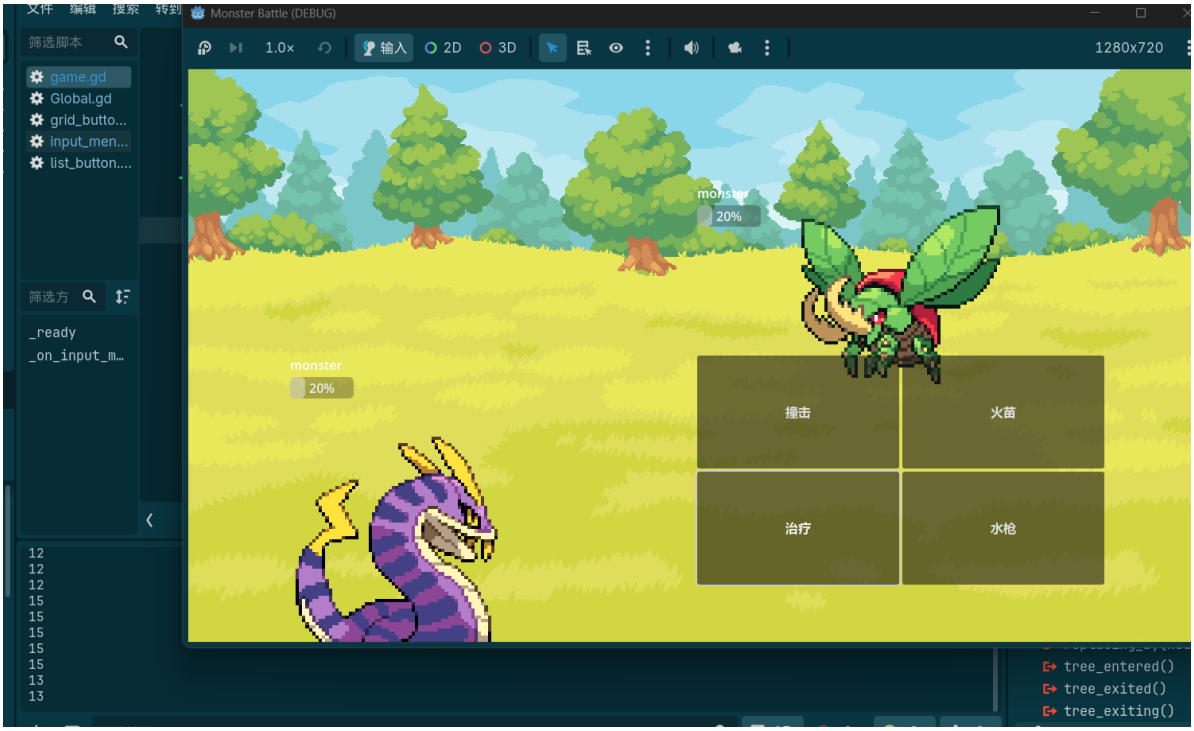
监听逻辑写在ButtonHandler内部，当前ButtonHandler只有在菜单状态为MAIN时才会生效，我们为其添加一个else分支，当其它菜单状态时触发，将发送一个信号到game场景，以此实现数据传递，如下。

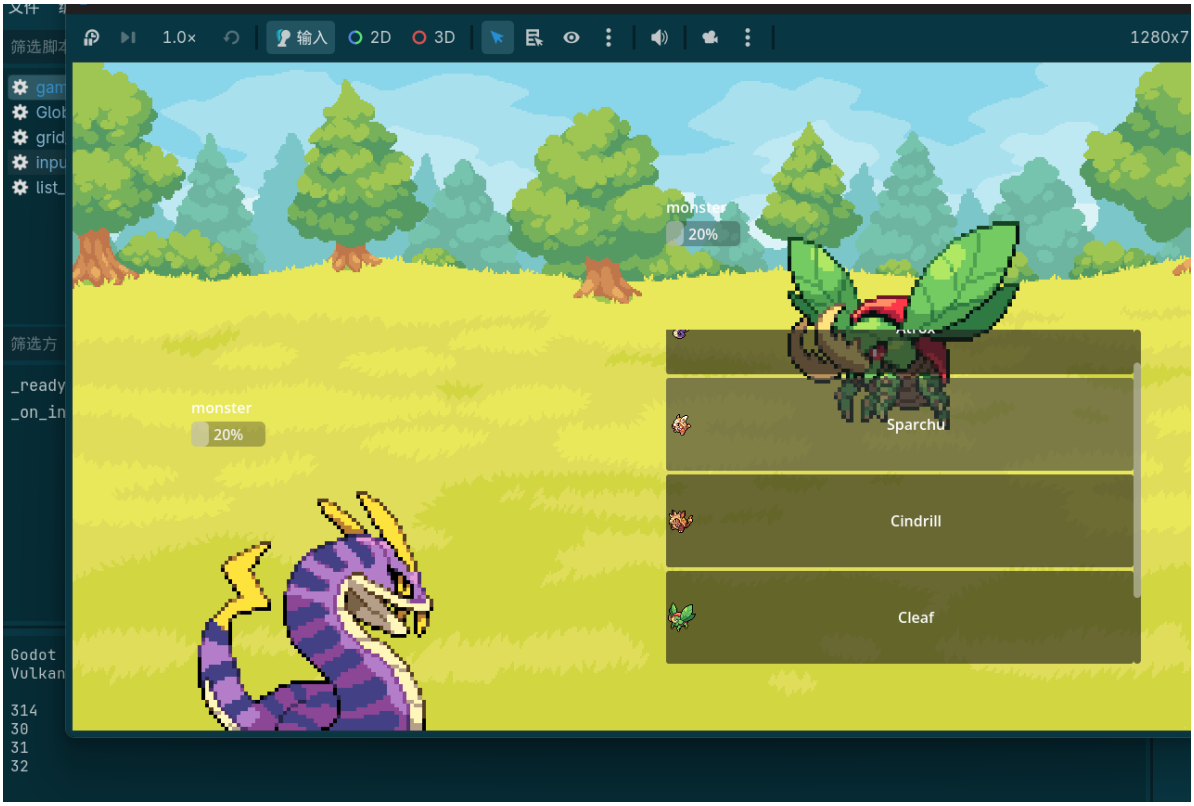
```
func button_handler(state, type):
    if state == Global.State.MAIN:
        current_state = type
    else:
        selected.emit(state,type)
```

完成信号定义后，我们来到游戏场景编写接收逻辑，游戏场景已挂载脚本，我们选中InputMenu节点后在信号面板选择selected信号，挂载到Game场景，为了直观，先让_on_input_menu_selected函数打印state和type，如下。

```
func _on_input_menu_selected(state: int, type: Variant) -> void:
    print(state,type)
```

运行效果如图，在下级面板触发按钮，打印的第一个数字是state，第二个数字是type，点击不同按钮输出不同数字，这就是我们需要的效果。





不过这里有一个特殊情况，当我们在主菜单，点击防御按钮时，不会有任何输出。因为按钮处理代码只会执行当前不在主界面的分支逻辑，但防御按钮属于主界面控件。

触发防御按钮时，我希望也触发selected信号并传递正确的参数，接下来完善这段逻辑，只需要在if分支中新增一个if逻辑判断即可，如下。

```
func button_handler(state, type):
    if state == Global.State.MAIN:
        current_state = type
        if type == Global.State.DEFEND:
            selected.emit(Global.State.DEFEND,type)
    else:
        selected.emit(state,type)
```

防御按钮本身不需要复杂逻辑。处理完防御输出后，我们来处理输入菜单选中信号的回调逻辑。

在game脚本中的_on_input_menu_selected函数中使用match来处理不同状态，首先处理ATTACK，需要先定义变量targe。

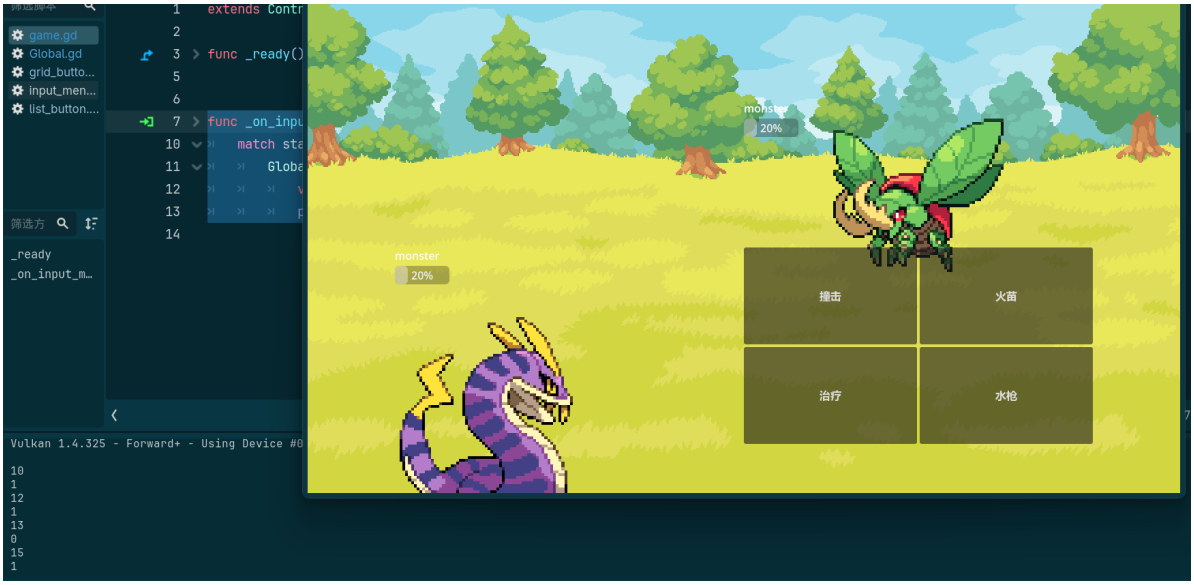
打开Global脚本可以在常量attack_data中找到target字段，target的值为1时表示技能目标为对方，为0时表示目标为自己，可以看到除了HEAL（治疗）外，其他技能的target值皆为1，如下。

```
const attack_data: Dictionary[Attack, Dictionary] = {
    Attack.CLAW:      {'name': '撞击',     'amount': 10, 'animation':
"res://graphics/attack effects/claw.png", 'target': 1},
    Attack.EXPLOSION: {'name': '爆炸','amount': 20, 'animation':
"res://graphics/attack effects/explosion.png", 'target': 1},
    Attack.FIRE:      {'name': '火苗',     'amount': 20, 'animation':
"res://graphics/attack effects/fire.png", 'target': 1},
    Attack.ICE:       {'name': '冰霜',      'amount': 20, 'animation':
"res://graphics/attack effects/ice.png", 'target': 1},
    Attack.WATER:     {'name': '水枪',    'amount': 20, 'animation':
"res://graphics/attack effects/water.png", 'target': 1},
    Attack.HEAL:      {'name': '治疗',     'amount': -20, 'animation':
"res://graphics/attack effects/heal.png", 'target': 0},
}
```

_on_input_menu_selected函数中target变量定义如下，先让其输出target值进行测试。

```
func _on_input_menu_selected(state: int, type: Variant) -> void:
    pass # Replace with function body.
    print(state,type)
    match state:
        Global.State.ATTACK:
            var target = Global.attack_data[type]['target']
            print(target)
```

运行游戏测试，可以看到正常打印了target值，如图。



判断逻辑没问题，但我们还需要对数据进行处理，我们要从怪物容器中取出玩家或敌方对应的TextureRect，用来播放攻击动画，target的作用就是决定是玩家还是敌方播放技能动画。

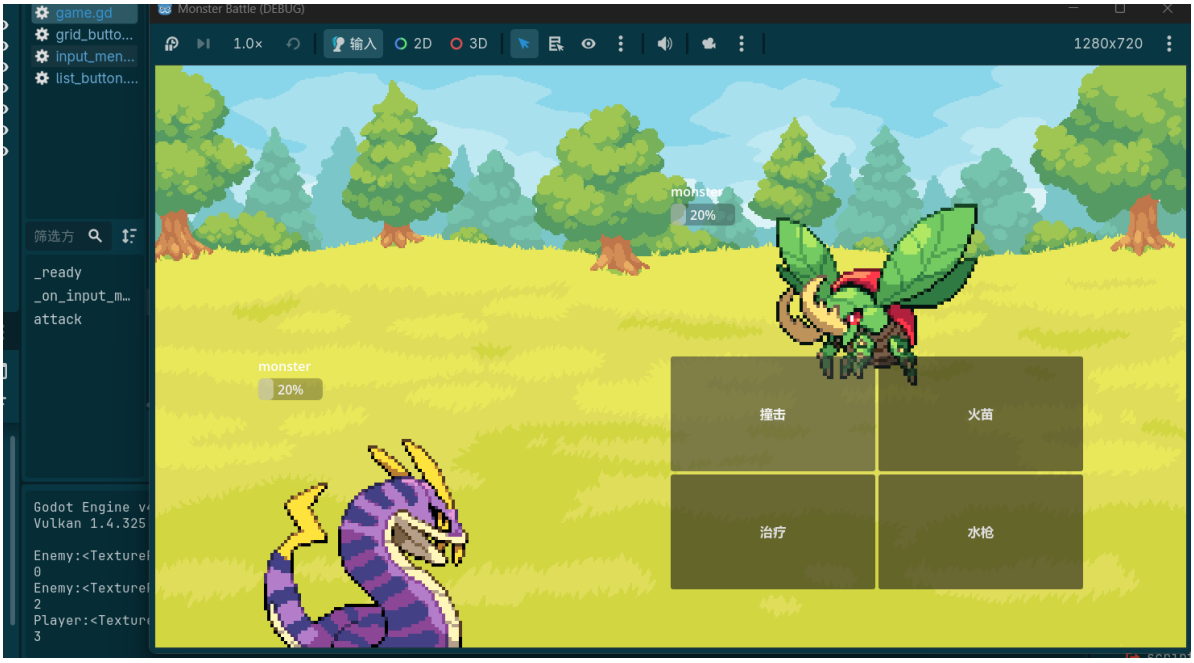
获取到对应的target后，调用attack函数，具体逻辑如下。

```
match state:
        Global.State.ATTACK:
            var target = $Monsters/Enemy if Global.attack_data[type]['target']
else $Monsters/Player
            attack(target, type)
```

目前还没有创建attack函数，所以在当前脚本中新建attack函数，两个参数分别为target: TextureRect和attack_type: Global.Attack，当前先让其打印两个参数，如下。

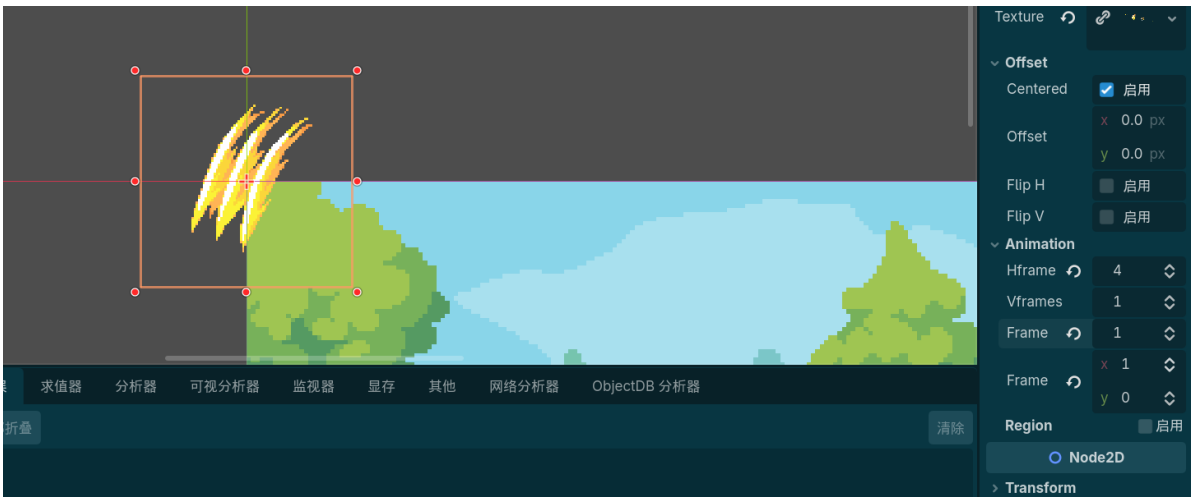
```
func attack(target: TextureRect, attack_type: Global.Attack):
    print(target)
    print(attack_type)
```

运行测试，触发对应攻击技能按键后分别输出目标的TextureRect和技能索引，如图。



接下来我们使用这些数据处理攻击动画，我在这里选用的方式是在Game场景挂载Sprite2D节点，能够在节点内部制作帧动画，只需要把控好坐标位置即可。

选择res://graphics/attack effects/claw.png作为sprite2D节点的贴图，素材一共包含4帧画面，因此横向帧数H frame固定填4，Vframe填1，之后就能通过设置frame值来循环切换每一帧画面，如图。



将这个Sprite2D命名为AttackSprite，方便区分节点用途。

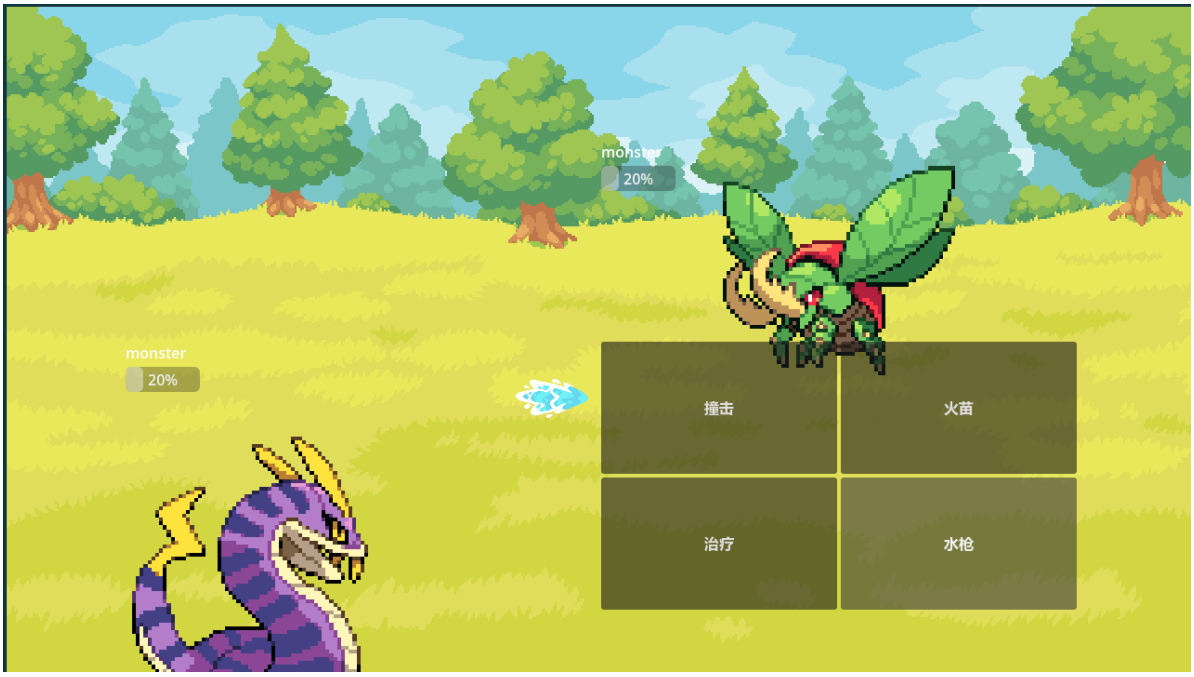
接下来编辑Attack函数内容，先获取AttackSprite节点，并将节点设置为显示状态，并将其当前帧数重置为0，保证动画永远从第一帧开始播放。

```
func attack(target: TextureRect, attack_type: Global.Attack):
    $AttackSprite.show()
    $AttackSprite.frame = 0
```

完成后我们加载对应的攻击贴图资源，也就是AttackSprite的Texture属性要与当前的攻击类型匹配，贴图参数可以从attack_data中通过attack_type参数获取，别忘了使用load函数加载贴图资源，如下。

```
func attack(target: TextureRect, attack_type: Global.Attack):
    $AttackSprite.show()
    $AttackSprite.frame = 0
    $AttackSprite.texture = load(Global.attack_data[attack_type]['animation'])
```

现阶段为了方便观察效果，将AttackSprite节点先挪到屏幕正中间，方便观察，运行游戏测试，点击不同攻击按钮，贴图会正确切换，如图。



接下制作动画，这段动画逻辑较为简单，不需要新建AnimationPlayer，直接使用Tween即可。

具体写法如下，创建tween对象后操作AttackSprite的frame属性从0到3，总时长0.6秒。

```
$AttackSprite.show()
    $AttackSprite.frame = 0
    $AttackSprite.texture = load(Global.attack_data[attack_type]['animation'])
    var tween = create_tween()
    tween.tween_property($AttackSprite, 'frame', 3, 0.6).from(0)
```

点击各攻击按钮，对应动画已经可以正常完整播放。

还需要调节几处细节，第一是动画尺寸，目前动画大小太小了，我们设置AttackSprite的Transform中的Scale属性为4，调整完后视觉效果会更好，如图。



除此之外，AttackSprite节点需要默认隐藏，也就是场景加载完成后，在ready函数中设置AttackSprite节点为隐藏，如下。

```
func _ready() -> void:
    Global.current_monster = Global.monsters[0]
    $AttackSprite.hide()
```

整套动画播放完毕后，也要将动画节点设置为隐藏，这里如果直接修改节点的visible属性，会导致动画无法正常播放，因此这里使用tween实现，在动画结束后立刻将其隐藏，如下。

```
func attack(target: TextureRect, attack_type: Global.Attack):
    $AttackSprite.show()
    $AttackSprite.frame = 0
    $AttackSprite.texture = load(Global.attack_data[attack_type]['animation'])
    var tween = create_tween()
    tween.tween_property($AttackSprite, 'frame', 3, 0.6).from(0)
    tween.tween_property($AttackSprite, 'visible', false, 0)
```

运行游戏测试，动画一开始不可见，触发攻击技能按钮后播放动画，每次触发只播放一次，播放完毕后动画节点自动隐藏，整体效果很不错。

最后还要解决一个问题，攻击特效的坐标定位。实现方也很简单，所有容器控制节点都自带一个很实用的函数：get_rect，该函数会返回一个Rect2，包含容器节点的坐标与尺寸，也就是这个控制节点的边界数据，我们可以依靠这份数据定位AttackSprite。

在attack函数中，执行补间动画之前调用get_rect函数获取target的边界数据，现阶段先打印数据，运行游戏后依次点击四个技能，控制台输出如图。



控制台输出包含坐标和尺寸两项数据，后一项数据表示所有TextureRect的尺寸都被统一设置为400*400，这是在Player和Enemy节点中先前设置的最小尺寸，如图。



而输出前一组数据代表矩形的左上角坐标，我们基于这个坐标摆放攻击特效贴图。先新建变量pos，赋值为get_rect返回值取出的位置坐标，随后将AttackSprite的位置值赋值为pos，如下。

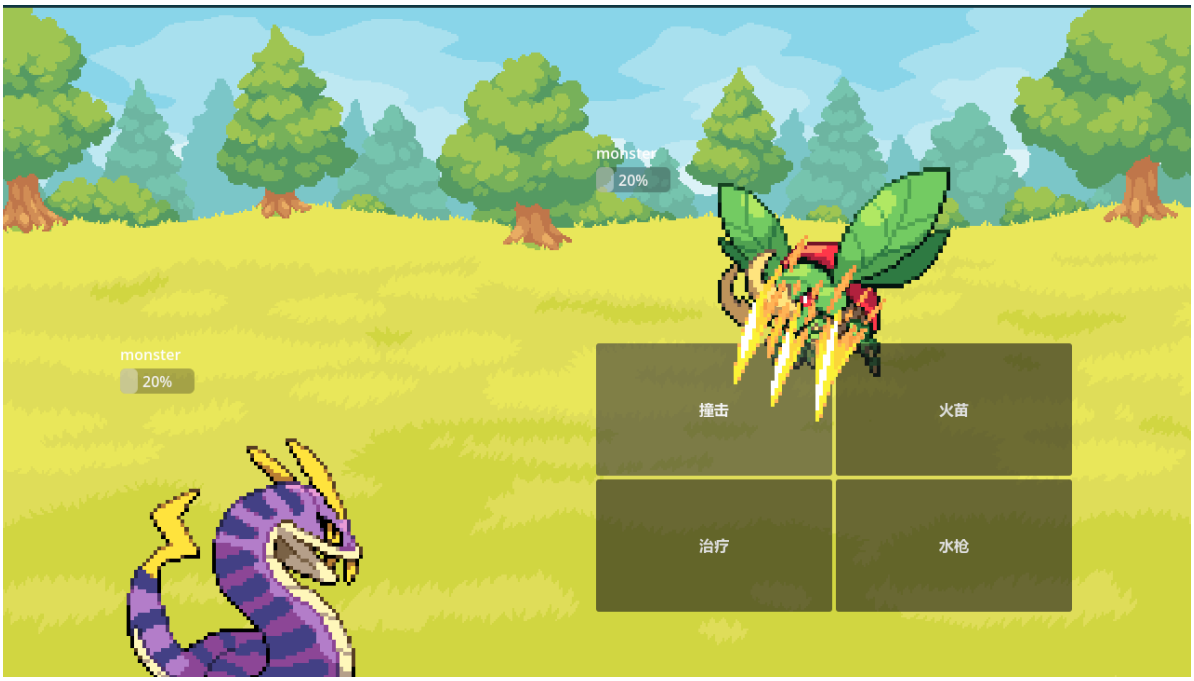
```
var pos = target.get_rect().position
$AttackSprite.position = pos
```

运行游戏测试，此时动画贴图位置已经接近理想位置，但仍有些偏移，因为get_rect返回的坐标是矩形左上角坐标，因此贴图位置偏左上，但我们需要特效居中显示。

处理方法也很简单，在原有的position基础上再加上矩形宽的一半、高的一半，如下。

```
var pos = target.get_rect().position + target.get_rect().size / 2
$AttackSprite.position = pos
```

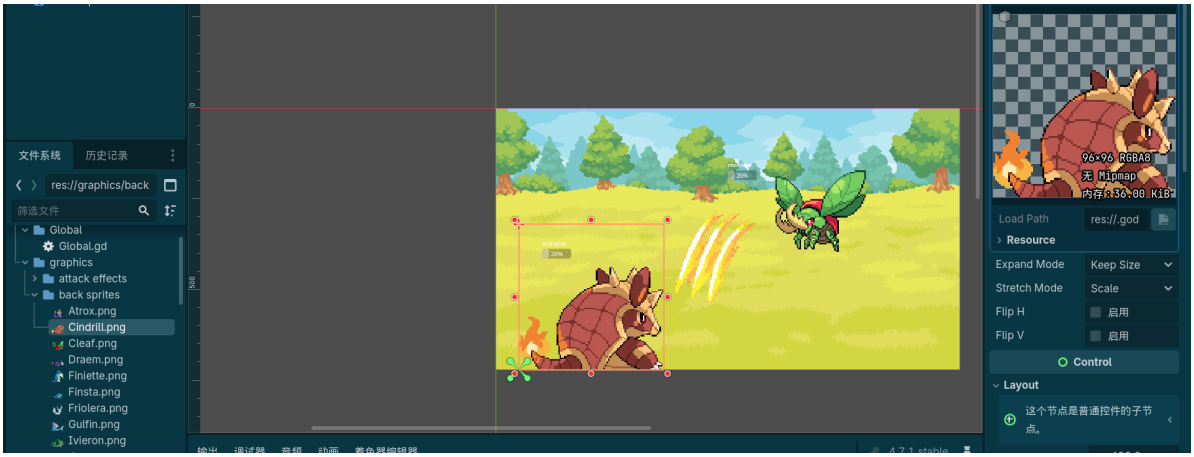
运行游戏测试，现在无论使用什么技能，特效都会精准的出现在怪物的正中心，如图。



9.怪物切换

我们接下来要实现的游戏机制是切换怪物形象，目前只针对玩家生效。

我们本质上只需要修改玩家TextureRect的纹理，举个例子，现在我们使用的是Atrox贴图，但我们可以打开图形资源文件夹，找到back sprites文件夹，替换成任意其他怪物，如图。



在游戏中理解就是，当我们触发输入菜单选中事件时，我们要在_on_input_menu_selected函数中判断另一个状态，也就是Global.State.SWAP，如果满足这个条件，我们先打印语句，如下

```
match state:
        Global.State.ATTACK:
            var target = $Monsters/Enemy if Global.attack_data[type]['target']
else $Monsters/Player
            attack(target, type)
        Global.State.SWAP:
            print("swap monster")
```

运行游戏测试，进入切换菜单后点击，控制台正常输出“swap monster”，如图。



这算是开了个好头，现在要做的是在这段代码中修改怪物的显示纹理。这段逻辑并不复杂，具体代码如下。

```
Global.State.SWAP:
    $Monsters/Player.texture = load(Global.monster_data[type]['back texture'])
```

完成后运行游戏测试，在切换菜单中选择不同的怪物，玩家怪物贴图也会切换为对应的怪物。

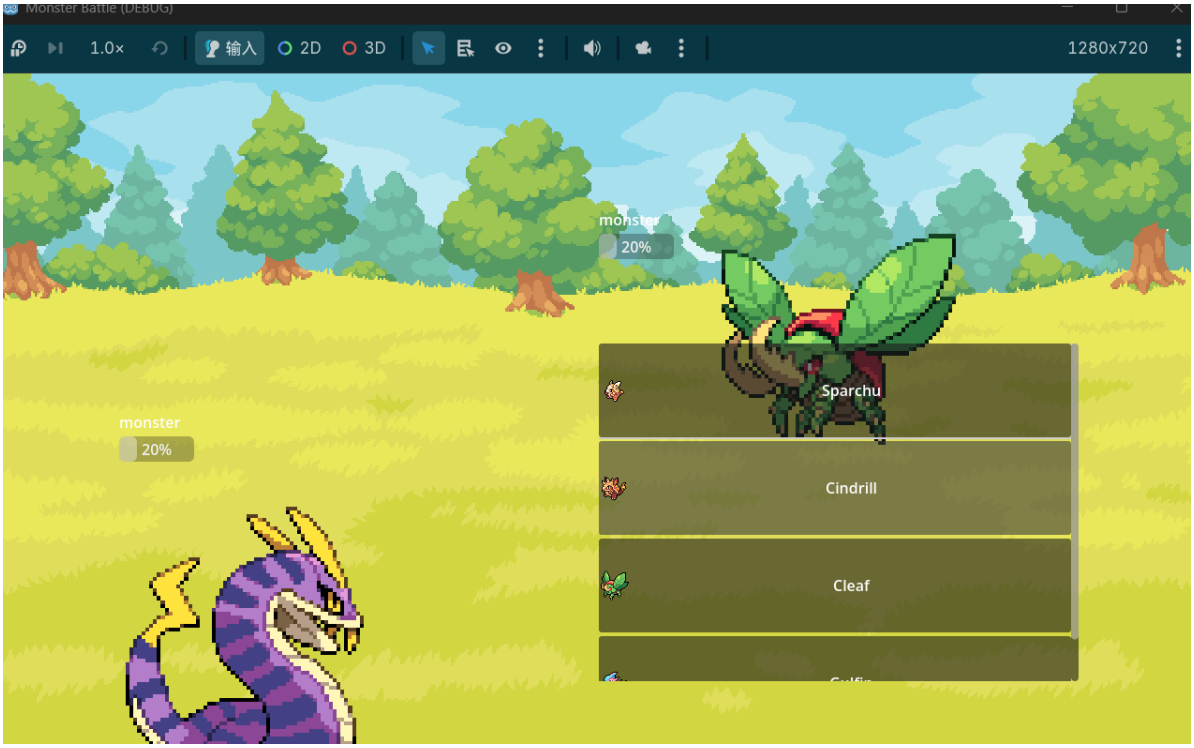
但是这里还有一个问题，游戏运行后，不会加载当前默认怪物贴图，而是显示Player节点上预设的贴图，我们需要在ready函数中也执行贴图加载语句，只是将type修改为Global.current_monster，如下。

```
$Monsters/Player.texture = load(Global.monster_data[Global.current_monster]
['back texture'])
```

这样游戏启动后就能加载正确的初始怪物，这一步效果没问题。但还有一个点，现在初始怪物是Atrox，但点开切换面板后，可以重复选择这个怪物，原因是我们只读取数组中第一个元素，但未将其从数组中移除，因此我们不要直接使用索引取值，改用pop方法读取索引0的元素，如下。

```
func _ready() -> void:
    Global.current_monster = Global.monsters.pop_at(0)#使用pop_at方法，将元素移出数组
```

现在，pop_at方法会取出索引0的值并将它从数组中移除，取出的值存入current_monster变量，现在再打开切换面板，初始怪物就不会出现在列表里，如图。



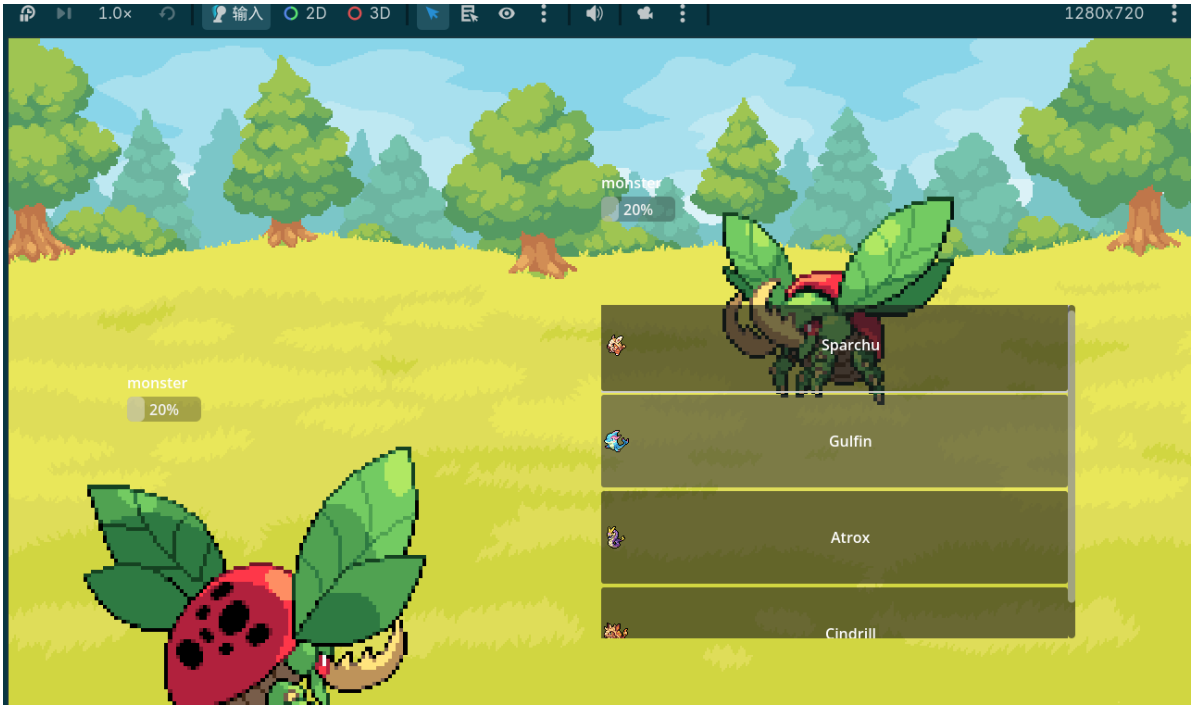
为了防止数据丢失，切换成其他怪物后无法选择Atrox，我们在输入菜单选择函数的SWAP处理中需要在切换成别的怪物之前调用append方法将current_monster记录的怪物存入Global.monster数组。为了攻击技能能够对应当前怪物，还需要将Global.current_monsters修改为当前传入的type，如下。

```
Global.monsters.append(Global.current_monster)
$Monsters/Player.texture = load(Global.monster_data[type]['back texture'])
Global.current_monster = type
```

此时运行游戏，会发现切换切换怪物后，怪物仍然会出现在列表中，我们需要再加上一句将当前选中怪物移出数组的语句，可以使用erase方法移除type对应的怪物，完整代码如下。

```
Global.State.SWAP:
    Global.monsters.append(Global.current_monster)#将先前移出数组的怪物添加回可选数组
    $Monsters/Player.texture = load(Global.monster_data[type]['back texture'])
    Global.current_monster = type#将当前怪物修改为选中怪物
    Global.monsters.erase(type)#将当前怪物移除可选怪物数组
```

现在再运行游戏，怪物切换功能已正常，切换后再打开切换菜单，当前怪物不会出现在选单，再次切换后怪物又会回到选单中，一切功能正常。



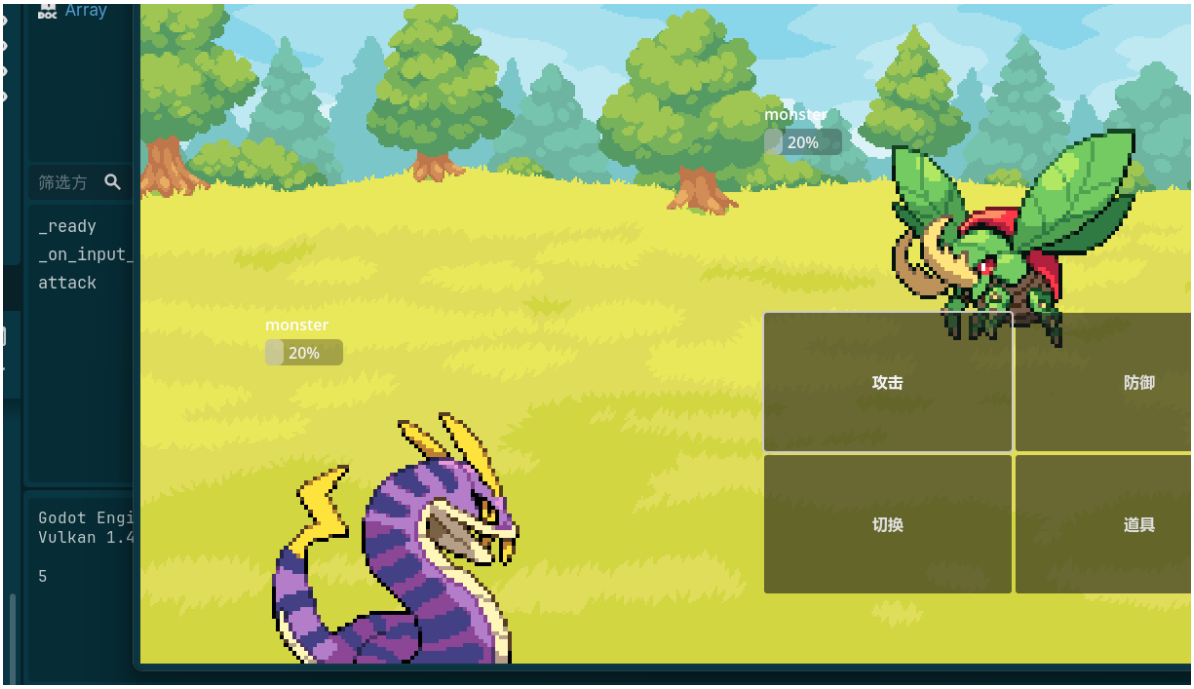
顺带完善一下功能，当游戏启动后随机分配一个怪物作为敌方单位，为此，我们现在ready函数中加上注释，注明玩家怪物初始化代码和敌方怪物初始化代码。

敌方怪物初始化部分，我们先通过values获取Global.Monster的值，再使用pick_random方法从中随机选择一个，将其值赋给Global.current_enemy，现阶段先让其打印Global.current_enemy进行测试，如下。

```
func _ready() -> void:
    #player monster setup
    Global.current_monster = Global.monsters.pop_at(0)
    $Monsters/Player.texture = load(Global.monster_data[Global.current_monster]
['back texture'])

    #enemy monster setup
    Global.current_enemy = Global.Monster.values().pick_random()
    print(Global.current_enemy)
```

运行游戏可以看到，每次运行，控制台都会输出不同的随机数字，如图。



我们还需要通过这个值来同步更新敌方怪物纹理，敌方怪物的纹理处理若直接套用玩家怪物纹理的操作方式，会出现异常，如下。

```
#enemy monster setup
Global.current_enemy = Global.Monster.values().pick_random()
$Monsters/Enemy.texture = load(Global.monster_data[Global.current_enemy]['front texture'])
```

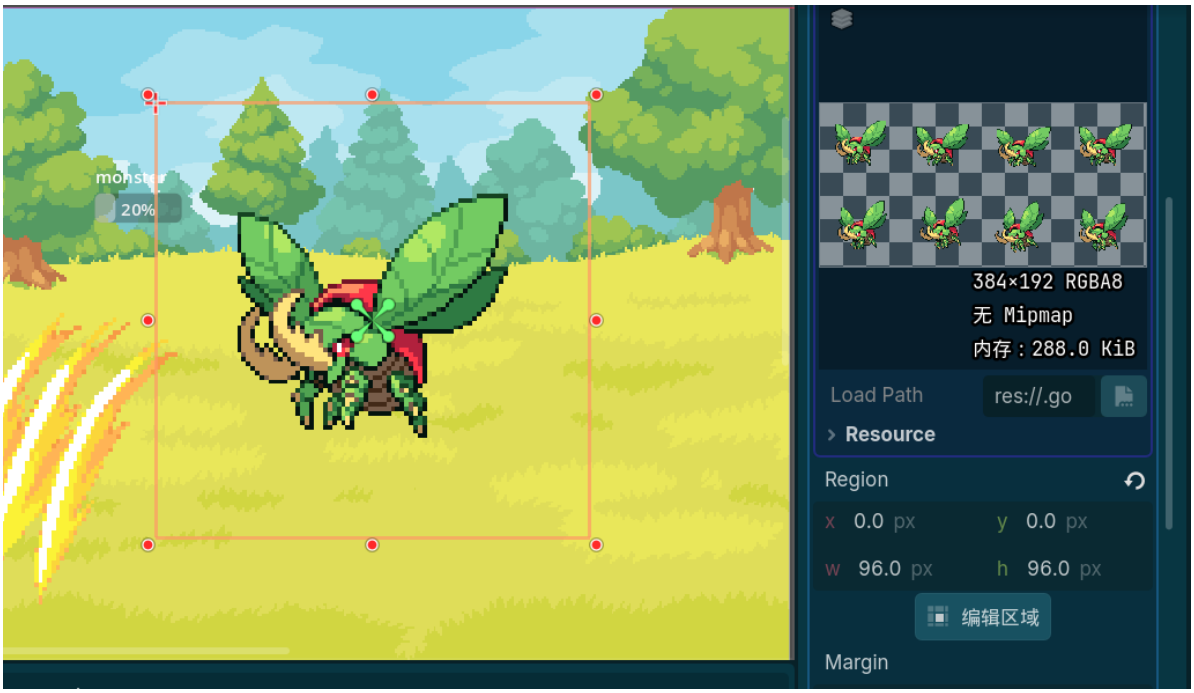
运行游戏可以看到哦，如果直接套用玩家怪物处理逻辑，虽然怪物贴图素材确实会切换成别的，但这张图需要转换成图集纹理，只截取其中一块区域，如图。



为此，我们新建一个图集资源，类型为AtlasTexture，为其创建AtlasTexture实例随后给这个实例赋值图集，也就是我们要加载的原始纹理贴图，赋值方式同样是使用load方法，如下。

```
    #enemy monster setup
    Global.current_enemy = Global.Monster.values().pick_random()
    var new_atlas: AtlasTexture = AtlasTexture.new()
    new_atlas.atlas = load(Global.monster_data[Global.current_enemy]['front
texture'])
```

此时运行游戏，显示效果依旧不对，原因是新建的图集纹理需要指定截取的矩形区域。Game场景中选中enemy节点后点击Texture打开图集可以看到，其大小为384x192，我们先前是手动设置了其区域为96x96，所以界面只会显示单张精灵图，但现在我们需要将全部逻辑都交由代码处理，没有手动设置区域的步骤，因此画面会显示整张图集，如图。

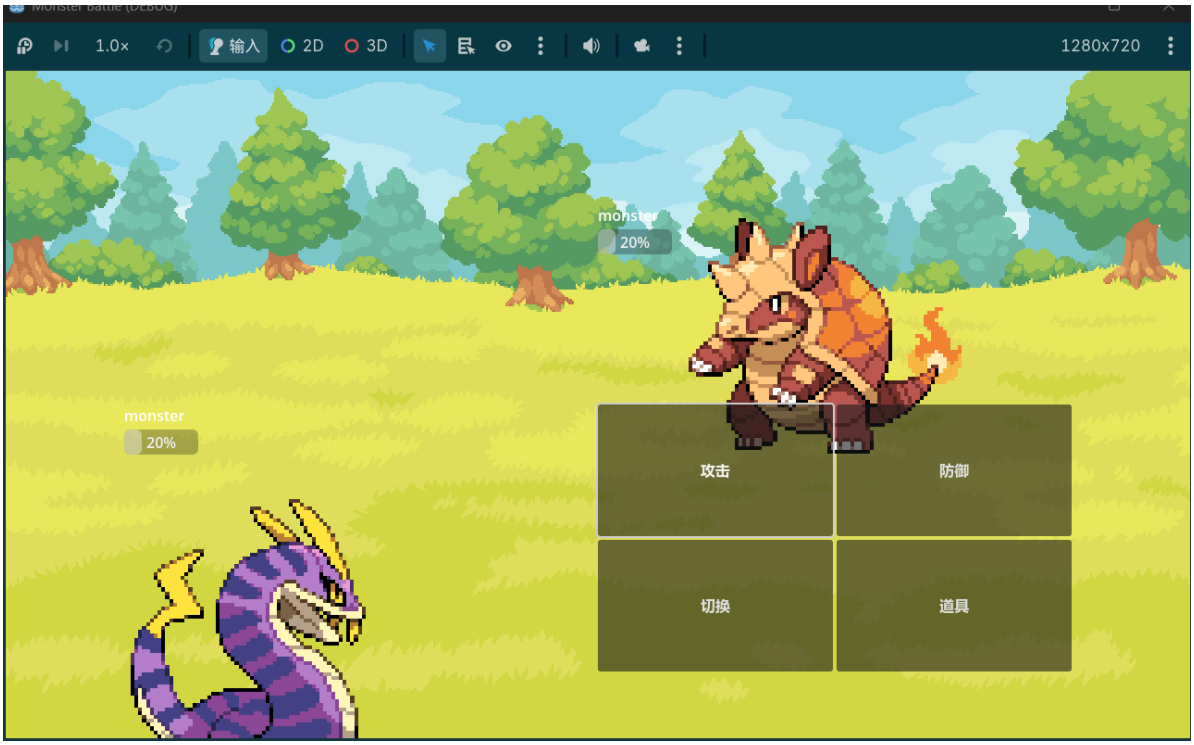




修复方法也很简单，只需要给图集设置region区域即可，也就是new_atlas.region，区域参数使用Rect2i，其中包含position和size，两者使用Vector2i定义，前者为Vector2i.ZERO，后者设置为Vector2i(96,96)，如下。

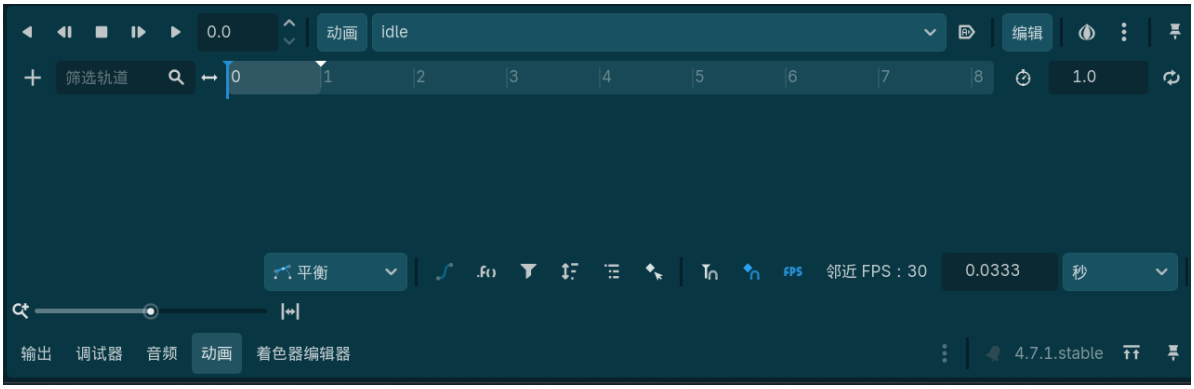
```
    #enemy monster setup
    Global.current_enemy = Global.Monster.values().pick_random()
    var new_atlas: AtlasTexture = AtlasTexture.new()
    new_atlas.atlas = load(Global.monster_data[Global.current_enemy]['front
texture'])
    $Monsters/Enemy.texture = new_atlas
    new_atlas.region = Rect2i(Vector2i.ZERO, Vector2i(96,96))
```

运行游戏测试，画面效果已经正常，如图。



趁现在，我们给敌方怪物加上动画，实现方式有很多，第一种是使用Tween循环切换图集中的不同精灵区块，图集一共8帧，上方4帧是待机动画，下方4帧是攻击动画，现阶段我们先只播放上方4帧，并循环播放。用Tween可以实现，但后续我们需要添加攻击动画，统一管理多套动画使用AnimationPlayer会更加方便。

在Game场景中添加AnimationPlayer节点，新建一条动画轨道，命名为idle，还需要绑定控制动画的属性，但问题是我们没办法直接读取敌方怪物图集纹理的区域参数，因此要换一套实现思路。



我的处理方案是在game脚本中新建变量animation_index，变量类型限定为整数，在process函数中实时更新这个数值。

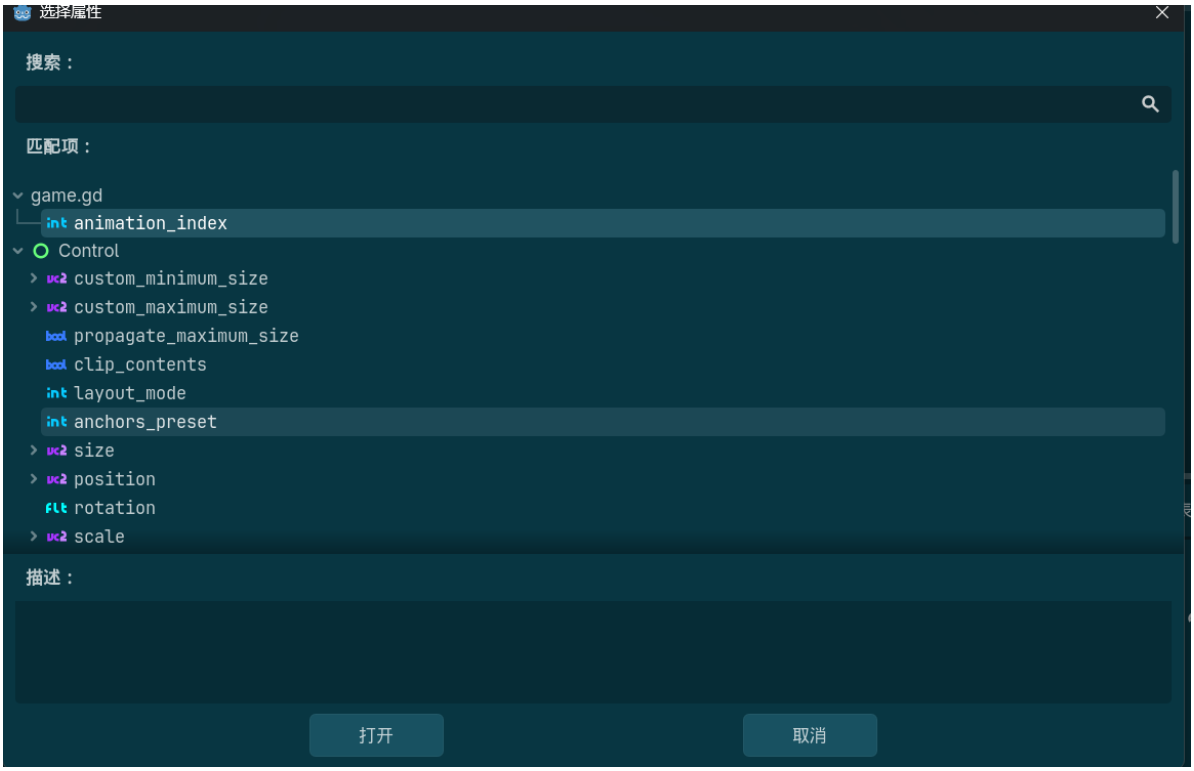
```
var animation_index: int
```

在process函数中，先获取敌方单位的图集纹理，可以直接从敌方节点对象获取，将类型限定为AtlasTexture。随后将设置纹理的region属性，和上方写法一致，size固定为Vector2i(96，96)，pos的y轴暂时固定为0，x轴的值为96 * animation_index，这样设置后，如果将animation_index设置为0，画面会显示第1张精灵，随着animation_index的值在0到3之间变化，画面也会从第1张精灵到第4张精灵变化，具体写法如下。

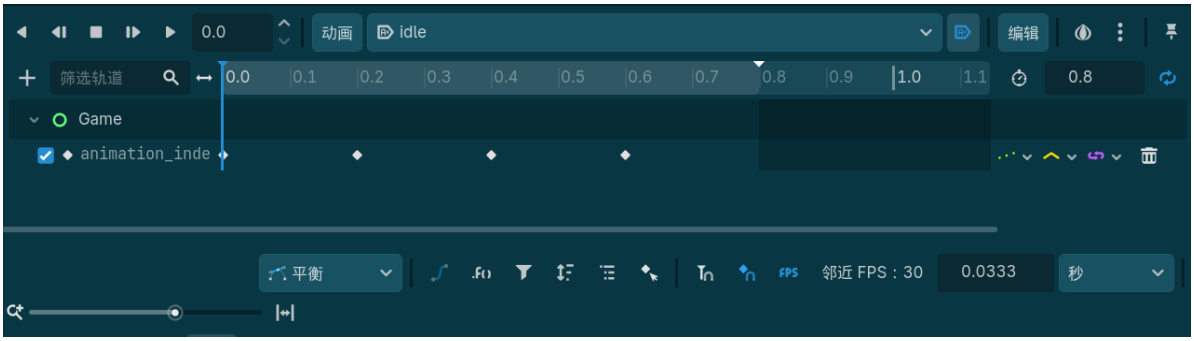
```
func _process(delta: float) -> void:
    var atlas = $Monsters/Enemy.texture as AtlasTexture
    atlas.region  = Rect2i(Vector2i(96 * animation_index, 0), Vector2i(96, 96))
```

这个逻辑只要观察精灵图集素材就很容易理解，每一张精灵的大小是96 x 96，坐标起点永远是区块的左上角，第1张精灵的pos的x和y都是0，想切换到第2张，只需要使用索引值1乘单张区块的大小96即可。

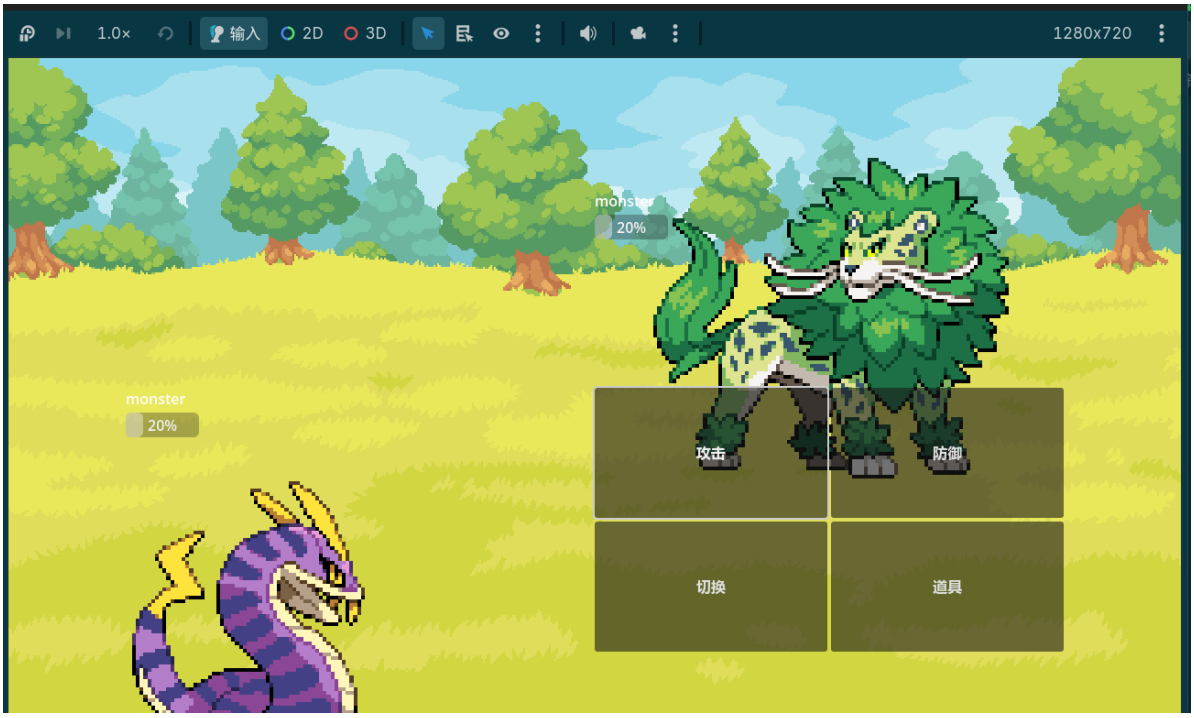
现在只剩下最后一个问题，如何自动变动animation_index？这里就需要用到AnimationPlayer，为了编辑器识别animation_index，我们使用export注解将其声明，随后选中AnimationPlayer节点，在动画界面重新添加属性轨道，目标节点选择game，即可看到game.gd中的animation_index，如图。



新建完属性轨道后，在轨道中0.0到0.6处每隔0.2秒插入一个关键帧，值从0到3，设置动画总时长为0.8秒，开启循环播放和自动播放，如图。



设置完成后运行游戏就能播放这套动画，画面效果非常流畅，游戏整体更加生动，如图。



10.回合战斗完整流程

现在基础内容已制作完毕，就可以将这个半成品改成完整游戏，包括攻击扣除血量、怪物血量为0则被击杀，击杀后替换怪物资源。

要实现这个功能，最关键的一点就是我们要完善怪物属性模块，就是之前创建用来展示怪物血量和名称的模块。

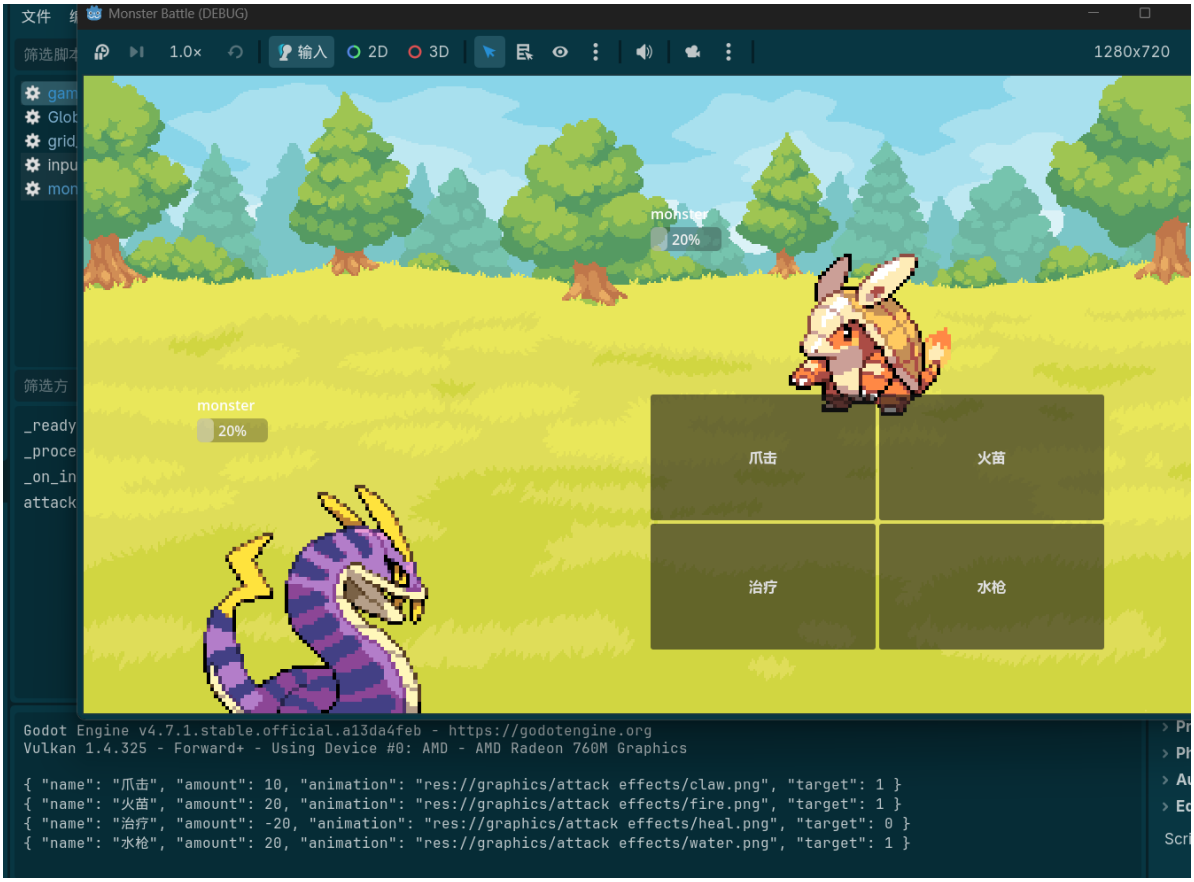
接下来我们编写MonsterStats场景的怪物属性的逻辑，这个节点需要挂载脚本才能交互，我们为其挂载脚本，并新建update函数，函数参数为attack_data，先简单打印我们获取到的内容做测试，如下。

```
func update(attack_data):
    print(attack_data)
```

接下来我们要调用这个函数，调用位置在Game场景的attack函数内，该函数内已经写好了播放动画的全部代码，除此之外，我们还要同步更新怪物属性。这里分为player_stats和enemy_stats两类，我们需要通过target来判断当前攻击目标属于哪一类，若目标是玩家，则调用Stats下PlayerStats的update方法，传入在Global.attack_data中通过attack_type得到的具体攻击参数；反之若目标是敌方，则改为调用EnemyStats的update方法即可，具体代码如下。

```
    if target == $Monsters/Player:
        $Stats/PlayerStats.update(Global.attack_data[attack_type])
    else:
        $Stats/EnemyStats.update(Global.attack_data[attack_type])
```

运行游戏，现在点击攻击列表中的技能，控制台会输出对应的完整数据，如图。



这就说明我们可以修改怪物属性中的数值逻辑了，最重要的一点是，怪物属性变更时，我们要同步刷新进度条节点，为此我们修改update方法的内容，让进度条的value属性减去传入的攻击参数的amount，如下。

```
func update(attack_data):
    $VBoxContainer/ProgressBar.value -= attack_data['amount']
```

运行游戏测试，双方初始血量均为20%，分别触发爪击和治疗后进度条同步变化，效果如图。



除此之外，还要在脚本中新建setup初始化函数，参数为monster（指定怪物）和player（怪物是否为玩家，这里暂时默认为true），通过参数获取name数据来修改怪物名称，如下。

```
func setup(monster: Global.Monster, player = true):
    var monster_data = Global.monster_data[monster]
    $VBoxContainer/Label.text = monster_data['name']
```

完成后，我们在game脚本的ready函数中调用setup方法，使游戏创建后就初始化怪物名称，如下。

```
func _ready() -> void:
    #player monster setup
    Global.current_monster = Global.monsters.pop_at(0)
    $Monsters/Player.texture = load(Global.monster_data[Global.current_monster]
['back texture'])
    $Stats/PlayerStats.setup(Global.current_monster)#初始化玩家怪物
```

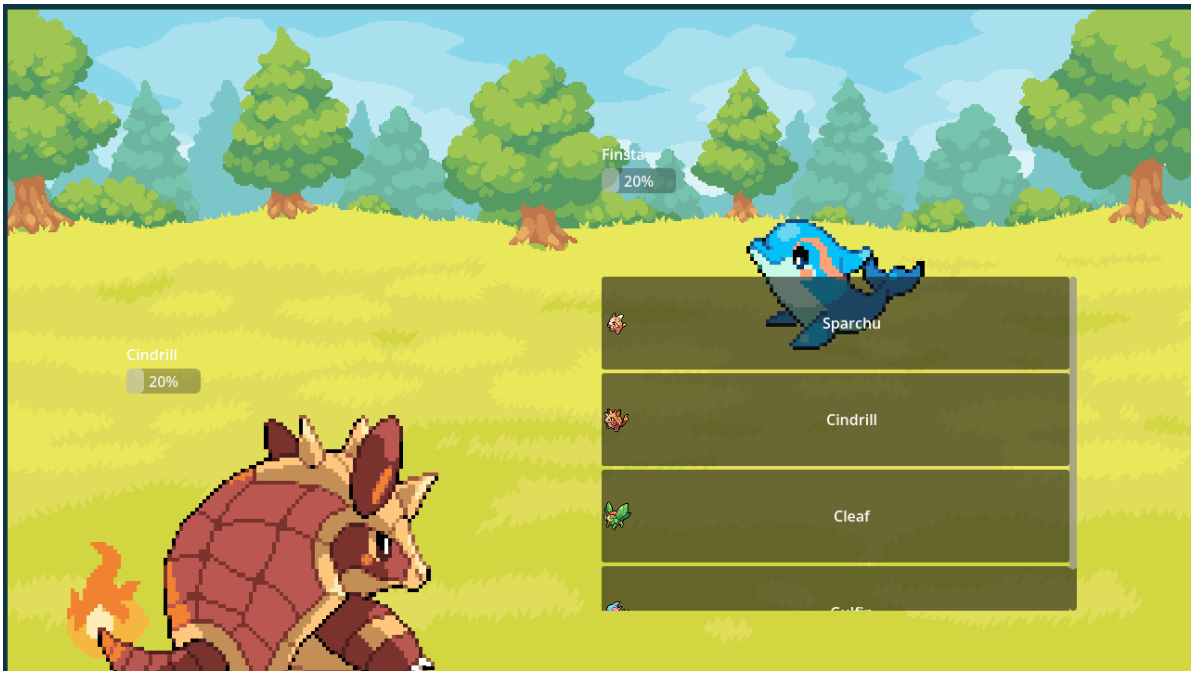
为了效果完整，为敌方怪物也写一套相同逻辑，player参数设置为false，如下。

```
$Stats/EnemyStats.setup(Global.current_enemy, false)
```

完成后运行游戏，可见初始玩家和敌方怪物名称均已正常运行，如图。



切换怪物后，我们也需要调用setup函数保证切换后的怪物名称正常显示，复制玩家的调用语句到_on_input_menu_selected函数的切换逻辑中，运行游戏后切换怪物测试，名称仍正常显示，如图。



还有一部分内容，打开Global脚本，可以看到每个怪物的数据都定义了max health最大血量属性。

```
'name': 'Friolera',
        'back texture': "res://graphics/back sprites/Friolera.png",
        'front texture': "res://graphics/battle sprites/Friolera.png",
        'icon': "res://graphics/menu sprites/Friolera.png",
        'attacks': [Attack.CLAW, Attack.ICE, Attack.EXPLOSION, Attack.HEAL],
        'max health': 120,
```

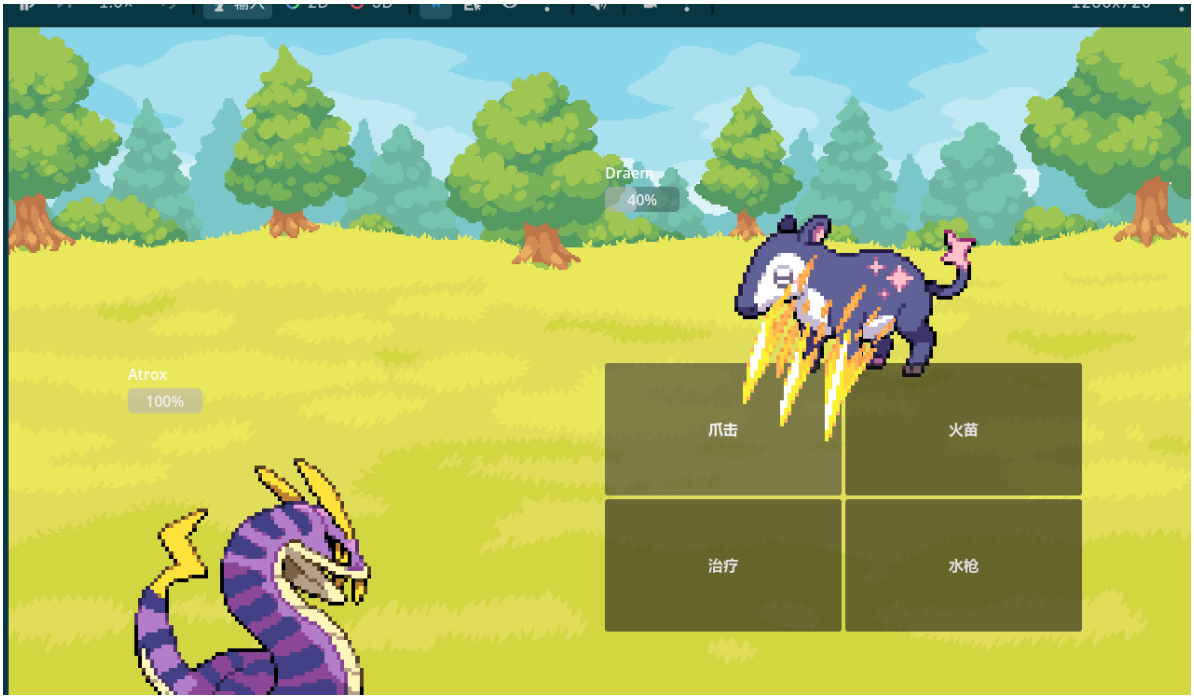
我们需要在monster_stats中使用该属性，，因此我们要在setup函数中设置ProcessBar的Max Value属性，作为怪物的血量，数值来源同样是怪物数据，取max health属性，如下。

```
$VBoxContainer/ProgressBar.max_value = monster_data['max health']
```

且每次调用setup函数时，都要将进度条当前值设置为max health值，保证每只怪物刚出场时都是最大血量，如下。

```
$VBoxContainer/ProgressBar.value = monster_data['max health']
```

运行游戏测试，血量效果正常，其余功能也能正常运行，如图。



现在我们可以实时监控怪物血量，同时也可以判断怪物血量归零被击败的时机，这里需要用到进度条自带的信号value_changed，它会在进度条的value变化时触发，绑定信号后，新建自定义信号defeat，value清零后发送defeat信号，方便后续在主脚本中编写逻辑，如下

```
func _on_progress_bar_value_changed(value: float) -> void:
    if value <= 0:
        defeat.emit()
```

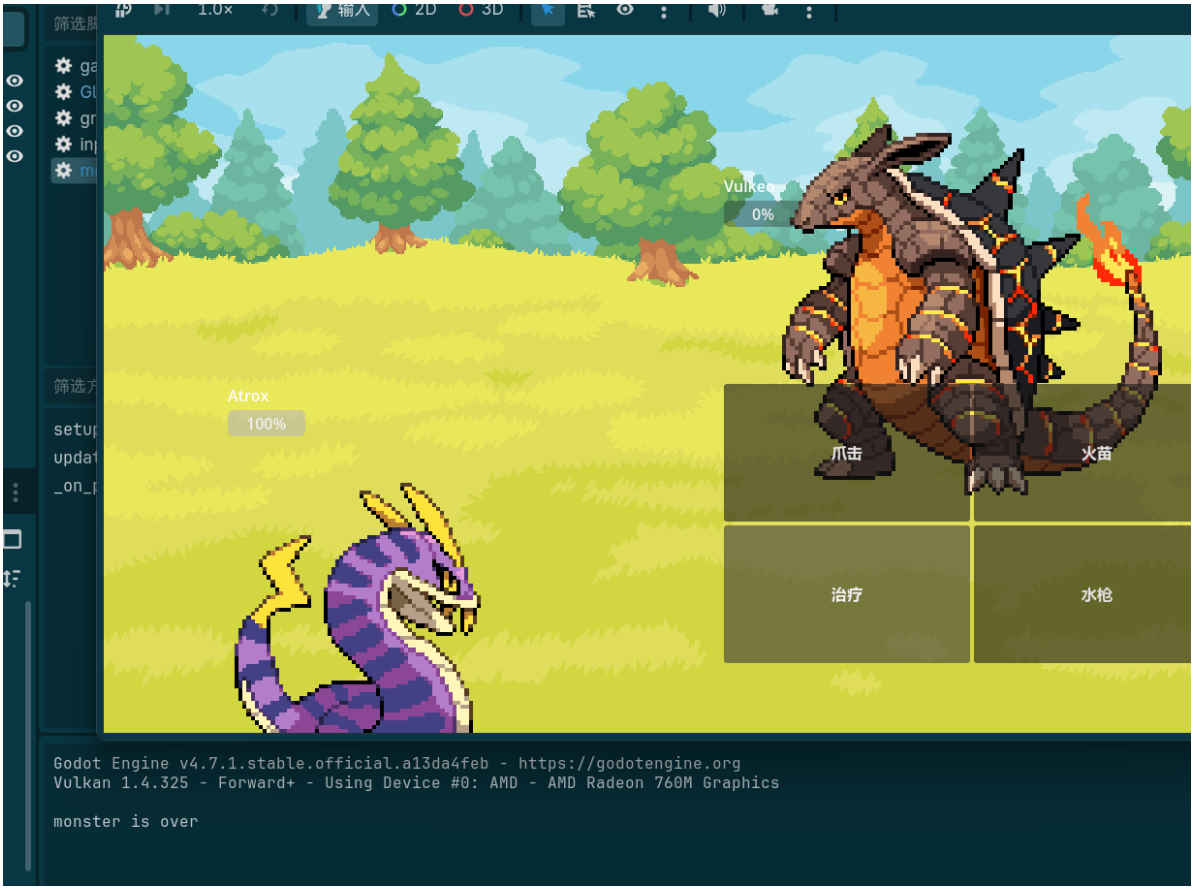
定义好信号后，回到game脚本，绑定玩家属性和敌人属性的defeat信号，手动绑定每个节点的信号过于繁琐，所以我们等待加载完成后，使用for循环批量绑定所有属性节点的信号，写法如下。

```
#connect the stats signals
    for  stats in $Stats.get_children():
        stats.connect('defeat', monster_swap)
```

信号绑定了monster_swap方法，我们现在新建这个方法，这个方法目前不需要参数，先让其打印“monster is over”语句。

```
func monster_swap():
    print('monster is over')
```

运行游戏测试，怪物血量为0后控制台正常输出，如图。



判定怪物被击败后，随机切换到另外一只怪物，实现逻辑也很简单，重新给Global.current_enemy赋值，赋值方式和初始化时保持一致，但直接粘贴先前代码造成代码复用，因此我们选择新建一个enemy_monster_setup敌人怪物初始化函数，将敌方怪物初始化逻辑全部封装入这个函数，初始化时直接调用即可，如下。

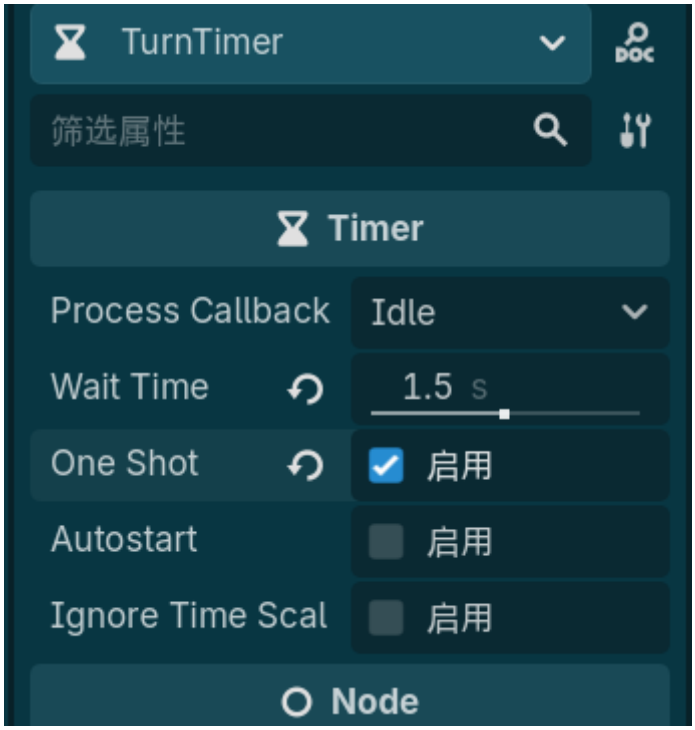
```
func enemy_monster_setup():
    Global.current_enemy = Global.Monster.values().pick_random()
    var new_atlas: AtlasTexture = AtlasTexture.new()
    new_atlas.atlas = load(Global.monster_data[Global.current_enemy]['front
texture'])
    $Monsters/Enemy.texture = new_atlas
    new_atlas.region = Rect2i(Vector2i.ZERO, Vector2i(96,96))
    $Stats/EnemyStats.setup(Global.current_enemy, false)

func monster_swap():
    enemy_monster_setup()
```

现在完整运行游戏测试，清空怪物的血量后，会自动刷新出另一只怪物，效果符合预期。

现在我们可以攻击敌人，但敌人还不会主动反击，反击的实现思路是，每次触发_on_input_menu_selected函数，不管是攻击还是切换等，需要隐藏输入菜单并触发敌人攻击。我们在Game场景新建一个计时器节点TurnTimer，设置为单次触发，时长设置为1.5秒，每次触发按键时开启，作为敌人攻击环节，如下。

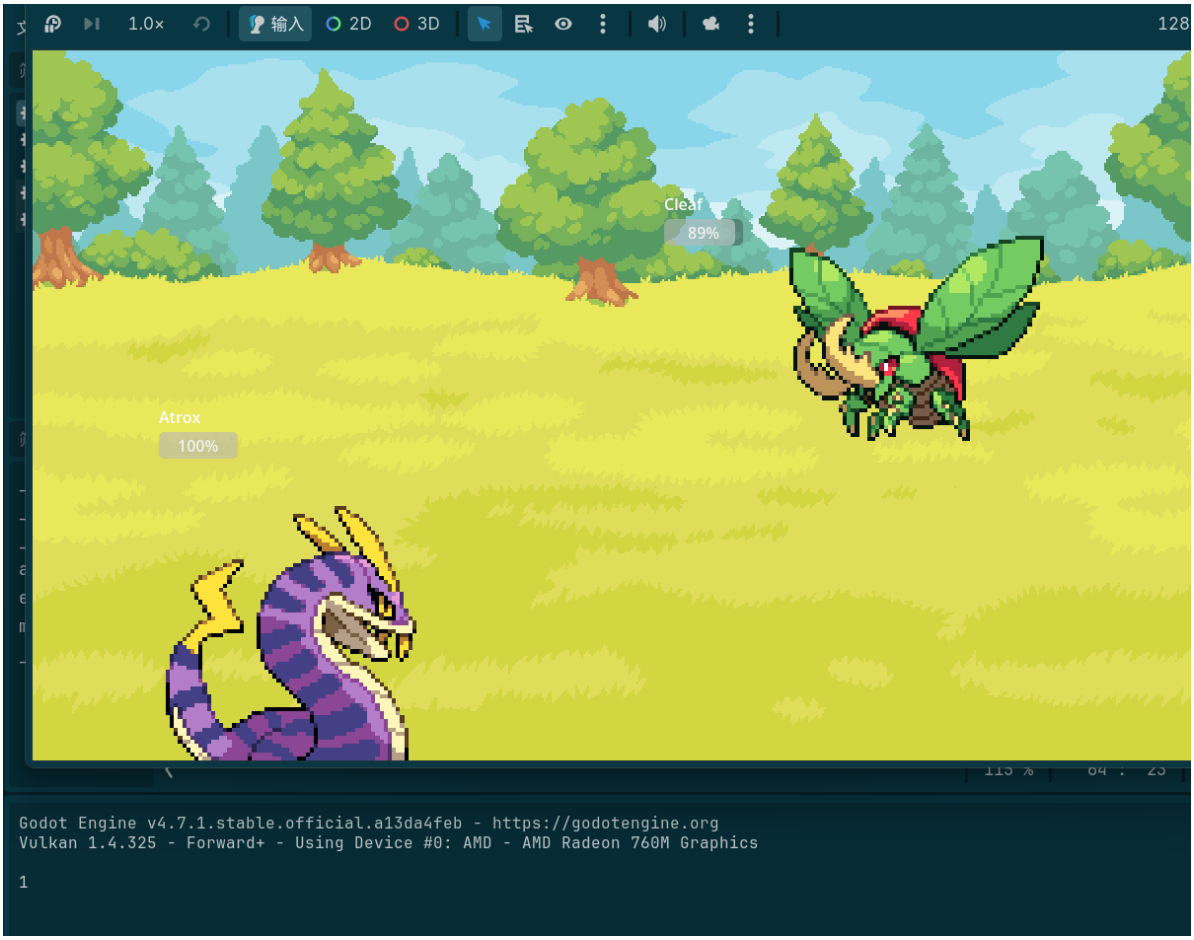
```
func _on_input_menu_selected(state: int, type: Variant) -> void:
    $InputMenu.hide()#隐藏菜单
    $TurnTimer.start()#触发计时器，进入敌人攻击环节
    match state:
        Global.State.ATTACK:
            var target = $Monsters/Enemy if Global.attack_data[type]['target']
else $Monsters/Player
            attack(target, type)
        Global.State.SWAP:
            Global.monsters.append(Global.current_monster)
            $Monsters/Player.texture = load(Global.monster_data[type]['back
texture'])
            Global.current_monster = type
            Global.monsters.erase(type)
            $Stats/PlayerStats.setup(Global.current_monster)
```



绑定计时器的timeout信号，在计时结束回调里，定义变量attack_type和target，attack_type值为当前敌方怪物的随机一个技能，现阶段先打印attack_type，如下。

```
func _on_turn_timer_timeout() -> void:
    var attack_type = Global.monster_data[Global.current_enemy]
['attacks'].pick_random()
    print(attack_type)
```

运行游戏测试，攻击敌人后，菜单隐藏并且控制台输出攻击类型值，如图。



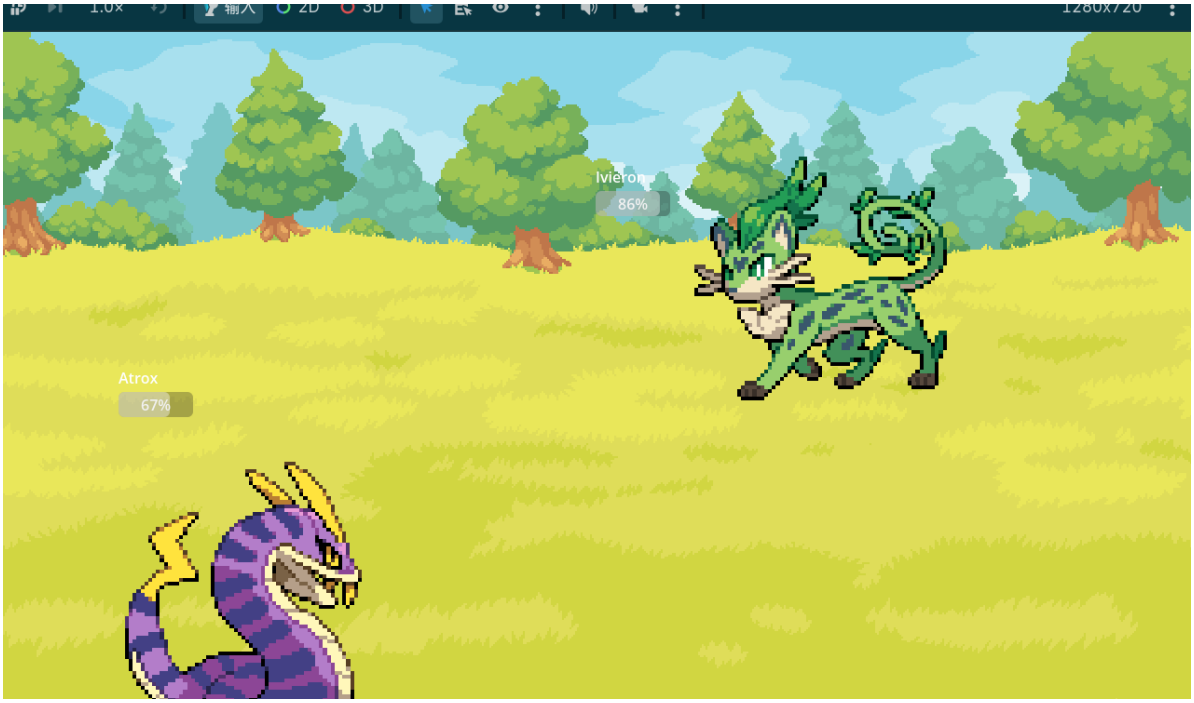
还有target的值，直接复用先前的写法即可，将type改为attack_type，只需要在if后加上not就可以将原先的攻击敌人改为攻击玩家，如下。

```
var target = $Monsters/Enemy if not Global.attack_data[attack_type]['target']
else $Monsters/Player
```

完整的回调函数如下，这个写法泛用性极强。

```
func _on_turn_timer_timeout() -> void:
    var attack_type = Global.monster_data[Global.current_enemy]
['attacks'].pick_random()
    var target = $Monsters/Enemy if not Global.attack_data[attack_type]
['target'] else $Monsters/Player
    attack(target,attack_type)
```

运行游戏测试，现在攻击敌人后，菜单消失且1.5秒以后敌方会随机攻击玩家，敌人动画效果也正常，玩家血量正常消失，如图。



整套反击流程设置完毕后，我们要重新显示菜单节点，进入下一个回合，在回调函数末尾添加代码如下。

```
$InputMenu.show()
```

但目前逻辑不完整，菜单重新显示后，会停留在攻击页，为此我们需要监听另一个信号。

来到input_menu脚本，选中InputMenu节点，绑定信号visibility_changed，在on_visibility_changed中设置current_state为Global.State.MAIN，这样每次菜单重新显示后会固定会到主菜单，如下。

```
func _on_visibility_changed() -> void:
    current_state = Global.State.MAIN
```

最后还需要完善逻辑，如果玩家操纵的怪物被击败，我们需要执行玩家怪物的替换逻辑，否则玩家被击败也会替换敌方怪物，这显然不合理。

要实现这个功能，我们要判断被击败的怪物是否是玩家怪物，在monster_swap中执行不同的逻辑。

在monster_stats脚本中，我们有判断是否为玩家的参数player，我们新建bool型变量is_player存储这个值，在setup函数中将player值赋予is_player，如下。

```
func setup(monster: Global.Monster, player = true):
    var monster_data = Global.monster_data[monster]
    $VBoxContainer/Label.text = monster_data['name']
    $VBoxContainer/ProgressBar.max_value = monster_data['max health']
    $VBoxContainer/ProgressBar.value = monster_data['max health']
    is_player = player#记录是否是玩家
```

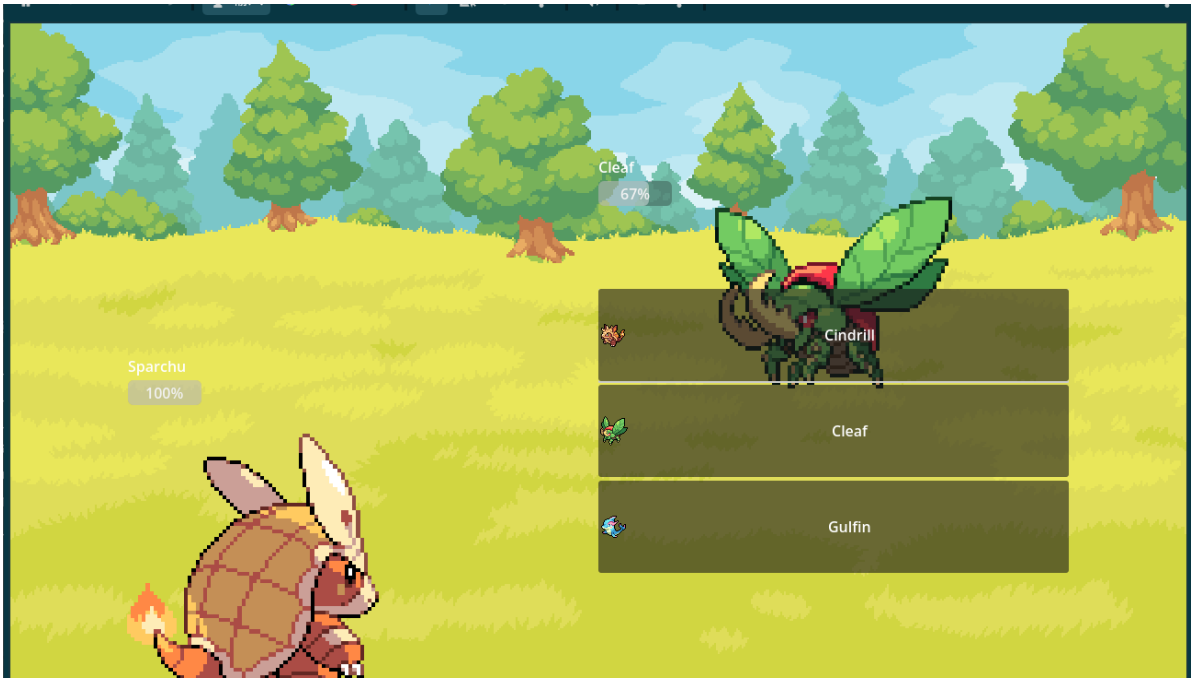
为信号defeat添加参数is_player，类型同样是bool，发送信号时，将is_player的值作为参数一同传出。

```
func _on_progress_bar_value_changed(value: float) -> void:
    if value <= 0:
        defeat.emit(is_player)
```

game脚本中的monster_swap同样增加对应参数，判断若不是player，则照旧执行敌方切换，否则执行玩家怪物切换逻辑，使用pop_at切换第一个怪物，并不将当前怪物加会列表，切换后怪物的初始化可以复用先前初始化玩家怪物的代码，如下。

```
func monster_swap(player: bool):
    if not player:
        enemy_monster_setup()
    else:
        Global.current_monster = Global.monsters.pop_at(0)
        $Monsters/Player.texture =
load(Global.monster_data[Global.current_monster]['back texture'])
        $Stats/PlayerStats.setup(Global.current_monster)
```

运行游戏测试，让敌方将玩家怪物血量清空后，会自动切换到下一个怪物，且切换列表中血量清空的怪物消失，如图。



怪物替换逻辑至此全部完成。

除此之外，还有一处细小逻辑需要补全，也就是实现防御和道具切换的菜单逻辑。防御比较简单，直接pass，接下来实现道具切换逻辑。

打开Global脚本，item_data中包含4个道具，道具信息包含道具名称、目标、数值，分为负数扣血和正数回血，蘑菇和毒素会对敌人造成伤害，药剂与精华能恢复我们方怪物血量，接下来我们在游戏脚本中调用这套道具数据。

```
    const item_data: Dictionary[Item, Dictionary] = {
    Item.POTION:   {'name': 'Potion', 'target': 0, 'attribute': 'health',
'amount': -20,  'icon': "res://graphics/item sprites/potion.png"},
    Item.ELIXIR:   {'name': 'Elixir', 'target': 0, 'attribute': 'health',
'amount': -50,  'icon': "res://graphics/item sprites/elixir.png"},
    Item.MUSHROOM: {'name': 'Mushroom', 'target': 1,  'attribute': 'health',
'amount': 20, 'icon': "res://graphics/item sprites/mushroom.png"},
    Item.POISON:   {'name': 'Poison', 'target': 1,  'attribute': 'health',
'amount': 50, 'icon': "res://graphics/item sprites/poison.png"}
}
```

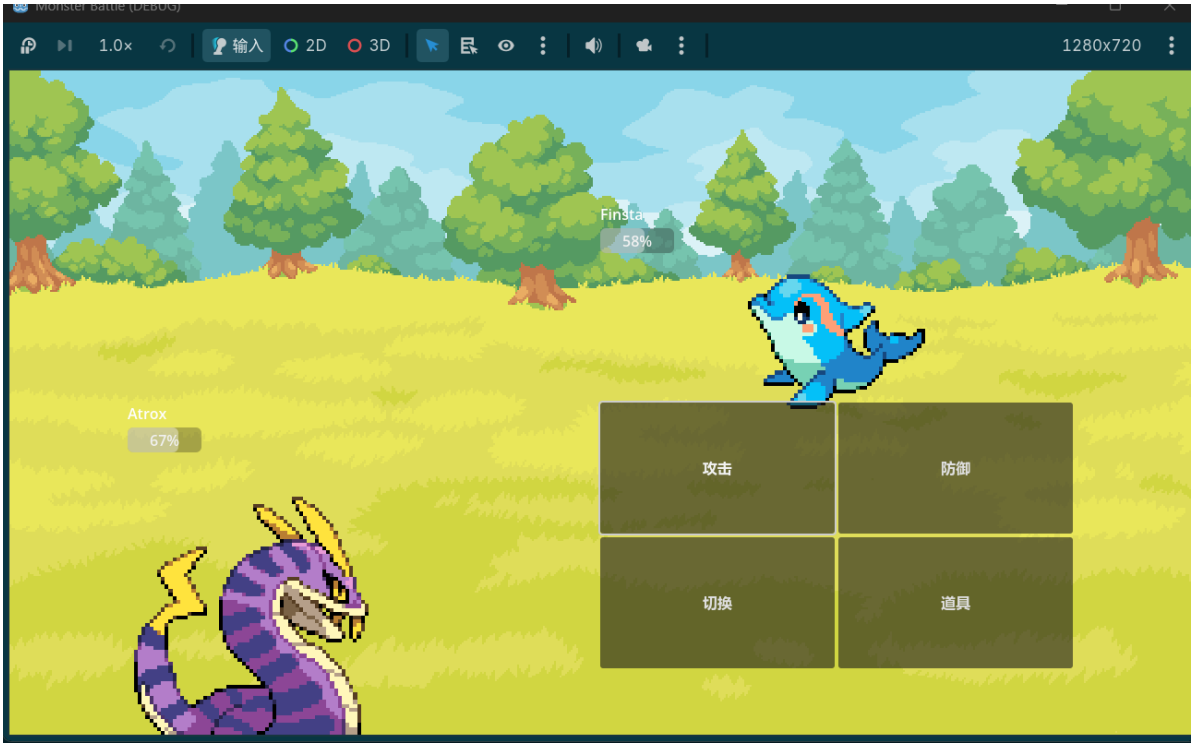
在_on_input_menu_selected的处理道具逻辑中，获取道具生效的target，并调用target的update方法，先前定义的update方法用于处理攻击，但它只读取传入参数的'amount'参数，因此可以直接使用无需修改，如下。

```
Global.State.DEFEND:
            pass
        Global.State.ITEM:
            var target = $Stats/PlayerStats if not Global.item_data[type]
['target'] else $Stats/EnemyStats
            target.update(Global.item_data[type])
```

为了便于理解，我们应该把update的参数统一改为data，而不是针对攻击数据的attack_data，且传入的data必须是字典类型，做一层格式约束，如下。

```
func update(data: Dictionary):
    $VBoxContainer/ProgressBar.value -= data['amount']
```

到这里，核心逻辑就写完了，运行游戏测试，使用道具后会根据道具类型来为玩家和敌方加减血量，如图。

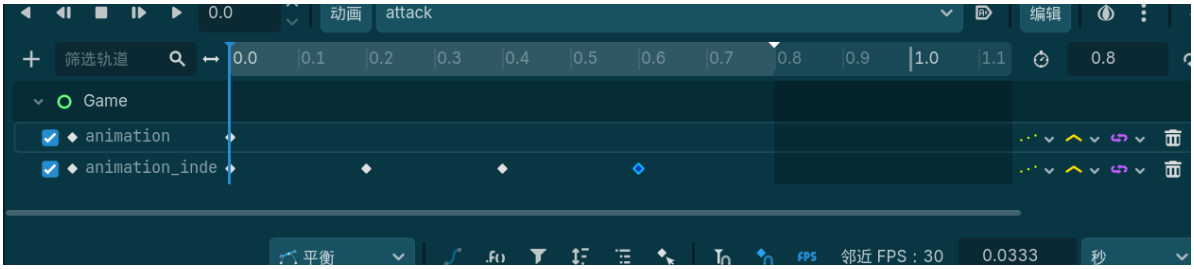


道具功能至此全部完成，最后我们实现攻击时播放敌方攻击动画的效果。

在game脚本中新建一个变量animation: int，之前我们播放的是图集资源中第一行的帧，使用这个变量在process帧更新函数中控制图集行切换。具体就是将pos的y轴值修改为96 * animation，当animation值为0，播放的是待机动画，值为1时，播放的是攻击动画。

```
func _process(_delta: float) -> void:
    var atlas = $Monsters/Enemy.texture as AtlasTexture
    atlas.region  = Rect2i(Vector2i(96 * animation_index, 96 * animation),
Vector2i(96, 96))
```

接下来使用AnimationPlayer控制切换。新键一段动画attack，添加属性轨道animation，插入关键帧数值设置为1，另外添加animation_index属性轨道，设置和先前的idle轨道一致，如图。



另外在idle待机动画中，也要新增animaition属性轨道，插入关键帧数值设为0。

到这里，动画制作就完成了，只需要在timeout计时结束函数中添加调用动画播放语句。

```
$AnimationPlayer.play("attack")
```

运行游戏测试，发现进入敌方攻击回合后，敌方会播放攻击动画，但不会自动切换会idle动画，画面停留在attack动画的最后一帧。这个问题也可以通过AnimationPlayer节点实现自动切换来解决。

AnimationPlayer节点自带animation_finished信号，我们绑定这个信号的回调函数，信号触发时，调用AnimationPlayer执行play播放idle动画。如下。

```
func _on_animation_player_animation_finished(anim_name: StringName) -> void:
    $AnimationPlayer.play("idle")
```

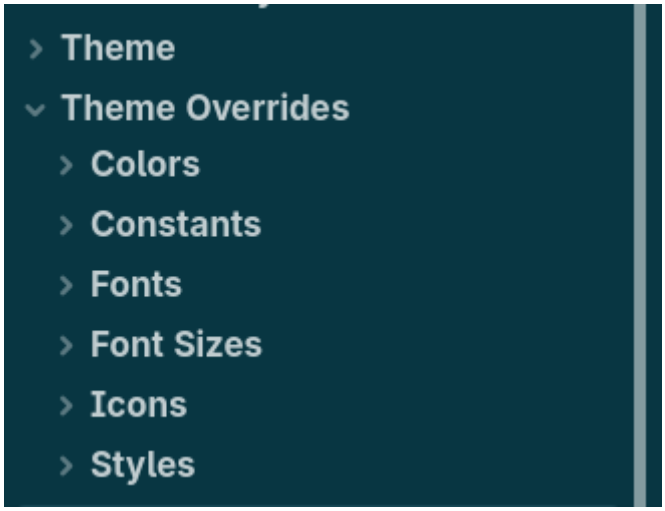
配置完成后，再次测试，敌方怪物攻击时正常播放attack动画，随后切换回idle动画。

美中不足的是我方怪物还未制作动画，但整体功能已经较为完善。

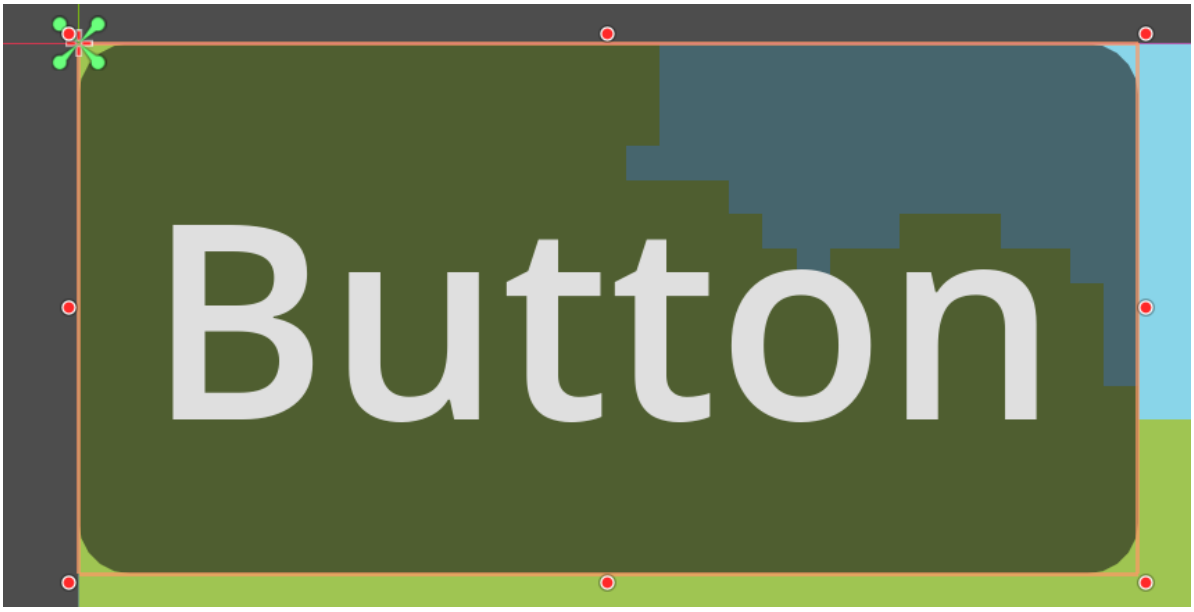
11.UI视觉样式美化

现在我们来收尾这款精灵对战游戏的制作。我们要给所有交互控件统一美化样式，包括全部按钮、进度条还有label。几乎所有组件都需要调整。

在Godot中实现这个效果的逻辑是，我们新建一个按钮作为演示，可以在检查器面板看到Theme和Theme Overrides，这里重点讲解Theme Overrides，如图。



其中包含颜色、字体等设置，我们放大画面，再给这个按钮预设文字，方便演示效果。



在这个按钮的颜色属性中，我们可以修改颜色，修改成黑色或是任意想要的颜色

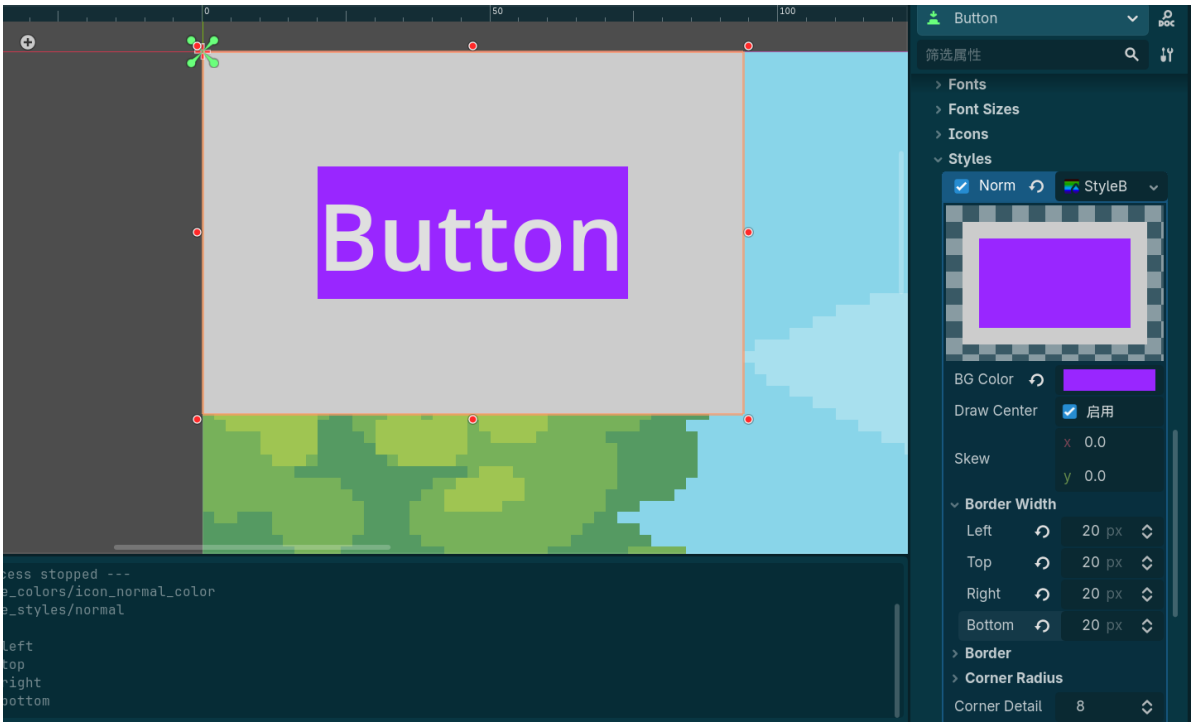


按钮颜色有很多种细分，包括触发时颜色、聚焦时颜色等，除此之外，还有style，里面包含了按钮所有可切换的交互状态，每个状态都能单独显示外观，normal是最基础的显示状态，可以给它绑定不同的样式资源来美化按钮，最常用的资源是StyleBoxFlat，如图。

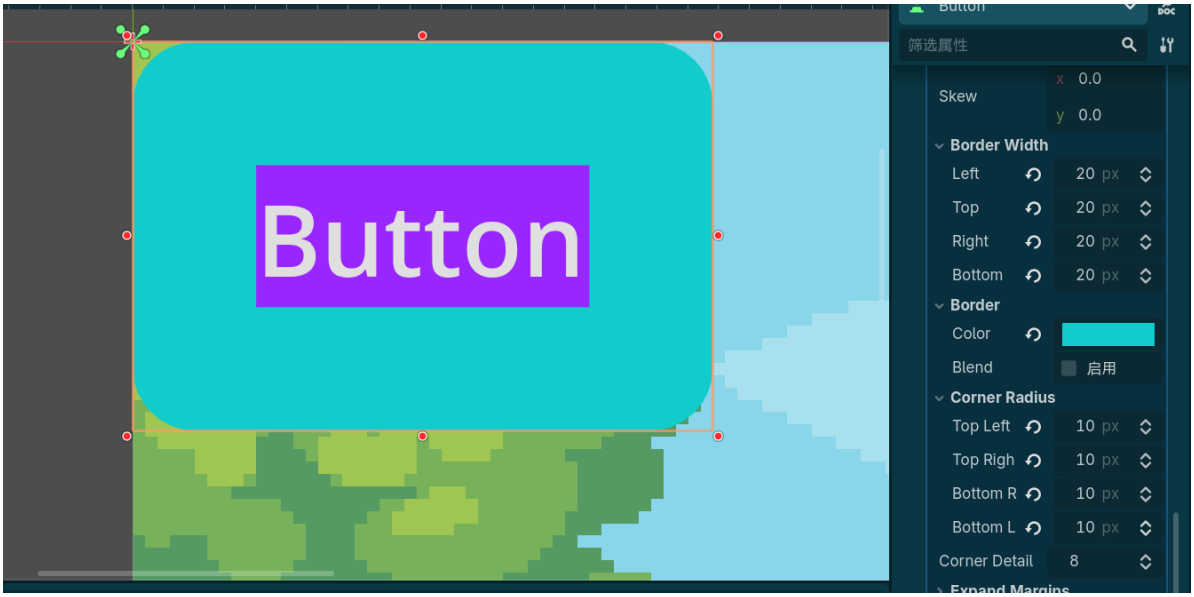


新建这个资源后，会出现一块纯色灰框，所有参数都能自定义。

举个例子，修改BG Color填充颜色，或是修改Border Width边框宽度上下左右为20，如图。



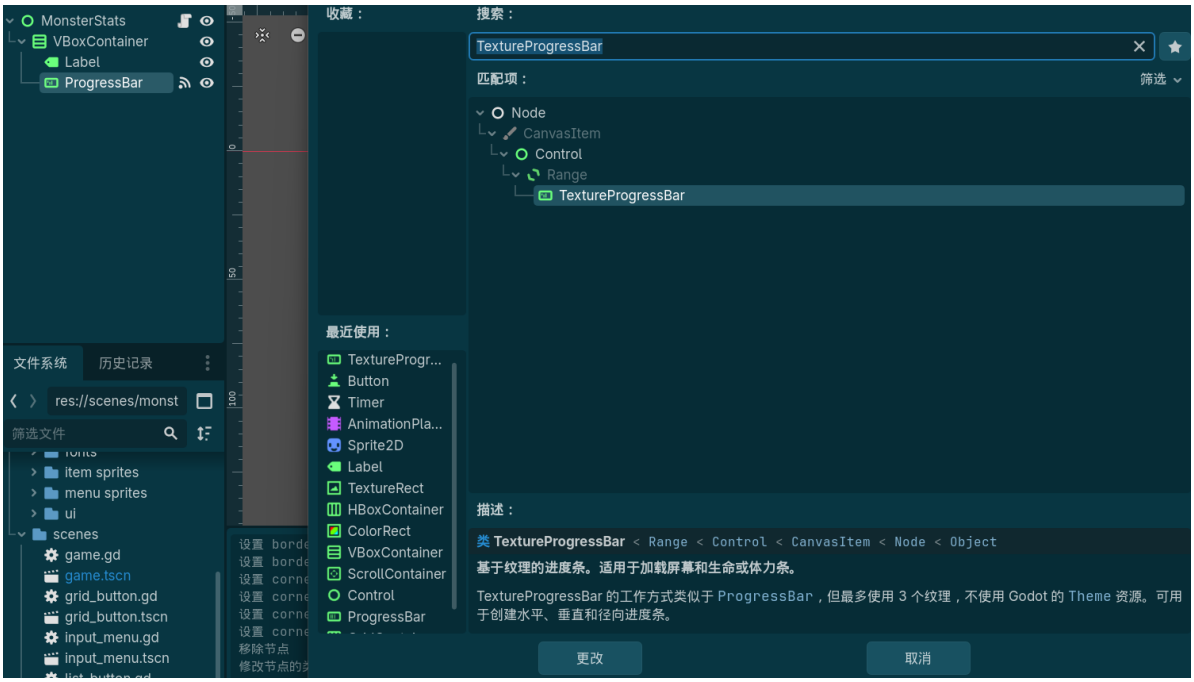
就连边框的颜色也能自由更换，还能调整圆角半径，如图。



除此之外还有非常多的参数，可以按需调整，这是最通用的界面美化方式，通过Theme Overrides修改的样式只会单独作用于当前选中的组件，但Theme不一样，可以创建一套完整的主题，批量套用到多个交互控件上。

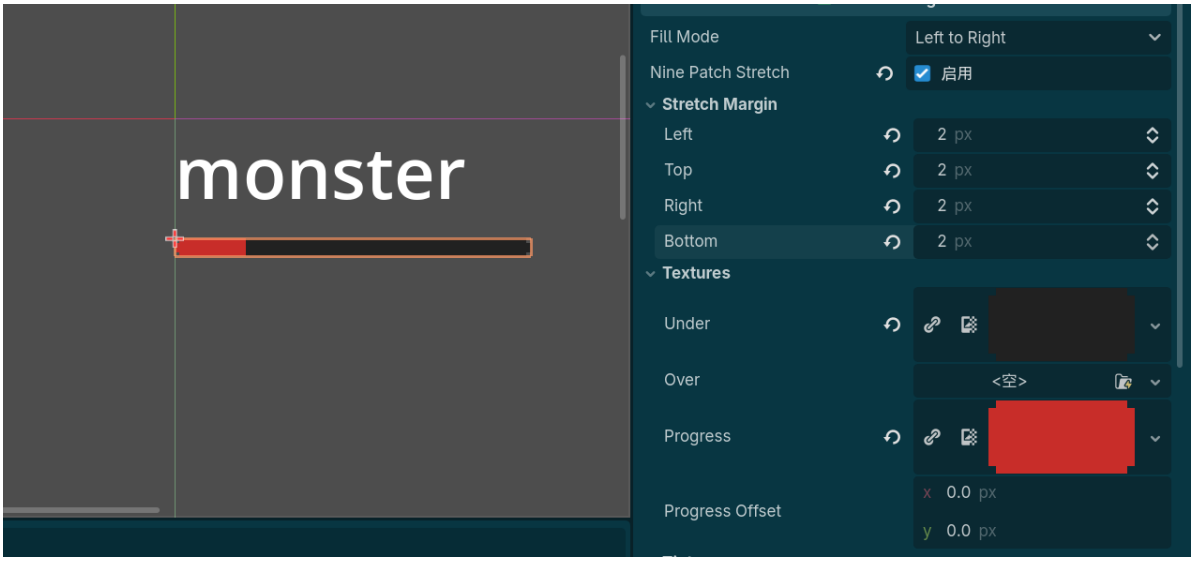
删除这个演示按钮，接下来正式设置monster_stats模块，需要对这个场景进行大量修改。

首先，修改当前进度条类型为TextureProgressBar，如图。

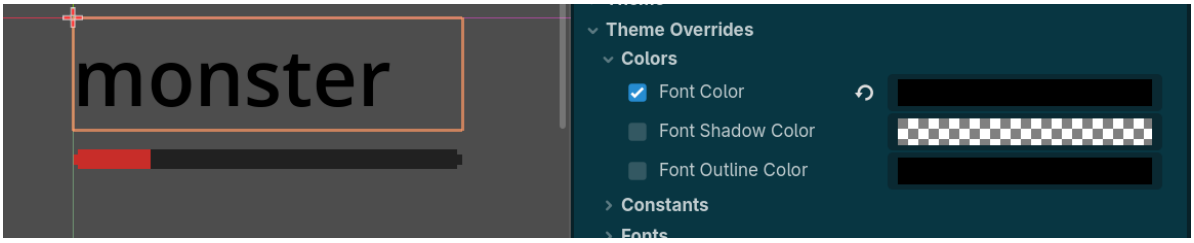


切换完成后画面会空白，因为这个节点需要两张贴图资源，资源文件存放在ui文件夹内，我们用res://graphics/ui/progress_bg.png作为Under底层贴图，res://graphics/ui/progress_main.png作为Progress进度贴图，这样一来，就不再是单调纯色进度条。

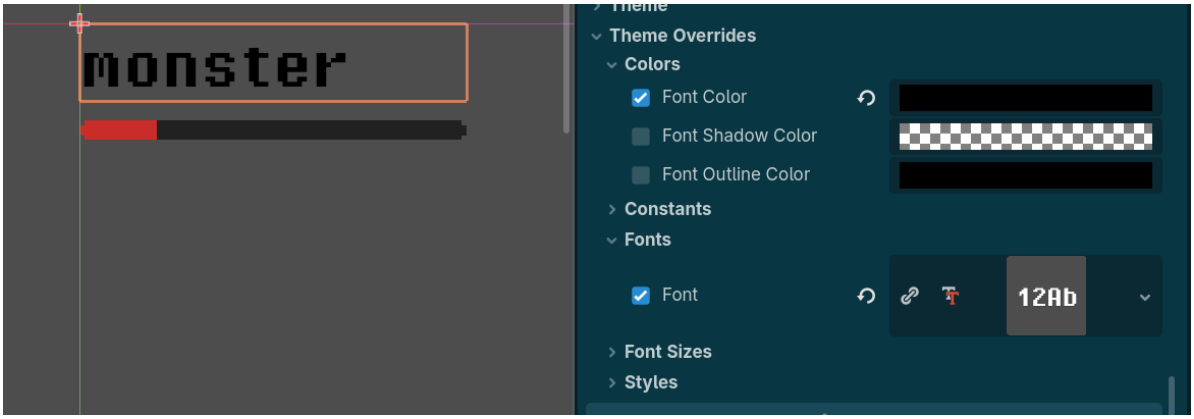
贴图需要拉伸铺满整个控件区域，实现这个效果要开启TextureProgress的Nine Patch Stretch，再配置Stretch Margin，四边统一设置为2像素，效果如图。



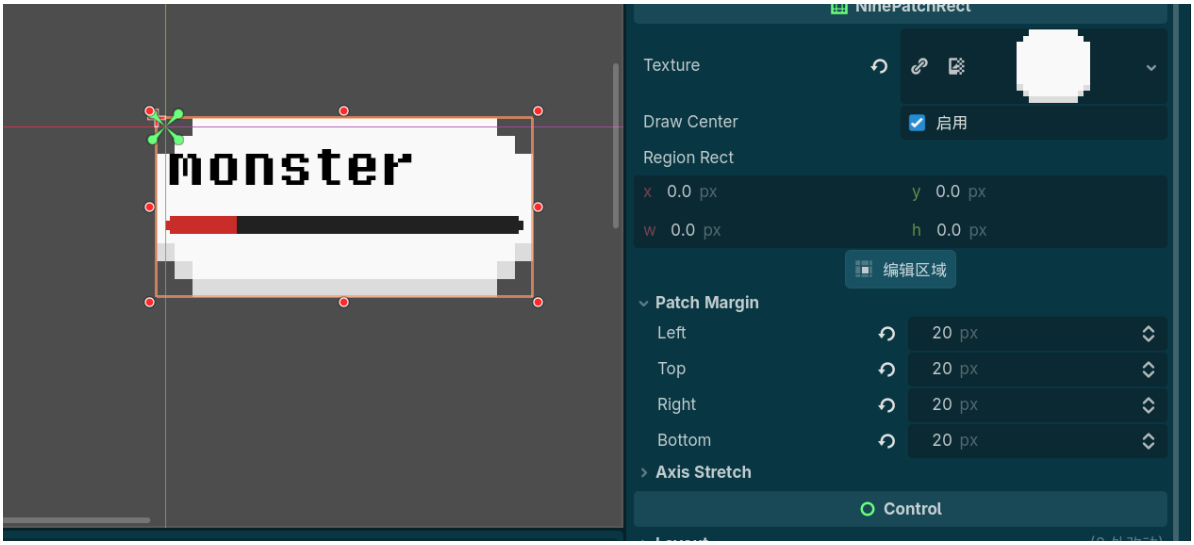
针对label，我们要在ThemeOverride里设置文字颜色，直接使用黑色即可。

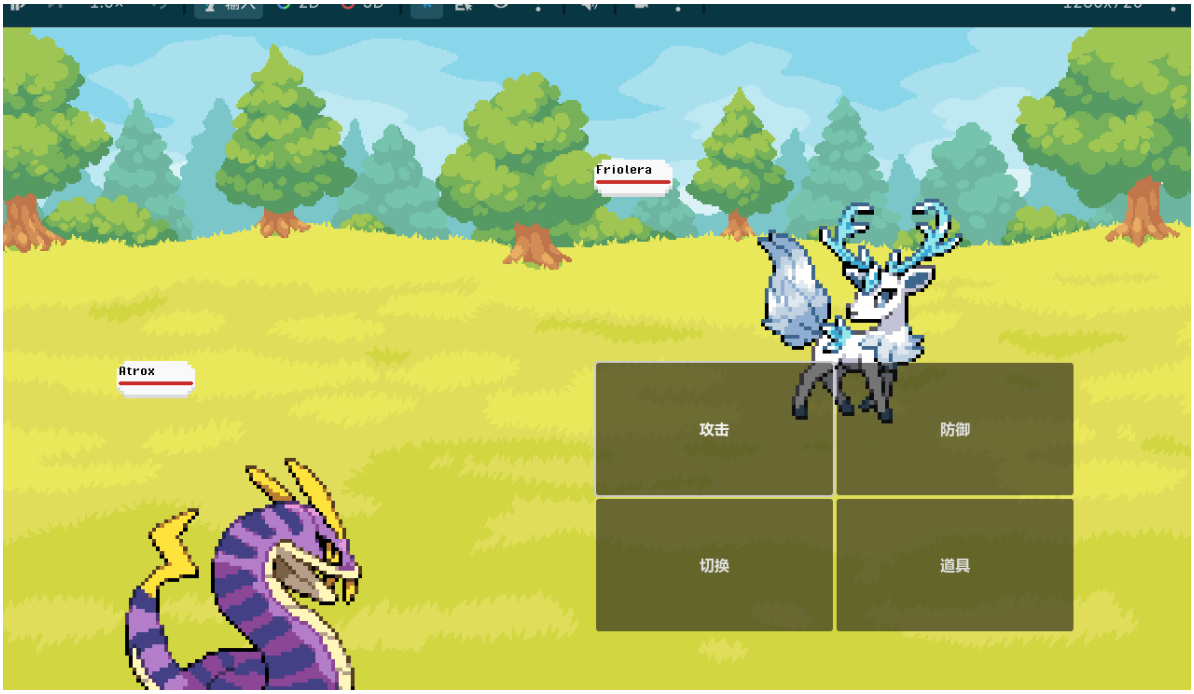


字体方面，我们加载res://graphics/fonts/PixelOperator-Bold.ttf，直接拖入字体对应属性栏即可，如图。

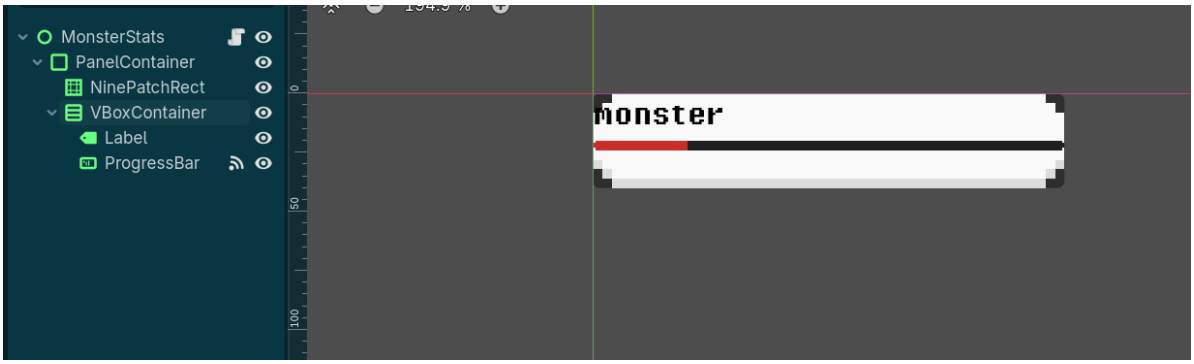


另外，我们还需要一张背景图，为此我们给MonsterStats场景新增一个NinePatchRect节点，我们需要给这个节点载入贴图，选用res://graphics/ui/frame.png，为了让贴图铺满整个节点，我们设置Patch Margin宫格边距上下左右均为20，如图。





可以看到现在ui样式还是不太完美，还需要微调尺寸。我们再为MonsterStats场景添加一个PanelContainer节点，给这个节点设置最小尺寸，宽度200px，接下来，将根节点下其他所有节点都移动到PanelContainer内部，如图。



接下来为了保证游戏正常运行，所有资源访问路径都需要重新修改。

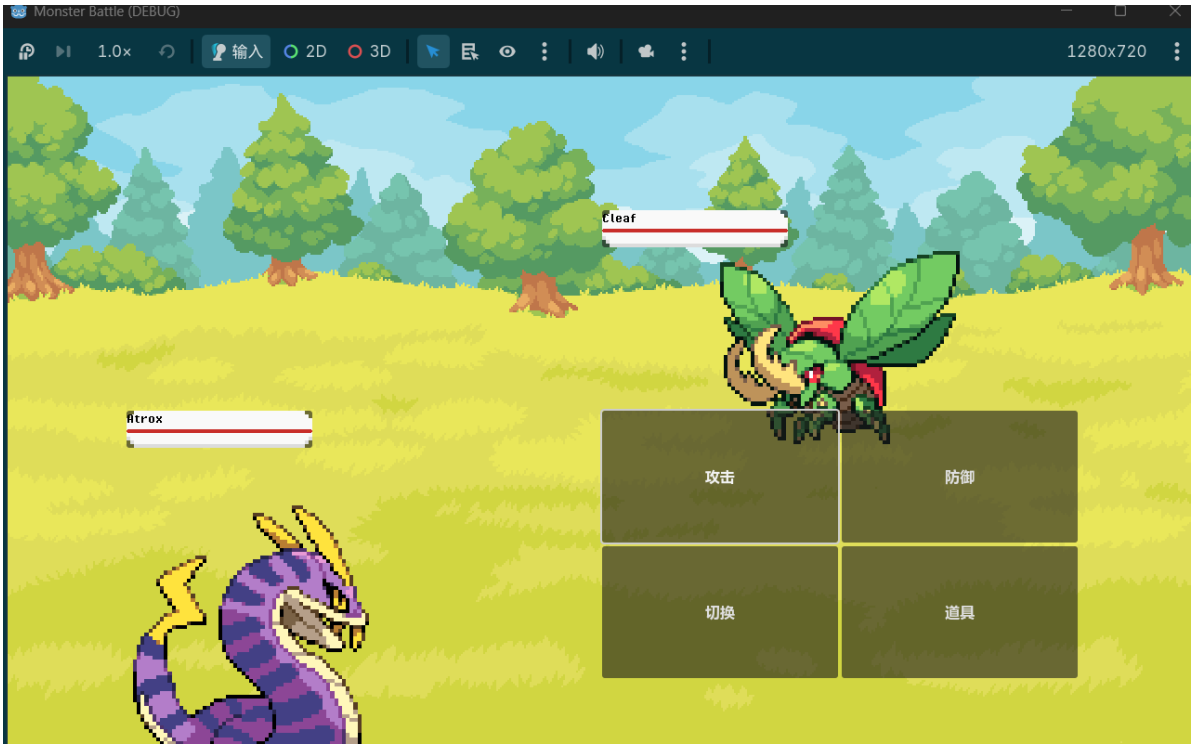
```gdscript
var is_player: bool
signal defeat(is_player: bool)


func setup(monster: Global.Monster, player = true):
    var monster_data = Global.monster_data[monster]
    $PanelContainer/VBoxContainer/Label.text = monster_data['name']
    $PanelContainer/VBoxContainer/ProgressBar.max_value = monster_data['max health']
    $PanelContainer/VBoxContainer/ProgressBar.value = monster_data['max health']
    is_player = player

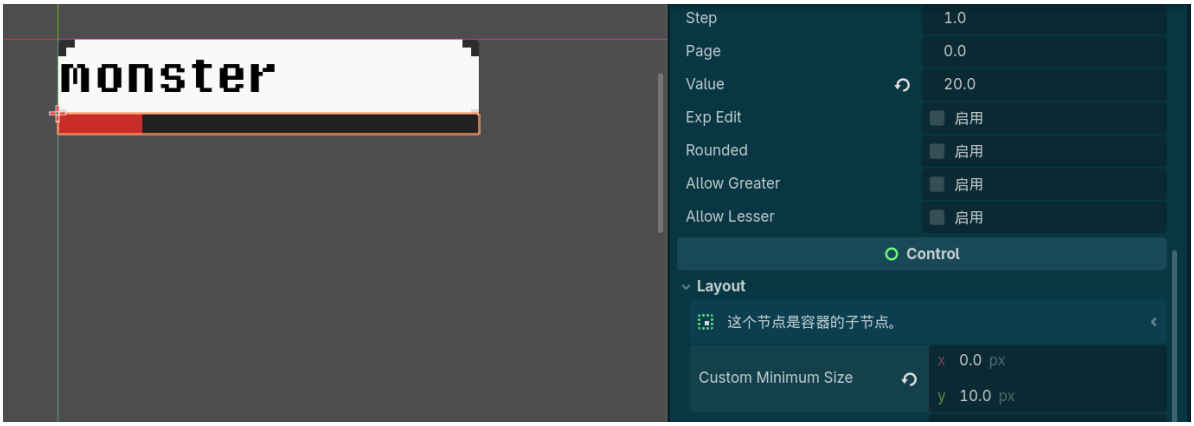
func update(data: Dictionary):
    $PanelContainer/VBoxContainer/ProgressBar.value -= data['amount']

func _on_progress_bar_value_changed(value: float) -> void:
    if value <= 0:
        defeat.emit(is_player)
```

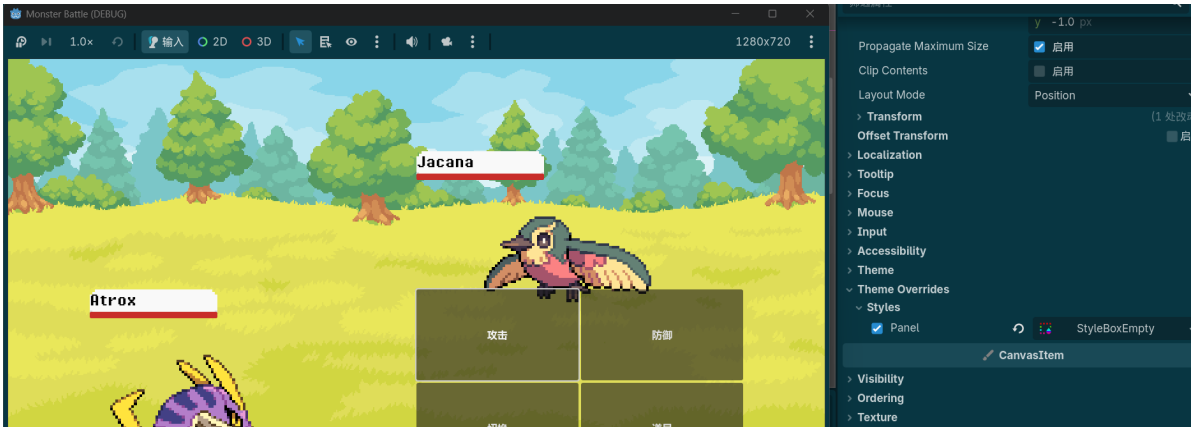
将所有路径修改完成后，运行游戏测试，如图。



当前整体尺寸还是偏小，解决办法是选中label，将字体大小设置为30，选中进度条设置其最小高度为10px，效果如图。



还有一些瑕疵，观察四个角可以看到PanelContainer自带的灰色底色，消除底色的办法是打开Theme Override，点开样式分类，给这个面板选择StyleBoxEmpty，这样这个容器的底色就会完全隐藏，如图。



除此之外，我想给所有显示内容预留内边距，实现方式是在PanelContainer下新建MarginContainer，将VBoxContainer拖入其下，完成后为MarginContainer设置上下左右边距，设置为10px，如图。



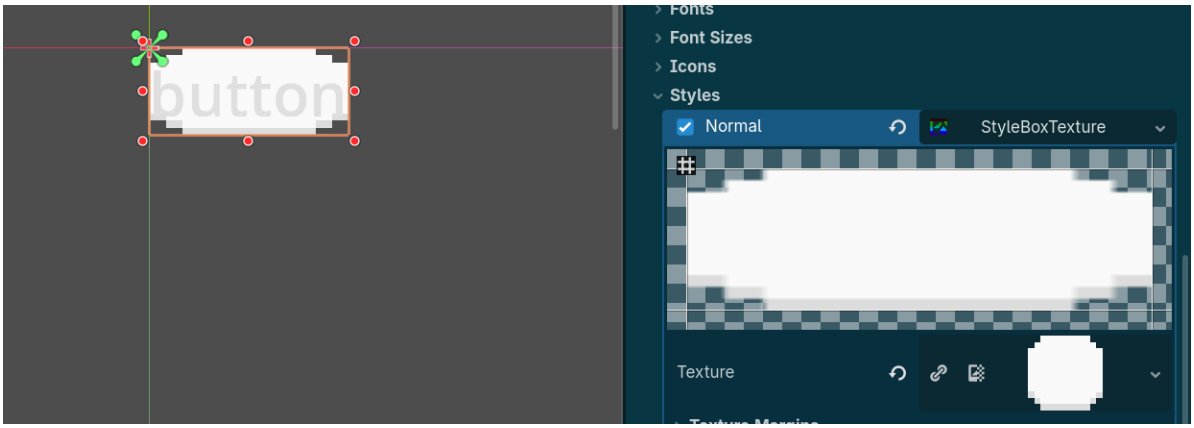
到这里我们又要重新修改全部资源访问路径，如图。

```
5
6
7  func setup(monster: Global.Monster, player = true):
8      var monster_data = Global.monster_data[monster]
9      $PanelContainer/MarginContainer/VBoxContainer/Label.text = monster_data['name']
10     $PanelContainer/MarginContainer/VBoxContainer/ProgressBar.max_value = monster_data['max health']
11     $PanelContainer/MarginContainer/VBoxContainer/ProgressBar.value = monster_data['max health']
12     is_player = player
13
14 func update(data: Dictionary):
15     $PanelContainer/MarginContainer/VBoxContainer/ProgressBar.value -= data['amount']
16
```

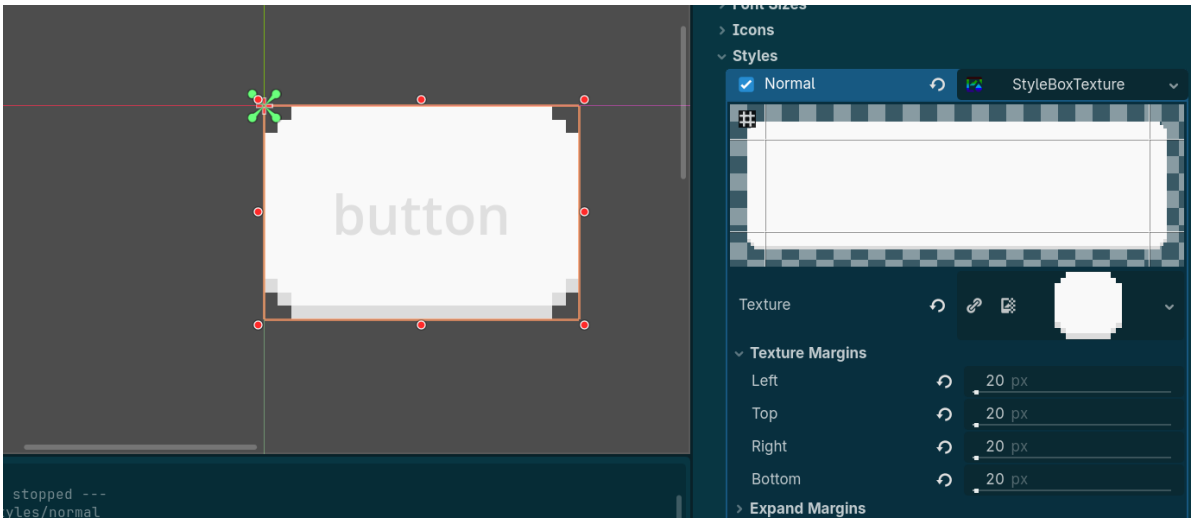
现在我们重新运行游戏，状态栏已正常显示，效果很好，如图。



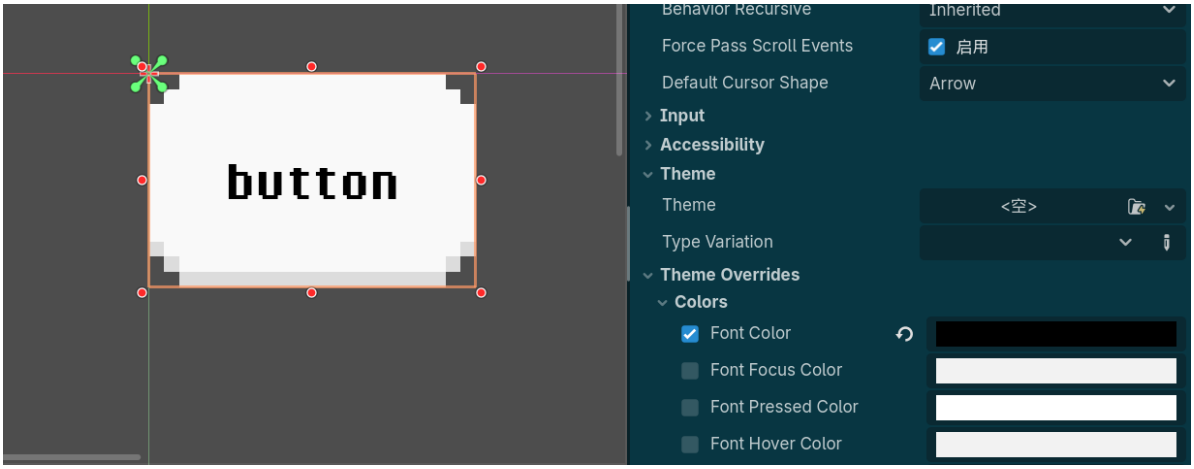
接下来，我们还需要完成grid_button和list_button的样式美化，选中girdbutton节点，在Theme Overrides中，为其styles中的Normal添加StyleBoxTexture，再次点击该StyleBoxTexture即可为其添加贴图资源，选择res://graphics/ui/frame.png，如图。



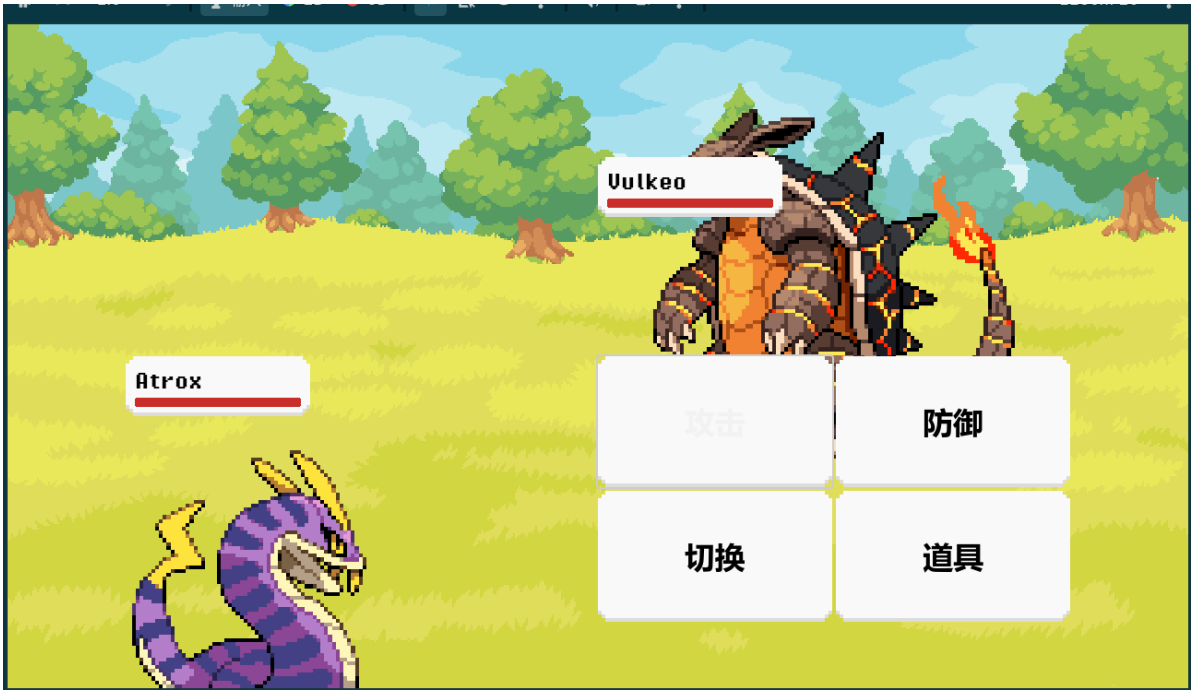
和状态栏一样，也为网格按钮设置边距，防止其拉伸变形四边统一设置为20，如图。



字体同样设置为res://graphics/fonts/PixelOperator-Bold.ttf，字体颜色设置为黑色，字体大小为33，如图。



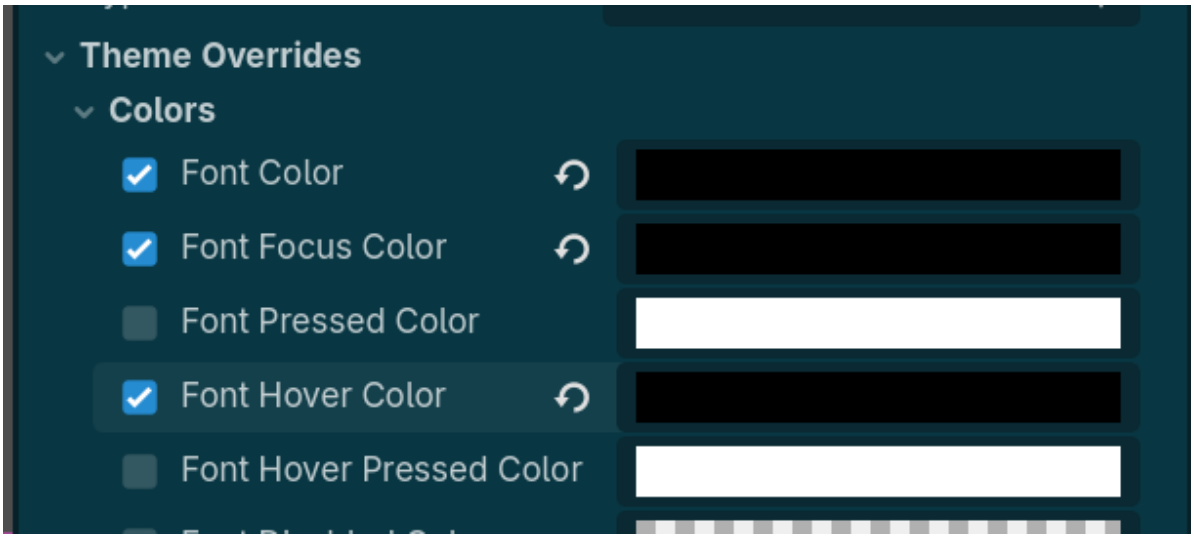
运行游戏测试，观感好了很多，但按键默认选中攻击，导致按钮选中高亮效果有问题，如图。



原因是我们只给Normal状态配置了样式，Hover按钮悬停时和Focus聚焦状态还有Pressed按下状态也要配置样式，样式容器选用StyleBoxTexture。



这四种按钮状态都要逐一设置，都是Hover和Focus的贴图选择res://graphics/ui/frame_selected.png，纹理边距统一设置为20。顺便设置字体颜色，Font Hover Color和Font Focus Color设置为黑色，如图。



运行游戏测试，观感大幅提升



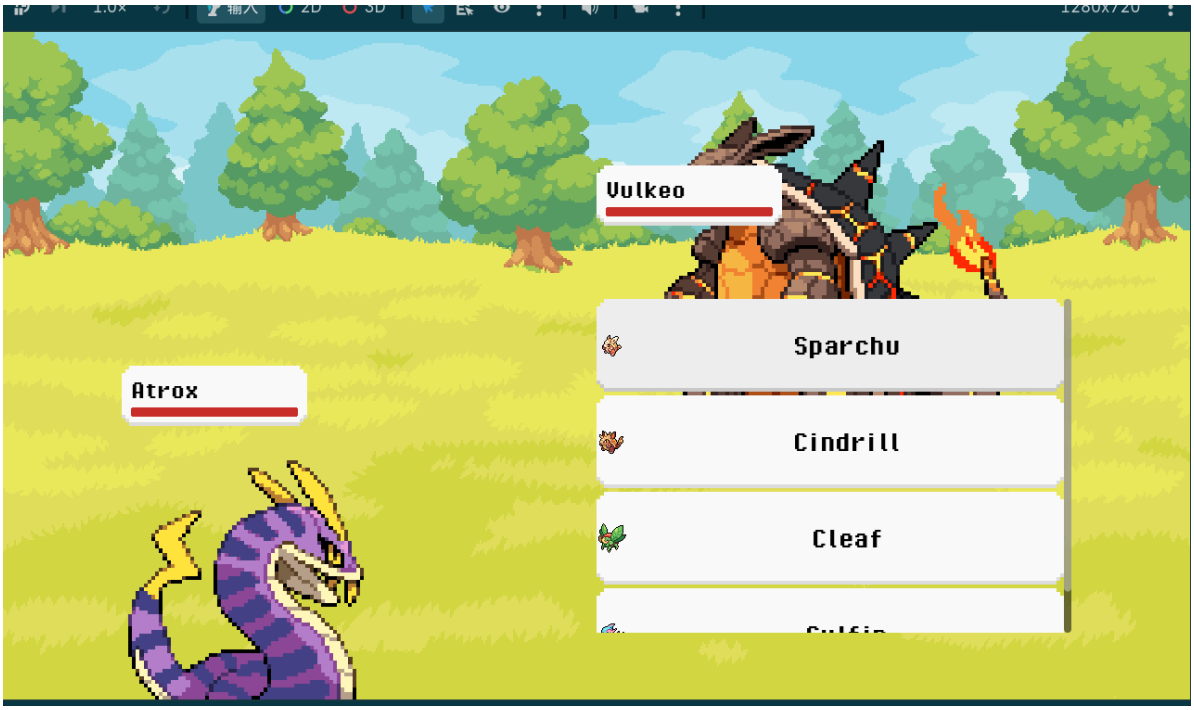
最后要配置的是pressed状态，贴图依旧选择res://graphics/ui/frame_selected.png，并设置边距，基础样式就配置的差不多了。

接下来是list_button的样式，包含按钮本体、图片、文字三个部分。

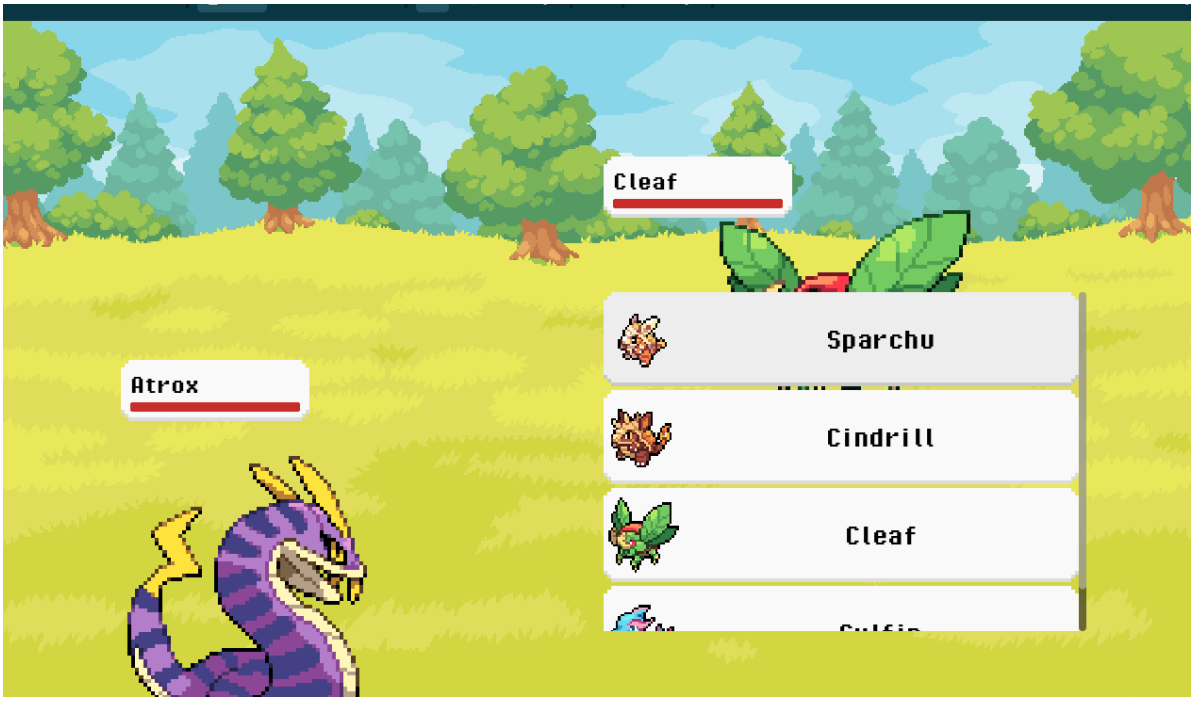
先选中Label，设置字体样式为res://graphics/fonts/PixelOperator-Bold.ttf，大小为33。

随后选中ListButton，调整按钮自身样式。和上面大同小异，配置Normal、Pressed、Hover和Focus的样式，搭配对应贴图边距，这一部分操作比较繁琐。

配置完成后运行游戏，整体效果提升非常明显，如图。



还有一处问题就是图标尺寸，当前图标太小了。选中TextureRect，设置最小尺寸为80*80，拉伸模式从keep改为Keep Aspect，运行游戏测试。



最后是滚动条，它和整体画风较为违和，我们可以在ScrollContainer的检查器面板找到Scrollbar设置，其中可以设置水平和垂直滚动条的显示和隐藏，且不会影响滚动效果。